

Посібник користувача панелі 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Версія 3X-UI: 3.5.0. Посібник складено за цією версією і є актуальним для неї. Зведення змін 3.5.0 відносно 3.4.2 – у розділі [«Що нового в 3.5.0»](#).

Докладний україномовний посібник з веб-панелі **3X-UI** (управління Xray-core): функції, налаштування та експлуатація, з розбором кожного поля і перемикача в інтерфейсі.

Назви та підписи відповідають інтерфейсу панелі; українські терміни наведено так, як вони відображаються в інтерфейсі. Слова *inbound* / *outbound* не перекладаються.

Зміст

- Що нового в 3.5.0
- 1. Вступ, вимоги та встановлення
 - 1.1. Що таке 3X-UI
 - 1.2. Підтримувані операційні системи та архітектури
 - 1.3. Способи встановлення
 - 1.4. Перший запуск і облікові дані за замовчуванням
 - 1.5. Розташування файлів
 - 1.6. Команда керування `x-ui` (меню скрипту)
 - 1.7. Підкоманди `x-ui` (без інтерактивного меню)
 - 1.8. Міграція SQLite → PostgreSQL
- 2. Вхід до панелі та безпека доступу
 - 2.1. Форма входу
 - 2.2. Двофакторна автентифікація (2FA / TOTP)
 - 2.3. Обмеження спроб входу (login limiter / захист від перебору)
 - 2.4. Зміна логіна та пароля адміністратора
 - 2.5. Секретний шлях (URI-шлях / webBasePath) і порт панелі
 - 2.6. Час життя сесії (таймаут)
 - 2.7. LDAP (синхронізація та автентифікація)
- 3. Огляд / Дашборд
 - 3.1. Загальні принципи збору даних
 - 3.2. ЦП (CPU)
 - 3.3. Пам'ять (RAM)
 - 3.4. Підкачка (Swap)
 - 3.5. Диск (Storage)
 - 3.6. Час роботи системи (Uptime)
 - 3.7. Навантаження системи (Load average)
 - 3.8. Мережа: швидкість та загальний обсяг трафіку
 - 3.9. IP-адреси сервера
 - 3.10. TCP/UDP з'єднання
 - 3.11. Статус Xray та керування процесом
 - 3.12. Оновлення панелі (3X-UI)
 - 3.13. Оновлення гео-файлів (GeoIP / GeoSite)
 - 3.14. Резервне копіювання та відновлення бази даних
 - 3.15. Додаткові елементи інтерфейсу
- 4. Inbounds: створення та загальні параметри
 - 4.1. Загальні поля форми
 - 4.2. Sniffing (Сніфінг)
 - 4.3. Allocate (стратегія розподілу портів)
 - 4.4. External Proxy (Зовнішній проксі)
 - 4.5. Fallbacks (Fallback'и)
 - 4.6. Періодичне скидання трафіку
 - 4.7. JSON входящего (advanced)
 - 4.8. Дії з inbound: QR / Edit / Reset / Delete та статистика

- 5. Протоколи
 - 5.1. Список підтримуваних протоколів
 - 5.2. Які протоколи підтримують TLS / REALITY / транспорт
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (прозорий форвардер)
 - 5.8. SOCKS / HTTP (протокол `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (за замовчуванням v2)
 - 5.11. MTPROTO (проксі для Telegram)
 - 5.12. Коротка шпаргалка щодо вибору протоколу
- 6. Транспорт (Stream Settings)
 - 6.1. Вибір мережі передачі
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Транспорт Hysteria (`hysteriaSettings`)
 - 6.9. Супутні параметри
- 7. Безпека з'єднання: TLS, XTLS та REALITY
 - 7.1. У чому різниця: TLS vs XTLS vs REALITY
 - 7.2. Режим «Немає» (`none`)
 - 7.3. Режим TLS
 - 7.4. Режим REALITY
 - 7.5. Практичні рекомендації щодо налаштування
- 8. Клієнти
 - 8.1. Поля клієнта
 - 8.2. Прив'язка до inbound
 - 8.3. Операції над клієнтом
 - 8.4. Масові операції
 - 8.5. Пошук, фільтри і сортування
 - 8.6. Значки і статуси
- 9. Групи клієнтів
 - 9.1. Що таке група клієнтів і навіщо вона потрібна
 - 9.2. Зв'язок групи з клієнтами, inbound, нодами та протоколами
 - 9.3. Довідник груп і «порожні» групи
 - 9.4. Поля та стовпці групи
 - 9.5. Створення групи
 - 9.6. перейменування групи
 - 9.7. Додавання клієнтів до групи
 - 9.8. Видалення клієнтів із групи (без видалення самих клієнтів)
 - 9.9. Скидання трафіку групи

- 9.10. Видалення групи та видалення клієнтів групи
- 9.11. Зв'язок зі сторінкою «Клієнти»
- 9.12. Зведення ендпоінтів API
- 9.13. Трафік за групою
- 10. Підписки (Subscription)
 - 10.1. Що таке subId і як формується посилання
 - 10.2. Налаштування сервера підписок
 - 10.3. Формати виведення
 - 10.4. Сторінка інформації про підписку та QR-коди
 - 10.5. Кастомні шаблони сторінки підписки
- 11. Xray: маршрутизація, outbounds, DNS та розширення
 - 11.1. Структура редактора: вкладки/режими
 - 11.2. Основні налаштування (General)
 - 11.3. Правила маршрутизації (routing)
 - 11.4. Outbounds (вихідні підключення)
 - 11.5. Балансувальники (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-проксі та TUN
 - 11.10. Логи та статистика (Stats, metrics)
 - 11.11. Збереження, перезапуск та автоматичні перетворення
 - 11.12. Outbound з підписки (з авто-оновленням)
 - 11.13. Ротація IP у WARP
- 12. Вузли (мультипанель, master/slave)
 - 12.1. Зведення вгорі списку
 - 12.2. Додавання та редагування вузла
 - 12.3. Перевірка TLS (для https-вузлів)
 - 12.4. Що показується по кожному вузлу
 - 12.5. Дії над вузлом
 - 12.6. Історія метрик
 - 12.7. Як синхронізуються inbounds і клієнти
 - 12.8. Ланцюжки вузлів (підвузли / транзитивні вузли)
 - 12.9. Виправлення 3.5.0 (стабільність вузлів)
 - 12.10. Вузли: нове в 3.3.0
- 13. Налаштування панелі
 - 13.1. Збереження та перезапуск панелі
 - 13.2. Загальні налаштування (вкладка «Панель» / *General*)
 - 13.3. Доступ до панелі: IP, порт, шлях, домен, сертифікат
 - 13.4. Сесія, проксі панелі та довірені проксі (вкладка «Проксі та сервер» / *Proxy and Server*)
 - 13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)
 - 13.6. Дата і час (вкладка «Дата і час» / *Date and Time*)
 - 13.7. Зовнішній трафік і поведінка Xray (вкладка «Зовнішній трафік» / *External Traffic*)
 - 13.8. Інше: шаблон конфігурації Xray та перевірочний URL
 - 13.9. Обліковий запис адміністратора та API-токени
 - 13.10. Зміни API в 3.3.0 (важливо для інтеграцій)

- 14. Telegram-бот
 - 14.1. Увімкнення та налаштування бота
 - 14.2. Головне меню та кнопки
 - 14.3. Команди бота
 - 14.4. Керування клієнтом (тільки адміністратор)
 - 14.5. Сповіщення та звіти
 - 14.6. Резервне копіювання та логи
 - 14.7. Особливості роботи
- 15. Гео-бази (geoip / geosite та користувацькі)
 - 15.1. Що таке geoip.dat і geosite.dat
 - 15.2. Стандартні гео-файли та їх оновлення
 - 15.3. Авто-оновлення гео-даних засобами Xray (Geodata Auto-Update)
 - 15.4. Валідація та обмеження
 - 15.5. Автоперевірка під час старту панелі
 - 15.6. Використання гео-баз у правилах маршрутизації
- 16. Експлуатація: резервні копії, логи, оновлення, CLI
 - 16.1. Резервне копіювання та відновлення бази даних
 - 16.2. Перегляд логів
 - 16.3. Рівень і налаштування логування Xray
 - 16.4. Управління Xray: зупинка і перезапуск
 - 16.5. Перезапуск і оновлення панелі
 - 16.6. Periodical tasks (cron)
 - 16.7. Консольне меню і CLI (`x-ui`)
 - 16.8. Видалення панелі
 - 16.9. Команда `x-ui migrateDB`

Що нового в 3.5.0

Версія 3.5.0 — великий реліз: MTProto переведено на мультиклієнтну модель (рушій mtg-multi, персональні секрети, квоти та ad-tag), керовані хости стали груповими (кілька inbound і адрес в одному записі), відновлення на PostgreSQL-панелі приймає SQLite-бекапи, в outbound з'явилися «Target Strategy», тест «Real delay» і стовпці Egress/Country, балансувальник може використовувати інший балансувальник як fallback. У комплекті — ядро Xray 26.7.11. Нижче — зміни відносно 3.4.2 за розділами посібника.

Зміни в розділі 1 — Вступ, вимоги та встановлення

- Ядро Xray оновлено до **26.7.11**. Супутні авто-міграції: шифри Shadowsocks `none / plain` і VMess `none / zero` видалені з ядра (збережені конфіги переписуються автоматично), незашифрований VLESS/Trojan-outbound на публічну адресу відхиляється при збереженні.
- Нова команда `x-ui pgclient [версія]` і пункт **10. Install/Upgrade client tools (pg_dump/pg_restore)** у меню PostgreSQL — встановлення/оновлення клієнтських інструментів PostgreSQL.
- Виправлення скриптів: встановлення PostgreSQL і fail2ban на RHEL-сімействі (EPEL), Arch без повного `распан -Syu`, коректне ім'я бінарника Xray на 32-бітному ARM (`xray-linux-arm32`), підтвердження авто-визначеного IPv4 перед випуском IP-сертифіката, виправлено хибне «Your input is invalid» при виборі ACME-порту за замовчуванням.

Зміни в розділі 2 — Вхід до панелі та безпека доступу

- Ліміт IP: «мертве» з'єднання тепер банять **один раз**, а не при кожному 10-секундному скануванні — лічильники fail2ban більше не накручуються, завищувати `maxretry` не потрібно.

Зміни в розділі 4 — Inbounds: створення та загальні параметри

- У списку inbound з'явився **пошук** (за приміткою, портом і протоколом), а випадючі списки вузлів («Развернуть на», фільтр «Вузли») стали пошуковими.

Зміни в розділі 5 — Протоколи

- **MTProto переведено на мультиклієнтну модель** (рушій mtg-multi): користувачі MTProto — тепер звичайні клієнти зі своїм секретом, квотою, терміном, ad-tag і персональним посиланням `tg://proxy`. Поле «Secret» на рівні inbound прибрано (наявні inbound автоматично конвертуються), «FakeTLS domain» став доменом за замовчуванням для нових секретів. Нові поля inbound: **Max connections** (обмеження з'єднань), **Public IPv4/IPv6** (для ad-tag middle proxy). Зміни клієнтів застосовуються «на льоту», не обриваючи чужі Telegram-сесії.
- WireGuard: меню inbound отримало повний набір клієнтських дій (Export All URLs, прив'язка/відв'язка, групи), експорт розділено на вкладки **Config** і **Links**, поле «Дозволені IP WireGuard» стало редагованим (кілька записів через кому), у конфігу клієнта вузлового inbound `Endpoint` тепер вказує на адресу вузла.

Зміни в розділі 7 — Безпека з'єднання: TLS, XTLS та REALITY

- Комбінація **Finalmask + REALITY** відхиляється при збереженні (вона призводила до падіння Xray-core при першому підключенні); плейсхолдер minClientVer оновлено до 26.3.27.
- Новий тип TCP-маски Finalmask — **ХМC (Minecraft)**: маскування потоку під трафік Minecraft (Hostname, Usernames, обов'язковий Password з автогенерацією).

Зміни в розділі 8 — Клієнти

- Новий стовпець «**Швидкість**» — жива швидкість кожного клієнта (↑/↓, ковзне ~5-секундне середнє).
- Пошук клієнтів знову знаходить за **Telegram ID**; у формі клієнта вимкнені inbound приховані зі списку прив'язки; виправлено накопичення дублікатів у вікні «Відв'язати».
- У MTPProto-клієнтів — свої поля: «**MTPProto secret**» (з регенерацією) і «**Ad-tag (спонсорський канал)**» (32 hex-символи); квота і термін тепер реально застосовуються до MTPProto.

Зміни в розділі 9 — Групи клієнтів

- У вікні інформації про клієнта тепер показується його **група**.

Зміни в розділі 10 — Підписки (Subscription)

- **Керовані хости стали груповими**: один запис охоплює **кілька inbound** (мульти-вибір) і **кілька адрес** (теги, у кожного запису можливий свій `port` ; автодоповнення адрес; порожньо — успадкування адреси inbound). Стовпці списку показують чіпи адрес та inbound (з «+N»), дії та API працюють із групами (`groupId`), з'явився масовий `POST /panel/api/hosts/bulk/add` . Сортювання хостів тепер глобальне (за порядком, потім за приміткою).
- Текст **оголошення** (`subAnnounce`) тепер показується банером на інфо-сторінці підписки; у кастомних шаблонах доступна змінна `announce` .
- Інфо-сторінка в браузері відкривається тепер і за **JSON/Clash-посиланнями** (не лише за основним).
- Host-налаштування **Final Mask** і **Allow insecure** тепер діють і в raw-посиланнях (`fm=`) та для **Hysteria2** (`insecure=1 / skip-cert-verify: true`) відповідно.
- Діапазон «Інтервали оновлення» (`subUpdates`) виправлено на **0–525600** (попередня межа UI 168 блокувала збереження налаштувань після апгрейду з 2.x).
- Нативні **WireGuard-клієнти тепер потрапляють у Clash- і JSON-підписки** (раніше — лише в raw).

Зміни в розділі 11 — Xray: маршрутизація, outbounds, DNS та розширення

- Outbound-редактор: нове поле «**Target Strategy**» (11 значень від `AsIs` до `ForceIPv4`), режим тесту «**Real delay**» (повний час зі встановленням тунелю; HTTP-режим тепер вимірюється за «теплим» з'єднанням), стовпці **Egress** (IP виходу за «оком») і **Country** (прапор + країна, мітка WARP) після HTTP/Real-тесту.
- **Fallback балансувальника може вказувати на інший балансувальник**: панель сама будує прихований loopback-об'єкт (`_bl_...`), захищає від циклів і від видалення використовуваного балансувальника; префікс `_bl_` зарезервований.
- Вкладка «Базова маршрутизація» отримала селектор «**Default Outbound**» — який outbound обробляє трафік, що не потрапив під жодне правило (вибраний переміщується на першу позицію).

- DNS-сервери на приватних IP більше не блокуються правилом `geoip:private` — панель сама підтримує кероване allow-правило.
- Happy Eyeballs у налаштуваннях dial (sockopt) тепер справді вмикається; «Try delay» за замовчуванням 250 мс, явний 0 зберігається.
- Імпорт підписок outbound: у `ss://` -посилань з `?plugin=` /кінцевим `/` коректно розбирається порт.

Зміни в розділі 12 — Вузли (мультипанель, master/slave)

- Пакет виправлень: збереження клієнта без змін більше не руйнує живий трафік вузлових inbound; Host-перевизначення вузла приймаються в майстер при першому прийнятті; авто-продовження відкриває свіже вікно квоти; видалення клієнта на майстері повністю видаляє його на вузлах; inbound вузла не вимітаються до першого прийняття; один некоректний inbound не зупиняє синхронізацію трафіку вузла; перевірка конфлікту портів обмежена своїм вузлом.

Зміни в розділі 14 — Telegram-бот

- У меню команд бота додано `/usage`, `/inbound`, `/restart` і нову адмін-команду `/clearall` (скидання трафіку всіх клієнтів, з підтвердженням).
- Список онлайн-клієнтів підписується як `email` — примітка `inbound`; повідомлення бекапу і бан-лога містять ім'я хоста; пошук за Telegram ID працює незалежно від форматування налаштувань.

Зміни в розділі 16 — Експлуатація: резервні копії, логи, оновлення, CLI

- **Відновлення на PostgreSQL-панелі приймає SQLite-файли:** звичайний бекап `.db` або міграційний `.dump` імпортується прямо в PostgreSQL (однією транзакцією, з перевітками до зупинки Xray). Діалог вибору файлу приймає `.dump`, `.db` на обох СУБД; кнопка «Завантажити файл міграції» залишилася лише на PostgreSQL-панелях.
- Перед відновленням `pg_dump` -архіву панель перевіряє читабельність дампа і при розбіжності версій підказує точну команду `x-ui pgclient <версія>`.
- Автовиправлення під час старту: перепоповнені лічильники трафіку затискаються і відновлюються; знімається застаріле обмеження UNIQUE на порт inbound (заважало multi-node).
- Логи Xray: новий джоб кожні 10 хвилин обрізає access- і error-лог при перевищенні **64 МБ**; щоденне очищення тепер чистить обидва.
- Docker: автопродовження сертифікатів полагоджено (crond запускається, стан `asme.sh` зберігається в томі).

1. Вступ, вимоги та встановлення

1.1. Що таке 3X-UI

3X-UI — це відкрита веб-панель керування для серверів [Xray-core](#). Панель надає єдиний багатомовний веб-інтерфейс для розгортання, налаштування та моніторингу широкого набору проксі- та VPN-протоколів: від одиночного VPS до розподілених конфігурацій із кількох вузлів (нод).

3X-UI — це розширений форк оригінального проекту X-UI. Порівняно з ним додано підтримку більшої кількості протоколів, підвищено стабільність, реалізовано обліковий (для кожного клієнта) облік трафіку та безліч зручних функцій.

Основні можливості:

- **Inbound різних протоколів** — VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN та **MTPROTO** (проксі Telegram, додано у 3.3.0).
- **Сучасні транспорти та шифрування** — TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade та XHTTP, захищені TLS, XTLS і REALITY.
- **Fallback** — обслуговування кількох протоколів на одному порту (наприклад, VLESS і Trojan на 443) засобами fallback в Xray.
- **Керування кожним клієнтом** — квоти трафіку, дати закінчення терміну дії, ліміти IP, відображення статусу «онлайн», запрошувальні посилання в один клік, QR-коди та підписки.
- **Статистика трафіку** — для кожного inbound, клієнта та outbound, з можливістю скидання.
- **Підтримка кількох вузлів (нод)** — керування та масштабування на кілька серверів з однієї панелі.
- **Outbound і маршрутизація** — WARP, NordVPN, користувацькі правила маршрутизації, балансувальники навантаження, ланцюжки проксі.
- **Вбудований сервер підписок** із кількома форматами виводу.
- **Telegram-бот** для віддаленого моніторингу та керування.
- **REST API** із вбудованою документацією Swagger.
- **Гнучке сховище** — SQLite (за замовчуванням) або PostgreSQL.
- **13 мов інтерфейсу**, темна та світла теми.
- **Інтеграція з Fail2ban** для застосування лімітів IP для кожного клієнта.

Важливо: проект призначено лише для особистого використання. Не рекомендується застосовувати його в незаконних цілях або у продакшен-середовищі.

1.2. Підтримувані операційні системи та архітектури

Операційні системи

Встановлювальний скрипт визначає дистрибутив за полем `ID` з `/etc/os-release` (або `/usr/lib/os-release`). Офіційно підтримуються:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, а також Windows.

На системах сімейства Alpine використовується служба OpenRC (`rc-service` / `rc-update`), на решті – `systemd`. Для CentOS 7 пакети встановлюються через `yum`, для новіших версій – через `dnf`. Якщо дистрибутив не розпізнано, скрипт за замовчуванням намагається використати менеджер пакетів `apt-get`.

Архітектури процесора

Архітектура визначається за виводом `uname -m` і зводиться до одного з підтримуваних значень:

Значення <code>uname -m</code>	Архітектура 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Якщо архітектура не входить до цього списку, скрипт виводить повідомлення «Unsupported CPU architecture!» і припиняє встановлення.

Базові залежності

Перед встановленням панелі скрипт автоматично встановлює базовий набір пакетів (назви різняться залежно від дистрибутива): `cron` / `cronie` / `dcron`, `curl`, `tar`, `tzdata` / `timezone`, `socat`, `ca-certificates`, `openssl`.

1.3. Способи встановлення

Спосіб 1. Встановлювальний скрипт (рекомендується)

Встановлення виконується однією командою від імені `root`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Скрипт обов'язково вимагає прав `root`: при запуску не від `root` виводиться «Please run this script with root privilege» і завершується з помилкою.

Що робить встановлювач покроково:

1. Визначає ОС та архітектуру.
2. Встановлює базові залежності.
3. Завантажує архів релізу `x-ui-linux-<arch>.tar.gz` і розпаковує його до каталогу `/usr/local/x-ui`.
4. Завантажує керуючий скрипт `x-ui.sh` і встановлює його як команду `/usr/bin/x-ui`.

5. Створює каталог логів `/var/log/x-ui`.
6. Запускає первинне налаштування: вибір бази даних, генерація облікових даних, вибір порту, опціональне налаштування SSL.
7. Встановлює і запускає службу автозапуску (systemd-юніт `x-ui.service` або init-скрипт OpenRC для Alpine).

Вибір бази даних під час встановлення. Встановлювач пропонує:

- 1) SQLite (за замовчуванням, рекомендується при кількості клієнтів < 500) — один файл `/etc/x-ui/x-ui.db`, не потребує налаштування.
- 2) PostgreSQL (рекомендується при великій кількості клієнтів або кількох нодах). PostgreSQL можна встановити локально (створюється виділений користувач і БД з іменем `xui`) або вказати DSN до вже наявного сервера. Параметри підключення записуються у файл оточення служби (`/etc/default/x-ui`, `/etc/conf.d/x-ui` або `/etc/sysconfig/x-ui` залежно від дистрибутива) у вигляді змінних `XUI_DB_TYPE` і `XUI_DB_DSN`.

Приклад: запис параметрів PostgreSQL у файл оточення служби. Після вибору PostgreSQL та вказівки DSN встановлювач додасть у файл оточення приблизно такі рядки:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Тут `xui` — ім'я користувача і бази, `127.0.0.1:5432` — адреса і порт сервера, `sslmode=disable` підходить для локального підключення (для віддаленого сервера зазвичай використовують `require`).

Встановлення конкретної (старої) версії. Можна явно вказати тег версії — встановлювач завантажить відповідний реліз:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Мінімально допустима версія для такого встановлення — `v2.3.5`; при вказівці старішої виводиться «Please use a newer version (at least v2.3.5)».

Встановлення dev-збірки. Окрім тега версії встановлювач приймає аргумент `dev-latest` (псевдонім `dev`) — це встановлює rolling dev-збірку за останнім комітом гілки `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Dev-збірка — це per-commit передреліз (тег `dev-latest`), а не стабільна версія, тому перевірка мінімально допустимої версії для неї не виконується. При запуску виводиться попередження «Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.». Без аргументу встановлювач ставить останній стабільний реліз. Використовувати dev-збірку має сенс лише для перевірки ще не випущених виправлень; у звичайній експлуатації встановлюйте стабільні версії.

Список 2. Docker

Запуск із базою SQLite за замовчуванням:

```
docker compose up -d
```

Для запуску із вбудованим сервісом PostgreSQL потрібно розкоментувати рядки `XUI_DB_*` у `docker-compose.yml` і запустити з профілем:

```
docker compose --profile postgres up -d
```

Образ включає Fail2ban (за замовчуванням активний) для застосування лімітів IP за клієнтами. Fail2ban блокує порушників через `iptables`, що вимагає можливості `NET_ADMIN`. У `docker-compose.yml` вона вже надана через `cap_add`. З 3.5.0 у контейнері покладено **автопродовження сертифікатів**, випущених через SSL-меню: `entrypoint` запускає `crond` і заново реєструє cron-завдання `asme.sh`, а `docker-compose.yml` отримав окремий том для стану `asme.sh` — продовження переживає перестворення контейнера (раніше сертифікати мовчки закінчувалися через ~90 днів). При ручному запуску через `docker run` можливості потрібно додати самостійно, інакше блокування будуть лише логуватися, але не застосовуватися:

Приклад: повна команда `docker run`. Мінімальний варіант із прокиданням порту панелі, мережевими можливостями та постійним томом для бази:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Том `/etc/x-ui` зберігає файл `x-ui.db` між перезапусками контейнера, інакше налаштування та акаунти будуть втрачатися.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

У Docker панель є основним процесом контейнера: автозапуск регулюється політикою перезапуску контейнера (наприклад, `restart: unless-stopped`), а не службою всередині контейнера.

1.4. Перший запуск і облікові дані за замовчуванням

При першому встановленні (коли ще використовуються облікові дані за замовчуванням) встановлювач **генерує випадкові значення** імені користувача, пароля та веб-шляху, а також порту:

Параметр	Як формується при встановленні	Примітка
Ім'я користувача (Username)	випадковий рядок з 10 символів	генерується автоматично
Пароль (Password)	випадковий рядок з 10 символів	генерується автоматично
Веб-шлях панелі (WebBasePath)	випадковий рядок з 18 символів	захищає панель від виявлення за кореневим URL
Порт панелі (Port)	за замовчуванням випадковий порт у діапазоні 1024–62000; за бажанням можна задати вручну	«заводське» значення <code>webPort</code> дорівнює <code>2053</code> , але встановлювач його перезаписує

Наприкінці встановлення скрипт виводить підсумкову зведену інформацію: ім'я користувача, пароль, порт, веб-шлях, API-токен і готове посилання для входу (Access URL) вигляду:

```
https://<домен-або-IP>:<порт>/<веб-шлях>
```

Якщо SSL-сертифікат не налаштовано, посилання буде за `http://`, і скрипт виведе попередження про необхідність налаштувати SSL (пункт меню 19).

Обов'язкова зміна облікових даних. Оскільки логін і пароль генеруються випадково, їх слід **зберегти одразу після встановлення**. Змінити їх у будь-який момент можна пунктом меню «Reset Username & Password» (див. нижче) або з веб-інтерфейсу в налаштуваннях панелі. Після скидання скрипт нагадує: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Після встановлення для відкриття меню керування використовується команда `x-ui` (див. розділ 1.6).

1.5. Розташування файлів

Шлях	Призначення
<code>/usr/local/x-ui/</code>	каталог встановлення панелі (бінарний файл <code>x-ui</code> , скрипт <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	бінарний файл Xray-core (на armv5/armv6/armv7 перейменовується на <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	керуючий скрипт (команда <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	файл бази даних SQLite (за замовчуванням)
<code>/var/log/x-ui/</code>	каталог логів панелі
<code>/etc/systemd/system/x-ui.service</code>	systemd-юніт служби (не для Alpine)
<code>/etc/init.d/x-ui</code>	init-скрипт OpenRC (тільки Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	файл змінних оточення служби (шлях залежить від дистрибутива); сюди записуються <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

Каталог бази даних можна перевизначити змінною оточення `XUI_DB_FOLDER` (за замовчуванням `/etc/x-ui`), а каталог бінарних файлів Xray – змінною `XUI_BIN_FOLDER` (за замовчуванням `bin` відносно каталогу панелі). Ім'я файлу БД – `x-ui.db`.

Приклад: винесення бази на окремий диск. Щоб зберігати `x-ui.db` не в `/etc/x-ui`, а, наприклад, на змонтованому диску `/data`, задайте змінну у файлі оточення служби та перезапустіть панель:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Повний шлях до бази стане `/data/x-ui/x-ui.db`.

Основні змінні оточення

Змінна	Призначення	За замовчуванням
<code>XUI_DB_TYPE</code>	бекенд БД: <code>sqlite</code> або <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	рядок підключення PostgreSQL (при <code>XUI_DB_TYPE=postgres</code>)	–
<code>XUI_DB_FOLDER</code>	каталог файлу БД SQLite	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	початковий URI-шлях веб-панелі (тільки при першій ініціалізації)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	максимум відкритих з'єднань (пул PostgreSQL)	–
<code>XUI_DB_MAX_IDLE_CONNS</code>	максимум простоючих з'єднань (пул PostgreSQL)	–
<code>XUI_ENABLE_FAIL2BAN</code>	увімкнути застосування лімітів IP через Fail2ban	<code>true</code>
<code>XUI_LOG_LEVEL</code>	рівень логування (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
<code>XUI_DEBUG</code>	режим налагодження	<code>false</code>

Приклад: тимчасово увімкнути докладне логування. Для діагностики проблеми підніміть рівень логів до `debug` і перезапустіть службу:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log # перегляд налагоджувального логу
```

Після діагностики поверніть значення `info`, щоб лог не розростався.

Початковий шлях веб-панелі через оточення. Змінна `XUI_INIT_WEB_BASE_PATH` задає URI-шлях веб-панелі (`webBasePath`) при первинній ініціалізації налаштувань. Це зручно при розгортанні в Docker або через `systemd`, щоб одразу зафіксувати шлях входу до панелі. Значення нормалізується автоматично – провідний і замикаючий слеші додаються за потреби, а пусте або складене з пробілів значення ігнорується (тоді застосовується шлях за замовчуванням `/`). Змінна впливає **лише на первинну ініціалізацію**: якщо налаштування вже створено, шлях змінюється у веб-інтерфейсі або пунктом меню «Reset Web Base Path».

1.6. Команда керування `x-ui` (меню скрипту)

Після встановлення команда `x-ui` (запускається від `root`) відкриває інтерактивне меню «3X-UI Panel Management Script». Вибір пункту здійснюється введенням його номера (діапазон 0–27). Багато пунктів доступні й у вигляді підкоманд для скриптів (див. розділ 1.7).

Меню розбито на тематичні блоки.

Встановлення та оновлення

- **1. Install** — встановлення панелі (запускає `install.sh`). Перед встановленням перевіряється, що панель ще не встановлена.
- **2. Update** — оновлення всіх компонентів `x-ui` до останньої версії. Дані при цьому не втрачаються; після оновлення панель перезапускається автоматично. Потребує підтвердження.
- **3. Update Menu** — оновлення лише керуючого скрипту (`x-ui.sh` / команди `x-ui`) до актуальної версії без перевстановлення панелі.
- **4. Legacy Version** — встановлення вказаної (старої) версії панелі. Скрипт запитує номер версії (наприклад, `2.4.0`) і завантажує відповідний реліз.
- **5. Uninstall** — повне видалення панелі **разом із Xray**. Зупиняється і вимикається служба, видаляються каталоги `/etc/x-ui/` і `/usr/local/x-ui/`, файл оточення служби та сам керуючий скрипт. Потребує підтвердження (за замовчуванням «ні»).

Облікові дані та налаштування

- **6. Reset Username & Password** — скидання імені користувача та пароля панелі. Можна ввести власні значення або залишити порожніми для випадкової генерації (випадкове ім'я — 10 символів, випадковий пароль — 18 символів). Додатково пропонується вимкнути двофакторну автентифікацію (2FA), якщо вона налаштована. Після скидання панель перезапускається.
- **7. Reset Web Base Path** — скидання веб-шляху панелі: генерується новий випадковий шлях (18 символів), панель перезапускається. Використовується, якщо попередній шлях скомпрометовано або забуто.
- **8. Reset Settings** — скидання всіх налаштувань панелі до значень за замовчуванням. **Облікові дані (ім'я користувача та пароль) і дані акаунтів при цьому не втрачаються.** Потребує підтвердження; після скидання панель перезапускається.
- **9. Change Port** — зміна порту веб-панелі. Запитується номер порту (1–65535); після встановлення потрібен перезапуск, щоб порт набрав чинності.
- **10. View Current Settings** — перегляд поточних налаштувань (`x-ui setting -show`). Показує, зокрема, використовуваний бекенд БД (SQLite або PostgreSQL з маскованим паролем у DSN) і готове посилання доступу (Access URL). Якщо SSL не налаштовано, пропонує випустити сертифікат Let's Encrypt для IP-адреси.

Керування службою

- **11. Start** — запуск служби панелі. Якщо панель вже працює, виводиться повідомлення, що повторний запуск не потрібен.
- **12. Stop** — зупинка служби панелі.

- **13. Restart** — перезапуск служби панелі.
- **14. Restart Xray** — перезапуск лише ядра Xray-core без перезапуску самої панелі (через `systemctl reload x-ui`, у Docker — сигналом USR1 процесу панелі).
- **15. Check Status** — перевірка стану служби (`systemctl status x-ui` або `rc-service x-ui status`).
- **16. Logs Management** — керування логами: перегляд налагоджувального логу (Debug Log, через `journalctl`) і, крім Alpine, очищення всіх логів (Clear All logs).

Автозапуск

- **17. Enable Autostart** — увімкнути автозапуск панелі при завантаженні ОС (`systemctl enable x-ui` або `rc-update add`).
- **18. Disable Autostart** — вимкнути автозапуск при завантаженні ОС.

У Docker автозапуск регулюється політикою перезапуску контейнера, тому ці пункти лише виводять відповідну підказку.

Безпека та мережа

- **19. SSL Certificate Management** — керування SSL-сертифікатами через acme.sh: випуск сертифіката для домену, відкликання, примусове оновлення, перегляд наявних доменів, вказівка шляхів до сертифіката для панелі, а також випуск короточасного (~6 днів, з автооновленням) сертифіката для IP-адреси.
- **20. Cloudflare SSL Certificate** — випуск SSL-сертифіката через DNS-валідацію Cloudflare.
- **21. IP Limit Management** — керування лімітами кількості IP за клієнтами (на базі Fail2ban): перегляд і зняття блокувань тощо.
- **22. Firewall Management** — керування міжмережевим екраном (відкриття/закриття портів і перегляд правил).
- **23. SSH Port Forwarding Management** — налаштування прокидання SSH-портів, щоб відкривати панель з локальної машини через SSH-тунель.

Продуктивність та обслуговування

- **24. Enable BBR** — увімкнення/вимкнення алгоритму керування перевантаженням TCP BBR (підменю з пунктами Enable BBR / Disable BBR).
- **25. Update Geo Files** — оновлення гео-баз (файли `.dat`) з вибором джерела: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) або All (всі одразу). Після оновлення панель перезапускається.
- **26. Speedtest by Ookla** — запуск тесту швидкості мережі через Speedtest by Ookla.
- **27. PostgreSQL Management** — керування вбудованим/пов'язаним екземпляром PostgreSQL (увімкнення та супутні операції).
- **0. Exit Script** — вихід з меню.

1.7. Підкоманди `x-ui` (без інтерактивного меню)

Для використання у скриптах команда `x-ui` підтримує прямі підкоманди (виведення `x-ui` без аргументів відкриває меню):

Команда	Дія
<code>x-ui</code>	відкрити меню керування
<code>x-ui start</code>	запустити панель
<code>x-ui stop</code>	зупинити панель
<code>x-ui restart</code>	перезапустити панель
<code>x-ui restart-xray</code>	перезапустити Xray
<code>x-ui status</code>	поточний стан служби
<code>x-ui settings</code>	поточні налаштування
<code>x-ui enable</code>	увімкнути автозапуск при завантаженні ОС
<code>x-ui disable</code>	вимкнути автозапуск
<code>x-ui log</code>	перегляд логів
<code>x-ui banlog</code>	перегляд логів блокувань Fail2ban
<code>x-ui update</code>	оновити панель
<code>x-ui update-all-geofiles</code>	оновити всі гео-файли
<code>x-ui migrateDB [file]</code>	конвертація <code>.db</code>  <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	встановити стару версію
<code>x-ui install</code>	встановити панель
<code>x-ui uninstall</code>	видалити панель

1.8. Міграція SQLite → PostgreSQL

Наявне встановлення на SQLite можна перенести на PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# потім задати XUI_DB_TYPE і XUI_DB_DSN у /etc/default/x-ui та перезапустити:
systemctl restart x-ui
```

Вихідний файл SQLite залишається незайманим — видаліть його вручну лише після перевірки роботи нового бекенду.

Приклад: перевірка переключення на PostgreSQL. Після міграції переконайтеся, що панель дійсно працює на новому бекенді, командою перегляду налаштувань — у виводі має бути вказано PostgreSQL (пароль у DSN маскується):

```
x-ui settings | grep -i -E 'db|dsn'
```

Якщо панель відкривається, а акаунти на місці, вихідний `x-ui.db` можна видалити.

2. Вхід до панелі та безпека доступу

Цей розділ описує все, що стосується автентифікації адміністратора панелі 3X-UI: форму входу, двофакторну автентифікацію (TOTP), захист від перебору пароля, зміну облікових даних, зміну секретного шляху та порту панелі, час життя сесії, а також синхронізацію/автентифікацію через LDAP.

2.1. Форма входу

Сторінка входу віддається за коренем секретного шляху панелі (`webBasePath`). Якщо користувач уже авторизований, він автоматично перенаправляється на `.../panel/`. На сторінці є перемикач теми, вибір мови інтерфейсу та власне форма.

Поля форми:

Поле	Підказка/заголовок (RU)	Обов'язково	Опис
Имя пользователя	«Имя пользователя»	Так	Логін адміністратора. Порожнє значення відхиляється ще на клієнті, а на сервері — повідомленням «Введите имя пользователя».
Пароль	«Пароль»	Так	Пароль адміністратора. Порожнє значення відхиляється повідомленням «Введите пароль».
Код 2FA	«Код 2FA»	Тільки за увімкненої 2FA	Поле з'являється тільки якщо на панелі увімкнено двофакторну автентифікацію. 6-значний код із застосунку-автентифікатора.

Кнопка **«Войти»** надсилає форму на `POST /login`.

Поведінка та повідомлення:

- За успішного входу показується «Вход выполнен успешно» і відбувається перехід на `.../panel/`.
- За будь-якої помилки облікових даних або невірної 2FA сервер повертає **єдине** повідомлення: «Неверные данные учетной записи.» (англ.: *Invalid username or password or two-factor code.*). Це зроблено навмисно — панель не підказує, що саме невірно (логін, пароль або код), щоб не полегшувати перебір.
- Поле «Код 2FA» панель показує або приховує на підставі запиту `POST /getTwoFactorEnable`, який повертає поточний статус 2FA ще до авторизації.
- Якщо минула серверна сесія, за наступного запиту показується «Сессия истекла. Войдите в систему снова», і користувач перенаправляється на сторінку входу.

Примітка про CSRF: перед надсиланням форми клієнт отримує CSRF-токен (`GET /csrf-token`); запити `/login` та `/logout` захищені CSRF-перевіркою.

Приклад: вхід через API. Коли 2FA вимкнена, достатньо логіна і пароля; за увімкненої 2FA додається поле `twoFactorCode` :

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

За успіху сервер поверне `Set-Cookie` із сесійною cookie — її й потрібно передавати в наступних запитах до `/panel/api/...`.

2.2. Двофакторна автентифікація (2FA / TOTP)

2FA у 3X-UI реалізована за стандартом **TOTP** і сумісна з будь-яким застосунком-автентифікатором (Google Authenticator, Aegis, FreeOTP тощо). Параметри жорстко задані: алгоритм **SHA1**, 6 цифр, період **30** секунд, емітент (issuer) `3x-ui`, мітка `Administrator`.

Приклад: otpauth-URI, який кодує QR-код. Якщо застосунок-автентифікатор не сканує камеру, токен можна додати вручну за таким посиланням (підставте свій Base32-секрет замість `JBSWY3DPENPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPENPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Параметри `algorithm=SHA1`, `digits=6`, `period=30` відповідають жорстко заданим значенням панелі — змінювати їх не потрібно.

Налаштування містяться в розділі **Настройки** → **Учетная запись**, вкладка **«Двухфакторная аутентификация»**.

Елемент	Текст (RU)	Опис
Перемикач	«Включить 2FA»	Вмикає/вимикає двофакторну автентифікацію.
Опис	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Підказка під перемикачем.

Як увімкнути 2FA

За увімкнення перемикача панель **локально генерує новий секрет** — випадковий рядок у кодуванні Base32 (алфавіт `A–Z` та `2–7`). Відкривається вікно «Включить двухфакторную аутентификацию» з покроковою інструкцією:

1. **«Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение».** Під QR-кодом відображається сам секрет у текстовому вигляді — за кліком на QR-код секрет копіюється до буфера обміну (спливає «Скопировано»).
2. **«Введите код из приложения»** — потрібно ввести 6-значний код, який згенерував застосунок. Код перевіряється **на стороні браузера**: панель сама обчислює поточний TOTP за щойно згенерованим

секретом і порівнює з уведеним. Якщо код невірний — «Неверный код»; поле приймає тільки рівно 6 цифр.

Тільки після успішного підтвердження секрет і прапорець увімкнення зберігаються. За збереження показується «Двухфакторная аутентификация была успешно установлена».

Важливо: зміни в розділі налаштувань застосовуються спільною кнопкою **«Сохранить»**, після чого, як правило, потрібен перезапуск панелі («Сохраните изменения и перезапустите панель для их применения»). За першого увімкнення 2FA сервер додатково **інвалідує всі активні сесії** (збільшує «login epoch»), тому після застосування налаштування знадобиться повторний вхід — уже з кодом 2FA.

Як вимкнути 2FA

Повторне натискання перемикача відкриває вікно «Отключить двухфакторную аутентификацию» з підказкою «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». Після введення вірного коду (з 3.4.2 він перевіряється **на сервері**) прапорець і секрет очищаються, показується «Двухфакторная аутентификация была успешно удалена».

Перевірка коду під час входу

Під час входу сервер бере збережений секрет і порівнює поточний TOTP із надісланим кодом 2FA. Розбіжність трактується як невдалий вхід, але користувачеві показується те саме об'єднане повідомлення «Неверные данные учетной записи.».

Відновлення доступу (recovery)

Окремого механізму «кодів відновлення» у 3X-UI **немає**. Якщо доступ до застосунку-автентифікатора втрачено, відновити вхід через інтерфейс панелі неможливо. Єдиний шлях — вимкнути 2FA напряму в базі даних на сервері: скинути ключ `twoFactorEnable` у `false` (і за потреби очистити `twoFactorToken`) у таблиці налаштувань, після чого перезапустити панель. Тому секрет (Base32-токен) за увімкнення 2FA рекомендується зберегти в надійному місці.

Приклад: аварійне вимкнення 2FA на сервері. Отримавши доступ до сервера за SSH, зупиніть панель, скиньте ключі в таблиці налаштувань і запустіть панель знову:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Після цього вхід виконується тільки за логіном і паролем, а 2FA за бажання налаштовується заново.

Зв'язок зі зміною облікових даних: за зміни логіна/пароля (див. 2.4) 2FA **автоматично вимикається** на сервері, щоб старий секрет не блокував доступ під новим обліковим записом.

2.3. Обмеження спроб входу (login limiter / захист від перебору)

Панель містить вбудований лімітер невдалих входів (аналог fail2ban на рівні застосунку). Параметри задані в коді й **не налаштовуються** через інтерфейс:

Параметр	Значення	Призначення
Максимум невдач	5	Скільки невдалих спроб допускається в межах вікна.
Вікно підрахунку	5 хвилин	Ковзне вікно, у якому накопичуються невдачі (старіші відкидаються).
Блокування (cooldown)	15 хвилин	На скільки блокується ключ після перевищення порога.

Як це працює:

- Ключ блокування будується зі **зв'язки «IP + логін»** (логін приводиться до нижнього регістру, пробіли обрізаються). Тобто блокування застосовується до конкретної пари «адреса + ім'я користувача», а не до всієї панелі.
- За кожної невдалої спроби (невірні логін/пароль або невірний код 2FA) лічильник зростає. Досягнувши **5** невдач за **5 хвилин**, ключ блокується на **15 хвилин**. Під час блокування будь-які спроби цієї пари негайно відхиляються тим самим повідомленням «Неверные данные учетной записи.», навіть якщо дані вірні.
- **Успішний вхід негайно скидає** лічильник і знімає блокування для цієї пари.
- IP-адреса клієнта визначається з урахуванням довірених проксі (див. `trustedProxyCIDRs`): заголовки `X-Real-IP` та `X-Forwarded-For` приймаються тільки якщо запит надійшов із довіреної адреси. Інакше використовується реальна адреса з'єднання, а за неможливості її витягти — рядок `unknown`.

Усі спроби логуються. Для невдалих пишеться попередження в лог сервера з ім'ям користувача, IP, причиною та, за блокування, часом `blocked_until`. Якщо увімкнено сповіщення про вхід через Telegram-бота (`tgNotifyLogin` — «Уведомление о входе»), адміністратор додатково отримує ім'я користувача, IP та час як успішних, так і невдалих і заблокованих спроб.

Приклад: сповіщення про вхід у Telegram. За увімкненого `tgNotifyLogin` після кожної спроби адміністратор отримує повідомлення приблизно такого вигляду:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Для заблокованої пари «IP + логін» у статусі буде вказано, що спробу відхилено лімітером.

2.4. Зміна логіна та пароля адміністратора

Розділ **Настройки** → **Учетная запись**, вкладка «**Учетные данные администратора**». Поля:

Поле	Текст (RU)	Опис
Текущий логин	«Текущий логин»	Чинне ім'я користувача. Має збігтися з поточним логіном, інакше зміна відхиляється.
Текущий пароль	«Текущий пароль»	Чинний пароль для підтвердження особи.
Новый логин	«Новый логин»	Нове ім'я користувача. Не може бути порожнім.
Новый пароль	«Новый пароль»	Новий пароль. Не може бути порожнім.

Зміна застосовується кнопкою **«Подтвердить»** і надсилається на `POST /panel/setting/updateUser`.

Логіка та повідомлення сервера:

- Якщо «Текущий логин» не збігається з реальним або «Текущий пароль» невірний — «Произошла ошибка при изменении учетных данных администратора.» з поясненням «Неверное имя пользователя или пароль».
- Якщо новий логін або новий пароль порожній — пояснення «Новое имя пользователя и новый пароль должны быть заполнены».
- За успіху — «Вы успешно изменили учетные данные администратора.». Пароль зберігається у вигляді bcrypt-хешу.

Підтвердження кодом 2FA (з 3.4.2). Якщо на панелі увімкнено двофакторну автентифікацію, зміна логіна/пароля додатково вимагає **ввести поточний код** із застосунку-автентифікатора. Відкривається вікно **«Зміна облікових даних»** з підказкою «Введите код из приложения, чтобы изменить учетные данные администратора.»; код (`twoFactorCode`) перевіряється на сервері, при невірному або порожньому коді зміна не застосовується. Те саме підтвердження тепер потрібне і при вимкненні 2FA (див. 2.2).

Приклад: зміна облікових даних через API. Запит вимагає чинної сесійної cookie (отримується за входу) та підтвердження поточних логіна/пароля:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Після успіху поточна сесія анулюється — знадобиться увійти знову вже з новими даними.

Важливі ефекти зміни облікових даних:

- **Усі наявні сесії анулюються** (збільшується лічильник `login_epoch` у користувача), тому після зміни панель автоматично виконує вихід і перенаправляє на сторінку входу — потрібно увійти знову.
- Якщо на момент зміни була увімкнена **2FA, вона автоматично вимикається** (прапорець і секрет скидаються). Двофакторну автентифікацію після зміни логіна/пароля доведеться налаштувати заново.

Якщо 2FA увімкнена, перед надсиланням форми відкривається вікно «Изменить учетные данные» з підказкою «Введите код из приложения, чтобы изменить учетные данные администратора.» — змінювати облікові дані можна лише підтвердивши поточний код 2FA.

2.5. Секретний шлях (URI-шлях / webBasePath) і порт панелі

Ці параметри містяться в розділі **Настройки** → **Панель** і безпосередньо впливають на «прихованість» та доступність панелі. Застосовуються після збереження та **перезапуску панелі**.

Поле	Текст (RU)	Значення за замовчуванням	Опис
Порт панелі	«Порт панелі» (<code>panelPort</code>), підказка «Порт, на котором работает панель»	2053	TCP-порт вебінтерфейсу.
URI-путь	«URI-путь» (<code>panelUrlPath</code>), підказка «Должен начинаться с '/' и заканчиваться '/'»	/	Секретний базовий шлях (<code>webBasePath</code>). Панель доступна тільки за ним (наприклад, <code>/мой-секрет/</code>).
IP-адрес для управління панелью	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), підказка «Оставьте пустым для подключения с любого IP»	порожньо	Адреса, на якій слухає панель. Порожньо = всі інтерфейси.
Домен панелі	«Домен панелі» (<code>panelListeningDomain</code>), підказка «Оставьте пустым для подключения с любых доменов и IP.»	порожньо	Обмеження доступу за доменом (Host).
Путь к публичному ключу сертификата панели	<code>publicKeyPath</code> , підказка «Введите полный путь, начинающийся с '/'»	порожньо	Сертифікат TLS для HTTPS-доступу до панелі.
Путь к приватному ключу сертификата панели	<code>privateKeyPath</code> , та сама підказка	порожньо	Приватний ключ TLS.

Поведінка базового шляху (`webBasePath`):

- Значення нормалізується автоматично: якщо не починається з `/` , символ додається на початок; якщо не закінчується на `/` , додається в кінець. Тобто фактично шлях завжди вигляду `/.../` .
- Базовий шлях застосовується до самої панелі, до асетів і до cookie сесії (cookie видається тільки для цього шляху).

Рекомендації безпеки (розділ «Предупреждения безопасности»): панель сама показує попередження, якщо конфігурація «надто публічна»:

- «Панель работает по обычному HTTP — настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен — измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

Іншими словами, для бойового сервера слід задати **нестандартний порт, нетривіальний URI-шлях і TLS-сертифікат**.

Приклад: «прихована» конфігурація панелі для продакшна. У розділі **Настройки** → **Панель** задайте приблизно такі значення:

Поле	Значення
Порт панелі	<code>34571</code> (випадковий, замість 2053)
URI-путь	<code>/aXf9Qm2/</code> (нетривіальний, починається і закінчується на <code>/</code>)
Путь к публичному ключу сертификата панелі	<code>/etc/letsencrypt/live/panel.example.com/fullchain.pem</code>
Путь к приватному ключу сертификата панелі	<code>/etc/letsencrypt/live/panel.example.com/privkey.pem</code>

Після збереження та перезапуску панель буде доступна тільки за `https://panel.example.com:34571/aXf9Qm2/`, а попередження безпеки зникнуть.

2.6. Час життя сесії (таймаут)

Поле **«Продолжительность сессии»** (`sessionMaxAge`) міститься серед налаштувань панелі/інтервалів.

Поле	Текст (RU)	Значення за замовчуванням	Одиниця	Опис
Продолжительность сессии	«Продолжительность сессии», підказка «Продолжительность сессии в системе (значение: минута)»	360	хвилини	Термін життя cookie-сесії адміністратора.

Поведінка:

- Значення задається у **хвилинах** (за замовчуванням 360 хвилин = 6 годин) і за налаштування cookie переводиться в секунди.
- Якщо значення **більше 0**, у cookie сесії виставляється відповідний `MaxAge`. Зі спливанням цього терміну cookie перестає діяти, і за наступного запиту користувач отримує «Сессия истекла. Войдите в систему снова».
- Сесія також стає недійсною достроково за зміни облікових даних або першого увімкнення 2FA (за рахунок механізму `login_epoch`, див. 2.4 і 2.2) та за явного виходу (`POST /logout`).
- Cookie сесії позначається `HttpOnly`, з політикою `SameSite=Lax`; прапорець `Secure` виставляється за прямого HTTPS-доступу до панелі.

Окрім самого таймауту є пов'язане сповіщення: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`, підказка «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», за замовчуванням `0`) — дозволяє отримувати попередження заздалегідь.

2.7. LDAP (синхронізація та автентифікація)

Розділ LDAP дає дві можливості: (1) автентифікувати вхід адміністратора за LDAP, якщо локальний пароль не підійшов, та (2) періодично синхронізувати стан клієнтів (увімкнено/вимкнено VLESS-прапорець) із каталогу.

Як використовується під час входу: сервер спершу перевіряє локальний bcrypt-хеш пароля. Якщо він **не збігся** і LDAP увімкнено, панель намагається автентифікувати користувача в каталозі: за заданого **Bind DN** виконується службовий bind, потім за фільтром і атрибутом шукається запис користувача, і робиться спроба bind під знайденим DN з уведеним паролем. Успіх означає вхід. (Після успішної LDAP-автентифікації, якщо увімкнена 2FA, все одно перевіряється код TOTP.)

Поля розділу:

Поле	Текст (RU)	Значення за замовчуванням	Опис
Включить LDAP-синхронізацію	«Включить LDAP-синхронізацію» (<code>enable</code>)	false	Головний вимикач LDAP-інтеграції.
LDAP-хост	«LDAP-хост» (<code>host</code>)	порожньо	Адреса LDAP-сервера.
Порт LDAP	«Порт LDAP» (<code>port</code>)	389	Порт. Для LDAPS зазвичай 636.
Использовать TLS (LDAPS)	«Использовать TLS (LDAPS)» (<code>useTls</code>)	false	За увімкнення використовується схема <code>ldaps://</code> (за замовчуванням — з перевіркою сертифіката сервера).
Пропускати перевірку TLS-сертифіката	«Пропускати перевірку TLS-сертифіката» (<code>ldapInsecureSkipVerify</code>)	false	Додано в 3.4.2. Вимикає перевірку сертифіката сервера при LDAPS — для внутрішніх/самопідписаних СА. Підказка: «Небезпечно — вимикає перевірку сертифіката сервера. Використовуйте лише з внутрішніми/недовіреними СА». Перемикач доступний лише при увімкненому «Использовать TLS (LDAPS)».
Bind DN	«Bind DN» (<code>bindDn</code>)	порожньо	DN службового облікового запису для первинного bind/пошуку. Якщо порожньо — bind не виконується (анонімний пошук).

Поле	Текст (RU)	Значення за замовчуванням	Опис
Пароль bind	підказки: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено — введите новое значение для замены»	порожньо	Пароль для Bind DN . Зберігається окремо; щоб залишити попередній, поле залишають порожнім.
Base DN	«Base DN» (baseDn)	порожньо	Корінь піддерева, у якому ведеться пошук (пошук рекурсивний, по всьому піддереву).
Фільтр користувача	«Фільтр користувача» (userFilter)	(objectClass=person)	LDAP-фільтр вибору облікових записів. За автентифікації логін підставляється у фільтр з екрануванням.
Атрибут користувача (username/ email)	«Атрибут користувача (username/ email)» (userAttr)	mail	Атрибут, що зіставляється з логіном/ідентифікатором клієнта (наприклад, mail або uid).
Атрибут VLESS-флага	«Атрибут VLESS-флага» (vlessField)	vless_enabled	Атрибут, який визначає, чи має VLESS-доступ клієнта бути увімкнено.
Общий атрибут флага (опц.)	«Общий атрибут флага (опц.)» (flagField), підказка «Если задано, переопределяет флаг VLESS — напр. shadowInactive.»	порожньо	Якщо заданий, використовується замість vless_enabled .
Truthy-значення	«Truthy-значення» (truthyValues), підказка «Через запятую; по умолчанию: true,1,yes,on»	true,1,yes,on	Список значень атрибута прапорця, які трактуються як «увімкнено».
Инвертировать флаг	«Инвертировать флаг» (invertFlag), підказка «Включите, когда атрибут означает «отключено» (напр. shadowInactive).»	false	Інвертує сенс прапорця.
Расписание синхронизации	«Расписание синхронизации» (syncSchedule), підказка «Строка типа cron, напр. @every 1m»	@every 1m	Періодичність синхронізації у cron-подібному форматі.
Теги входящих	«Теги входящих» (inboundTags), підказка «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	порожньо	Обмежує, на яких inbound дозволені авто-операції. Якщо inbound немає: «Входящие не найдены. Сначала создайте входящий.»
Авто-создание клиентов	«Авто-создание клиентов» (autoCreate)	false	Створювати клієнта у вказаних inbound, якщо він з'явився в каталозі.

Поле	Текст (RU)	Значення за замовчуванням	Опис
Авто-удаление клиентов	«Авто-удаление клиентов» (<code>autoDelete</code>)	false	Видаляти клієнта, якщо він зник із каталогу.
Объём по умолчанию (ГБ)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	0	Ліміт трафіку для авто-створюваних клієнтів (0 = без ліміту).
Срок по умолчанию (дни)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	0	Термін дії для авто-створюваних клієнтів (0 = безстроково).
Лимит IP по умолчанию	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	0	Обмеження кількості одночасних IP (0 = без обмеження).

Особливості логіки прапорця синхронізації: за читання атрибута прапорця (`flagField` , за замовчуванням `vless_enabled`) значення вважається «увімкнено», якщо воно входить до списку `truthy-значень`; за увімкненої інверсії результат змінюється на протилежний. Атрибут користувача (`userAttr`) використовується як ключ зіставлення (email/ім'я) – записи без значення цього атрибута пропускаються.

Безпека: рекомендується вмикати **TLS (LDAPS)**, щоб паролі bind і паролі, що перевіряються, не передавалися відкритим текстом, а для Bind DN використовувати обліковий запис із мінімально необхідними правами на читання.

Приклад: типова конфігурація LDAP-синхронізації (Active Directory). Заповнення полів розділу для каталогу, де статус доступу зберігається в атрибуті `userAccountControl` -подібного прапорця, а зіставлення йде за поштою:

Поле	Значення
LDAP-хост	<code>ldap.example.com</code>
Порт LDAP	<code>636</code>
Использовать TLS (LDAPS)	увімкнено
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Фильтр пользователя	<code>(objectClass=person)</code>
Атрибут пользователя (username/email)	<code>mail</code>
Атрибут VLESS-флага	<code>vless_enabled</code>
Truthy-значения	<code>true,1,yes,on</code>
Расписание синхронизации	<code>@every 5m</code>

За такого налаштування кожні 5 хвилин панель пройде по піддереву `OU=Users` , зіставить клієнтів за `mail` і увімкне/вимкне VLESS-доступ за значенням `vless_enabled` .

3. Огляд / Дашборд

Дашборд («Дашборд», в англійському інтерфейсі — *Overview*) — це стартова сторінка панелі. Вона в режимі реального часу показує стан сервера та процесу Xray. Всі показники надходять із боку сервера. Фоновий планувальник перезбирає знімок **кожні 2 секунди** та надсилає його всім відкритим вкладкам через WebSocket; раз на хвилину накопичені ряди метрик скидаються на диск. HTTP-ендпоінт `GET /status` повертає останній кешований знімок.

Нижче розібрано кожен показник та кожен елемент керування на сторінці.

У бічному меню панелі (і в мобільній шторці) з версії 3.4.2 є кнопка «Документація» (іконка книги) — відкриває офіційну документацію <https://docs.sanaei.dev/>. Також у 3.4.2 проведено наскрізне покращення **доступності**: значки-кнопки і клікабельні елементи отримали aria-підписи і активацію по Enter/Space (для скрінрідерів і навігації з клавіатури).

3.1. Загальні принципи збору даних

- Знімок збирається бібліотекою `gopsutil`. Якщо конкретний замір не вдавсь, поле залишається нульовим, а в лог записується попередження (`get cpu percent failed`, `get uptime failed` тощо) — це не руйнує весь дашборд, просто відповідна плитка покаже 0/N-A.
- «Миттєві» швидкості (CPU %, мережа, диск I/O) обчислюються як різниця між поточним і попереднім знімком, поділена на інтервал у секундах. Тому при першому завантаженні сторінки значення швидкостей можуть бути нульовими, поки не накопичиться другий замір.
- Історію можна переглянути в розділі «Історія системи» (*System History*) — графіки будуються за тими самими рядками даних, що описані нижче (див. п. 3.12).

3.2. ЦП (CPU)

Плитка «ЦП» (CPU) показує поточне навантаження процесора у відсотках, а також параметри самого процесора.

Показник	Опис
Завантаження ЦП, %	Частка зайнятого процесорного часу за останній інтервал. Згладжується експоненціальним середнім (ЕМА, коефіцієнт <code>alpha = 0.3</code>), щоб стрибки не смикали індикатор. Значення завжди обмежене діапазоном 0–100 %. При самому першому замірі повертається 0 (ініціалізація базової точки).
Логічні процесори	Кількість логічних ядер — тобто з урахуванням Hyper-Threading.
Фізичні ядра	Кількість фізичних ядер.
Частота	Базова частота процесора в МГц. Запитується ліниво та кешується: перший успішний замір зберігається, повторна спроба робиться не частіше ніж раз на 5 хвилин, а сам запит обмежений таймаутом 1,5 с (на деяких системах запит частоти відповідає повільно).

Навантаження CPU алгоритмічно розраховується так: за наявності нативної платформенної реалізації використовується вона, інакше — розрахунок за дельтами лічильників процесорного часу (`busy / total`). Guest- та GuestNice-час виключаються, щоб не враховувати його двічі.

3.3. Пам'ять (RAM)

Плитка «Пам'ять» (*RAM*) показує використано та всього. Відображається у вигляді «використано / всього» та/або відсотка заповнення. В історію записується відсоток.

3.4. Підкачка (Swap)

Плитка «Підкачка» (*Swap*) показує використано та всього. Якщо файл/розділ підкачки не налаштовано (всього = 0), показник дорівнює нулю; в історичний ряд за відсутності swap записується 0.

3.5. Диск (Storage)

Плитка «Диск» (*Storage*) показує використано та всього, при цьому враховується **лише кореневий розділ** /. В історію «Використання диска» (*Disk Usage*) записується відсоток заповнення. Окремо збирається дисковий ввід-вивід (читання / запис, байт/с) як дельта лічильників за інтервал — він відображається на вкладці «Диск I/O» історії.

3.6. Час роботи системи (Uptime)

Показник «Час роботи системи» (*Uptime*). Це час з моменту завантаження **всього сервера** (у секундах), а не час роботи панелі або Xray. Окремо зберігається аптайм процесу Xray (див. п. 3.9), а також кількість потоків панелі (у перекладі — «Потоки» / *Threads*).

Пам'ять, що займає панель

Поруч із показниками процесу панелі відображається обсяг оперативної пам'яті, який займає сам процес 3X-UI. Це значення береться з реального RSS процесу (як його бачить операційна система) і збігається з тим, що показують системні утиліти. Число знижується в міру звільнення пам'яті. Раніше панель виводила внутрішній лічильник Go, який завищував витрату пам'яті (наприклад, ~300 МБ на простоюючому сервері з одним клієнтом) і ніколи не зменшувався — тепер цього артефакту немає. Додатково періодичний фоновий процес повертає невикористану пам'ять операційній системі, щоб показник відображав фактичне споживання.

3.7. Навантаження системи (Load average)

Блок «Навантаження на систему» (*System Load*) — масив із трьох чисел [Load1, Load5, Load15]. Підпис-підказка: «Середнє навантаження системи за останні 1, 5 і 15 хвилин» (*System load average for the past 1, 5, and 15 minutes*). Графік історії називається «Середнє навантаження системи (1 / 5 / 15 хв)». В історичні ряди значення записуються окремо: load1, load5, load15.

Це стандартний Unix-показник: середня кількість процесів, що перебувають у черзі на виконання. Орієнтир — порівнювати з кількістю ядер: навантаження, що стабільно перевищує кількість фізичних ядер, свідчить про перевантаження.

3.8. Мережа: швидкість та загальний обсяг трафіку

Враховуються **лише фізичні інтерфейси**. Віртуальні та тунельні інтерфейси виключаються: це `lo / lo0`, а також усе, що починається з `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Значення підсумовуються по всіх інтерфейсах, що залишилися.

Загальна швидкість (*Overall Speed*) — миттєва швидкість, дельта лічильників за інтервал:

Показник	Опис
Завантаження / віддача (підпис «Завантажка» / <i>Upload</i>)	Вихідна швидкість, байт/с.
Скачування / приймання (підпис «Скачати» / <i>Download</i>)	Вхідна швидкість, байт/с.

Загальний обсяг трафіку (*Total Data*) — накопичені лічильники з моменту старту системи:

Показник	Опис
Надіслано (підпис «Відправлено» / <i>Sent</i>)	Всього надіслано байт.
Отримано (підпис «Отримано» / <i>Received</i>)	Всього отримано байт.

Додатково збираються швидкості пакетів (пакетів/с) та підсумкові лічильники пакетів — вони показуються на вкладці «Мережеві пакети» (*Network Packets*) історії. Ряди історії по мережі: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. IP-адреси сервера

Блок «IP-адреси сервера» (*IP Addresses*) показує `IPv4` та `IPv6`. Зовнішні адреси визначаються через сторонні сервіси (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` для `IPv4` та аналогічні для `IPv6`). Список перебирається по черзі до першої успішної відповіді; таймаут на кожен запит — 3 с.

Особливості:

- Результат **кешується** на час життя процесу: успішно визначена адреса повторно не запитується.
- Якщо жоден сервіс не відповів, у полі залишається `N/A`. Для `IPv6` при першому `N/A` запити `IPv6` взагалі вимикаються, щоб не витратити час на мережах без `IPv6`.
- Поруч є кнопка-«око» для приховування/показу адрес — підказка «Приховати або показати IP-адреси сервера» (*Toggle visibility of the IP*). Це лише візуальне приховування в інтерфейсі (наприклад, для скріншотів), на самі адреси воно не впливає.

3.10. TCP/UDP з'єднання

Блок «Кількість з'єднань» (*Connection Stats*) показує загальне число активних TCP- та UDP-з'єднань на сервері (по всій системі, не лише Xray). Графік історії — «Активні з'єднання (TCP / UDP)» (*Active Connections*), ряди `tcpCount`, `udpCount`.

3.11. Статус Xray та керування процесом

Картка «Xray» показує стан процесу Xray-core та дає змогу ним керувати.

Стани

Значення	Підпис	Переклад	Коли встановлюється
running	«Запущено»	Running	Процес Xray запущено.
stop	«Зупинено»	Stopped	Процес не запущено, і при цьому немає зафіксованої помилки запуску.
error	«Помилка»	Error	Процес не запущено, але зафіксовано помилку запуску. Текст помилки показується у спливаючому вікні із заголовком «Помилка при запуску Xray» (<i>An error occurred while running Xray</i>).
—	«Невідомо»	Unknown	Відображається, поки статус ще не отримано.

Поруч зі статусом відображається **версія Xray**.

Кнопки керування

- **Стоп (Stop)**. Викликає `POST /stopXrayService`. При успіху панель надсилає по WebSocket новий стан `stop` та повідомлення «Xray успішно зупинено» (*Xray service has been stopped*), при помилці — стан `error` з текстом. Важливо: якщо панель доступна *через* сам Xray, зупинка Xray може перервати зв'язок із панеллю — при прямому підключенні до панелі проблем немає.
- **Перезапуск (Restart)**. Викликає `POST /restartXrayService`. Перед дією показується підтвердження «Перезапустити xray?» з поясненням «Перезавантажує сервіс xray зі збереженою конфігурацією». При успіху — стан `running` та повідомлення «Xray успішно перезапущено» (*Xray service has been restarted successfully*). Перезапуск застосовує поточну збережену конфігурацію — використовуйте його після зміни налаштувань.

Примітка. У цьому форку для дашборда додано повноцінне керування Start / Stop / Restart для всіх типів авторизації; у вихідному UI 3x-ui окремої кнопки «старт» немає — запуск виконується перезапуском.

Кнопка перегляду логів Xray

На картці Xray є кнопка перегляду логів Xray (*Logs*). Вона з'являється лише тоді, коли в конфігурації Xray налаштовано access-лог: вбудований переглядач читає саме цей файл, тому без access-логу кнопка прихована. Видимість кнопки прив'язана до окремої ознаки `accessLogEnable` і більше не залежить від ліміту IP — онлайн-список і ліміт IP-адрес продовжують працювати навіть без access-логу (див. п. 8).

Вибір версії Xray

Розділ «Вибір версії» (*Version*) дозволяє перемкнути Xray-core на інший реліз. Список версій завантажується через `GET /getXrayVersion`:

- Джерело — GitHub API репозиторію `XTLS/Xray-core` (`/releases`). Запити кешуються на **15 хвилин**; при збої GitHub повертається останній вдало отриманий список, щоб пікер не спустів.
- У список потрапляють лише релізи виду `X.Y.Z` і **не старіші за 26.4.25**.

Підказки: «Виберіть потрібну версію» (*Choose the version you want to switch to.*) та попередження «Важливо: старі версії можуть не підтримувати поточні налаштування» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Перемикання: `POST /installXray/:version`. Сценарій:

Приклад. Перемкнутися на конкретну версію Xray-core (cookie сесії вже має бути отримано авторизацією):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Тут `v25.6.8` — тег зі списку, який повертає `GET /getXrayVersion`. Версія обов'язково має бути присутня в цьому списку, інакше панель відповість відмовою. З версії 3.4.2 мінімально допустима для встановлення версія Xray піднята до **26.6.27**, а в комплекті 3.5.0 йде ядро **Xray 26.7.11**. Супутні зміни ядра: шифри `Shadowsocks none/plain` і `VMess none/zero` видалені (збережені конфіги переписуються автоматично: `SS` — на підтримуваний шифр, `VMess` — на `auto`), а незашифрований `VLESS/Trojan-outbound` на публічну адресу відхиляється при збереженні — ядро з таким конфігом не запустилося б.

1. Вибрана версія перевіряється на наявність в актуальному списку релізів (інакше — відмова).
2. Xray зупиняється.
3. Під поточну ОС та архітектуру скачується архів `Xray-<os>-<arch>.zip` з GitHub (підтримуються `amd64/64`, `arm64-v8a`, `arm32-v7a/v6/v5`, `386/32`, `s390x`; для Windows — `xray.exe`). Розмір архіву та бінарника обмежені 200 МБ.
4. Бінарник замінюється атомарно (через тимчасовий файл + перейменування) та позначається як виконуваний.
5. Xray запускається знову.

Перед перемиканням показується діалог «Перемкнути версію Xray» (*Do you really want to change the Xray version?*) з описом «Це змінить версію Xray на `#version#`». При успіху — повідомлення «Xray успішно оновлено» (*Xray updated successfully*).

3.12. Оновлення панелі (3X-UI)

Блок перевірки оновлень панелі. Дані надходять через `GET /getPanelUpdateInfo`:

Поле	Опис
Поточна версія панелі	Версія встановленої панелі.

Поле	Опис
Остання версія панелі	Останній реліз 3x-ui, отриманий з GitHub.
Доступне оновлення	Ознака того, що остання версія новіша за поточну. Якщо оновлення не потрібне – показується «Панель оновлена» / «Оновлено».

Кнопка **«Оновити панель»** (*Update Panel*) запускає `POST /updatePanel`. Підказка: «Це оновить 3X-UI до останнього релізу та перезапустить сервіс панелі». Перед запуском – підтвердження «Ви дійсно хочете оновити панель?» з текстом «Це оновить 3X-UI до версії #version# та перезапустить сервіс панелі».

Особливості та обмеження:

- Самооновлення підтримується **лише на Linux** (на інших ОС повертається помилка).
- Скрипт-оновлювач скачується з офіційного репозиторію (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, ліміт 2 МБ) та запускається через `bash`, по можливості ізольовано через `systemd-run`.
- При успішному запуску показується «Оновлення панелі розпочато» (*Panel update started*); якщо перевірка оновлення не вдалася – «Перевірка оновлення панелі не вдалася». Під час встановлення відображається попередження «Встановлення в процесі. Не оновлюйте сторінку».

3.13. Оновлення гео-файлів (GeoIP / GeoSite)

Кнопка/діалог оновлення гео-баз викликає `POST /updateGeofile` (всі файли) або `POST /updateGeofile/:fileName` (один файл). Оновлення працює за суворим білим списком імен та джерел:

Файл	Джерело
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Поведінка:

- Ім'я файлу валідується: заборонені `..`, слеші, абсолютні шляхи; допустимі лише `[a-zA-Z0-9._-]+.dat`. Файли поза білим списком не скачуються.
- Використовується умовний запит `If-Modified-Since`: якщо на сервері-джерелі файл не змінювався (HTTP 304), повторно він не качається, лише оновлюється відмітка часу.
- Після завантаження Xray **перезапускається** (щоб підхопити нові бази).

Приклад. Оновити лише російські гео-бази, не чіпаючи інші файли:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Щоб оновити одразу всі файли з білого списку — викличте `POST /updateGeofile` без імені файлу.

- Діалоги: «Ви дійсно хочете оновити геофайл?» з «Це оновить файл #filename#» для одного файлу та «Це оновить всі геофайли» для кнопки «Оновити всі». Успіх — «Геофайли успішно оновлено».

3.14. Резервне копіювання та відновлення бази даних

Блок «Бекап і відновлення» (*Backup & Restore*). Поведінка залежить від використовуваної СУБД (SQLite за замовчуванням або PostgreSQL).

Експорт бази (Резервна копія)

Кнопка «Експорт бази даних» / «Резервна копія» (*Back Up*) викликає `GET /getDb`. Файл повертається як вкладення:

- **SQLite**: спочатку виконується checkpoint (скидання WAL), потім скачується файл `x-ui.db`. Підказка: «Натисніть, щоб завантажити файл .db, що містить резервну копію вашої поточної бази даних...».
- **PostgreSQL**: скачується дамп `x-ui.dump` у кастомному форматі (`pg_dump --format=custom --no-owner --no-privileges`). На сервері мають бути встановлені клієнтські інструменти PostgreSQL; інакше — помилка про відсутність `pg_dump`.

Імпорт бази (Відновлення)

Кнопка «Імпорт бази даних» / «Відновлення» (*Restore*) завантажує файл через `POST /importDB` (поле форми `db`). Підказка: «Натисніть, щоб вибрати та завантажити файл .db... для відновлення бази даних з резервної копії».

Сценарій для **SQLite** безпечний, з відкатом:

1. Файл перевіряється на формат SQLite та зберігається у тимчасовий файл, потім перевіряється цілісність.
2. Храу зупиняється, поточна БД закривається та перейменовується в `*.backup` (фолбек).
3. Новий файл стає на місце робочої БД, виконується ініціалізація та міграція. Якщо щось пішло не так — відновлюється фолбек.
4. Храу запускається знову.

Для **PostgreSQL** з 3.5.0 «Відновлення» приймає **три види файлів** (тип визначається за вмістом, а не за розширенням): архів `pg_dump` (PGDMP) — застосовується через `pg_restore --clean --if-exists --single-transaction ...`; **SQLite-бази** `.db` (звичайний бекап) і **SQLite-міграційний** `.dump` — вони перевіряються, за потреби Perezбираються та імпортується в PostgreSQL **однією транзакцією** тим самим рушієм, що й `x-ui migrate-db --dsn` (при помилці PostgreSQL залишається недоторканим). Діалог вибору файлу приймає `.dump`, `.db` на обох СУБД; завантаження `pg_dump` -архіву на SQLite-панель дає зрозумілу помилку. Підказка прямо попереджає: «Це замінить усі поточні дані».

Повідомлення: «База даних успішно імпортована», «Сталася помилка при імпорті бази даних», «...при читанні бази даних», «...при отриманні бази даних».

Файл міграції (між SQLite та PostgreSQL)

Кнопка «Завантажити файл міграції» (*Download Migration*) викликає `GET /getMigration`. З 3.5.0 вона показується **лише на PostgreSQL-панелях** і скачує `x-ui.db` — готову SQLite-базу, зібрану з даних PostgreSQL (шлях «назад на SQLite»). На SQLite-панелях кнопку прибрано як надлишкову: звичайний бекап `.db` тепер і так відновлюється прямо в PostgreSQL-панель.

3.15. Додаткові елементи інтерфейсу

- **Індикатор онлайн-клієнтів.** Дашборд веде ряд `online` (*Online Clients* / «Клієнти онлайн») — кількість клієнтів з активним з'єднанням. Підраховується при працюючому Xray (інакше 0) та записується в історію на тому ж 2-секундному такті. Графік — вкладка «Онлайн».
- **Історія системи (графіки).** Кнопка/розділ «Графіки» → «Історія системи» з вкладками: «Пропускна здатність», «Пакети», «Диск I/O», «Онлайн», «Навантаження», «З'єднання», «Використання диска». Дані тягнуться по `GET /history/:metric/:bucket`; допустимі інтервали агрегації (bucket, сек): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, на вкладку надходить до 60 точок. У самому селекторі діапазону на сторінці доступні кнопки **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (bucket'и 2, 60, 180, 360, 720, 1440, 2880, 10080 відповідно). На довгих діапазонах **2d** та **7d** підписи часу по осі доповнені датою у форматі `MM-DD HH:MM`. Зберігання організовано триступеневим проріджуванням (rollup): свіжі дані зберігаються з кроком 2 с за останню **годину**, далі усереднюються до кроку 1 хв за **48 годин** і до кроку 10 хв за **7 діб**. Тому графіки (CPU, RAM, трафік, пакети, з'єднання, диск, онлайн, навантаження) можна переглядати за період **до 7 діб** (раніше — до 48 годин), причому чим далі в минуле, тим грубіша деталізація. Допустимі метрики: `cpu`, `mem`, `swap`, `netUp`, `netDown`, `pktUp`, `pktDown`, `diskRead`, `diskWrite`, `diskUsage`, `tcpCount`, `udpCount`, `online`, `load1`, `load5`, `load15`. Підпис «Останні 2 хвилини» відповідає bucket = 2 (режим реального часу).

Приклад. Отримати ряд завантаження ЦП за останні ~2 хвилини (bucket = 2 с, до 60 точок) та той самий ряд, агрегований по 5 хвилин (bucket = 300 с):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Метрику можна замінити на будь-яку допустиму (`mem`, `netUp`, `tcpCount`, `load1` тощо). Bucket поза білим списком 2, 30, 60, 180, 360, 720, 1440, 2880, 10080 буде відхилено.

- **Метрики Xray** — окремий блок із споживанням пам'яті та збиранням сміття Xray (ряди `xrAlloc`, `xrSys`, `xrHeapObjects`, `xrNumGC`, `xrPauseNs`) та «Обсерваторією» (стан вихідних з'єднань). Працюють, лише якщо в конфігурації Xray задано блок `metrics` (`listen 127.0.0.1:11111`, `ter metrics_out`); інакше показується «Кінцеву точку метрик Xray не налаштовано». У вікні метрик Xray власний селектор діапазону з кнопками **2m, 1h, 3h, 6h, 12h** (bucket'и 2, 60, 180, 360, 720).

Приклад блоку, який вмикає плитку метрик Xray. У розділі налаштувань Xray мають бути присутні одночасно `metrics` (з тегом) та `inbound`, який цей тег слухає:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Адреса `127.0.0.1:11111` навмисно не виставляється назовні — панель опитує її локально.

- **Перемикач темної теми.** Знаходиться в загальному меню/шапці, а не в самому дашборді. Варіанти: «Тема» (*Theme*) з варіантами «Темна» та «Дуже темна» (*Ultra Dark*). Це суто візуальне налаштування оформлення, на роботу панелі не впливає.
- **Інші посилання** в оточенні дашборда (з меню/нижньої панелі): «Логи», «Конфігурація» — перегляд підсумкового JSON Xray (GET /getConfigJson), «Документація».

4. Inbounds: створення та загальні параметри

Розділ **«Входящие»** (англ. *Inbounds*) — це список усіх точок входу Xray, через які підключаються клієнти. Кожен inbound зберігає як «панельні» поля (примітка, ліміт трафіку, розклад скидання), так і сирі JSON-блоки конфігурації Xray (`settings` , `streamSettings` , `sniffing`).

Створення виконується кнопкою **«Создать подключение»** (*Add Inbound*), редагування — **«Изменить подключение»** (*Modify Inbound*). Обидві операції надсилаються на API-ендпоінти `POST /add` та `POST /update/:id`.

Нижче розібрано всі поля форми, які **не** стосуються налаштувань конкретного протоколу (клієнти, шифрування, REALITY/TLS) та **не** стосуються транспорту/поток (вкладки **«Поток»**, **«Безопасность»**) — це теми окремих розділів.

4.1. Загальні поля форми

Remark (Примітка)

Параметр	Значення
Поле	<code>remark</code>
Тип	рядок
За замовчуванням	порожньо

Зрозуміле для людини ім'я inbound, що відображається у списку та в заголовках діалогів (**«Удалить подключение "{remark}"?»** тощо). Підпис поля — **«Примечание»**. На роботу Xray не впливає, потрібне лише для зручності адміністрування; рекомендується задавати унікальні осмислені імена, оскільки вони підставляються в назви експортованих файлів та в підтвердження масових операцій.

Protocol (Протокол)

Параметр	Значення
Поле	<code>protocol</code>
Підпис	«Протокол»
Валідація	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Випадний список протоколу inbound. Допустимі значення:

Значення	Примітка
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Значення	Примітка
shadowsocks	
wireguard	
hysteria	Hysteria v2 – це hysteria з streamSettings.version = 2 , окремого протоколу немає
http	
mixed	socks/http на одному порту
tunnel	
tun	приймається валідатором, окремої константи протоколу немає

Поле обов'язкове (required). Вибір протоколу визначає, які поля налаштувань клієнтів і який транспорт будуть доступні (див. протокол-специфічні розділи).

Важливо: під час збереження сервіс нормалізує streamSettings . Транспортні налаштування залишаються лише для протоколів vmess , vless , trojan , shadowsocks , hysteria ; для решти (http , mixed , tunnel , wireguard , tun) поле streamSettings **примусово очищується**.

Для inbound типу tunnel /TProxy, у яких блок streamSettings не містить ключа security (безтранспортний варіант), форма відкривається та зберігається без помилки валідації streamSettings.security Invalid input .

Listen IP (IP прослуховування)

Параметр	Значення
Поле	listen
Тип	рядок
За замовчуванням	порожньо → Xray слухає на 0.0.0.0 (усі IP)

IP-адреса, на якій inbound приймає з'єднання. Підказка до поля:

«Оставьте пустым для прослушивания всех IP-адресов».

Під час генерації конфігурації Xray порожнє значення замінюється на 0.0.0.0 . Окрім IP, поле приймає **шлях Unix-сокета** – підказка:

«Можно также указать путь Unix-сокета (например, /run/xray/in.sock) или имя абстрактного сокета с префиксом @ (например, @xray/in.sock), чтобы слушать сокет вместо TCP-порта – в этом случае задайте порт 0».

Таким чином, поле приймає дві форми Unix-сокета: шлях у файловій системі (`/run/xray/in.sock`) та абстрактне ім'я сокета з префіксом `@` (`@xray/in.sock`). В обох випадках задайте `Port` рівним `0`.

Змінюють це поле, коли потрібно обмежити inbound одним інтерфейсом (наприклад, `127.0.0.1` для inbound, що працює лише як fallback-ціль за Nginx) або коли inbound слухає Unix-сокет.

Приклад. Inbound, який слухає лише локальний інтерфейс (типова fallback-ціль за Nginx) та Unix-сокет:

```
listen = 127.0.0.1   порт = 8443
listen = /run/xray/in.sock   порт = 0
```

Port (Порт)

Параметр	Значення
Поле	<code>port</code>
Підпис	«Порт»
Валідація	<code>gte=0, lte=65535</code>
За замовчуванням	— (задається користувачем)

TCP/UDP-порт прослуховування. Допустимі значення від `0` до `65535`. Значення `0` використовується лише в парі з прослуховуванням на Unix-сокеті (див. вище).

Під час збереження сервіс перевіряє конфлікт порту: два inbound не можуть одночасно займати порти `listen:port`, що перетинаються, для одного й того самого транспорту (TCP/UDP). Транспорт обчислюється з протоколу та `streamSettings / settings`: наприклад, `hysteria` та `wireguard` завжди займають UDP, `kcp / quic` — UDP, а більшість інших — TCP. У разі конфлікту збереження відхиляється з помилкою.

Окремо панель не дає зайняти **зарезервований порт внутрішнього Xray API** (тег `api`, за замовчуванням `62789` на `127.0.0.1`): локальний TCP-inbound, чия адреса прослуховування перетинається з цим портом на loopback, відхиляється з тією самою помилкою конфлікту порту. Реальний порт API читається з шаблону конфігурації Xray (із запасним значенням `62789`). На вузлах (nodes) це обмеження не діє — у них власний Xray.

Тег Xray (Tag , унікальний) генерується автоматично з порту й транспорту у форматі `in-<порт>-<tcp|udp|tcpudp|any>`; для inbound, розгорнутого на вузлі, додається префікс `n<nodeId>-`. У разі колізії до тега дописується `-2`, `-3` тощо. Користувач зазвичай тег не редагує.

Total traffic (Усього трафіку, GB)

Параметр	Значення
Поле	<code>total</code> (у байтах)
Підпис	«Общий расход»

Параметр	Значення
За замовчуванням	0

Сумарний ліміт трафіку inbound. У формі значення вводиться в гігабайтах, у БД зберігається в байтах.

Підказка до поля:

«= Безлимит. (єдиница: ГБ)».

Тобто **0 означає безліміт**. Це ліміт на рівні всього inbound (а не окремих клієнтів); фактичний витрачений трафік зберігається в полях `up` (надіслано) та `down` (отримано) і порівнюється з `total`.

Expiry date / Duration (Дата закінчення / строк)

Параметр	Значення
Поле	<code>expiryTime</code> (Unix-мітка часу)
Підпис	«Дата закінчення» (англ. <i>Duration</i>)
За замовчуванням	порожньо / 0

Строк дії inbound. Підказка:

«Оставьте пустым, чтобы было бесконечным».

Порожнє значення (0) означає безстроковий inbound. Значення зберігається як Unix-мітка; форма дозволяє задавати як конкретну дату, так і строк у днях (відносний відлік від поточного моменту — англ. підпис поля *Duration*).

Enabled (Увімкнути)

Параметр	Значення
Поле	<code>enable</code>
Підпис	«Включить» (англ. <i>Enabled</i>)
За замовчуванням	задається при створенні

Ознака активності inbound. Перемикач цього прапорця у списку обробляється окремим «легким» ендпоінтом `POST /setEnable/:id`, а не повним оновленням — це зроблено спеціально, щоб не пересеріалізувати весь блок `settings` (усіх клієнтів) при кожному клацанні по перемикачу на inbound із тисячами клієнтів. При вимкненні inbound видаляється з працюючого Xray, при увімкненні — додається назад.

Node / Deploy to (Вузол / Розгорнути на)

Параметр	Значення
Поле	nodeId
Підпис	«Развернуть на», «Локальная панель»
За замовчуванням	порожньо (локальна панель)

Вибір, де фізично працює inbound: на локальній панелі чи на одному із зареєстрованих вузлів. З 3.5.0 список «Развернуть на» (як і фільтр «Вузли» над списком inbound) підтримує **пошук за введенням**, а над самим списком inbound з'явилося **поле пошуку** (за приміткою, портом і протоколом; працює разом із фільтром вузлів). Особливість реалізації: `nodeId = 0` нормалізується в `nil`, оскільки `0` — це не валідний id вузла, а артефакт прив'язки форми; `nil / 0` означає локальну панель. При збереженні inbound на офлайн-вузлі можливий тост «изменение синхронизируется, когда узел переподключится». З версії 3.4.2 цей тост показується лише коли вузол справді офлайн або вимкнений (раніше він міг з'являтися і при збереженні на онлайн-вузол).

Стратегія адреси для посилань (Share address strategy)

Параметр	Значення
Поле	стратегія + (опційно) користувацька адреса
Підпис	«Стратегия адреса для ссылок» (англ. <i>Share address strategy</i>)
За замовчуванням	«Адрес прослушивания inbound» (<i>Inbound listen</i>)

Випадний список визначає, яка адреса підставляється в **експортовані посилання-шерингу та QR-коди** цього inbound. Значення:

Значення	Підпис	Що підставляється
node	«Адрес узла» (<i>Node address</i>)	адреса вузла, на якому працює inbound
listen	«Адрес прослушивания inbound» (<i>Inbound listen</i>)	адреса прослуховування самого inbound
custom	«Пользовательская» (<i>Custom</i>)	своя адреса з поля «Пользовательский адрес для ссылок» (<i>Custom share address</i>)

При виборі «Пользовательская» з'являється поле «Пользовательский адрес для ссылок»; у нього вводять хост або IP **без схеми та порту** (значення валідується). Варіант «Адрес узла» показується у списку лише якщо існує увімкнений вузол, на якому цей inbound може працювати; інакше він прихований, а значення приводиться до «Адрес прослушивания inbound».

Ця стратегія впливає **лише** на прямі посилання-шерингу та QR-коди. На видачу підписки вона **не** впливає — там адреса, як і раніше, визначається звичайною логікою панелі.

4.2. Sniffing (Сніфінг)

Вкладка «Сніффінг» редагує блок `sniffing` конфігурації Xray, який зберігається як сирий JSON. Sniffing дозволяє Xray «підглядати» реальне доменне ім'я/протокол усередині з'єднання для цілей маршрутизації.

Підполе	Підпис	Призначення
<code>enabled</code>	(перемикач вкладки)	Вмикає/вимикає сніфінг для inbound
<code>destOverride</code>	—	Список протоколів, для яких перехоплюється адреса призначення: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Только метаданные»	Використовувати лише метадані з'єднання, без читання корисного навантаження
<code>routeOnly</code>	«Только маршрутизация»	Застосовувати результат сніфінгу лише для маршрутизації, не переписуючи адресу призначення
<code>domainsExcluded</code>	«Исключённые домены»	Домени, що виключаються зі сніфінгу
(виключені IP)	«Исключённые IP»	IP-адреси, що виключаються зі сніфінгу

- **`destOverride`** — набір сніферів: `http` (визначає домен із HTTP-заголовка Host), `tls` (із SNI), `quic` (із QUIC ClientHello), `fakedns` (зіставлення з FakeDNS-пулом). Зазвичай для визначення домену вмикають `http` та `tls`.

Приклад блоку `sniffing` (визначати домен за HTTP та TLS, використовувати результат лише для маршрутизації, не чіпаючи локальну мережу):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — коли увімкнено, Xray не зчитує вміст першого пакета й спирається лише на метадані; корисно, щоб не порушувати протоколи, у яких не можна «підглядати» дані.
- **`routeOnly`** — результат сніфінгу використовується лише правилами маршрутизації; адреса з'єднання в outbound при цьому не переписується на розпізнаний домен.

Примітка: панель зберігає `sniffing` як непрозорий JSON-блок і під час збереження нічого до нього не додає — усі значення за замовчуванням для цих галочок формуються на стороні застосунку-клієнта. У сирому вигляді блок можна відредагувати через розділ «JSON входящего» (див. нижче).

4.3. Allocate (стратегія розподілу портів)

Блок `allocate` у `streamSettings` керує тим, як Xray розподіляє порти прослуховування. Це частина конфігурації Xray; панель зберігає та передає його як частину `streamSettings /JSON inbound`. Параметри (за термінологією Xray-core):

Підполе	Призначення	Значення / за замовчуванням
<code>strategy</code>	Стратегія виділення портів	<code>always</code> – завжди слухати заданий порт (за замовчуванням); <code>random</code> – періодично змінювати прослуховувані порти всередині діапазону
<code>refresh</code>	Інтервал зміни портів (хвилини) при <code>random</code>	ціле число хвилин (рекомендується 5; мінімум – 2)
<code>concurrency</code>	Скільки портів тримати відкритими одночасно при <code>random</code>	ціле (за замовчуванням 3; не більше третини ширини діапазону портів)

`strategy: always` залишає inbound на одному порту (стандартний режим). `strategy: random` потрібен для сценаріїв anti-blocking, коли inbound періодично «стрибає» по діапазону портів; у цьому випадку мають сенс `refresh` та `concurrency`. Змінювати ці значення потрібно лише при свідомому використанні режиму випадкових портів.

Приклад блоку `allocate` у `streamSettings` (режим випадкових портів: тримати 3 порти відкритими, змінювати кожні 5 хвилин):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Щоб це працювало, `port inbound` задають діапазоном (наприклад, `20000–20100`).

4.4. External Proxy (Зовнішній проксі)

Поле «**External Proxy**» стосується налаштувань формування посилань-запрошень і зберігається в `streamSettings inbound`. Воно задає список альтернативних зовнішніх адрес (хост/порт, за потреби з примусовим TLS – «**Принудительный TLS**»), які підставляються в клієнтські посилання замість реального `listen:port inbound`.

Використовується, коли клієнти мають підключатися не напряму до сервера, а через зовнішній проксі/реверс/CDN: тоді в загальних посиланнях вказується публічна адреса такого фронтенду. На сам процес прийому з'єднань Xray це не впливає – це «косметика» генерованих посилань. Пов'язані поля форми: «**Принудительный TLS**», «**Fingerprint**», підписи кожного запису.

4.5. Fallbacks (Fallback'и)

Розділ **«Fallback'и»** задає правила перенаправлення з'єднань, що не збіглися з жодним клієнтом inbound. Доступний для master-inbound на TLS-транспорті (VLESS/Trojan TCP-TLS). Керується через ендпоінти `GET /:id/fallbacks / POST /:id/fallbacks`.

Підказка до розділу:

«Когда соединение на этом инбаунде не совпадает ни с одним клиентом, оно перенаправляется в другое место. Выберите дочерний инбаунд ниже, чтобы поля маршрутизации (SNI / ALPN / Path / xver) заполнились автоматически из его транспорта, либо оставьте выбор пустым и задайте Dest напрямую (например, 8080 или 127.0.0.1:8080), чтобы перенаправить на внешний сервер, такой как Nginx. Каждый дочерний инбаунд должен слушать на 127.0.0.1 с security=none».

Розділ fallback'ів показується лише для inbound VLESS/Trojan поверх RAW (TCP) з безпекою TLS або REALITY. Новий inbound стартує з `security=none`, тому розділ спочатку може виглядати відсутнім. У цьому стані (VLESS/Trojan, RAW/TCP, безпека ще не налаштована) замість розділу виводиться вбудована підказка: fallback'и стануть доступні після вибору TLS або Reality на вкладці **«Безопасность»**.

Поля рядка fallback

Поле	За замовчуванням	Опис
(дочірній inbound)	—	Вибір дочірнього inbound (підпис «Выберите инбаунд»). Якщо вибрано, поля Name/Alpn/Path/Dest можуть авто-заповнитися з його транспорту
Name	порожньо (= будь-який)	Умова збігу за іменем (SNI/іменем). Підпис «любой» — «любой»
Alpn	порожньо	Умова збігу за ALPN
Path	порожньо	Умова збігу за шляхом (для WS/HTTP-транспортів дочірнього inbound)
Dest	авто	Куди перенаправляти. Плейсхолдер «авто (listen:порт дочернего)» . Можна вказати порт (8080) або <code>host:port</code> (127.0.0.1:8080)
Xver	0	Версія PROXY-протоколу («Xver»): 0 — вимкнено, 1 або 2 — відповідна версія PROXY protocol
(порядок)	за позицією	Порядок застосування правил; задається кнопками «Вверх»/«Вниз»

Логіка збереження: весь список фолбеків майстра замінюється атомарно. Рядок, у якого немає ні вибраного дочірнього inbound (`childId <= 0`), ні заданого Dest , **пропускається**. Якщо вибраний дочірній inbound дорівнює id самого майстра, він обнуляється. Під час генерації підсумкового JSON: якщо Dest порожній, він обчислюється з дочірнього inbound як `listen:port`, причому `0.0.0.0 / :: / ::0` замінюються на `127.0.0.1`; порожні поля `name / alpn / path` у вихідний JSON не потрапляють; `xver` додається лише якщо він більший за 0.

Приклад підсумкового settings.fallbacks (трафік з `alpn=h2` іде на WS-ціль шляхом `/ws`, усе інше — на локальний Nginx на порту 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Останній рядок без `name / alpn / path` — це правило «за замовчуванням», що ловить усе інше.

Кнопки та підказки розділу fallbacks

- **«Добавить фолбэк»** — додати рядок; **«Фолбэков пока нет»** — порожній стан.
- **«Быстро добавит все подходящие» / «Добавить все»** — додає рядок fallback для кожного відповідного inbound, ще не підключеного. Результат: «Добавлено {n} фолбэк(ов)» або «Нет новых подходящих инбаундов».
- **«Заполнить из дочернего»** — перепідтягнути поля маршрутизації (SNI/ALPN/Path/xver) з транспорту вибраного дочірнього inbound; після виконання — «Заполнено из дочернего».
- **«Изменить поля маршрутизации» / «Скрыть расширенные»** — показати/приховати тонкі поля рядка.
- Підписи **«Маршрутизирует, когда»** та **«По умолчанию — ловит всё остальное»** пояснюють умову спрацювання кожного рядка.

Після збереження фолбеків сервер викликає рестарт Xray, щоб нові `settings.fallbacks` набули чинності.

4.6. Періодичне скидання трафіку

Блок **«Сброс трафика»** налаштовує автоматичне скидання лічильників трафіку inbound за розкладом. Опис:

«Автоматический сброс счетчика трафика через указанные интервалы».

Параметр	Значення
Поле	<code>trafficReset</code>
Валідація	<code>omitempty,oneof=never hourly daily weekly monthly</code>
За замовчуванням	<code>never</code>
Супутнє поле	<code>lastTrafficResetTime</code> — мітка останнього скидання (підпис «Последний сброс»)

Випадний список:

Значення	Підпис
<code>never</code>	«Никогда»
<code>hourly</code>	«Ежечасно»
<code>daily</code>	«Ежедневно»
<code>weekly</code>	«Еженедельно»

Значення	Підпис
monthly	«Ежемесячно»

Для кожного періоду зареєстровано cron-джоб, що запускається за відповідним розкладом (@hourly , @daily , @weekly , @monthly). Джоб вибирає всі inbound із заданим trafficReset і для кожного скидає лічильники самого inbound (up=0 , down=0) та трафік усіх його клієнтів. Тобто періодичне скидання зачіпає й inbound, і його клієнтів.

Приклад значення поля. Щоб лічильники обнулялися першого числа кожного місяця, у формі вибирають «Ежемесячно», що зберігається як:

```
{ "trafficReset": "monthly" }
```

Значення never (за замовчуванням) повністю вимикає автоскидання.

4.7. JSON входящего (advanced)

Розділ «Разделы JSON входящего» дає прямий доступ до сирих JSON-блоків inbound. Опис:

«Полный JSON входящего и отдельные редакторы для settings, sniffing и streamSettings».

Доступні редактори:

Вкладка	Підпис	Що редагує
Всё	«Полный объект входящего со всеми полями в одном редакторе»	весь об'єкт Inbound
Настройки	«Обёртка блока settings Xray»	поле settings
Sniffing	«Обёртка блока sniffing Xray»	поле sniffing
Stream	«Обёртка блока stream Xray»	поле streamSettings

Ці поля серіалізуються як вкладені JSON-об'єкти: порожні блоки віддаються як null , а текст, що не є валідним JSON, обертається в рядок, щоб дані не губилися. Помилки парсингу при збереженні виводяться з префіксом «Расширенный JSON».

Вікно перегляду «JSON входящего», як і вікно імпорту inbound, використовує повноцінний редактор коду з підсвічуванням синтаксису JSON (замість звичайного текстового поля): перегляд конфігурації — у підсвіченому режимі лише для читання, а імпорт — у редагованому, що спрощує читання та правку.

4.8. Дії з inbound: QR / Edit / Reset / Delete та статистика

У списку та в картці inbound доступні такі дії (меню «Меню»):

Статистика трафіку

Відображається агрегований трафік inbound: «Отправлено/получено» (поля up / down), «Всего трафика», «Всего подключений». У картці також — «Создано», «Обновлено», «Дата окончания».

У списку inbounds є колонка **Speed** з поточною швидкістю трафіку по кожному inbound (віддача/завантаження), що обчислюється з приростів лічильників між опитуваннями; та сама жива швидкість показується у вікні статистики inbound. Коли чергове опитування не дає приросту, значення швидкості скидається.

У зведенні клієнтів на сторінці inbounds статус визначається за пріоритетом «вичерпано/завершено»: клієнти, у яких збіг строк або вичерпано трафік (і яким авто-завдання зняло enable), належать до статусу «Исчерпан/завершён» (Depleted/Ended), а не до сірого «Отключён» (Disabled), і не рахуються двічі. Класифікація збігається з тією, що показана в картці самого клієнта, і коректно враховує клієнтів, прив'язаних до кількох inbound.

QR-код та копіювання посилань

- «Подробнее» — розкриває посилання підключення та підписки.
- QR-код клієнта: підказка «Нажмите на QR-код, чтобы скопировать».
- «Копировать ссылку» (англ. Copy URL), «Экспорт ссылок». З версії 3.4.2 сторінкова дія «Експортувати всі посилання» (GET /panel/api/inbounds/allLinks ; вікно «Експорт усіх посилань inbound», файл «All-Inbounds») формує посилання через рушій підписок — застосовує шаблон ремарки до кожного клієнта і надає перевагу керуванню Host-ендпоінтам (раніше використовувалася фіксована ремарка inbound-email).

Edit (Змінити)

«Изменить подключение» — відкриває форму редагування (POST /update/:id). При оновленні сервіс перечитує наявний рядок, переносить змінені поля, за потреби регенерує тег (якщо старий тег був авто-згенерований) та синхронізує рантайм Xray. Успіх — тост «Подключение успешно обновлено».

Reset Traffic (Скинути трафік)

«Сбросить трафик» — обнуляє лічильники up / down саме цього inbound (POST /:id/resetTraffic , ставить up=0, down=0). Підтвердження:

«Сбросить трафик "{remark}"?» / «Сбрасывает счётчики отправки/получения этого подключения до 0».

Скидання трафіку inbound не чіпає лічильники його клієнтів (для них є окремі дії «Сброс трафика клиентов»). Після скидання ініціюється рестарт Xray. Успіх — тост «Входящий трафик сброшен». Є й масовий варіант — «Сброс трафика всех подключений» (POST /resetAllTraffic).

Delete (Видалити)

«Удалить подключение» (POST /del/:id). Підтвердження:

«Удалить подключение "{remark}"?» / «Подключение и все его клиенты будут удалены. Это действие нельзя отменить».

Видалення знімає inbound із працюючого Xray (за потреби з рестартом). Успіх — тост **«Подключение успешно удалено»**. Масове видалення — `POST /bulkDel`, з пер-елементною звітністю та не більше ніж одним рестартом Xray.

Інші дії з клієнтами inbound

У меню також доступні: **«Клонировать»** (копія inbound з новим портом і порожнім списком клієнтів), **«Удалить всех клиентов»** (`POST /:id/delAllClients` — видаляє всіх клієнтів, сам inbound зберігається), **«Удалить отключенных клиентов»**, **«Привязать/Отвязать клиентов»**, **«Импортировать»/«Экспорт подключений»** (`POST /import`). Деталі клієнтських операцій належать до розділу про клієнтів.

5. Протоколи

При створенні вхідного з'єднання (inbound) першим вибирається **Протокол** («Protocol»). Протокол визначає, який спосіб автентифікації та шифрування трафіку Xray-core застосовуватиме до цього inbound, який набір полів у `settings` потрібно заповнити, а також які транспорти (`network`) і види безпеки (TLS / REALITY) для нього доступні.

Поле протоколу задається один раз при створенні inbound і **не змінюється при редагуванні** (у формі редагування випадаючий список заблокований). Щоб змінити протокол, потрібно створити новий inbound.

5.1. Список підтримуваних протоколів

Сервер приймає такий набір значень поля `Protocol` :

```
oneof=vless vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Починаючи з версії **3.3.0** до переліку додано значення `mtproto` (проксі Telegram).

Значення у конфігу	Призначення	Клієнтська модель
<code>vless</code>	Основний проксі-протокол (за замовчуванням при створенні inbound)	Клієнти з UUID, підтримка flow та пост-квантового шифрування
<code>vmess</code>	Класичний проксі-протокол Xray	Клієнти з UUID і параметром <code>security</code>
<code>trojan</code>	Проксі, що маскується під звичайний HTTPS	Клієнти з паролем
<code>shadowsocks</code>	Проксі Shadowsocks (включаючи SIP022 / 2022-blake3)	Один користувач або декілька (2022)
<code>wireguard</code>	Inbound WireGuard	Peer'и (а не клієнти)
<code>hysteria</code>	Inbound Hysteria (за замовчуванням версія 2)	Клієнти з токеном <code>auth</code>
<code>http</code>	Класичний HTTP-проксі (forward proxy)	Облікові записи user/pass, без урахування трафіку
<code>mixed</code>	Поєднаний SOCKS + HTTP-проксі	Облікові записи user/pass
<code>tunnel</code>	Прозорий форвардер (xray dokodemo-door)	Без клієнтів
<code>tun</code>	TUN-інтерфейс (лише рендеринг наявних)	Без клієнтів
<code>mtproto</code>	Проксі Telegram (MTPROTO), додано у 3.3.0; обслуговується окремим процесом <code>mtg</code> , а не Xray	Без клієнтів (доступ за секретом)

Примітка про `tun` : значення залишено у списку для сумісності та **відображення** раніше збережених inbound, але в актуальній версії backend його створення не рекомендується — підтримку визнано застарілою. Створення нових inbound цього типу не має сенсу.

Примітка про Hysteria 2: окремого протоколу «hysteria2» немає. Це протокол `hysteria` з полем `streamSettings.version = 2`. Схема посилання `hysteria2://` при генерації share-посилань вибирається автоматично, коли версія потоку дорівнює 2.

Не всі протоколи підтримують розподіл по вузлах (nodes). На вузли можна розгортати лише: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Протоколи `http`, `mixed`, `tunnel`, `tun`, `mtproto` працюють лише на локальній панелі.

5.2. Які протоколи підтримують TLS / REALITY / транспорт

Можливість увімкнути той чи інший шар безпеки та транспорт залежить від протоколу та вибраної мережі (`streamSettings.network`):

Можливість	Доступна для протоколів	Допустимі мережі (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (а також завжди для <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
<code>flow (xtls-rprx-vision)</code>	лише <code>vless</code>	лише <code>tcp</code> , при <code>security = tls</code> або <code>reality</code>
Stream / транспорт (вкладка «Поток»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	—

Для протоколів `http`, `mixed`, `tunnel`, `tun`, `wireguard` вкладка транспорту недоступна — у них немає stream-налаштувань Xray.

5.3. VLESS

Призначення: основний сучасний проксі-протокол. Підтримує XTLS-Vision (`flow`), REALITY, а також пост-квантове шифрування на рівні самого VLESS (поля `decryption` / `encryption`). Використовується за замовчуванням для нових inbound.

Поля блоку `settings`:

Поле	Значення за замовчуванням	Опис
<code>clients</code>	<code>[]</code>	Список клієнтів. У кожного: <code>id</code> (UUID), <code>email</code> (обов'язковий), <code>flow</code> , ліміти (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Параметр розшифрування на стороні сервера. Підпис у UI: «Розшифрування» (англ. «Decryption»)
<code>encryption</code>	<code>none</code>	Парний параметр шифрування (потрапляє до клієнтського посилання). Підпис: «Шифрування» (англ. «Encryption»)

Поле	Значення за замовчуванням	Опис
<code>fallbacks</code>	<code>[]</code>	Список fallback'ів (див. розділ про fallback'и); доступний, коли <code>network = tcp</code> і <code>security = TLS</code> або <code>REALITY</code>
<code>testseed</code>	(4 числа: 900, 500, 900, 256)	«Vision testseed» — 4 позитивних цілих для XTLS-Vision padding. Застосовується лише до клієнтів з flow <code>xtls-rprx-vision</code> , інакше ігнорується

flow (`xtls-rprx-vision`)

`flow` задається **на клієнті**, а не на inbound, і приймає одне з трьох значень:

Значення	Зміст
<code>""</code> (порожньо)	Без XTLS-flow (за замовчуванням)
<code>xtls-rprx-vision</code>	XTLS-Vision — рекомендований режим для VLESS поверх TCP+TLS/REALITY
<code>xtls-rprx-vision-udp443</code>	Той самий Vision, але з обробкою UDP/443 (QUIC)

Поле `flow` доступне для вибору лише коли виконані всі умови: протокол `vless`, `network = tcp` і `security = tls` або `reality`. Поле **Vision testseed** у формі показується лише за тих самих умов.

Виняток для ХНТТР: при VLESS поверх `network = xhttp` з увімкненою пост-квантовою автентифікацією VLESS (`encryption / decryption, vlessenc`) `flow xtls-rprx-vision` також допустимий — незалежно від шару безпеки, у тому числі з REALITY. У цьому випадку панель коректно передає `xtls-rprx-vision` у share-посилання та підписки (включаючи формат Clash/Mihomo), тому клієнт отримує конфігурацію саме з Vision.

Розшифрування / Шифрування (пост-квантова автентифікація VLESS)

Поля `decryption` і `encryption` — це автентифікація на рівні самого VLESS (окремо від транспортного TLS/REALITY). За замовчуванням обидва дорівнюють `none`. У формі під цими полями розташований блок **«Генерація ключів»** — випадючий список режиму і кнопка **«Згенерувати»** (поруч — кнопка **«Очистити»**). Випадючий список містить шість варіантів: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** — тобто два типи ключа (класичний X25519 і пост-квантовий ML-KEM-768), кожен у трьох режимах:

- **native** — базова пара ключів вибраного типу;
- **xorpub** — похідний режим з додатковою обробкою публічної частини;
- **random** — похідний режим з випадковою складовою.

Виберіть потрібний режим у списку та натисніть **«Згенерувати»**: панель заповнить **обидва** поля (`decryption` і `encryption`) готовою парою значень для цього режиму. Кнопка **«Очистити»** скидає обидва поля назад до `none`.

Під блоком виводиться рядок стану **«Вибрано: ...»**, який розпізнає за згенерованим рядком як тип ключа (X25519 або ML-KEM-768), так і режим (native / xorpub / random) і показує їх. Порожні поля або `none` відображаються як «None».

Технічно кнопки звертаються до `GET /panel/api/server/getNewVlessEnc` (генерація ключів через `xray vlessenc`) і заповнюють **обидва** поля парними значеннями вигляду `mlkem768x25519plus.native.<rtt>.<role>` (наприклад, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). Параметр `decryption` залишається на сервері, `encryption` іде у клієнтське посилання.

Важливо: при генерації конфігурації inbound для Xray панель видаляє зайве: якщо у `settings` залишається `encryption` (що належить до клієнтської сторони), воно **вирізається** з серверного конфігу. На самому сервері залишається лише `decryption`.

Коли вибирати VLESS: це рекомендований варіант за замовчуванням для нового inbound, особливо у зв'язці з REALITY (без власного сертифіката) або з TLS + XTLS-Vision.

Приклад: блок `settings VLESS-inbound` з одним клієнтом і XTLS-Vision. Поле `flow` стоїть на клієнті, `decryption` залишається на сервері:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Для зв'язки REALITY відповідний блок `streamSettings` (вкладка «Transport» → Security: REALITY) виглядає так:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватний ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Призначення: класичний проксі-протокол Xray. Автентифікація за UUID, на клієнті додатково налаштовується метод шифрування корисного навантаження (`security`).

Поля блоку `settings` :

Поле	Значення за замовчуванням	Опис
<code>clients</code>	<code>[]</code>	Список клієнтів

У кожного клієнта VMess (крім загальних полів `email` , лімітів, `enable` , `tgId` , `subId` , `comment` , `reset`):

Поле клієнта	Значення за замовчуванням	Опис
<code>id</code>	—	UUID клієнта
<code>security</code>	<code>auto</code>	Метод шифрування корисного навантаження VMess. Допустимі значення: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

Значення `security` :

- `auto` — Xray сам вибирає шифр залежно від платформи (рекомендується);
- `aes-128-gcm` , `chacha20-poly1305` — фіксовані AEAD-шифри;
- `none` — без шифрування корисного навантаження (має сенс лише поверх TLS);
- `zero` — без шифрування та без автентифікації корисного навантаження.

Історична сумісність: старі записи могли зберігати `security: ""` — при читанні порожній рядок зводиться до `auto` . При генерації серверного конфігу поле `security` у клієнтів VMess **видаляється з `settings`** , оскільки для inbound воно не потрібне.

Коли вибирати VMess: для сумісності зі старими клієнтами або наявними конфігураціями. Для нових розгортань зазвичай кращий VLESS.

5.5. Trojan

Призначення: проксі, що імітує звичайний HTTPS-трафік. Автентифікація за паролем. Як і VLESS, підтримує fallback'и та (при `network = tcp`) REALITY/TLS.

Поля блоку `settings` :

Поле	Значення за замовчуванням	Опис
<code>clients</code>	<code>[]</code>	Список клієнтів
<code>fallbacks</code>	<code>[]</code>	Список fallback'ів (доступний при <code>network = tcp</code> і TLS/REALITY)

У кожного клієнта Trojan ключове поле:

Поле клієнта	Значення за замовчуванням	Опис
password	—	Пароль клієнта (обов'язковий, мінімум 1 символ)
email	—	Унікальний ідентифікатор клієнта

Решта полів клієнта — загальні (limitIp , totalGB , expiryTime , enable , tgId , subId , comment , reset).

Коли вибирати Trojan: коли потрібне маскування під HTTPS на порту 443, у тому числі з fallback'ами на веб-сервер (Nginx) для непрошених підключень.

Приклад: блок settings Trojan з fallback на локальний веб-сервер. Непрошені підключення (без дійсного пароля) надходять на Nginx, який слухає 127.0.0.1:8080 :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Для fallback потрібні network = tcp і Security = TLS або REALITY; інакше поле fallbacks недоступне.

5.6. Shadowsocks

Призначення: легкий проксі Shadowsocks. Підтримує як застарілі AEAD-шифри, так і нові методи SIP022 (2022-blake3-*). Може працювати в однокористувацькому або багатокористувацькому режимі.

Поля блоку settings :

Поле	Значення за замовчуванням	Опис
method	2022-blake3-aes-256-gcm	Метод шифрування inbound. Підпис у UI: «Метод шифрування» (англ. «Encryption method»)
password	""	Пароль inbound (для 2022-методів генерується автоматично під вибраний метод)
network	tcp,udp	Транспорт. Підпис: «Мережа» (англ. «Network»). Варіанти: tcp,udp (TCP, UDP), tcp , udp
clients	[]	Список клієнтів
ivCheck	false (вимкн.)	Перемикач «ivCheck» — захист від повторного використання IV

Методи шифрування (method)

Допустимий набір:

Метод	Категорія
aes-256-gcm	Застарілий AEAD
chacha20-poly1305	Застарілий AEAD
chacha20-ietf-poly1305	Застарілий AEAD
xchacha20-ietf-poly1305	Застарілий AEAD
2022-blake3-aes-128-gcm	SS 2022 (рекомендується)
2022-blake3-aes-256-gcm	SS 2022 (за замовчуванням)
2022-blake3-chacha20-poly1305	SS 2022, однокористувацький

Логіка панелі щодо методів:

- **2022-методи** (2022-blake3-*) вважаються «SS 2022». Метод 2022-blake3-chacha20-poly1305 — **однокористувацький** (multi-user не підтримується); інші 2022-методи допускають кількох клієнтів. Поле пароля (з кнопкою генерації, що підганяє довжину ключа під метод) показується у формі саме для 2022-методів.
- **Застарілі шифри** (aes-* , chacha20-*) працюють за класичною схемою «один метод + один пароль».

Нормалізація перед запуском Xray: для застарілих шифрів кожен клієнт має нести method , що збігається з методом inbound (інакше Xray падає з «unsupported cipher method:»). Для 2022-методів навпаки — поле method у клієнта має бути **порожнім** (інакше Xray відхиляє inbound з «users must have empty method»). Панель сама приводить дані до порядку при перемиканні методу.

Перегенерація клієнтських ключів при зміні розміру ключа: для Shadowsocks-2022 при зміні методу шифрування на метод з іншим розміром ключа (наприклад між 2022-blake3-aes-256-gcm і 2022-blake3-aes-128-gcm) панель автоматично перегенерує клієнтські PSK під нову довжину при збереженні inbound. Інакше старі ключі залишилися б попередньої довжини, і Xray відхилив би їх. Наслідок: зачепленим клієнтам потрібно заново отримати підписку — колишні посилання перестануть підключатися.

Коли вибирати Shadowsocks: для простих розгортань без TLS-маскування; сучасний вибір — 2022-blake3-методи.

Приклад: блок settings Shadowsocks для методу 2022-blake3 (багатокористувацький режим). У inbound — власний пароль (ключ base64 потрібної довжини), у кожного клієнта — свій пароль, поле method клієнта **порожнє**:


```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Для legacy-шифрів (`aes-256-gcm` тощо) — навпаки: пароль один на inbound, а `method` клієнта має збігатися з методом inbound.

5.7. Dokodemo-door / Tunnel (прозорий форвардер)

Призначення: прозорий форвардер (у панелі — протокол `tunnel`, що реалізує поведінку `dokodemo-door`). Приймає трафік і переадресовує його на задану адресу/порт, без автентифікації та клієнтів.

Поля блоку `settings`:

Поле	Значення за замовчуванням	Опис
<code>rewriteAddress</code>	(немає)	«Переписати адресу» (англ. «Rewrite address») — цільова адреса, на яку перенаправляється трафік
<code>rewritePort</code>	(немає)	«Переписати порт» (англ. «Rewrite port») — цільовий порт (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Дозволена мережа» (англ. «Allowed network»). Варіанти: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Відповідність портів» — карта порт→порт (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (вимкн.)	«Слідувати redirect» (англ. «Follow redirect») — використовувати початкову адресу призначення з перехопленого з'єднання

Вкладка «Transport» для Tunnel: у inbound цього типу доступна вкладка **«Transport»**, обмежена налаштуванням `sockopt` — цього достатньо для режиму **TProxy** (прозоре проксіювання/redirect через `sockopt.tproxy`). Випадаючий список вибору транспорту (`network`) та вкладка «Security» для Tunnel приховані, оскільки TLS/REALITY цим типом не підтримуються.

Коли вибирати: для прозорого проксіювання/перенаправлення портів на внутрішні сервіси.

Поле «Переписати порт» (`rewritePort`) можна залишати порожнім: при очищенні значення просто виключається з налаштувань inbound, а не викликає помилку збереження. (Раніше очищення цього поля призводило до помилки валідації `settings.rewritePort` і блокувало збереження, у тому числі через вкладку JSON.)

5.8. SOCKS / HTTP (протокол `mixed`)

У цій збірці окремого протоколу `socks` немає – SOCKS і HTTP-проксі об'єднані у протокол `mixed` (поєднаний SOCKS + HTTP). Крім того, є окремий чистий `http`-проксі.

5.8.1. Mixed (SOCKS + HTTP)

Поля блоку `settings` :

Поле	Значення за замовчуванням	Опис
<code>auth</code>	<code>password</code>	«Auth» – режим автентифікації. Варіанти: <code>password</code> (за логіном/паролем) або <code>noauth</code> (без авторизації)
<code>accounts</code>	(опціонально)	«Облікові записи» – список пар user/pass. При <code>auth = noauth</code> поле не записується у конфіг
<code>udp</code>	<code>false</code> (вимкн.)	Перемикач «UDP» – підтримка UDP через SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – локальна адреса для UDP-асоціацій. Поле показується лише при увімкненому <code>udp</code>

Облікові записи додаються кнопкою «Додати»; при додаванні генеруються випадкові логін (8 символів) і пароль (12 символів), які можна відредагувати.

5.8.2. HTTP (чистий проксі)

Призначення: класичний форвард-проксі HTTP. На рівні Xray не відстежує клієнтів як «білінгових» (немає email/лімітів) – є лише список облікових записів.

Поля блоку `settings` :

Поле	Значення за замовчуванням	Опис
<code>accounts</code>	<code>[]</code>	«Облікові записи» – список пар user/pass (обидва поля обов'язкові)
<code>allowTransparent</code>	<code>false</code> (вимкн.)	«Дозволити прозорий» (англ. «Allow transparent») – пересилати запити з оригінальним заголовком Host

Коли вибрати SOCKS/HTTP: для локального або службового проксі-доступу без складного маскуванню. `mixed` зручний тим, що один порт обслуговує і SOCKS, і HTTP-клієнтів.

5.9. WireGuard (inbound)

Призначення: inbound WireGuard – VPN-тунель WireGuard, а не маскований проксі. Транспорт і TLS/REALITY до нього незастосовні.

З версії 3.4.2 WireGuard переведено на мультиклієнтну модель (як VLESS/VMess/Trojan/Shadowsocks/Hysteria). Сам inbound зберігає лише серверну частину; пристрої-піри тепер додаються як звичайні **клієнти** (див. розділ 8) — з автоматичною видачею адреси в тунелі, підтримкою підписок, лімітів трафіку/терміну та груп. Попередній інлайн-список «Реєр'и / Додати реєр» з форми inbound прибрано: новий inbound створюється без пірів, а кожному клієнту адреса видається автоматично.

Поля inbound (блок `settings`):

Поле	Значення за замовчуванням	Опис
<code>secretKey</code>	—	Приватний ключ сервера (обов'язковий). Поруч кнопка генерації; публічний ключ (<code>Public Key</code>) виводиться з нього автоматично (лише для читання; окремо збережений <code>pubKey</code> прибрано)
<code>mtu</code>	1420	MTU інтерфейсу
<code>dns</code>	1.1.1.1, 1.0.0.1	Нове в 3.4.2. DNS, що записується в рядок <code>DNS =</code> клієнтського <code>.conf</code>
<code>noKernelTun</code>	<code>false</code> (вимкн.)	«TUN без kernel» — userspace-TUN замість ядерного
<code>domainStrategy</code>	(опціонально)	«Domain Strategy»: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>

Параметри WireGuard у клієнта — вкладка «Облікові дані» у вікні додавання/редагування клієнта (показуються, коли прив'язаний inbound — WireGuard):

Поле	Опис
«Приватний ключ WireGuard»	Приватний ключ клієнта (редаговане, є кнопка «Перегенерувати»); при введенні з нього автоматично виводиться публічний ключ
«Публічний ключ WireGuard»	Лише для читання; обчислюється з приватного
«Спільний ключ WireGuard» (PSK)	Необов'язковий додатковий спільний ключ
«Дозволені IP WireGuard»	З 3.5.0 — редаговане (плейсхолдер <code>10.0.0.2/32</code> , підказка «Залиште порожнім для авто-видачі; записи розділяйте комами»). Порожньо — адреса видається автоматично; сервер перевіряє кожен запис (IP або CIDR), одиночні адреси нормалізує до <code>/32</code> і відхиляє адресу, зайняту іншим клієнтом цього inbound

Адреса в тунелі видається сервером автоматично з підмережі inbound (за замовчуванням `10.0.0.0/24` : сервер займає `.1` , клієнти — починаючи з `.2`); якщо наявні клієнти використовують іншу `/24` , нові адреси видаються в тій самій підмережі.

Domain Strategy і вкладка Transport

Крім перелічених, у WireGuard-inbound є поле **Domain Strategy** (стратегія розв'язання доменів: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`). Поле необов'язкове і записується у конфіг, лише якщо задане.

Поле **Workers** (workers , кількість робочих потоків) прибрано з форм WireGuard (як inbound, так і outbound): починаючи з xray-core v26.6.22 рушій його більше не використовує і покладається на внутрішній механізм wireguard-go. Раніше збережені конфіги працюють без змін — при розборі поле просто відкидається, міграція не потрібна.

Для WireGuard також доступна вкладка **«Transport»** — але в урізаному вигляді: на ній налаштовуються лише sockopt і обфускація **Finalmask**. Випадаючий список вибору транспорту (network) прихований, оскільки WireGuard завжди слухає по UDP. У шумових записках (noise) Finalmask окремим полем задається **Rand Range** (діапазон байт 0–255, з валідацією), а як метод обфускації для WireGuard і Hysteria доступний **Salamander**.

Дії inbound та експорт (з 3.5.0)

Меню рядка WireGuard-inbound тепер містить повний «мультиклієнтний» набір дій — **«Експортувати всі посилання»**, експорт за підпискою, **прив'язка/відв'язка клієнтів**, додавання до групи і видалення всіх клієнтів (раніше були лише експорт/скидання/клонування/видалення), а в списку відображається лічильник клієнтів. Для WireGuard вікно «Експорт усіх посилань» розділено на дві вкладки: **Config** (склеєні .conf -блоки, по одному на клієнта) і **Links** (персональні посилання wireguard://); «Копіювати» і «Завантажити» діють на відкриту вкладку. Для inbound, розміщеного на вузлі, Endpoint у клієнтському .conf /QR тепер вказує **адресу вузла**, а не головної панелі.

Конфігурація та шеринг клієнта WireGuard

У WireGuard-клієнта у вікнах «Інформація»/QR доступна **згортна картка конфігурації**: повний .conf ([Interface] з PrivateKey / Address / DNS / MTU і [Peer] з PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive) з кнопками **Копіювати**, **Завантажити** і **QR**. Підтримується також посилання виду wireguard:// / wg:// , яке клієнт-застосунок розбирає в готовий .conf .

Коли вибирати WireGuard: коли потрібен саме VPN-тунель WireGuard, а не маскований проксі.

5.10. Hysteria (за замовчуванням v2)

Призначення: inbound Hysteria поверх QUIC. Панель за замовчуванням працює з версією 2. Кожен клієнт автентифікується токеном auth замість UUID/пароля. TLS для Hysteria доступний завжди (див. таблицю можливостей у 5.2).

Поля блоку settings :

Поле	Значення за замовчуванням	Опис
version	2	Версія протоколу (мінімум 1; панель за замовчуванням 2)
clients	[]	Список клієнтів

У кожного клієнта ключове поле — `auth` (токен, обов'язковий) плюс загальні поля (`email`, `ліміти`, `enable`, `tgId`, `subId`, `comment`, `reset`).

Додаткові параметри задаються у `streamSettings.hysteriaSettings`:

Поле	Значення / варіанти	Опис
<code>version</code>	зафіксовано 2 (поле заблоковано)	«Версія» (англ. «Version»)
<code>udpIdleTimeout</code>	(ціле ≥ 1 , сек.)	«UDP idle timeout (с)» — тайм-аут простою UDP
<code>masquerade</code>	вимкнений за замовчуванням	«Masquerade» — маскуванню під звичайний веб-сервер при «непрошених» запитах

При увімкненні `masquerade` доступний вибір типу (`type`):

- `` — default (сторінка 404);
- `proxy` — зворотній проксі (поля «Upstream URL», «Переписати Host», «Пропустити TLS verify»);
- `file` — роздача каталогу (поле «Директорія», наприклад `/var/www/html`);
- `string` — фіксована відповідь (поля «Код статусу», «Body», «Заголовки»).

Коли вибирати Hysteria: при потребі у QUIC-транспорті та стійкості на нестабільних/мобільних каналах; маскуванню підвищує прихованість точки входу.

5.11. MTPROTO (проксі для Telegram)

Додано у версії **3.3.0**. Значення протоколу — `mtproto`.

MTPROTO — протокол власного проксі Telegram. У 3X-UI такий inbound **обслуговується не Хґау, а окремим процесом `mtg`**, яким керує сама панель. Панель періодично звіряє увімкнені MTPROTO-inbound із запущеними процесами `mtg`: піднімає відсутні, зупиняє зайві та знімає лічильники трафіку з метрик `mtg`. Тому **облік трафіку** за таким inbound працює як у звичайних протоколів.

З версії 3.5.0 MTPROTO — мультиклієнтний протокол (рушій змінено на форк `mtg-multi`): один inbound обслуговує багато користувачів, кожен з яких — звичайний **клієнт** (див. розділ 8) зі своїм FakeTLS-секретом, квотою трафіку, терміном, перемикачем увімкнення, необов'язковим ad-tag і персональним посиланням `tg://proxy`. Попередню модель «один inbound = один секрет» скасовано: поле «Secret» на рівні inbound з форми прибрано, а при оновленні панелі наявний одиночний секрет автоматично перетворюється на першого клієнта цього inbound (ручних дій не потрібно).

Наслідки:

- Вкладка **«Транспорт» (Stream Settings)** до цього inbound не застосовується; клієнти ж тепер керуються як у решти протоколів — прив'язка/відв'язка, ліміти, групи, підписки.
- Зміни клієнтів (додавання, видалення, зміна секрету, увімкнення/вимкнення, ad-tag) застосовуються **«на льоту»** через management-API `mtg`, не розриваючи Telegram-сесії решти користувачів; лише структурні зміни inbound (адреса, domain fronting, ліміт з'єднань, публічні IP) перезапускають процес.

- MTPROTO-inbound запускається **лише на основній панелі**; на дочірні вузли (nodes) він не розгортається (inbound із заданим NodeID пропускаються).
- Вкладка **«Sniffing»** для MTPROTO прихована — цей протокол обслуговується процесом `mtg`, а не Xray, тому сніфінг до нього незастосовний.

Поля. Зберігаються у `settings inbound`:

Поле у UI	Ключ	Опис
Remark	<code>remark</code>	Мітка inbound.
Listen IP	<code>listen</code>	IP для прослуховування (порожньо = усі інтерфейси).
Port	<code>port</code>	Порт проксі.
Домен прикриття (FakeTLS)	<code>settings.fakeTlsDomain</code>	За замовчуванням <code>www.cloudflare.com</code> з 3.5.0 — домен за замовчуванням для генерації секретів нових клієнтів (підказка: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); кожен клієнт може прикриватися власним доменом через свій секрет.
Max connections	<code>settings.throttleMaxConnections</code>	Нове в 3.5.0. Обмеження одночасних з'єднань на всіх користувачів із чесним розподілом; <code>0</code> — без обмеження.
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	Нове в 3.5.0. Публічна адреса сервера для ad-tag middle проху (плейсхолдери <code>1.2.3.4</code> / <code>2001:db8::1</code>); порожньо — <code>mtg</code> визначить сам.

Формат секрету (FakeTLS). Секрет тепер живе у кожного клієнта (поле «MTPROTO secret» у формі клієнта, з кнопкою регенерації; поруч — необов'язковий «Ad-tag (спонсорський канал)», рівно 32 hex-символи). Вигляд секрету попередній: `ee` + 32 hex-символи + hex-код домену прикриття (`ee<hex32><hex(домен)>`); при прив'язці нового клієнта до MTPROTO-inbound секрет генерується автоматично від домену inbound за замовчуванням. Квота трафіку і термін дії клієнта з 3.5.0 **реально застосовуються** (через ліміти `mtg-multi`): вичерпаний або прострочений клієнт відключається, а скидання трафіку повертає доступ одразу.

Domain-fronting і розширені опції `mtg`

У MTPROTO-inbound є додаткові параметри процесу `mtg`. Поля **Domain fronting IP**, **Domain fronting port** і **Domain fronting PROXY protocol** задають, куди `mtg` надсилає не-Telegram трафік (наприклад, на фейковий сайт NGINX): якщо IP залишити порожнім, використовується FakeTLS-домен через DNS, порт за замовчуванням — `443`. Додатково доступні **Accept PROXY protocol** (для слухача), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) і **Debug logging**. Кожне значення записується у файл `mtg-<id>.toml`, лише якщо воно задане.

Маршрутизація Telegram-трафіку через Xray

Перемикач **«Route through Xray»** (за замовчуванням вимкнений) і необов'язкове поле **Outbound** дозволяють підпорядкувати egress Telegram маршрутизатору Xray. При увімкненні панель вбудовує у конфіг Xray локальний SOCKS-міст із тегом самого inbound, і `mtg` надсилає Telegram-трафік через нього. Після

цього трафік можна зіставляти правилами на вкладці «Routing» або примусово направити у вибраний outbound чи балансувальник через поле **Outbound** (якщо поле порожнє, вирішують правила маршрутизації).

Як роздати користувачу. Для MTProto-inbound панель генерує посилання-запрошення:

Приклад: секрет FakeTLS і готове посилання. Якщо домен прикриття — `www.cloudflare.com`, секрет складається як `ee` + 32 hex-символи + hex-код домену, наприклад:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Готове посилання-запрошення (його та QR-код надсилають користувачу у Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f75646
66c6172652e636f6d
```

```
tg://proxy?server=<адреса>&port=<порт>&secret=<секрет>
```

(еквівалент — `https://t.me/proxy?server=...&port=...&secret=...`). З 3.5.0 посилання **персональне** — будується із секрету конкретного клієнта, а фрагмент `#remark` з нього прибрано (нестрогі парсери Telegram склеювали його із секретом і ламали імпорт); примітка показується окремою міткою у вікні інформації. Це посилання та QR-код потрібно надіслати користувачу Telegram — при відкритті проксі одразу додається до застосунку. Посилання також повертається через сервер підписок.

Коли використовувати. Штатний спосіб обходу блокувань Telegram; FakeTLS-маскування (домен прикриття) робить трафік схожим на звичайний відвідинний указаного сайту.

5.12. Коротка шпаргалка щодо вибору протоколу

- **VLESS** — вибір за замовчуванням; найкращий варіант з REALITY або TLS + XTLS-Vision, підтримує пост-квантову автентифікацію.
- **Trojan** — маскування під HTTPS з fallback'ами на веб-сервер.
- **VMess** — сумісність зі старими клієнтами.
- **Shadowsocks** — простий проксі без TLS; сучасний вибір — методи `2022-blake3-*`.
- **Hysteria** — QUIC, стійкість на поганих каналах.
- **mixed / http** — службові SOCKS/HTTP-проксі.
- **WireGuard** — повноцінний VPN-тунель.
- **tunnel** — прозоре перенаправлення портів.
- **MTProto** — проксі для обходу блокувань Telegram (FakeTLS); окремий процес `mtg`.

6. Транспорт (Stream Settings)

Транспорт (в інтерфейсі панелі — поле **«Транспорт»**, англ. *Transmission*) визначає спосіб, у який Xray-core передає дані всередині inbound: який мережевий протокол використовується поверх TLS/Reality і як саме кадрється трафік. Ці параметри зберігаються в об'єкт `streamSettings` конфігурації Xray і задаються на вкладці транспорту в редакторі inbound. Шифрування (TLS / Reality) розглядається в окремому розділі — тут описується лише вибір мережі та її параметри.

6.1. Вибір мережі передачі

Мережа вибирається у випадному списку **«Транспорт»** (`streamSettings.network`). Значення за замовчуванням — `tcp` (у списку відображається як **RAW**). Доступні такі варіанти:

Значення в списку	Поле <code>network</code>	Транспорт
RAW	<code>tcp</code>	Звичайний TCP (у нових версіях Xray перейменований на RAW), опційно з HTTP-обфускацією
mKCP	<code>kcp</code>	Надійний UDP-транспорт mKCP
WebSocket	<code>ws</code>	WebSocket поверх HTTP(S)
gRPC	<code>grpc</code>	gRPC-тунель (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP — сучасний мультиплексований транспорт

Під час зміни значення панель очищає блок налаштувань попередньої мережі та заповнює блок нової мережі значеннями за замовчуванням з її схеми, тому кожне поле під-форми завжди має осмислене початкове значення.

Важливо. У цій збірці панелі **транспорт HTTP/2 (h2) у списку відсутній** — його було вилучено з набору мереж; для двоспрямованого HTTP/2-подібного тунелю використовується gRPC, а для сучасного HTTP-маскованого транспорту — XHTTP. Транспорт **Hysteria (hysteria)** не вибирається через цей список: він жорстко прив'язаний до протоколу Hysteria і з'являється автоматично, коли сам inbound створюється з протоколом Hysteria (див. п. 6.8).

Нижче кожен мережу та кожне її поле розібрано окремо.

6.2. RAW / TCP (`tcpSettings`)

Базовий TCP-транспорт. За замовчуванням трафік передається «як є»; опційно його можна замаскувати під звичайний HTTP/1.1-обмін.

Поле	Значення за замовчуванням	Опис
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (вимк.)	Приймати заголовок PROXY protocol від вищого балансувальника/проксі
HTTP-обфускація (<code>header.type</code>)	<code>none</code> (вимк.)	Вмикає маскування трафіку під HTTP/1.1

Proxy Protocol

Перемикач «**Proxy Protocol**» (`acceptProxyProtocol`). Коли увімкнений, Xray очікує на вхідному з'єднанні заголовок PROXY protocol і витягує з нього реальний IP клієнта. Вмикають лише якщо перед панеллю стоїть зворотний проксі/балансувальник (наприклад, HAProxy або nginx зі `send-proxy`), який цей заголовок додає. За замовчуванням вимкнений.

HTTP-обфускація (camouflage)

Перемикач «**HTTP Обфускація**». Керує полем `header` :

- **Вимкнений** → `header.type = "none"` (на дроті поле `header` просто відсутнє). Чистий TCP.
- **Увімкнений** → `header.type = "http"` . Трафік кадрується під виглядом HTTP/1.1-запиту та відповіді. Під час увімкнення панель одразу заповнює під-об'єкти `request` і `response` значеннями за замовчуванням.

За увімкненої HTTP-обфускації з'являються поля налаштування імітованих запиту та відповіді.

Заголовок запиту (`header.request`):

Поле	Ключ	Значення за замовчуванням	Опис
Версія запиту	<code>request.version</code>	<code>1.1</code>	Версія HTTP у стартовому рядку запиту
Метод запиту	<code>request.method</code>	<code>GET</code>	HTTP-метод імітованого запиту
Шлях запиту	<code>request.path</code>	<code>/</code>	Шлях(и). Вводиться списком значень через кому; на дроті це масив рядків. Якщо залишити порожнім, підставляється <code>/</code>
Заголовки запиту	<code>request.headers</code>	<code>{}</code> (порожньо)	Таблиця «Ім'я/Значення» HTTP-заголовків. Зберігається як карта <code>ім'я → [значення]</code> (одному імені може відповідати кілька значень)

Заголовок відповіді (`header.response`):

Поле	Ключ	Значення за замовчуванням	Опис
Версія відповіді	<code>response.version</code>	<code>1.1</code>	Версія HTTP у стартовому рядку відповіді

Поле	Ключ	Значення за замовчуванням	Опис
Статус відповіді	<code>response.status</code>	<code>200</code>	HTTP-код статусу імітованої відповіді
Причина відповіді	<code>response.reason</code>	<code>OK</code>	Текстовий опис статусу (reason-phrase)
Заголовки відповіді	<code>response.headers</code>	<code>{}</code> (порожньо)	Таблиця «Ім'я/Значення» заголовків відповіді (карта ім'я → [значення])

Поля заголовків редагуються по рядках — кожен рядок задає ім'я заголовка (Ім'я) та його значення (Значення). Ці параметри використовують лише для маскування зовнішнього вигляду трафіку; на криптографію вони не впливають. Значення за замовчуванням (GET / HTTP/1.1 , відповідь 200 OK) підходять для більшості сценаріїв — змінювати їх варто, лише якщо потрібно імітувати конкретний сайт/сервіс.

Приклад `streamSettings` для RAW з HTTP-обфускацією:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Зверніть увагу: `path` на дроті — масив рядків, а кожен заголовок — масив значень (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP — надійний транспорт поверх UDP. Корисний на каналах із втратами пакетів і високою затримкою, але створює підвищений службовий трафік. Усі значення за замовчуванням збігаються з рекомендованими в xray-core.

Поле	Ключ	За замовчуванням	Допустимо	Опис
MTU	<code>mtu</code>	<code>1350</code>	576–1460	Максимальний розмір пакета (байт). Зменшують за проблем фрагментації
TTI (мс)	<code>tti</code>	<code>20</code>	10–100	Інтервал передачі (ms). Менше — нижча затримка, але вищі накладні витрати
Uplink (МБ/с)	<code>uplinkCapacity</code>	<code>5</code>	≥ 0	Розрахункова пропускна здатність на віддачу (МБ/с)
Downlink (МБ/с)	<code>downlinkCapacity</code>	<code>20</code>	≥ 0	Розрахункова пропускна здатність на прийом (МБ/с)
Множник CWND	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Множник вікна перевантаження (congestion window)
Макс. вікно відправлення	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Максимальний розмір вікна відправлення

Зауваження щодо полів:

- **Uplink / Downlink capacity** задають, наскільки агресивно mKCP займає канал. Їх виставляють відповідно до реальної ширини каналу: завищені значення ведуть до зайвого трафіку, занижені — до недовикористання каналу.
- **TTI** напряму впливає на компроміс «затримка [?] накладні витрати»: менші значення знижують затримку, але збільшують обсяг службових пакетів.
- **MTU** обмежує розмір одного пакета mKCP; зниження допомагає на каналах, де великі UDP-пакети ріжуться або втрачаються.

У цій збірці поле «seed» (пароль обфускації mKCP) та випадний список **типу заголовка/обфускації** (`none` , `srtp` , `utp` , `wechat-video` , `dtls` , `wireguard`) у під-формі mKCP **відсутні як окремі поля** — обфускацію транспортного рівня винесено в загальний механізм «FinalMask» (включно з режимом `mKCP-legacy`), описуваний у відповідному розділі. Параметр «congestion» як окрема галочка також не виведений; керування перевантаженням задається через `cwndMultiplier` і `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

WebSocket-транспорт поверх HTTP(S). Добре проходить через CDN та зворотні проксі, маскується під звичайний вебтрафік.

Поле	Ключ	За замовчуванням	Опис
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Приймати заголовок PROXY protocol від вищого проксі (див. п. 6.2)
Хост	<code>host</code>	<code>""</code> (порожньо)	Значення HTTP-заголовка <code>Host</code> . Указують під час роботи через CDN/домен-фронтиг

Поле	Ключ	За замовчуванням	Опис
Шлях	<code>path</code>	<code>/</code>	Шлях у рядку запиту WebSocket-рукописання
Період heartbeat	<code>heartbeatPeriod</code>	<code>0</code>	Інтервал відправлення heartbeat-кадрів (у секундах). <code>0</code> вимикає heartbeat
Заголовки	<code>headers</code>	<code>{}</code> (порожньо)	Додаткові HTTP-заголовки рукописання. Карта «Ім'я → Значення» (лише рядкові значення, без масивів)

Зауваження:

- **Шлях** повинен збігатися на сервері (inbound) та в клієнта. Часто за цим шляхом на боці вебсервера маскують точку входу.
- **Хост** має сенс задавати, якщо inbound стоїть за CDN або використовується фронтинг за доменом; інакше можна залишити порожнім.
- **Період heartbeat** тримає з'єднання «живим» під час проходження через проксі/CDN, що агресивно розривають неактивні сесії. За замовчуванням (`0`) heartbeat вимкнений.
- На відміну від RAW, таблиця заголовків WebSocket використовує «плоский» формат `ім'я → значення` (один рядок значення на заголовок).

Приклад `streamSettings` для WebSocket за CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Значення `host` і `path` повинні збігатися в клієнта; на відміну від RAW значення заголовка тут — звичайний рядок, а не масив.

6.5. gRPC (`grpcSettings`)

Найлегший за кількістю параметрів транспорт. Тунелює трафік усередині gRPC-викликів (поверх HTTP/2); добре сумісний із CDN, що підтримують gRPC. Обфускації заголовків немає.

Поле	Ключ	За замовчуванням	Опис
Ім'я сервісу (<code>Service Name</code>)	<code>serviceName</code>	<code>""</code> (порожньо)	Ім'я gRPC-сервісу (фактично — «шлях» тунелю). Повинно збігатися в сервера та клієнта

Поле	Ключ	За замовчуванням	Опис
Authority	<code>authority</code>	<code>""</code> (порожньо)	Значення псевдо-заголовка <code>:authority</code> (аналог <code>Host</code> для HTTP/2). Указують під час роботи через CDN/домен
Multi Mode	<code>multiMode</code>	<code>false</code> (вимк.)	Вмикає мультиплексування кількох паралельних gRPC-потоків усередині одного з'єднання

Зауваження:

- **Service Name** — основний ідентифікатор gRPC-каналу; він повинен бути однаковим на обох сторонах. Порожнє значення припустиме, але зазвичай задають неочевидний рядок для маскування.
- **Authority** впливає на те, який `:authority` відправляється у HTTP/2-кадрах; потрібен насамперед під час проксіювання через CDN.
- **Multi Mode** дозволяє кільком логічним потокам іти через одне фізичне з'єднання; вмикають для підвищення продуктивності, коли і сервер, і клієнт це підтримують.

Приклад `streamSettings` для gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (тут `GunService`) виконує роль «шляху» тунелю і повинен збігатися в сервера та клієнта.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Транспорт на основі механізму HTTP Upgrade (як у WebSocket, але без самого WebSocket-протоколу). Також добре проходить через проксі та CDN. Набір полів повторює WebSocket, але **без** періоду heartbeat.

Поле	Ключ	За замовчуванням	Опис
Proху Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Приймати заголовок PROXY protocol від вищого проксі
Хост	<code>host</code>	<code>""</code> (порожньо)	Значення HTTP-заголовка <code>Host</code>
Шлях	<code>path</code>	<code>/</code>	Шлях HTTP-запиту із заголовком <code>Upgrade</code>
Заголовки	<code>headers</code>	<code>{}</code> (порожньо)	Додаткові HTTP-заголовки. «Плоска» карта ім'я → значення (як у WebSocket)

Призначення полів **Хост**, **Шлях** і **Заголовки** збігається з WebSocket (п. 6.4). Heartbeat для HTTPUpgrade не передбачений — це специфіка WebSocket.

6.7. ХНТТP / SplitHTTр (xhttpSettings)

ХНТТP (він же SplitHTTр) — сучасний мультиплексований HTTP-транспорт xray-core. Розділяє висхідний та низхідний потоки на окремі HTTP-запити, що добре підходить для CDN і середовищ з обмеженнями за тривалістю з'єднань. Поля видимі в редакторі не всі одразу: частина з них з'являється залежно від обраного режиму (mode) та увімкнених перемикачів.

Базові поля (видні завжди)

Поле	Ключ	За замовчуванням	Опис
Хост	host	"" (порожньо)	Значення HTTP-заголовка Host
Шлях	path	/	Базовий шлях HTTP-запитів
Режим (Mode)	mode	auto	Режим передачі (див. нижче)
Server Max Header Bytes	serverMaxHeaderBytes	0	Ліміт розміру заголовків запиту на сервері (байт). 0 — значення за замовчуванням xray-core
Padding Bytes	xPaddingBytes	100–1000	Діапазон випадкового «набивного» паддингу (у байтах, формат мін–макс) для ускладнення аналізу розмірів
Заголовки	headers	{ } (порожньо)	Додаткові HTTP-заголовки. «Плоска» карта ім'я → значення
HTTP-метод Uplink	uplinkHTTPMethod	"" (Default = POST)	HTTP-метод висхідних запитів. Варіанти: порожньо (за замовчуванням POST), POST , PUT , GET (останній доступний лише в режимі packet–up)
Padding Obfs Mode	xPaddingObfsMode	false (вимк.)	Вмикає розширену обфускацію паддингу та відкриває додаткові поля (див. нижче)
No SSE Header	noSSEHeader	false (вимк.)	Не відправляти заголовок Content–Type: text/event–stream (SSE). Вмикають, якщо він заважає проходженню через проміжні вузли

Поле «Режим» (mode)

Випадний список зі значеннями:

Значення	Опис
auto	Авто-вибір режиму (за замовчуванням)
packet–up	Висхідний потік розбивається на окремі HTTP-запити (по пакету на запит)
stream–up	Висхідний потік передається одним тривалим потоковим запитом
stream–one	Один загальний двоспрямований потоковий запит

Вибір режиму визначає, які додаткові поля стають видимими.

Поля, видимі лише за `mode = packet-up` :

Поле	Ключ	За замовчуванням	Опис
Макс. буферизоване завантаження	<code>scMaxBufferedPosts</code>	<code>30</code>	Максимум одночасно буферизованих POST-запитів висхідного потоку
Макс. розмір завантаження (байт)	<code>scMaxEachPostBytes</code>	<code>1000000</code>	Максимальний розмір одного висхідного POST-запиту (байт)
Uplink Data Placement	<code>uplinkDataPlacement</code>	<code>""</code> (Default = <code>body</code>)	Де розмішувати дані висхідного потоку: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	<code>uplinkDataKey</code>	<code>""</code>	Ім'я ключа/заголовка для даних uplink. З'являється, лише якщо <code>uplinkDataPlacement</code> заданий і не дорівнює <code>body</code>

Поле, видиме лише за `mode = stream-up` :

Поле	Ключ	За замовчуванням	Опис
Stream-Up Server	<code>scStreamUpServerSecs</code>	<code>20-80</code>	Діапазон часу утримання серверного потокового з'єднання (у секундах, формат <code>мін-макс</code>)

Поля обфускації паддингу (видні за `xPaddingObfsMode = увімк.`)

Поле	Ключ	За замовчуванням	Опис
Padding Key	<code>xPaddingKey</code>	<code>""</code> (placeholder <code>x_padding</code>)	Ім'я ключа для паддингу
Padding Header	<code>xPaddingHeader</code>	<code>""</code> (placeholder <code>X-Padding</code>)	Ім'я HTTP-заголовка, у якому передається паддинг
Padding Placement	<code>xPaddingPlacement</code>	<code>""</code> (Default = <code>queryInHeader</code>)	Де розмішувати паддинг: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	<code>""</code> (Default = <code>repeat-x</code>)	Метод генерації паддингу: <code>repeat-x</code> або <code>tokenish</code>

Розміщення сесії та послідовності (видні завжди)

Поле	Ключ	За замовчуванням	Опис
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = <code>path</code>)	Де передавати ідентифікатор сесії: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>

Поле	Ключ	За замовчуванням	Опис
Session ID Key	<code>sessionIDKey</code>	"" (placeholder <code>x_session</code>)	Ім'я ключа сесії. З'являється, лише якщо <code>sessionIDPlacement</code> заданий і не дорівнює <code>path</code>
Session ID Table	<code>sessionIDTable</code>	"" (placeholder <code>Base62</code>)	Набір символів для генерації ідентифікаторів сесії. Можна вибрати визначений наперед з випадного списку-автодоповнення (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) або ввести довільний ASCII-рядок. Порожньо – значення за замовчуванням <code>хгау-соре</code>
Session ID Length	<code>sessionIDLength</code>	"" (порожньо)	Довжина або діапазон (наприклад <code>8–16</code>) генерованих ідентифікаторів. Показується, лише коли заданий <code>Session ID Table</code> ; мінімум повинен бути більше 0
Sequence Placement	<code>seqPlacement</code>	"" (Default = <code>path</code>)	Де передавати порядковий номер пакета: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	"" (placeholder <code>x_seq</code>)	Ім'я ключа послідовності. З'являється, лише якщо <code>seqPlacement</code> заданий і не дорівнює <code>path</code>

Поля сесії перейменовано під `хгау-соре v26.6.22`: раніше вони називалися **Session Placement / Session Key** (`sessionPlacement` / `sessionKey`) – тепер це **Session ID Placement / Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); старого імені ядро більше не розуміє. Inbound, збережені до оновлення, мігруються на нові ключі автоматично – пересохраняти їх не потрібно.

Рекомендації:

- Для більшості установок достатньо залишити **Режим** = `auto` , задати **Шлях/Хост** і (під час роботи через CDN) узгодити їх із клієнтом.
- Поля розміщення (`*Placement` / `*Key`) та обфускації паддингу потрібні лише для тонкого підлаштування під конкретний анти-DPI/CDN-сценарій; за порожніх значень використовуються значення за замовчуванням `хгау-соре`, указані в дужках.
- Параметри, що стосуються боку клієнта/outbound (наприклад, інтервали повторних POST, розміри чанків), у формі inbound не виводяться – сервер-слухач їх ігнорує. Мультиплексор XMUX, навпаки, у формі inbound доступний (див. нижче).
- **Службові дефолти не проставляються.** Панель більше не записує в конфіги ХНТТР службові значення за замовчуванням `scMaxEachPostBytes` і `scMinPostsIntervalMs` – застосовуються внутрішні значення `хгау-соре`. Це усуває постійну DPI-сигнатуру (літерал `scMinPostsIntervalMs=30`), за якою раніше міг блокуватися трафік. Для вже збережених inbound значення, що збігаються з дефолтами `хгау-соре`, не виводяться в посиланнях і підписці (пересохраняти inbound не потрібно); задані вручну значення зберігаються.

Приклад `streamSettings` для ХНТТР (режим `auto`):


```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Для більшості установок достатньо цих чотирьох полів; поля розміщення сесії/послідовності та обфускації паддингу залишають порожніми — тоді використовуються значення за замовчуванням xray-core.

XMUX (мультиплексування з'єднань)

Перемикач **XMUX** (`enableXmux`) вмикає шар мультиплексування, який розподіляє паралельні запити за невеликим пулом фізичних з'єднань. Під час увімкнення розкриваються шість налаштовуваних полів (зберігаються в `xhttpSettings.xmux`):

Поле	Ключ	За замовчуванням	Опис
Max Concurrency	<code>maxConcurrency</code>	16-32	Максимум одночасних запитів на одне з'єднання (діапазон <code>мін-макс</code>)
Max Connections	<code>maxConnections</code>	6	Максимум фізичних з'єднань (<code>0</code> — без обмеження). З версії 3.4.2 нові inbound створюються зі значенням <code>6</code> ; раніше створені зберігають своє.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (порожньо)	Скільки разів перевикористовувати з'єднання
Max Request Times	<code>hMaxRequestTimes</code>	600-900	Максимум запитів на з'єднання (діапазон)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800-3000	Час, протягом якого з'єднання придатне для повторного використання (секунди, діапазон)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (порожньо)	Період keep-alive для утримання з'єднання

Важливо. Задавати одночасно **Max Connections** і **Max Concurrency** не можна — xray-core відхилить такий конфіг. За замовчуванням під час увімкнення XMUX панель пропонує `Max Concurrency = 16-32` ; якщо ви задасте **Max Connections** (значення більше `0`), панель прибере значення `Max Concurrency` за замовчуванням, щоб уникнути конфлікту.

6.8. Транспорт Hysteria (`hysteriaSettings`)

Транспорт **Hysteria** не вибирається в списку «Транспорт»: він автоматично активується, коли inbound створюється з протоколом Hysteria, і для інших протоколів прихований (під час відходу з протоколу Hysteria мережа примусово повертається до `tcp`). Параметри:

Поле	Ключ	За замовчуванням	Опис
Версія	<code>version</code>	2 (зафіксовано, поле заблоковано)	Версія Hysteria. Підтримується лише Hysteria 2
UDP idle timeout (с)	<code>udpIdleTimeout</code>	60	Таймаут простою UDP-сесії (секунди). Допустимий діапазон — 2–600; xray-core відхиляє значення поза цим інтервалом під час запуску
Masquerade	<code>masquerade</code>	вимк. (відсутнє)	Вмикає маскування слухача під HTTP/3-сервер під час зондування

За увімкненого **Masquerade** з'являється вибір типу (`type`) та залежні від нього поля:

- **"" — default (404 page)**: віддається стандартна сторінка 404 (дод. поля не потрібні).
- **proxy (reverse proxy)**: зворотне проксіювання на зовнішній сайт.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) — цільова адреса;
 - `rewriteHost` (**Переписати Host**, за замовчуванням `false`) — підміняти заголовок `Host` ;
 - `insecure` (**Пропустити TLS verify**, за замовчуванням `false`) — не перевіряти TLS-сертифікат апстріму.
- **file (serve directory)**: роздача файлів з каталогу.
 - `dir` (**Директорія**, placeholder `/var/www/html`).
- **string (fixed body)**: фіксована HTTP-відповідь.
 - `statusCode` (**Код статусу**, за замовчуванням `0`, діапазон 0–599);
 - `content` (**Body**) — тіло відповіді;
 - `headers` (**Заголовки**) — карта ім'я → значення .

Masquerade дозволяє inbound на основі Hysteria виглядати як звичайний HTTP/3-сервер для активних проб, що підвищує скритність. За замовчуванням маскування вимкнене.

Приклад `hysteriaSettings` зі зворотним проксіюванням (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Тут під час зондування слухач віддає відповідь від `https://www.example.com` , маскуючись під звичайний HTTP/3-сайт.

6.9. Супутні параметри

Окрім вибору мережі, на тій самій вкладці доступні два загальні блоки, що не залежать від конкретного транспорту (детально — у відповідних розділах):

- **External Proxy** (`externalProxy`) — список зовнішніх адрес/портів, які підставляються в посилання-підписки замість адреси самої панелі.
- **Sockopt** (`sockopt`) — низькорівневі опції сокета (TCP Fast Open, mark, домен-стратегія, прозоре проксіювання тощо).

Real client IP (визначення реального IP за CDN/релеєм)

Коли inbound стоїть за посередником (CDN на кшталт Cloudflare, L4-тунель/релей або інша панель), Xray бачить адресу посередника, а не реального відвідувача. Ця адреса потрапляє в список онлайн-клієнтів і за нею рахується ліміт IP на клієнта, через що обидва стають марними за проксі. Щоб відновити реальний IP, у секції **Sockopt** форми inbound є вибір пресету **Real client IP**, що об'єднує налаштування `acceptProxyProtocol` і `trustedXForwardedFor`:

Пресет	Що робить	Коли застосовувати
Off / direct	Очищає обидва поля.	Inbound доступний клієнтам напряму
Cloudflare CDN	Встановлює <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC за CDN Cloudflare (помаранчева хмара)
L4 relay / Spectrum (PROXY)	Вмикає <code>acceptProxyProtocol = true</code> .	L4-тунель/релей перед inbound або Cloudflare Spectrum

Пресети взаємовиключні: вибір одного очищає поле іншого, тому застарілий `trustedXForwardedFor` не перебиває відновлений за PROXY-протоколом IP. Нижче пресету залишаються видні «сирі» перемикач **Proxy Protocol** і список **Trusted X-Forwarded-For** — пресет лише заповнює їх за вас, а за потреби їх правлять вручну. Якщо обраний пресет не підтримується поточним транспортом (наприклад, PROXY-протокол на mKCP), форма показує попередження. Ці поля стосуються лише боку сервера і **ніколи не відправляються клієнтам у підписах**.

Використовуйте щось одне. `acceptProxyProtocol` читає реальний IP з L4-заголовка PROXY-протоколу, а `trustedXForwardedFor` — з HTTP-заголовка запиту; змішувати їх вручну варто, лише якщо цього вимагає ваш ланцюжок посередників.

- **FinalMask** (`finalmask`) — загальний механізм обфускації транспортного рівня (включно з legacy-обфускацією mKCP), який замінив собою окремі поля «seed»/«header type» усередині під-форм мереж.

7. Безпека з'єднання: TLS, XTLS та REALITY

Кожен inbound, що підтримує передачу через transport-потік (VMess, VLESS, Trojan, Shadowsocks, Hysteria), має вкладку «Безпека» у редакторі. На ній налаштовується, як саме шифрується та маскується транспортний канал. Доступні три режими, що перемикаються кнопками-радіо:

Режим	Підпис у UI	Коли доступний
none	Немає	Завжди (крім Hysteria, де TLS є обов'язковим)
tls	TLS	Для VMess/VLESS/Trojan/Shadowsocks на мережах tcp, ws, http, grpc, httpupgrade, xhttp; для Hysteria — завжди
reality	Reality	Лише для VLESS/Trojan на мережах tcp, http, grpc, xhttp

Кнопка **Немає** не відображається, якщо протокол — Hysteria (для нього TLS є обов'язковим). Кнопка **Reality** з'являється лише при допустимій комбінації протоколу та мережі (див. таблицю вище).

При зміні режиму панель повністю перебудовує блок `streamSettings`: видаляються `tlsSettings` і `realitySettings` від попереднього режиму та підставляються значення за замовчуванням для обраного. Зокрема, при виборі **Reality** панель одразу автоматично: підставляє випадкову пару `target` + `serverNames` (SNI) зі вбудованого списку популярних доменів, генерує випадкові `shortIds`, а також надсилає запит до сервера за свіжою парою ключів X25519 (`privateKey/publicKey`).

7.1. У чому різниця: TLS vs XTLS vs REALITY

- **TLS** — класичне шифрування транспорту за протоколом TLS 1.2/1.3. На сервері має лежати дійсний сертифікат (власний домен + ланцюжок). Трафік виглядає як звичайний HTTPS, але для активного цензора характерне розпізнаване TLS-рукописання до вашого домену; при «відстрілі» за SNI або за відсутності довіреного сертифіката з'єднання блокується/видає помилку.
- **XTLS (Vision)** — це не окремий режим у списку «Безпека», а *flow*-механізм поверх TLS або Reality. Вмикається на стороні клієнта inbound через поле **Flow** = `xtls-rprx-vision` (або `xtls-rprx-vision-udp443`). Vision доступний для VLESS на мережі `tcp` при `security = tls` або `security = reality`, а також для VLESS поверх транспорту `xhttp` при увімкненому VLESS-шифруванні (`vlessenc` / ML-KEM) — у цьому випадку поле **Flow** також можна встановити в `xtls-rprx-vision`, і значення коректно потрапляє до посилання `vless://` (`flow=xtls-rprx-vision`). Vision знижує «подвійне шифрування» (TLS-in-TLS), передаючи корисне навантаження напряму після рукописання, що прискорює передачу і покращує маскування. Тому зв'язка **VLESS + Reality + Flow xtls-rprx-vision** вважається рекомендованою сучасною конфігурацією.

Автоматичне відновлення flow Vision. Якщо у VLESS/XHTTP-inbound шифрування (ML-KEM, поля decryption/encryption) вмикається вже після того, як були додані клієнти, inbound стає flow-придатним. У цій ситуації панель сама відновлює `flow = xtls-rprx-vision` у тих клієнтів, яким він належить, але в кого поле **Flow** виявилося порожнім. Раніше при такому сценарії Vision мовчки зникав із конфігів, share-посилань і підписок (особливо помітно на вузлових inbound). Жодних ручних дій не потрібно: виправлення застосовується автоматично при збереженні inbound і одноразово при

оновленні панелі. Поведінка консервативна — панель не вигадує flow і не перезаписує явно задане клієнтом значення.

- **REALITY** — механізм маскуванню без власного сертифіката. Сервер «позичає» TLS-рукописання реального стороннього сайту (`target` / `serverNames`), тому для спостерігача з'єднання невідмінне від звернення до цього сайту, а сертифікат не потрібен взагалі. Автентифікація будується на парі ключів X25519 і наборі `shortIds`. REALITY стійкий до активних зондів (`active probing`) і блокування за SNI, оскільки SNI вказує на справжній зовнішній домен. Ціна — суворіші вимоги до налаштування (коректний `target` з портом, синхронізація ключів із клієнтом).

Коротке правило вибору:

- є власний домен і валідний сертифікат, потрібен простий HTTPS-вигляд → **TLS** (за можливості з Vision);
- немає домену/сертифіката або потрібна максимальна прихованість від DPI → **REALITY** (з Vision для VLESS/TCP).

7.2. Режим «Немає» (`none`)

Транспорт передається без TLS-обгортки: блоки `tlsSettings` і `realitySettings` з `streamSettings` виключаються. Додаткових полів у режиму немає. Підходить, коли:

- `inbound` слухає лише на `127.0.0.1` і служить fallback-ціллю (за правилом панелі дочірній `inbound` для fallback має слухати на `127.0.0.1` із `security=none`);
- шифрування/маскування забезпечує зовнішній шар (наприклад, зворотний проксі Nginx перед панеллю);
- використовується протокол із власним шифруванням (Shadowsocks) у внутрішній мережі.

Для `inbound`, доступних ззовні, режим «Немає» не рекомендується.

Приклад: блок `streamSettings` для TLS на мережі `tcp` (VLESS/Trojan/VMess). Так виглядає результат після вибору режиму **TLS** і заповнення SNI та шляхів до сертифіката:

```

"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}

```

7.3. Режим TLS

Поля блоку `tlsSettings`. Значення за замовчуванням взяті зі схеми панелі.

Основні параметри

Поле (підпис)	Значення за замовчуванням	Опис
SNI (<code>serverName</code>)	<code>""</code> (порожньо)	Server Name Indication — ім'я домену, що пред'являється під час TLS-рукоштовування. Має збігатися з доменом сертифіката. Англ. підказка-плейсхолдер: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Авто	Список дозволених наборів шифрів. За замовчуванням порожньо — вибір надається на розсуд Xray/Go (варіант Авто). Змінювати лише за необхідності явно обмежити шифри.
Мін/Макс версія (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Мінімальна та максимальна версії TLS. Доступні значення: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Рекомендується залишити <code>1.2</code> – <code>1.3</code> ; знижувати мінімум до 1.0/1.1 небажано (застарілі, небезпечні версії).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (у формі — пункт None = <code>""</code> доступний)	Відбиток TLS клієнтського hello, що імітується (uTLS fingerprint), щоб рукоштовування виглядало як у популярного браузера. Див. список нижче. У TLS перший пункт списку — None (<code>""</code>), що викликає імітацію.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Список протоколів прикладного рівня, що узгоджуються в TLS (множинний вибір). Допустимі значення: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . За замовчуванням пропонуються <code>h2</code> і <code>http/1.1</code> .

Можливі значення **uTLS fingerprint** (однакові для TLS і REALITY): `chrome`, `firefox`, `safari`, `ios`, `android`, `edge`, `360`, `qq`, `random`, `randomized`, `randomizednoalpn`, `unsafe`. У TLS-формі додатково доступний порожній варіант **None** (імітація відбитка не застосовується).

Доступні значення **Cipher Suites** (TLS 1.3 і ECDHE-набори): `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`.

Перемикачі TLS

Перемикач	За замовчуванням	Опис
Відхилити невідомий SNI (<code>rejectUnknownSni</code>)	вимк. (<code>false</code>)	Якщо увімкнено, сервер розриває з'єднання, коли пред'явлений клієнтом SNI не збігається з іменем у сертифікаті. Підвищує прихованість (сервер не відповідає на «чужі» запити), але потребує точного збігу SNI у клієнта.
Вимкнути System Root (<code>disableSystemRoot</code>)	вимк. (<code>false</code>)	Вимикає використання системного сховища довірених кореневих сертифікатів.
Відновлення сесії (<code>enableSessionResumption</code>)	вимк. (<code>false</code>)	Вмикає відновлення TLS-сесії (session resumption / session tickets).

Додаткові параметри TLS (vsn, криві, лог ключів, ECH Sockopt)

Під основними налаштуваннями TLS доступні додаткові поля.

Поле (підпис)	За замовчуванням	Опис
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Імена (через кому), за якими клієнт перевіряє сертифікат сервера замість SNI. Це сучасна заміна видаленого з Xray після 2026-06-01 поля <code>allowInsecure</code> . Значення лише для панелі: на сервер у конфіг xray не записується, але передається в посилання-запрошення і підписки (<code>vsn=...</code>), щоб клієнт застосував його сам. Плейсхолдер: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Обмеження та порядок кривих обміну ключами TLS, у порядку переваги (наприклад <code>X25519MLKEM768</code> , <code>X25519</code>). Порожньо – беруться значення за замовчуванням Xray-core.

Поле (підпис)	За замовчуванням	Опис
Master Key Log (masterKeyLog)	""	Шлях для запису TLS master keys у форматі <code>SSLKEYLOGFILE</code> (для розшифровки трафіку у Wireshark при налагодженні). Плейсхолдер: <code>/path/to/sslkeylog.txt</code> . У продакшені залишати порожнім — файл дозволяє розшифрувати весь трафік.
ECH Sockopt (echSockopt)	вимк.	Перемикач із параметрами сокета для з'єднання, через яке Xray запитує ECH config list. При увімкненні доступні: Dialer Proxy (dialerProxy — пропустити запит через зазначений outbound за тегом), Domain Strategy (domainStrategy), TCP Fast Open (tcpFastOpen), Multipath TCP (tcpMptcp). Без необхідності залишати вимкненим.

Поля `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog` і `echSockopt` розташовуються на верхньому рівні `tlsSettings` і зберігаються при «обрізці» панельних полів під час збереження конфігурації.

Сертифікати

Розділ **SSL-сертифікат** (у UI заголовок «SSL-сертифікат») задається списком: кнопкою **+** додається новий запис сертифіката, кнопкою **– Видалити** — прибирається (кнопка видалення доступна лише якщо записів більше одного). За замовчуванням при увімкненні TLS створюється один порожній запис.

Для кожного запису перемикач режиму введення (`useFile`):

- **Шлях до сертифіката** (значення `useFile = true`, за замовчуванням) — вказуються шляхи до файлів на сервері:
 - **Публічний ключ** (`certificateFile`) — шлях до файлу сертифіката (`.crt` / `.pem`);
 - **Приватний ключ** (`keyFile`) — шлях до файлу приватного ключа (`.key`).
- **Вміст сертифіката** (значення `useFile = false`) — вміст вставляється безпосередньо в поля (багаторядкові текстові області):
 - **Публічний ключ** (`certificate`) — PEM-вміст сертифіката;
 - **Приватний ключ** (`key`) — PEM-вміст ключа.

Під полями режиму «Шлях до сертифіката» доступні дві кнопки:

- **Встановити сертифікат панелі** — підставляє в поля шляхи до власного SSL-сертифіката панелі. Для inbound на центральній панелі береться її сертифікат (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`); для inbound, призначеного на ноду, — сертифікат самої ноди (`GET /panel/api/nodes/webCert/{nodeId}`), оскільки шляхи центральної панелі на ноді не існують. Якщо сертифікат не налаштований, виводиться попередження: «Для панелі не налаштовано сертифікат. Спочатку встановіть його у Налаштуваннях.» (сам сертифікат панелі задається у розділі «Налаштування → Безпека»).
- **Очистити** — стирає обидва шляхи.

Додаткові поля кожного запису сертифіката:

Поле	За замовчуванням	Опис
OCSF Stapling (ocsfStapling)	0 (вимк.)	Інтервал оновлення OCSF-стейплінгу в секундах (мінімум 0). Для нових inbound за замовчуванням вимкений (0): це прибирає помилки в логах хгау для сертифікатів без OCSF-респондера (наприклад, Let's Encrypt, що відмовився від OCSF). Вмикайте лише для сертифікатів, які підтримують stapling.
Одноразове завантаження (oneTimeLoading)	вимк. (false)	Якщо увімкнено, сертифікат читається з диска один раз при старті і не перерисовується при зміні файлу.
Опція використання (usage)	encipherment	Призначення сертифіката. Допустимо: encipherment (шифрування – звичайний серверний сертифікат), verify (перевірка), issue (випуск – сервер сам підписує/випускає сертифікати).
Build Chain (buildChain)	вимк. (false)	Відображається лише при usage = issue . Добудовувати ланцюжок сертифікатів.

Самопідписаного сертифіката як окремої кнопки в редакторі inbound немає: панель не генерує самопідписаний сертифікат на льоту для inbound. Сертифікат або вказується шляхом/вмістом, або підтягується з налаштувань панелі кнопкою «Встановити сертифікат панелі». Випуск/отримання SSL-сертифіката самої панелі (включно із завантаженням файлів і прив'язкою до домену) виконується у розділі **Налаштування** → **Безпека**; ACME/Let's Encrypt-ендпоінтів для inbound тут немає.

ECH та закріплення сертифіката (розширені поля TLS)

Поле	За замовчуванням	Опис
ECH key (echServerKeys)	""	Серверні ключі Encrypted Client Hello.
ECH config (settings.echConfigList)	""	ECH config list (клієнтська частина, потрапляє до посилання).
SHA-256 сертифіката піра (settings.pinnedPeerCertSha256)	[]	SHA-256-хеші сертифіката піра (hex-рядки, через кому). Дослівна підказка: «SHA-256-хеші сертифіката піра у вигляді шістнадцяткового рядка (напр. e8e2d3...), через кому. Лише для панелі – не записується в конфіг хгау сервера, але включається в посилання-запрошення, щоб клієнти могли закріпити сертифікат.»

Кнопки: Поряд із полем **SHA-256 сертифіката піра** є дві кнопки автозаповнення:

- **Fill from this inbound's certificate** (іконка щита) – підставляє SHA-256-хеш сертифіката самого цього inbound (береться локально через ендпоінт getCertHash).
- **Fetch the hash by pinging the SNI (xray tls ping)** (іконка завантаження) – отримує хеш живого сертифіката сервера, виконавши TLS-підключення за зазначеним SNI (на сервері викликається getRemoteCertHash). Поле **SNI** (serverName) має бути заповнене – інакше виводиться підказка «Set the SNI (serverName) first to ping the remote certificate.»

Отримані хеші додаються до поля (через кому) і потрапляють до посилань-запрошень, щоб клієнт міг закріпити сертифікат.

- **Отримати новий ECH-сертифікат** – запитує у сервера нову ECH-пару під поточний SNI (`POST /panel/api/server/getNewEchCert` , на сервері виконується `xray tls ech --serverName <SNI>`); заповнює поля **ECH key** і **ECH config**.
- **Очистити** – обнуляє обидва ECH-поля.

7.4. Режим REALITY

Поля блоку `realitySettings` . REALITY не використовує SSL-сертифікат: замість нього – позичене TLS-рукописання зовнішнього домену та пара ключів X25519.

Параметри маскування

Поле (підпис)	Значення за замовчуванням	Опис
Показати (<code>show</code>)	вимк. (<code>false</code>)	Відладковий вивід REALITY в логи Xray. Зазвичай залишають вимкненим.
Xver (<code>xver</code>)	0	Версія PROXY-протоколу, що передається на бекенд (0 – вимкнено). Мінімум 0 .
uTLS (<code>settings.fingerprint</code>)	chrome	Відбиток TLS, що імітується (той самий список, що в TLS, але без порожнього варіанту None).
Ціль (<code>target</code>)	"" (з 3.4.2 при увімкненні REALITY залишається порожнім)	Обов'язкове поле. Реальний домен, чиє TLS-рукописання позичає REALITY. Дослівна підказка: «Обов'язково. Має містити порт (наприклад, <code>example.com:443</code>). Без порту Xray-core не запускається.» Валідація в панелі перевіряє наявність і коректність порту; інакше виводяться помилки «Ціль REALITY обов'язкова» / «Ціль REALITY має містити порт...» / «У цілі REALITY вказано неприпустимий порт». Поруч із полем – кнопки «Сканувати» (перевірити поточну ціль «наживо») і «Знайти цілі» (відкрити сканер цілей REALITY); див. нижче.
SNI (<code>serverNames</code>)	[] (підставляється разом із ціллю)	Список допустимих SNI (множинне введення тегами). Має відповідати домену з Ціль . При успішному скануванні цілі SNI заповнюється за її сертифікатом.
Макс. різниця у часі (мс) (<code>maxTimediff</code>)	0	Максимально допустиме розходження годин клієнта і сервера в мілісекундах (0 – без обмеження). Мінімум 0 .
Мін. версія клієнта (<code>minClientVer</code>)	""	Мінімальна версія Xray-клієнта (плейсхолдер з 3.5.0 – 26.3.27). Порожньо – без обмеження.
Макс. версія клієнта (<code>maxClientVer</code>)	""	Максимальна версія Xray-клієнта. Порожньо – без обмеження.
Short IDs (<code>shortIds</code>)	[] (генеруються при увімкненні)	Список коротких ідентифікаторів (hex), що розрізняють клієнтів. Множинне введення тегами; кнопка-оновлення генерує випадковий набір.

Поле (підпис)	Значення за замовчуванням	Опис
SpiderX (settings.spiderX)	/	Шлях «павука» (клієнтська частина REALITY), що використовується при імітації звернення до зовнішнього сайту. Потрапляє до посилання-запрошення.

З версії 3.4.2 **Ціль** (target) і **SNi** (serverNames) при увімкненні REALITY більше **не** заповнюються автоматично – обидва поля залишаються порожніми, а ціль підбирається живим сканером (див. нижче). Обирайте «важкий», стабільний сторонній HTTPS-сайт з підтримкою TLS 1.3 і HTTP/2, що не знаходиться за вашим же сервером.

Пошук цілі REALITY (живий сканер)

З версії 3.4.2 попередня кнопка «випадкова ціль» (зі статичним вбудованим списком) замінена двома діями поруч із полем **Ціль**:

- **«Сканувати»** – «наживо» перевіряє поточну ціль: встановлює TLS-з'єднання і, якщо ціль придатна, підставляє SNi з її сертифіката. Тости: «Ціль придатна – заповнено ціль і SNi.» / «Ціль доступна, але не придатна для REALITY.» / «Не вдалося просканувати ціль REALITY.».
- **«Знайти цілі»** – відкриває вікно **«Сканер цілей REALITY»**: можна вказати домен, **IP** або **діапазон CIDR** (тоді панель знаходить цілі за сертифікатами живих хостів) або залишити поле порожнім (перевіряються вбудовані кандидати). Результати ранжуються; стовпці – «Статус»/«Придатно», «Обмін ключами», «Сертифікат», TLS , ALPN , «Затримка». Кнопкою **«Використати»** вибраний рядок підставляється в поля inbound.

Ціль вважається **придатною**, якщо сервер узгоджує **TLS 1.3, HTTP/2 (ALPN h2)**, обмін ключами **X25519/X25519MLKEM768** і пред'являє **довірений** (не wildcard) сертифікат. Сканування виконується на стороні сервера панелі (POST /panel/api/server/scanRealityTarget і .../scanRealityTargets).

Приклад: блок streamSettings для REALITY на мережі tcp (VLESS). Сертифікат не потрібен – замість нього позичений домен та пара ключів X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Тут поле **Ціль** (`target`) панелі відповідає `dest` у готовому конфізі Xray. Якщо REALITY-inbound був створений із `destination` у ключі `dest` (старими версіями панелі, через API або зовнішніми інструментами), панель при розборі нормалізує `dest` → `target`, коли `target` порожній, — тому такий inbound коректно завантажується, поле **Ціль** не виявляється порожнім, і повторне збереження не ламає робочий REALITY.

Ключі REALITY (X25519)

Поле	За замовчуванням	Опис
Публічний ключ (<code>settings.publicKey</code>)	""	Публічний ключ X25519 (його клієнт кладе у свою конфігурацію/посилання).
Приватний ключ (<code>privateKey</code>)	""	Приватний ключ X25519 (зберігається лише на сервері).

Кнопки під ключами:

- **Отримати новий сертифікат** — запитує у сервера нову пару ключів (`GET /panel/api/server/getNewX25519Cert` ; на сервері виконується `xray x25519`), заповнює **Приватний ключ** і **Публічний ключ**. Ця сама пара автоматично генерується при першому увімкненні режиму REALITY.

Приклад: отримати пару ключів X25519 через API (поза формою, наприклад для скрипту). Запит повертає приватний і публічний ключі:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Відповідь:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

`cookie.txt` — файл cookie сесії, отриманий після входу через `POST /login`.

- **Очистити** — обнуляє обидва ключі.

Пост-квантовий підпис ML-DSA-65 (mldsa65)

Додатковий (необов'язковий) шар пост-квантової автентифікації REALITY:

Поле	За замовчуванням	Опис
mldsa65 Seed (<code>mldsa65Seed</code>)	""	Серверний seed ключа ML-DSA-65.
mldsa65 Verify (<code>settings.mldsa65Verify</code>)	""	Перевірочне значення (клієнтська частина, потрапляє до посилання).

Кнопки:

- **Отримати новий Seed** — запитує нову пару (`GET /panel/api/server/getNewmldsa65` ; на сервері виконується `xray mldsa65`), заповнює **mldsa65 Seed** і **mldsa65 Verify**.
- **Очистити** — обнуляє обидва поля.

Обмеження швидкості fallback та лог ключів REALITY

У налаштуваннях REALITY доступне обмеження швидкості fallback-трафіку — воно не дає активним зондам використовувати сервер як безкоштовний канал до позиченого домену. Налаштування задається окремо для двох напрямків — **Limit Fallback Upload** і **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), кожен з однаковим набором полів:

Поле (підпис)	За замовчуванням	Опис
After Bytes (<code>afterBytes</code>)	0	Скільки байт пропустити на повній швидкості перед початком обмеження. 0 — обмежувати з першого байта.
Bytes Per Sec (<code>bytesPerSec</code>)	0	Стеля швидкості fallback-трафіку в байтах на секунду після порогу. 0 — без обмеження (вимикає цей напрямок).
Burst Bytes Per Sec (<code>burstBytesPerSec</code>)	0	Запас для короточасних сплесків понад постійну швидкість (розмір token-bucket). Якщо менше Bytes Per Sec , підіймається до його значення.

Там же додано поле **Master Key Log** (`masterKeyLog`) — шлях для запису TLS master keys у форматі `SSLKEYLOGFILE` для налагодження у Wireshark; у продакшені залишати порожнім.

7.5. Практичні рекомендації щодо налаштування

1. **VLESS + Reality (рекомендується)**: створіть VLESS-inbound на мережі `tcp`, на вкладці «Безпека» виберіть **Reality** — панель сама підставить випадкові `target/SNI`, `shortIds` і згенерує ключі X25519. За необхідності натисніть «Отримати новий сертифікат» для своєї пари ключів. Для клієнтів VLESS увімкніть **Flow** = `xtls-rprx-vision` (XTLS Vision) — це дасть максимальну продуктивність і прихованість.

Приклад: підсумкове клієнтське посилання VLESS + Reality + Vision. Так виглядає посилання-запрошення, яке панель видає для такого inbound (значення ключів/ID — ілюстративні):

```
vless://uuid-клієнта@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛІЧНИЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Тут `pbk` — публічний ключ X25519, `sni` — позичений домен із **Ціль**, `sid` — один із **Short IDs**, `flow=xtls-rprx-vision` — увімкнений XTLS Vision. 2. **TLS із власним доменом**: виберіть **TLS**, заповніть **SNI** ім'ям домену, додайте сертифікат (шляхом до файлів або вмістом), або натисніть «Встановити сертифікат панелі», якщо домен і сертифікат вже налаштовані в «Налаштування → Безпека». Залиште **Мін/Макс версія** = `1.2 - 1.3` і **uTLS** = `chrome` для маскуванню під звичайний браузер. 3. Не залишайте режим **Немає** для inbound, відкритих назовні, — використовуйте його лише для локальних fallback-цілей (`127.0.0.1`) або коли TLS забезпечує зовнішній проксі. 4. Порада з інтерфейсу: для більшості розширених полів діє підказка «Рекомендується залишити налаштування за замовчуванням» — змінюйте їх лише розуміючи наслідки.

8. Клієнти

Клієнт — це обліковий запис користувача VPN: набір облікових даних (UUID або пароль), прив'язаний до одного або кількох inbound, з власною квотою трафіку, терміном дії та лімітом одночасних підключень. У цьому форку клієнт є самостійною сутністю (таблиця `clients`): один і той самий клієнт може бути прив'язаний до кількох inbound одночасно, зберігаючи спільний UUID/пароль і спільний лічильник трафіку. Розділ **Клієнти** показує всі облікові записи панелі незалежно від inbound, з пошуком, фільтрами, сортуванням і масовими операціями.

8.1. Поля клієнта

Нижче розібрано кожне поле редактора **Додати клієнта** / **Змінити клієнта**.

Форма клієнта розбита на дві вкладки: **Основні** (email, прив'язка до inbound, ліміти, термін, група, коментар, зворотний тег) і **Облікові дані** (UUID/пароль/auth, Flow, VMess Security). У підписах полів квота вказана як **Ліміт трафіку (ГБ)**, а терміни — як **Тривалість (днів)** і **Автопродовження (днів)**; у полів **Ліміт трафіку (ГБ)** і **Ліміт IP** є підказки, що пояснюють, що `0` означає «без обмежень». При редагуванні вже існуючого клієнта кнопка генерації випадкового email прихована, а кнопка журналу IP винесена безпосередньо до поля **Ліміт IP** і показує кількість зафіксованих адрес.

Поле	Ключ JSON	За замовчуванням	Опис
Email	<code>email</code>	— (обов'язково)	Унікальний ідентифікатор клієнта
UUID	<code>id</code>	генерується	Ідентифікатор для VMess/VLESS
Пароль	<code>password</code>	генерується	Пароль для Trojan/Shadowsocks
Авторизація	<code>auth</code>	генерується	Пароль для Hysteria
Flow	<code>flow</code>	пусто	Flow control (XTLS), тільки VLESS
VMess Security	<code>security</code>	<code>auto</code>	Метод шифрування VMess
Ліміт IP	<code>limitIp</code>	<code>0</code> (без ліміту)	Максимум одночасних IP
Всього відправлено/отримано (ГБ)	<code>totalGB</code>	<code>0</code> (без ліміту)	Квота трафіку
Термін дії	<code>expiryTime</code>	<code>0</code> (безстроково)	Дата закінчення
Автопродовження	<code>reset</code>	<code>0</code> (вимк.)	Період скидання трафіку, дні
ID користувача Telegram	<code>tgId</code>	<code>0</code> (немає)	Числовий Telegram ID
ID підписки	<code>subId</code>	генерується	Ідентифікатор підписки
Група	<code>group</code>	пусто	Логічна мітка групування
Коментар	<code>comment</code>	пусто	Довільна нотатка
Увімкнено	<code>enable</code>	<code>true</code>	Чи активний обліковий запис

Email (ідентифікатор)

Поле **Email** — головний і обов'язковий ідентифікатор клієнта. Попри назву, це не обов'язково поштова адреса: підійде будь-яка текстова мітка (ім'я користувача, номер). Значення має бути **унікальним** в межах панелі; спроба створити другого клієнта із зайнятим email відхиляється (`email already in use`), за винятком випадку, коли збігається й `subId` (це трактується як прив'язка того самого клієнта).

Email **не можна залишати порожнім** (`client email is required`) і він **не може містити пробіли, символи /, \ і керуючі символи** («Email не може містити пробіли, /, \ або керуючі символи»). Email бере участь в обліку трафіку, в журналі IP, в онлайн-списку та в іменах операцій — змінювати його заднім числом не рекомендується.

UUID / Пароль / Авторизація (облікові дані)

Конкретне поле облікових даних залежить від протоколу inbound, до якого прив'язується клієнт. Значення підставляються автоматично, якщо залишити поле порожнім:

- **UUID** (поле `id`) — для протоколів **VMess** і **VLESS**. Якщо не задано, генерується випадковий UUID v4.
- **Пароль** (поле `password`) — для **Trojan** і **Shadowsocks**. Для Trojan за замовчуванням генерується UUID без дефісів. Для Shadowsocks генерується ключ потрібної довжини в Base64 залежно від методу шифрування inbound: 16 байт для `2022-blake3-aes-128-gcm`, 32 байти для `2022-blake3-aes-256-gcm` і `2022-blake3-chacha20-poly1305`; для інших методів — UUID без дефісів. Якщо введений вручну ключ не підходить під метод 2022-blake3, він буде замінений згенерованим.
- **Авторизація** (поле `auth`) — пароль для **Hysteria**. За замовчуванням UUID без дефісів.

Оскільки один клієнт може бути прив'язаний до inbound різних протоколів, у запису клієнта можуть одночасно бути присутні й UUID, і пароль, і auth — для кожного протоколу використовується своє поле.

Приклад: як облікові дані клієнта виглядають у settings різних inbound. Один і той самий клієнт у VLESS-inbound ідентифікується за `id`, у Trojan — за `password`, у Shadowsocks — за `password` (ключ Base64):

```
// фрагмент settings.clients у VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// той же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// той же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmbL0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (поле `flow`) — керування потоком XTLS. Застосовне **тільки до VLESS** і тільки коли inbound сконфігурований для XTLS Vision: VLESS поверх транспорту **TCP** із security **tls** або **reality**. Допустиме значення — `xtls-rprx-vision` (а також історичний `xtls-rprx-vision-udp443`); порожнє значення означає відсутність flow.

Якщо inbound не підтримує XTLS-flow, заданий flow **мовчки обнуляється** при збереженні клієнта: для одного й того самого клієнта, прив'язаного до кількох inbound, flow застосовується тільки там, де це допустимо. Змінювати варто лише якщо ви цілеспрямовано використовуєте VLESS-Vision.

VMess Security

VMess Security (поле `security`) — метод шифрування корисного навантаження для VMess. Значення за замовчуванням — `auto` (Xray сам обирає шифр). Допустимі значення — стандартні для VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Для інших протоколів поле не використовується.

Ліміт IP (одночасні підключення)

Ліміт IP (поле `limitIp`) — максимальна кількість **різних IP-адрес**, з яких клієнт може бути підключений одночасно. Значення за замовчуванням — `0`, що означає **відсутність обмеження**. При позитивному значенні панель відстежує активні IP клієнта і, при перевищенні ліміту, вимикає обліковий запис фоновим завданням. (Починаючи з **3.3.1** підрахунок IP йде через online-stats API ядра Xray і **не потребує** журналу доступу; на старих версіях ядра панель повертається до читання журналу доступу, який тоді має бути увімкнений.) Використовуйте, щоб заборонити роздачу однієї підписки на багато пристроїв: наприклад, `2` — дозволити два пристрої.

Ліміт IP застосовується засобами **Fail2ban**, тому поле **Ліміт IP** активне тільки при встановленому й працездатному Fail2ban (панель перевіряє його статус через `GET /panel/api/server/fail2banStatus`). Якщо Fail2ban не встановлено, поле редактора клієнта (і форми масового додавання) блокується, а при наведенні показується підказка з пропозицією встановити Fail2ban із bash-меню `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); на Windows підказка повідомляє, що Fail2ban там недоступний («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), а якщо функцію вимкнено на сервері — «The IP limit feature is disabled on this server.». При оновленні панелі у клієнтів на серверах без Fail2ban збережений ліміт IP обнуляється одноразовою міграцією, оскільки він там усе одно не застосовувався.

Приклад значень. `limitIp: 0` — без обмеження; `limitIp: 1` — строго один пристрій одночасно; `limitIp: 3` — до трьох різних IP. При четвертому активному IP фонове завдання вимкне клієнта (`enable = false`), поки ви не виконаєте **Скинути ліміт IP**.

Пов'язані операції: **Журнал IP** показує список зафіксованих IP клієнта; кожен запис містить, крім самого IP, час останнього звернення та мітку ноди (`@ім'я_ноди`), через яку зафіксовано підключення, — у мультипанельній конфігурації видно, через який вузол підключався клієнт. **Скинути ліміт IP** очищає накопичений журнал IP, щоб клієнт знову зміг підключитися, не чекаючи природного закінчення записів.

Всього відправлено/отримано (ГБ) — квота трафіку

Всього відправлено/отримано (ГБ) (поле `totalGB`) — сумарна квота трафіку (відправка + приймання). Значення за замовчуванням — `0` — означає **безліміт**. При досягненні квоти (`up + down >= total`) клієнт вважається **вичерпаним** (depleted) і відключається. В UI зазвичай вводиться в гігабайтах; у базі зберігається в байтах.

У списку клієнтів стовпець **Трафік** показує кольорову смугу використання: обсяг витраченого трафіку, мітку ліміту (або знак ∞ при безліміті) і підказку при наведенні з розбивкою на відправлено/отримано та залишок. Той самий компактний індикатор відображається в картках клієнтів на телефоні.

Термін дії (Expiry)

Термін дії (поле `expiryTime`) задає момент закінчення облікового запису. Поле має три режими:

- **Безстроково** — `0`. Клієнт ніколи не закінчується за часом.
- **Конкретна дата** — позитивний Unix-timestamp (у мілісекундах). При настанні (`expiryTime <= зараз`) клієнт вважається таким, що закінчився (expired), і відключається. В UI зазвичай задається вибором дати або тривалістю в днях (**Тривалість**, одиниця — **Дні**).
- **Старт після першого використання** — **від'ємне** значення, що кодує тривалість. Поки клієнт не передав жодного байта, термін залишається від'ємним («відкладений старт»). На першому ж тіку обліку трафіку панель конвертує його в абсолютну дату: `зараз + |тривалість|`. Це дозволяє продати, наприклад, «30 днів з моменту першого підключення», не знаючи наперед, коли клієнт активується. Конвертація виконується один раз на email, щоб усі прив'язані inbound отримали один і той самий термін.

Приклад кодування терміну. Фіксована дата 1 березня 2026, 00:00 UTC → `expiryTime: 1772323200000` (позитивний timestamp у мілісекундах). «30 днів з першого підключення» → `expiryTime: -2592000000` (від'ємне значення, $30 \times 24 \times 60 \times 60 \times 1000$); при першому байті трафіку панель замінить його на `зараз + 2592000000`. Безстроково → `expiryTime: 0`.

Автопродовження (період скидання трафіку клієнта)

Поле **Автопродовження** (поле `reset`) — це період автоматичного продовження/скидання в днях. Підказка: «Автоматичне продовження після закінчення. (0 = вимкнено) (одиниця: день)».

- `0` — автопродовження **вимкнено** (значення за замовчуванням). Після закінчення терміну клієнт просто стає вичерпаним.
- `> 0` — фонове завдання при закінченні терміну **скидає лічильники трафіку в нуль** (`up = down = 0`), **зсуває термін дії вперед** на `reset` днів (якщо потрібно — на кілька періодів, поки новий термін не опиниться в майбутньому) і за необхідності знову **вмикає** клієнта. Це реалізує періодичну підписку (наприклад, щомісячну). Автопродовження **не застосовується до inbound на вузлах-нодах** (`node_id IS NOT NULL`).

Важливий наслідок: клієнти з `reset > 0` **виключаються** з поняття «вичерпаний» у операціях масового видалення — їх трафік/термін очікувано обнуляються автопродовженням, а не роблять обліковий запис кандидатом на видалення.

ID користувача Telegram

ID користувача Telegram (поле `tgId`) — числовий Telegram-ідентифікатор користувача для прив'язки до вбудованого Telegram-бота панелі (сповіщення, самостійний перегляд статистики). Підказка: «Числовий ID користувача Telegram (0 = немає)». Значення за замовчуванням `0` — прив'язка відсутня. За цим полем доступна фільтрація (**Є / Немає**).

ID підписки (subId)

ID підписки (поле `subId`) — ідентифікатор, під яким клієнт включається до **підписки** (subscription). Всі клієнти з однаковим `subId` віддаються за одним посиланням підписки. Якщо поле залишено порожнім при створенні, панель **автоматично генерує випадковий** `subId` (UUID). Значення має бути **унікальним** серед клієнтів з іншим email (`subId already in use`) і підпорядковується тим самим обмеженням символів, що й email («ID підписки не може містити пробіли, '/', '' або керуючі символи»).

Без `subId` посилання підписки для клієнта недоступне («У цього клієнта немає subId, посилання для спільного доступу недоступне.»).

Вкладка Links (зовнішні посилання і підписки)

Крім вкладок **Основні** і **Облікові дані**, в редакторі клієнта є третя вкладка **Links** (підказка: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). На ній кнопкою **Add External Link** додають сторонні share-посилання (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), а кнопкою **Add External Subscription** — URL віддалених підписок (наприклад, `https://provider.example/sub/...`).

Усе перелічене підмішується до видачі підписки даного клієнта (формати raw, JSON і Clash): посилання додаються як є, а віддалені підписки панель періодично завантажує (з кешем і коротким таймаутом) і об'єднує їх конфігурації з власними. Так в одному посиланні підписки клієнта можна віддавати разом зі своїми серверами ще й зовнішні конфігурації.

Група

Група (поле `group`) — логічна мітка для об'єднання пов'язаних клієнтів. Підказка: «Логічна мітка для групування пов'язаних клієнтів (наприклад, команда, клієнт, регіон). Фільтрується з панелі інструментів.», плейсхолдер — «наприклад, customer-a». Поле необов'язкове (за замовчуванням порожнє). За групою можна фільтрувати список і виконувати масові операції; для очищення мітки у клієнта використовується дія **Розгрупувати**.

Зняти групу можна й безпосередньо в редакторі одного клієнта: якщо очистити поле **Група** і зберегти, мітка коректно знімається, і клієнт перестає відображатися під попередньою групою.

Коментар

Коментар (поле `comment`) — довільна текстова нотатка для адміністратора (за замовчуванням порожнє). Вміст потрапляє в пошук і доступний для фільтрації (**Є** / **Немає** коментар).

Увімкнено

Увімкнено (поле `enable`) — прапор активності облікового запису. За замовчуванням **увімкнено** (`true`); при створенні, навіть якщо прапор не переданий, панель примусово ставить `true`. Вимкнений клієнт (`enable = false`) не може підключитися і в зведенні відноситься до категорії **неактивних** (deactive). Панель сама вимикає клієнтів, що вичерпали квоту, закінчилися або перевищили ліміт IP.

Поля тільки для читання

У картці клієнта також відображаються службові поля: **Створено** (`created_at`) і **Оновлено** (`updated_at`) — часові мітки створення і останньої зміни, заповнюються автоматично і не редагуються. Поле **Зворотний тег** (`reverse`) — необов'язковий Reverse tag для простого зворотного проксі VLESS («Необов'язковий Reverse tag»).

8.2. Прив'язка до inbound

Кожен клієнт має бути прив'язаний хоча б до одного inbound — при створенні потрібен мінімум один (`at least one inbound is required`). В редакторі це поле **Прив'язані вхідні** з підказкою **Виберіть один або кілька вхідних**.

- **Прив'язати** — додати клієнта до обраних inbound (той самий UUID/пароль і спільний трафік). Існуючі прив'язки зберігаються.
- **Відв'язати** — прибрати клієнта з обраних inbound. Сам запис клієнта зберігається (для повного видалення використовуйте **Видалити**). Пари, де клієнт не був прив'язаний, тихо пропускаються.

При збереженні клієнта, прив'язаного до кількох inbound, поля, несумісні з конкретним протоколом/транспорт (наприклад, Flow поза VLESS-Vision), автоматично приводяться до допустимих значень для кожного inbound.

Над списком вибору inbound (у формі клієнта, при масовому додаванні клієнтів і у вікнах масового прикріплення/відкріплення) є кнопки **Вибрати всі** і **Очистити**. У цих списках кожен inbound підписується своїм примітком (remark), якщо воно задано, інакше — тегом inbound.

8.3. Операції над клієнтом

Для окремого клієнта (через картку **Інформація про клієнта** або контекстне меню **Дії**) доступні:

Перегляд інформації, QR-коду і посилання

- **Інформація про клієнта** — картка з усіма полями, використаним/залишковим трафіком (**Залишок**), терміном дії та прив'язаними inbound. З 3.5.0 картка також показує **групу** клієнта (рядок «Група» з міткою, над «Коментар»), коли групу задано.

Запит клієнта через API (`GET /panel/api/clients/get/:email`) поруч із полями `client` і `inboundIds` додатково повертає `usedTraffic` — фактично витрачений трафік (відправлено + отримано, з урахуванням даних вузлів), що спрощує зіставлення витрат із квотою `totalGB`.

- **QR-код і Посилання** — конфігураційне посилання клієнта для імпорту в клієнтський застосунок. Формується за всіма прив'язаними inbound із підтримуваним протоколом (`GET /links/:email`). Якщо відповідних посилань немає: «Немає посилань для спільного доступу — спочатку прив'яжіть клієнта до вхідного з підтримуваним протоколом.»
- **Посилання підписки** — URL підписки за `subId` (`GET /subLinks/:subId`). Доступне тільки якщо у клієнта є `subId` і сервіс підписки увімкнено в **Налаштування панелі** → **Підписка** (інакше «Сервіс підписки вимкнено»). Додатково віддається **URL JSON-підписки**.

Скинути трафік

Скинути трафік (`POST /resetTraffic/:email`) обнуляє лічильники відправки/приймання (`up` , `down`) конкретного клієнта. Квота (`totalGB`) і термін дії **не зачіпаються** — обнуляється тільки використаний обсяг. Тост: «Трафік скинуто». Якщо клієнт не прив'язаний до жодного inbound: «Спочатку прив'яжіть цього клієнта до вхідного.».

Кнопка **Скинути трафік** доступна і з форми редагування клієнта — в нижній панелі, поруч із **Скасувати / Зберегти** (перед скиданням запитується підтвердження). Якщо клієнт був вимкнений через вичерпання трафіку, скидання (як одиночне, так і масове) автоматично знову **вмикає** його (`enable = true`) і відразу розсилає цю зміну на вузли-ноди — повторно вмикати клієнта вручну на майстері та нодах більше не потрібно.

Скинути ліміт IP

Очищає накопичений журнал IP клієнта (`POST /clearIps/:email`), щоб зняти тимчасове блокування через перевищення ліміту одночасних підключень. Тост: «Лог було очищено».

Видалити

Видалити (`POST /del/:email`) — повне видалення клієнта. Діалог підтвердження: заголовок «Видалити клієнта {email}?», текст «Клієнт буде видалений з усіх прив'язаних вхідних, а його запис трафіку буде знищено. Цю дію не можна скасувати.». Видалення знімає клієнта з **усіх** inbound і знищує його запис трафіку. Тост: «Клієнт видалено».

8.4. Масові операції

У списку клієнтів можна відмітити кілька записів (**Вибрати всі**, **Очистити всі**); лічильник — «{count} вибрано». Над вибраними доступні:

- **Видалити ({count})** (`POST /bulkDel`) — групове видалення. Підтвердження: «Видалити {count} клієнтів?», «Кожен вибраний клієнт видаляється з усіх прив'язаних вхідних, його запис трафіку знищується. Цю дію не можна скасувати.». Тост: «Видалено клієнтів: {count}», при частковій невдачі — «Видалено: {ok}, не вдалося: {failed}».
- **Змінити ({count}) / Коригування** (`POST /bulkAdjust`) — масова зміна терміну та/або квоти. Діалог «Змінити {count} клієнтів» з підказкою «Позитивні значення додають, від'ємні — зменшують. Клієнти з необмеженим терміном або трафіком пропускаються для відповідного поля.». Поля: **Додати дні**, **Додати трафік (ГБ)** і **Set flow**. Логіка:
 - **Термін:** клієнти з безстроковим терміном (`expiryTime == 0`) пропускаються («unlimited expiry»); для клієнтів із датою термін зсувається на вказану кількість днів; для клієнтів у режимі «після першого використання» (від'ємний термін) коригується тривалість очікування. Зменшення, що виходить за межі залишку, пропускається («reduction exceeds remaining time/delay window»).
 - **Трафік:** клієнти з безлімітом (`totalGB == 0`) пропускаються («unlimited traffic»); інакше квота змінюється на вказаний обсяг, не опускаючись нижче нуля.
 - **Flow:** випадючий список **Set flow** дозволяє встановити або скинути XTLS flow відразу у всіх вибраних клієнтів. За замовчуванням вибрано **No change** (без змін). Варіант **Disable (clear flow)** скидає flow, а значення `xtls-rprx-vision` і `xtls-rprx-vision-udp443` задають відповідний

vision-flow. Встановлення vision-flow застосовується тільки до inbound, що підтримують flow; невідповідні inbound залишаються без змін і позначаються як пропущені, тоді як скидання flow допускається завжди.

- **Авто-увімкнення (з 3.4.2):** клієнт, вимкнений **лише через вичерпання** (минул термін або перевищено квоту), при продовженні автоматично знову вмикається — локально і на його вузлі, — якщо коригування повертає його в межі лімітів. Вимкнені вручну або все ще вичерпані залишаються вимкненими (поле `enable` тепер записується явно, тому стан зберігається і у клієнтів без inbound).
- Якщо не вказані ні дні, ні трафік, ні flow: «Вкажіть дні, трафік або flow перед застосуванням.». Тост: «Змінено: {count}» / «Змінено: {ok}, пропущено: {skipped}».

Приклад: продовжити вибраних клієнтів на 30 днів і додати 50 ГБ. У діалозі **Змінити** вкажіть **Додати дні** = 30 , **Додати трафік (ГБ)** = 50 . Щоб, навпаки, відняти тиждень і урізати квоту на 10 ГБ, введіть від'ємні значення: **Додати дні** = -7 , **Додати трафік (ГБ)** = -10 (клієнти з безстроковим терміном або безлімітом за відповідним полем будуть пропущені).

- **Прив'язати ({count}) / Відв'язати ({count})** (`POST /bulkAttach / bulkDetach`) — масове прив'язування/відв'язування вибраних клієнтів до вибраних inbound. Цілі — тільки багатокористувацькі inbound. Результат відв'язування: «Відв'язано {detached}, пропущено {skipped}». З 3.5.0 селектор inbound у формі клієнта **приховує вимкнені inbound** (крім тих, до яких клієнт уже прив'язаний), а у вікні «Відв'язати» усунуто накопичення дублів записів одного клієнта (дані виправляються автоматично при старті панелі).
- **Sub-посилання ({count})** — зведена таблиця URL підписок і JSON-підписок вибраних клієнтів із кнопкою **Копіювати всі**. Якщо ні в кого немає subId: «Жоден із вибраних клієнтів не має ID підписки.».
- **Додати до групи і Розгрупувати** — призначення і зняття мітки групи.
- **Увімкнути ({count}) / Вимкнути ({count})** (`POST /bulkEnable / bulkDisable`) — масове увімкнення і вимкнення вибраних клієнтів. **Увімкнути** активує кожного вибраного клієнта на всіх прив'язаних inbound; клієнти з вичерпаною квотою трафіку або закінченим терміном будуть автоматично вимкнені знову. **Вимкнути** одразу позбавляє клієнтів доступу, але їхні записи й накопичений трафік зберігаються. Перед виконанням панель запитує підтвердження, а після операції показує сповіщення з кількістю оброблених клієнтів і, за наявності, з кількістю тих, для яких дія не вдалася.

Скидання трафіку і видалення за статусом

- **Скинути трафік усіх клієнтів** (`POST /resetAllTraffics`) — обнуляє лічильники `up / down` у **всіх** клієнтів панелі. Підтвердження: «Скинути трафік усіх клієнтів?» і «Лічильники відправки/приймання всіх клієнтів скидаються в нуль. Квоти і термін дії не зачіпаються. Цю дію не можна скасувати.». Тост: «Трафік усіх клієнтів скинуто».
- **Видалити вичерпаних** (`POST /delDepleted`) — видаляє всіх клієнтів, у яких **вичерпана квота** (`total > 0 and up + down >= total`) **або закінчився термін** (`expiry_time > 0 and expiry_time <= зараз`), за умови `reset = 0` (клієнти з автопродовженням не зачіпаються). Підтвердження: «Видалити вичерпаних клієнтів?», «Видаляються всі клієнти, у яких вичерпана квота трафіку або закінчився термін. Цю дію не можна скасувати.». Тост: «Видалено вичерпаних клієнтів: {count}».

Експорт, імпорт і видалення неприв'язаних клієнтів

Коли нічого не виділено, у меню **Ще** на сторінці **Клієнти** доступні три операції.

Експорт клієнтів (GET /clients/export) відкриває переглядач із JSON-списком усіх клієнтів у форматі {client, inboundIds} з кнопками копіювання і завантаження (файл clients-export.json). **Імпорт клієнтів** (POST /clients/import) відкриває редактор, в який вставляють такий самий JSON і натискають **Import**: клієнти з inboundIds створюються й прив'язуються до inbound, клієнти без прив'язок відновлюються як окремі «голі» записи, а вже існуючі email **ніколи не перезаписуються** — вони потрапляють до списку пропущених. Тости: «{count} clients imported», «{ok} imported, {failed} skipped».

Видалити неприв'язаних клієнтів (POST /clients/delOrphans) — небезпечна операція: видаляє всіх клієнтів, не прив'язаних до жодного inbound, разом із їхнім записом трафіку, журналом IP і зовнішніми посиланнями. Підтвердження: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.». Тост: «{count} unattached clients deleted». Дія незворотна.

8.5. Пошук, фільтри і сортування

Над списком є рядок пошуку («Пошук email, коментаря, sub ID, UUID, пароля, auth, Telegram ID...») — він шукає за email, коментарем, subId, UUID, паролем, auth і (знову з 3.5.0) за **Telegram ID**. Лічильник результатів: «Показано {shown} з {total}».

Список клієнтів оновлюється автоматично: панель раз на кілька секунд підтягує актуальну поточну сторінку, тому щойно підключені клієнти й змінений порядок сортування з'являються без ручного оновлення (індикатор завантаження при фоновому опитуванні не мигає).

З 3.5.0 у таблиці є стовпець **«Швидкість»** (між «Трафік» і «Залишок»): жива швидкість клієнта — синя мітка  віддача /  завантаження (ковзне ~5-секундне середнє, оновлюється кожні 5 секунд); за відсутності трафіку — сіра риска. На мобільних швидкість показується рядком у картці клієнта.

Панель **Фільтр клієнтів** дозволяє відбирати за статусом (категорії), протоколом, прив'язаним inbound, діапазоном терміну дії, діапазоном використаного трафіку, наявністю автопродовження (**Є/Немає**), наявністю Telegram ID і коментаря, а також за групою. На панелях із вузлами з'являється мультиселект **Вузли**: можна обмежити список клієнтами вибраних вузлів; окремий пункт **Локальна панель** відбирає клієнтів inbound без прив'язки до вузла (фільтр видно тільки за наявності вузлів). Сортування: **Спочатку старі/нові, Нещодавно оновлені, Нещодавно в мережі, Email А→Я / Я→А, Більше трафіку, Більше залишку, Скоро закінчується**.

8.6. Значки і статуси

Пріоритет статусів: вичерпаний/закінчений → неактивний → скоро закінчується → активний.

- **В мережі / Не в мережі** — клієнт з активним підключенням (є в поточному онлайн-списку) і **увімкнений**. Онлайн-список оновлюється окремими запитами (/onlines , /onlinesByGuid).
- **Вичерпаний** (depleted) — квота вибрана (up + down >= totalGB) **або** термін закінчився (expiryTime <= зараз). Такий клієнт автоматично вимикається і підпадає під дію **Видалити вичерпаних**.

- **Скоро закінчується** (expiring) – увімкнений клієнт, у якого до закінчення терміну залишилося менше порогового інтервалу **або** до вичерпання квоти залишилося менше порогового обсягу (пороги задаються в налаштуваннях панелі). Не враховується, якщо клієнт вже вичерпаний/вимкнений.
 - **Неактивний** (deactive) – клієнт з `enable = false` (вимкнений вручну або фоновим завданням).
 - **Активний** (active) – увімкнений, не вичерпаний, термін не закінчився і до порогів ще далеко.
-

9. Групи клієнтів

Це можливість даного форку 3X-UI. В оригінальному проєкті 3x-ui (MHSanaei) поняття «група клієнтів» немає — тут додано окрему таблицю груп, сторінку **Групи** в меню панелі та відповідні методи API. Якщо ви переносите конфігурацію на оригінальний 3x-ui, мітка групи просто ніде не враховуватиметься.

9.1. Що таке група клієнтів і навіщо вона потрібна

Група — це іменована логічна мітка (label), яку можна навісити на одного або кількох клієнтів. Вона не створює нового способу підключення і не є ні inbound, ні нодою — це суто організаційний ярлик, за яким клієнтів зручно фільтрувати та обробляти масово.

Ключова ідея моделі клієнтів у цьому форку: **клієнт — це сутність верхнього рівня, що ідентифікується за email** (поле `email` у таблиці `clients` має унікальний індекс). Один і той самий клієнт (один email з одними й тими самими обліковими даними) може одночасно числитися в кількох inbound і навіть на кількох нодах, зокрема з різними протоколами. Мітка групи зберігається **один раз на клієнта**, тому вона автоматично поширюється на всі його прив'язки до inbound одразу.

Мітка групи — це логічна мітка для групування:

Шар	Де зберігається	Поле
Запис клієнта (БД)	таблиця <code>clients</code>	<code>group_name</code> (за замовчуванням порожній рядок <code>''</code>)
Довідник груп (БД)	таблиця <code>client_groups</code>	<code>name</code> (унікальний індекс, не порожній)
Налаштування inbound (Xray)	JSON <code>settings.clients[].group</code>	дублюється в кожен клієнтський об'єкт кожного inbound, де перебуває клієнт

Навіщо це потрібно на практиці:

- **Один клієнт на кількох inbound/нодах.** Якщо клієнт «продається» як доступ одразу до кількох inbound (наприклад, різні протоколи або різні ноди), група дозволяє керувати ним як єдиним цілим: скинути трафік, видалити, перейменувати мітку — однією операцією по всіх його inbound.
- **Масові операції та фільтрація.** На сторінці **Клієнти** список можна фільтрувати за групою; на сторінці **Групи** доступні масові дії над усіма учасниками групи.
- **Організація великого парку клієнтів.** Мітки на кшталт `vip`, `trial`, `team-A` допомагають розкласти тисячі клієнтів по логічних кошиках, не плодячи при цьому окремих inbound.

9.2. Зв'язок групи з клієнтами, inbound, нодами та протоколами

Це найважливіший для розуміння підрозділ, оскільки синхронізація мітки нетривіальна.

Група описує клієнта, а не inbound. Мітка живе в записі клієнта (`clients.group_name`). Коли клієнт приєднаний до кількох inbound, за будь-якої зміни групи панель проходить по **всіх** inbound, у яких цей

клієнт перебуває, і проставляє/прибирає поле `group` всередині їхніх Xray-налаштувань (`settings.clients[]`). Технічно це робиться так: за email клієнта знаходяться всі inbound, у яких він перебуває, потім у JSON-налаштуваннях кожного такого inbound правиться клієнтський об'єкт із цим email. Тому:

- Група **не залежить від протоколу**. Один email може бути VLESS-клієнтом в одному inbound і Hysteria-клієнтом в іншому — мітка групи в нього все одно одна й застосується до обох (облікові дані при цьому в кожного протоколу свої та зберігаються окремо).
- Група **охоплює ноди**. Inbound, що належать нодам, відрізняються від inbound головної панелі полем `nodeId` (у inbound головної панелі воно `null / 0`). Мітка групи поширюється на клієнтські об'єкти в inbound незалежно від того, головний це inbound чи нодовий — аби лише клієнт із таким email там був присутній.

Мітка групи стійка до синхронізації з нод і до перебудови налаштувань. Це поведінка спеціально закладена:

- Коли нода надсилає знімок трафіку, її дані **не затирають** локальні `group_name` і `comment` клієнта в БД панелі. Група та коментар вважаються «панель-локальними» полями — нода ними не керує.
- При перебудові налаштувань inbound порожнє значення `group` у даних, що надходять, **не скидає** вже збережену мітку. Групою керують саме через сторінку **Групи** (а не через редагування Xray-налаштувань інбаунда), тому «порожня група» при звичайній перебудові налаштувань трактується як «не чіпати», а не як «очистити».

Практичний висновок: єдині операції, які **навмисно очищають** мітку, — це видалення групи та явне видалення клієнта з групи (див. нижче). Звичайне редагування inbound або фонові синхронізація з нодою групу не втраять.

9.3. Довідник груп і «порожні» групи

Список груп на сторінці формується злиттям двох джерел:

- Похідні групи (derived)** — усі непорожні значення `group_name`, що реально трапляються в клієнтів, з підрахунком кількості клієнтів.
- Збережені групи (stored)** — записи з таблиці `client_groups`.

Це об'єднання дає важливий ефект: група може існувати **без жодного клієнта**. Така група створюється явною кнопкою «Додати групу» (запис у `client_groups`) і в списку відображається з лічильником `0`. Ці записи й вважаються **порожніми групами**. Список завжди відсортований за іменем без урахування регістру.

Зведені лічильники на сторінці:

Поле (RU)	Що показує
Всего групп	Загальна кількість груп (збережених і похідних разом).
Клиенты с группой	Скільки клієнтів мають непорожню мітку групи.
Пустые группы	Скільки груп існує без клієнтів (лічильник <code>0</code>).
Клиентов в группе	Кількість клієнтів у конкретній групі (стовпець таблиці).

9.4. Поля та стовпці групи

Запис групи в таблиці `client_groups` містить:

Поле	Тип	За замовчуванням	Опис
<code>Id</code>	int	автоінкремент	Первинний ключ запису групи.
<code>Name</code>	string	— (обов'язково)	Ім'я групи. Унікальний індекс, не може бути порожнім. В UI — стовпець Имя .
<code>CreatedAt</code>	int64 (мс)	час створення	Момент створення запису групи.
<code>UpdatedAt</code>	int64 (мс)	час зміни	Момент останньої зміни.

У таблиці на сторінці відображаються щонайменше стовпці **Имя** та **Клиентов в группе**, а також кнопки дій (див. нижче).

9.5. Створення групи

Кнопка **Добавить группу**.

Кроки:

1. Натисніть **Добавить группу**.
2. Введіть ім'я групи.
3. Підтвердьте.

Поведінка бекенду (`POST /panel/api/clients/groups/create` , тіло `{"name": "..."}`):

- Ім'я обрізається від пробілів по краях. Порожнє ім'я відхиляється з помилкою «group name is required».
- Якщо група з таким іменем уже є — помилка «group already exists».
- За успіху створюється запис у `client_groups` (спочатку без клієнтів — це порожня група).

Повідомлення про успіх: **«Группа «{name}» создана.»**

Приклад: створити порожню групу через API. Підготуйте набір міток заздалегідь, ще до наповнення клієнтами:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Відповідь за успіху:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Повторний виклик з тим самим іменем поверне `"success": false` і повідомлення `group already exists`.

Створення порожньої групи заздалегідь зручне, коли ви хочете підготувати набір міток і потім наповнювати їх клієнтами через «Добавить клиентов...».

9.6. Переименовання групи

Кнопка **Переименовать**, заголовок діалогу — «**Переименовать {name}**».

Поведінка (`POST /panel/api/clients/groups/rename`, тіло `{"oldName": "...", "newName": "..."}:`

- Обидва імені обрізаються від пробілів. Порожнє старе ім'я — помилка «old group name is required», порожнє нове — «new group name is required».
- Якщо нове ім'я збігається зі старим — нічого не робиться (зачеплено 0 клієнтів).
- Інакше перейменування виконується атомарно:
 - запис у `client_groups` перейменовується;
 - у всіх клієнтів з `group_name = oldName` поле оновлюється на `newName`;
 - у **всіх inbound**, де перебувають зачеплені клієнти (включно з нодовими), у Xray-налаштуваннях значення `group` правиться зі старого на нове.
- Після перейменування панель позначає Xray як такий, що потребує перезапуску, і розсилає сповіщення про зміну клієнтів.

Повідомлення:

- Успіх: «Группа переименована для {count} клиент(ов).»
- Конфлікт імен в UI: «Группа с именем «{name}» уже существует.»

9.7. Додавання клієнтів до групи

Кнопка **Добавить клиентов...**, заголовок — «**Добавить клиентов в группу «{name}»**».

Дослівна підказка в діалозі:

«Выберите клиентов для добавления в эту группу. Существующие привязки к входящим сохраняются; меняется только метка группы. Клиенты, уже входящие в эту группу, не показываются.»

Якщо додавати нікого, показується «**Нет других клиентов для добавления.**»

Поведінка (`POST /panel/api/clients/groups/bulkAdd`, тіло `{"emails": [...], "group": "..."}:`

- Ім'я групи обов'язкове (інакше помилка «group name is required»); порожній список email — операція нічого не робить.
- Якщо такої групи ще немає ні в `client_groups`, ні серед похідних — її буде створено автоматично.
- Для вибраних email у клієнтів проставляється `group_name = group`; **прив'язки клієнтів до inbound не змінюються** — зачіпається лише мітка. Потім у всіх inbound цих клієнтів проставляється поле `group`.

- Повертається кількість зачеплених записів клієнтів; Xray позначається на перезапуск.

Повідомлення про успіх: **«Додано {count} клієнт(ов) в {name}»**

Приклад: позначити кількох клієнтів групою одним запитом. Клієнти вказуються за email, прив'язки до inbound при цьому не змінюються:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Якщо групи `vip` ще немає, її буде створено автоматично. Після запиту в цих клієнтів у записі проставиться `group_name = "vip"`, а в Xray-налаштуваннях кожного їхнього inbound клієнтський об'єкт отримає поле `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Видалення клієнтів із групи (без видалення самих клієнтів)

Кнопка **Удалить клиентов...**, заголовок — **«Удалить клиентов из группы «{name}»»**.

Дослівна підказка:

«Выберите участников для удаления из этой группы. Сами клиенты сохраняются (используйте «Удалить клиентов группы» для полного удаления).»

Поведінка (`POST /panel/api/clients/groups/bulkRemove`, тіло `{"emails": [...]}`): технічно це те саме, що «Додати в групу» з порожньою групою. У вибраних клієнтів очищається `group_name`, а в їхніх inbound поле `group` видаляється з Xray-налаштувань. Самі клієнти та їхні прив'язки до inbound залишаються.

Повідомлення про успіх: **«Удалено {count} клієнт(ов) из {name}»**

9.9. Скидання трафіку групи

Кнопка **Сбросить трафик**.

Діалог підтвердження:

- Заголовок: **«Сбросить трафик группы {name}?»**
- Текст: **«Сбрасывается только счётчик трафика группы. Счётчики отдельных клиентов не затрагиваются.»**

Поведінка: обнуляється **лише відображуваний лічильник самої групи** — панель запам'ятовує поточну суму `up/down` групи як базову відмітку і далі в списку груп показує `max(0, сума - базова відмітка)`. Лічильники `up / down`, квоти і поле `enable` окремих клієнтів **не змінюються**, перезапуск Xray не потрібен. Це зміна поведінки відносно попередніх версій, де скидання обнуляло трафік усіх клієнтів групи.

Запит: `POST /panel/api/clients/groups/resetTraffic` з тілом `{"name": "<ім'я групи>"}`.

Повідомлення про успіх: **«Трафик группы {name} сброшен.»**

9.10. Видалення групи та видалення клієнтів групи

На сторінці є **дві принципово різні операції видалення** — їх легко переплутати, тому відмінність критична.

9.10.1. Видалити групу (зберегти клієнтів)

Кнопка **«Удалить группу (сохранить клиентов)»**.

Діалог:

- Заголовок: **«Удалить группу {name}?»**
- Текст: **«Это удаляет группу и очищает её метку у {count} клиент(ов). Сами клиенты не удаляются.»**

Поведінка (`POST /panel/api/clients/groups/delete`, тіло `{"name": "..."}`): запис групи видаляється з `client_groups`, у всіх її клієнтів `group_name` очищається, і з їхніх inbound прибирається поле `group`. **Клієнти, їхні підключення та трафік зберігаються**. Храй позначається на перезапуск.

Повідомлення про успіх: **«Группа очищена у {count} клиент(ов).»**

9.10.2. Видалити клієнтів групи (повне видалення)

Кнопка **«Удалить клиентов группы»**.

Діалог:

- Заголовок: **«Удалить всех клиентов в {name}?»**
- Текст: **«Это безвозвратно удаляет {count} клиент(ов) вместе с их записями трафика. Метка группы также очищается. Это нельзя отменить.»**

Це деструктивна операція: вона видаляє самих клієнтів (через масове видалення за email, ендпоінт `POST /panel/api/clients/bulkDel`), включно з їхніми записами трафіку, і тим самим прибирає їх з усіх inbound.

Повідомлення:

- Успіх: **«Удалено {count} клиент(ов).»**
- Частковий результат: **«{ok} удалено, {failed} пропущено»**

Якщо група порожня, дії над її учасниками недоступні — виводиться **«В этой группе пока нет клиентов.»**

9.11. Зв'язок зі сторінкою «Клієнти»

Мітка групи видима й використовується також поза сторінкою **Групи**:

- У компактному записі клієнта є поле `group`, тому в списку клієнтів відображається належність до групи.
- Список клієнтів (`/panel/api/clients/list/paged`) приймає параметр фільтра `group`: можна передати одне ім'я або кілька через кому. Зіставлення виконується за принципом «АБО» всередині поля, без урахування регістру. Особливий випадок: порожній елемент у списку груп фільтра означає «клієнти без групи» (у яких `group` порожня).
- У відповіді сторінки клієнтів віддається масив `groups` — повний перелік імен наявних груп, щоб UI міг побудувати випадний список фільтра.

Приклад: фільтрація клієнтів за групами. Запит віддає лише клієнтів з мітками `vip` або `trial` (кілька імен — через кому, «АБО»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Щоб отримати клієнтів **без** групи, передайте порожній елемент у списку — наприклад, значення фільтра `group=` (порожній рядок) або `group=vip,` (мітка `vip` плюс клієнти без групи).

9.12. Зведення ендпоінтів API

Усі маршрути груп змонтовані під `/panel/api/clients`:

Метод і шлях	Призначення	Тіло запиту
GET <code>/panel/api/clients/groups</code>	Список груп з лічильниками клієнтів	—
GET <code>/panel/api/clients/groups/:name/emails</code>	Email усіх учасників групи (сортування за email)	—
POST <code>/panel/api/clients/groups/create</code>	Створити порожню групу	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Перейменувати групу	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Видалити групу, зберігши клієнтів (очищення мітки)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Додати клієнтів до групи (за email)	<code>{"emails":[...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Прибрати клієнтів із групи (очищення мітки)	<code>{"emails":[...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	Повне видалення клієнтів (використовується «Удалить клиентов группы»)	<code>{"emails":[...], "keepTraffic"}</code>

Приклад: типовий сценарій життєвого циклу групи через API.

```
# 1. Создать метку trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Навесить её на двух клиентов
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Обнулить трафик всех участников (по email из /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Удалить группу, но сохранить клиентов (только очистка метки)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Крок 4 видаляє запис групи та очищає `group_name` у її клієнтів, але самі клієнти, їхні підключення та трафік залишаються. Для безповоротного видалення самих клієнтів замість цього використовується `bulkDel`.

Операції, що змінюють мітку в клієнтів (`rename` , `delete` , `bulkAdd` , `bulkRemove`), позначають Xray як такий, що потребує перезапуску, і розсилають сповіщення про зміну клієнтів.

9.13. Трафік за групою

Новинка версії 3.3.0: у розділі **Групи** (сторінка «Клієнти», вкладка керування групами) таблиця груп тепер показує не лише кількість клієнтів у кожній групі, а й сумарний витрачений трафік групи. Колонка підписана **«Использованный трафик»**.

Що показує колонка

Для кожного рядка-групи виводиться сума трафіку по всіх клієнтах, що входять до цієї групи — тобто складені `up` + `down` (відданий + прийнятий трафік) усіх її учасників. Це дає швидку відповідь на питання «скільки сумарно завантажила/роздала вся група», без необхідності відкривати клієнтів по одному та складати вручну.

Поряд у таблиці груп виводяться:

Колонка	Що означає
Имя	Ім'я групи
Клиенты	Скільки клієнтів позначено цією групою (раніше стовпець називався «Клиентов в группе»)
Отправлено	Сумарний <code>up</code> (відданий трафік) по всіх клієнтах групи
Получено	Сумарний <code>down</code> (прийнятий трафік) по всіх клієнтах групи
Использованный трафик	Сумарний <code>up</code> + <code>down</code> по всіх клієнтах групи

Відданий і прийнятий трафік виводяться окремими стовпцями **Отправлено** та **Получено**, а стовпець **Использованный трафик** показує їхню суму. Стовпець кількості клієнтів називається просто **Клиенты**.

Зведення над таблицею додатково показує агрегати по всіх групах — **«Всего групп»** і **«Клиенты с группой»**, а сумарний трафік розбито на дві картки: **«Всего отправлено / получено»** (зі стрілками вгору/вниз —

окремо відданий і прийнятий трафік усіх груп) та **«Общий трафик»** (з іконкою діаграми — їхній сумарний підсумок).

Як рахується

Розрахунок виконується одним SQL-запитом до таблиці клієнтів із приєднанням (`LEFT JOIN`) таблиці обліку трафіку:

- за полем-міткою групи (`group_name`) клієнти групуються, рахується їхня кількість — це і є «Клиентов в группе»;
- трафік береться як сума `up + down` із приєднаної таблиці `client_traffic` . Тобто підсумовуються і віддані (`up`), і прийняті (`down`) байти по кожному клієнту;
- оскільки email унікальний і в таблиці клієнтів, і в таблиці трафіку, приєднання не задвоює трафік одного клієнта.

Особливості значень:

- **Клієнти без запису трафіку** враховуються в лічильнику учасників, але додають до суми 0, тому щойно створена група показує трафік `0` .
- **Порожні групи** (створені, але без клієнтів) також присутні в списку з нульовим лічильником і нульовим трафіком: окрім груп, «виведених» з міток клієнтів, у результат підмішуються явно збережені групи, після чого список сортується за іменем без урахування регістру.
- Клієнти без мітки групи (`group_name` порожній) у розрахунок не потрапляють.

Пов'язані дії

З таблиці груп, як і раніше, доступні дії над групою в цілому, зокрема **«Сбросить трафик»** — обнуляє **лише лічильник самої групи** (базову відмітку), не чіпаючи трафік окремих клієнтів (див. 9.9). Після такого скидання колонка «Использованный трафик» для цієї групи показує `0` .

10. Підписки (Subscription)

Підписка (subscription) — це механізм, який дозволяє видати клієнту одне постійне посилання (URL), за яким VPN-клієнт сам завантажує і періодично оновлює повний набір конфігурацій. Замість того щоб вручну пересилати користувачу окреме посилання на кожен inbound, йому передається єдина адреса виду `https://домен:порт/sub/<subId>`. За цією адресою панель на льоту збирає всі конфігурації, прив'язані до даного клієнта, і повертає їх у потрібному клієнту форматі. При зміні налаштувань на сервері (нова адреса, ротація Reality-ключів, додавання inbound) клієнт отримує актуальну конфігурацію при наступному автоматичному оновленні, не потребуючи жодних дій від користувача.

Підписку обслуговує окремий HTTP/HTTPS-сервер усередині панелі, який запускається незалежно від вебпанелі і слухає на власному порту. Це зроблено з міркувань безпеки: порт підписки можна відкрити назовні, не відкриваючи порт самої панелі.

10.1. Що таке subId і як формується посилання

Кожен клієнт в inbound має поле `subId` (в інтерфейсі — «ID підписки»). Саме це значення є ключем підписки: панель шукає в усіх inbound клієнтів, у яких `subId` збігається із запитаним, і об'єднує їхні конфігурації в одну відповідь.

- Якщо у кількох клієнтів (в одному або в різних inbound) задано однаковий `subId`, їхні конфігурації потраплять в одну підписку. Це штатний спосіб видати одному користувачу одразу кілька серверів/протоколів за одним посиланням.

Приклад: один користувач — два сервери за одним посиланням. Припустимо, є два inbound (VLESS на сервері А і Trojan на сервері В). Щоб видати користувачу обидві конфігурації одним посиланням, задайте обом його клієнтам однаковий `subId`:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Тоді за адресою `https://sub.example.com:2096/sub/ivan2025` панель поверне обидві конфігурації одразу. Додасте пізніше третій inbound з тим самим `subId` — він з'явиться у користувача при наступному автооновленні підписки, без пересилання нового посилання.

- Якщо у клієнта поле `subId` порожнє, поділитися посиланням для загального доступу неможливо. В інтерфейсі на це вказує підказка: «У цього клієнта немає subId, посилання для загального доступу недоступне.»

Зовнішні посилання та підписки клієнта (вкладка «Links»)

У формі клієнта є вкладка «**Links**», де для окремого клієнта можна прикріпити додаткові джерела конфігурацій, які домішуються саме до його підписки (формати RAW, JSON і Clash):

- **Add External Link** — стороннє share-посилання (`vless://`, `trojan://`, `ss://` тощо). Додається до видачі як є, а для JSON/Clash додатково розбирається в конфігурацію.

- **Add External Subscription** — адреса зовнішньої підписки. Панель сама завантажує її (з кешуванням і коротким таймаутом) і впливає отримані конфігурації до загального списку клієнта.

Це зручно, щоб видати клієнту додаткові сервери поверх ваших inbound за тим самим єдиним посиланням. Якщо відповідь віддаленої підписки занадто велика, вона більше не обрізається мовчки: панель повертає помилку і продовжує використовувати останнє успішно закешоване значення.

- Значення `subId` не можна задати довільно: при збереженні перевіряється, що в ньому немає пробілів, символів `/`, `\` та керуючих символів. Відповідна підказка валідації: «ID підписки не може містити пробіли, '/', '\' або керуючі символи».

Підсумкове посилання будується як `<база>/<subPath>/<subId>` (див. розділ про налаштування сервера підписок і поле «URI зворотного проксі»). Якщо за `subId` не знайдено жодного клієнта (клієнт видалено, `subId` не існує), сервер повертає HTTP 404 без тіла. При внутрішній помилці — HTTP 500. VPN-клієнти орієнтуються лише на код відповіді, тому тіло помилки навмисно порожнє.

Порядок посилань inbound у підписці

У кожного inbound є поле **«Порядок у підписці»** (`subSortIndex`) — число від 1, що задає позицію посилань цього inbound у видачі підписки. Менші значення йдуть першими; при рівних значеннях зберігається початковий порядок створення (за id). Порядок застосовується до всіх форматів видачі — сирий текст, сторінка підписки, JSON і Clash. На порядок inbounds у самій панелі це поле не впливає.

Поле редагується у формі inbound поруч із налаштуваннями адреси в посиланні (share address) і синхронізується на вузли за звичайними правилами. Якщо хоча б у одного inbound порядок відрізняється від 1, у списку Inbounds з'являється компактна колонка **«Порядок»**.

10.2. Налаштування сервера підписок

Усі параметри підписки знаходяться в розділі налаштувань панелі на вкладці **«Підписка»**. Нижче розібрано кожен параметр; у дужках вказано внутрішній ключ налаштування і значення за замовчуванням.

Сам розділ розбитий на вкладки: **«Налаштування панелі»**, **«Інформація»**, **«Профіль»**, **«Сертифікати»**, **«Нарр»** і **«Clash / Mihomo»**. Поля заголовка підписки, URL підтримки, сторінки профілю, оголошення та каталогу теми знаходяться на вкладці «Профіль»; правила маршрутизації Нарр і Clash/Mihomo — на однойменних вкладках; інтервал оновлення підписки — на вкладці «Інформація».

Основні параметри

Поле (UI)	Ключ	Значення за замовчуванням	Опис
Увімкнуті підписку	<code>subEnable</code>	<code>true</code> (увімкнено)	Запускає окремих сервер підписок. Підказка: «Функція підписки з окремою конфігурацією». Якщо вимкнено — сервер підписок не стартує, і жодне з посилань не працює.
Прослуховування IP	<code>subListen</code>	порожньо	IP-адреса, на якій сервер підписок приймає з'єднання. Підказка: «Залиште порожнім за замовчуванням, щоб відстежувати всі IP-адреси».

Поле (UI)	Ключ	Значення за замовчуванням	Опис
Порт підписки	subPort	2096	TCP-порт сервера підписок. Підказка: «Номер порту для обслуговування служби підписки не повинен використовуватися на сервері» – порт має бути вільним і не конфліктувати з панеллю або Xray.
URI-шлях	subPath	/sub/	Шлях, за яким видаються звичайні підписки. Підказка: «Має починатися з '/' і закінчуватися на '/'».
Домен прослуховування	subDomain	порожньо	Домен, за яким дозволено доступ до підписки (валідація Host). Підказка: «Залиште порожнім за замовчуванням, щоб слухати всі домени та IP-адреси». Якщо задано – запити з іншим Host відхиляються.

Важливо щодо безпеки: шлях за замовчуванням `/sub/` (і `/json/` для JSON) широко відомий і легко вгадується. Панель показує попередження: «Шлях підписки за замовчуванням `/sub/` широко відомий – змініть його.» і аналогічне для JSON. Рекомендується задати власний неочевидний шлях.

TLS / сертифікат

Поле (UI)	Ключ	За замовчуванням	Опис
Шлях до файлу публічного ключа сертифіката підписки	subCertFile	порожньо	Повний шлях до файлу сертифіката (<code>.crt</code> / <code>fullchain</code>). Підказка: «Введіть повний шлях, що починається з '/'».
Шлях до файлу приватного ключа сертифіката підписки	subKeyFile	порожньо	Повний шлях до файлу приватного ключа. Підказка: «Введіть повний шлях, що починається з '/'».

Якщо задано обидва шляхи і сертифікат успішно завантажується, сервер підписок працює по **HTTPS**. Якщо поля порожні або сертифікат не вдалося прочитати – сервер повертається на **HTTP** (помилка записується в лог). Наявність валідного TLS також впливає на формування базового URL: при порті 443 з TLS і порті 80 без TLS номер порту в посиланні опускається.

Інтервал оновлення

Поле (UI)	Ключ	За замовчуванням	Опис
Інтервали оновлення підписки	subUpdates	12	Як часто (у годинах) клієнтський застосунок повинен перезапитувати підписку. Підказка: «Інтервал між оновленнями в клієнтському застосунку (у годинах)». З 3.5.0 поле приймає 0–525600 і показує діапазон (попередня межа інтерфейсу 168 не збігалася із сервером і після апгрейду з 2.x блокувала збереження будь-яких налаштувань).

Значення передається клієнту в HTTP-заголовку `Profile-Update-Interval`; сучасні клієнти використовують його як період автооновлення конфігурації.

Формат і інформація у відповіді

Поле (UI)	Ключ	За замовчуванням	Опис
Кодувати	subEncrypt	true	Підказка: «Шифрувати повернуті конфіги в підписці». Технічно це не шифрування, а Base64-кодування всього тіла звичайної підписки (формат, який очікує більшість клієнтів). При вимкненні посилання видаються відкритим текстом, по одному на рядок.
Показати інформацію про використання	subShowInfo	true	Підказка: «Відображати залишок трафіку та дату закінчення після назви конфігурації». Коли увімкнено, до назви (remark) кожної конфігурації додаються маркери залишку трафіку () і терміну дії (наприклад, 5D, 3H); при вичерпаному/недоступному клієнті показується N/A .
Включати Email у назву	subEmailInRemark	true	Підказка: «Включати email клієнта в назву профілю підписки». Додає email клієнта в remark профілю.

Шаблон ремарки (Remark Template)

Відображувана назва (remark) кожної конфігурації в підписці формується за **шаблоном ремарки** — поле **«Шаблон примітки»** (remarkTemplate) на вкладці **«Інформація»** налаштувань підписки. Попередній конструктор моделі примітки (окремий вибір частин inbound/email/external проху і символу-розділювача) з інтерфейсу прибрано; тепер ви пишете довільний формат імені та вставляєте в нього змінні. Значення за замовчуванням — `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` (тобто за замовчуванням ім'я профілю містить email клієнта). Якщо поле залишити порожнім, застосовується попередня (не налаштовувана через інтерфейс) модель ремарки.

Змінні згруповані за розділами **Client**, **Traffic** і **Time & status** і показуються поруч з полем як натискувані чіпи `{{VAR}}` з підказкою при наведенні; клік вставляє токен у шаблон, доступний живий попередній перегляд. Кожна змінна підставляється індивідуально для конкретного клієнта в момент генерації підписки. Допускається і спрощений запис в одинарних дужках (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` тощо) — панель сама приводить його до внутрішнього формату `{{...}}` .

Доступні змінні:

- **Ідентифікація клієнта:** `{{EMAIL}}` (синонім — `{{USERNAME}}`), `{{INBOUND}}` (ремарка самого inbound), `{{HOST}}` (ремарка хоста), `{{ID}}` (UUID), `{{SHORT_ID}}` (перші 8 символів UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Трафік:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (і їхні *_BYTES -варіанти в точних байтах), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Термін і статус:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (PPPP-MM-DD), `{{JALALI_EXPIRE_DATE}}` (дата за календарем джалалі), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .
- **Підключення (Connection):** `{{PROTOCOL}}` — протокол (VLESS, VMess, Trojan тощо), `{{TRANSPORT}}` — транспортна мережа (tcp, ws, grpc тощо), `{{SECURITY}}` — безпека транспорту (TLS, REALITY, NONE; виводиться великими літерами). Як і змінні витрати і терміну, ці три змінні діють лише в тілі підписки і

автоматично прибираються з ремарки на відображуваних посиланнях у панелі (QR/«Інформація») і на сторінці інформації про підписку.

Шаблон можна розбити на сегменти вертикальною рисою `|`. Сегмент, у якому змінна дає «безлімітне» значення (∞) — наприклад `{{TRAFFIC_LEFT}}` або `{{DAYS_LEFT}}` у клієнта без обмежень, — автоматично приховується. Крім того, блок з витратою трафіку і терміном показується один раз, на першому посиланні клієнта, щоб не дублюватися в кожній конфігурації.

Приклад. Шаблон `{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` дасть у клієнта із залишком 42 ГБ і 7 днями ім'я виду `ivan@vpn 42.00GB 7D`, а у безлімітного клієнта — просто `ivan@vpn` (сегменти з ∞ опущені).

З версії 3.4.2 змінна `{{EMAIL}}` (і синонім `{{USERNAME}}`) виводиться лише на **першому** посиланні клієнта в тілі підписки — як і блок залишку трафіку/терміну; на решті посилань того самого клієнта вона опускається.

На відображуваних у панелі посиланнях (QR-код і вікна «Інформація» на сторінці «Клієнти») і на сторінці інформації про підписку email клієнта присутній у назві профілю: вид «inbound-host-email» при заданому хості або «inbound-email» без хоста. Відомості про трафік і термін (а також змінні групи «Підключення») в ці відображувані імена не підставляються — вони працюють лише в тілі підписки, яке отримує VPN-клієнт.

Якщо рядок статистики трафіку клієнта «осиротів» після видалення і повторного створення inbound, змінна `{{TRAFFIC_USED}}` (та інші показники витрати) більше не показує `0.00B`: панель додатково шукає статистику за email клієнта і підставляє коректний використаний трафік. | Шаблон ремарки | `remarkTemplate | {{INBOUND}}| {{TRAFFIC_LEFT}}| {{DAYS_LEFT}}D` | Вільний шаблон відображуваної назви (remark) кожної конфігурації з підстановкою змінних `{{VAR}}`. Підставляється індивідуально для кожного клієнта при генерації підписки. Попередній конструктор «моделі примітки» (вибір inbound/email/external проху і розділювача) прибрано з інтерфейсу і використовується лише як запасний варіант, якщо поле залишити порожнім. Детальніше — див. «Шаблон ремарки (Remark Template)» нижче. |

Метадані профілю (заголовки відповіді)

Ці рядки передаються клієнту в HTTP-заголовках відповіді та відображаються у VPN-клієнті як метадані профілю. Усі вони за замовчуванням порожні.

Поле (UI)	Ключ	Заголовок	Опис
Заголовок підписки	<code>subTitle</code>	<code>Profile-Title</code> (y Base64)	«Назва підписки, яку бачить клієнт у VPN-клієнті». Для Clash також використовується як ім'я імпортованого профілю через <code>Content-Disposition</code> .
URL підтримки	<code>subSupportUrl</code>	<code>Support-Url</code>	«Посилання на технічну підтримку, що відображається у VPN-клієнті».
URL профілю	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Посилання на ваш сайт, що відображається у VPN-клієнті». Якщо не задано, підставляється фактичний URL запиту підписки.

Поле (UI)	Ключ	Заголовок	Опис
Оголошення	subAnnounce	Announce (у Base64)	«Текст оголошення, що відображається у VPN-клієнті». З 3.5.0 текст також показується інформаційним банером угорі інформаційної сторінки підписки (коли непорожній), а кастомним шаблоном доступна змінна <code>announce</code> .

Крім того, у кожній відповіді передається заголовок `Subscription-UserInfo` з агрегованими даними трафіку клієнта: `upload`, `download`, `total` і `expire` (момент закінчення в секундах). За ним клієнт показує залишок трафіку і термін дії.

Маршрутизація (лише для клієнта Haproxy)

Поле (UI)	Ключ	За замовчуванням	Опис
Увімкнути маршрутизацію	subEnableRouting	false	«Глобальне налаштування для увімкнення маршрутизації у VPN-клієнті. (Лише для Haproxy)». Передається в заголовку <code>Routing-Enable</code> .
Правила маршрутизації	subRoutingRules	порожньо	«Глобальні правила маршрутизації для VPN-клієнта. (Лише для Haproxy)». Передаються в заголовку <code>Routing</code> .

| Приховати налаштування сервера | `subHideSettings` | false | «Приховувати налаштування сервера в підписці (лише для Haproxy)». Коли увімкнено, у клієнті Haproxy приховується можливість переглядати і змінювати параметри сервера. Опція діє лише для клієнта Haproxy.

Маршрутизація Incy (лише для клієнта Incy)

Для VPN-клієнта **Incy** в налаштуваннях підписки є окрема вкладка **«Incy»** з двома полями: перемикач **«Увімкнути маршрутизацію»** (`subIncyEnableRouting`, за замовчуванням вимкнено) і текстове поле **«Правила маршрутизації»** (`subIncyRoutingRules`) формату `incy://routing/onadd/<base64>`. Коли маршрутизацію увімкнено і поле заповнено, цей рядок дописується окремим рядком до тіла підписки (формат raw) — так профіль маршрутизації доставляється клієнту Incy, не конфліктуючи із заголовком `Routing` клієнта Haproxy. Налаштування діють лише для клієнта Incy.

URI зворотного проксі

Поле (UI)	Ключ	За замовчуванням	Опис
URI зворотного проксі	subURI	порожньо	«Змінити базовий URI URL-адреси підписки для використання за проксі-серверами».

Якщо поле порожнє, базову адресу посилання панель будує сама з домену і порту підписки (з урахуванням TLS). Якщо ж підписка роздається через зовнішній реверс-проксі/CDN на іншому домені або шляху, в це поле задають підсумковий базовий URI, і всі посилання будуватимуться від нього. Аналогічні окремі поля є для JSON (`subJsonURI`) і Clash (`subClashURI`).

Якщо задано лише загальний `subURI`, а окремі поля для JSON і Clash залишені порожніми, посилання цих форматів на сторінці підписки успадковують схему і хост із `subURI` (а не порт sub-сервера і `http`) – так вони збігаються з адресою зворотного проксі.

Приклад: підписка за реверс-проксі. Сама підписка слухає на `2096`, але зовні доступна через nginx/CDN на `https://cfg.example.com/u/`. Щоб посилання у відповіді будувалися від зовнішньої адреси, а не від внутрішньої `домен:2096`, в поле «Reverse proxy URI» задають підсумковий базовий URI:

Reverse proxy URI: `https://cfg.example.com/u`

Тоді кінцеве посилання матиме вигляд `https://cfg.example.com/u/ivan2025`. Для JSON- і Clash-форматів при необхідності заповнюють окремі поля `subJsonURI` і `subClashURI` тим самим способом.

10.3. Формати виведення

Підписка може видаватися в трьох незалежних форматах, кожен зі своїм ендпоінтом, який можна вмикати/вимикати окремо.

Адреса сервера та вузли у видачі

Адреса сервера в посиланнях підписки підставляється за тією самою стратегією адреси в посиланні, що й звичайні посилання і QR-коди в панелі: «listen» – маршрутизована адреса прив'язки, «custom» – задана користувацька адреса (`shareAddr`), «node» (за замовчуванням) – адреса вузла. Для inbound без явно заданої стратегії видача підписки не змінюється. Це дозволяє вузловому inbound, прив'язаному до конкретного публічного IP, видавати клієнтам досяжну адресу. Стратегія застосовується до сирого, JSON і Clash форматів.

Ім'я вузла (Node) не додається до назви (remark) профілю в підписці: у клієнтському застосунку показується лише ремарка inbound, задана адміністратором, без внутрішнього суфікса виду `@ім'я-вузла`. Щоб розрізнити однойменні записи в мультивузловій підписці, задавайте їм різні ремарки вручну або використовуйте керовані хости (Hosts) з власними Remark.

Якщо через розсинхронізацію між вузлами один і той самий клієнт потрапив у службовий JSON inbound двічі, видача підписки автоматично усуває такі дублікати за email у всіх трьох форматах, тому повторювані профілі у видачі не з'являються.

Керовані хости (Hosts)

Розділ **Hosts** (пункт бокового меню; зведена сторінка з кількістю Total/Enabled/Disabled і списком) задає перевизначення адреси для посилань підписки. Для кожного inbound можна додати один або кілька **хостів** – ендпоінтів, які підставляються у видавані клієнту subscription-посилання **замість адреси, порту і TLS-параметрів самого inbound**. Це зручно, щоб роздавати трафік через CDN або релей, не змінюючи сам inbound.

З версії 3.5.0 хост – це «група»: один запис охоплює **кілька inbound і кілька адрес** одразу. Дії списку (увімкнення/вимкнення/видалення/перестановка) та API працюють із групами (рядковий `groupId`);

з'явився масовий ендпоінт `POST /panel/api/hosts/bulk/add ({"inboundIds": [...], "hosts": [...], ...})`, а `list / get / update / del / setEnable` оперують об'єктами групи.

У кожного хоста (групи) задаються:

- **Remark** і опис (Description), прив'язка до **Inbounds** (мульти-вибір з пошуком; мінімум один), перемикач **Enable** і призначення на вузли (**Nodes**).
- **Address** — з 3.5.0 це **список адрес** (введення тегами; розділювачі — кома, крапка з комою, пробіл; плейсхолдер `cdn.example.com, cdn2.example.com:443`). У кожного запису може бути свій вбудований `:порт` (зокрема IPv6 у дужках `::1:443`); випадająca підказка пропонує адреси, вже використані іншими хостами. Порожній список — успадковується адреса самого inbound (у списку це помаранчева мітка **Inherits**). **Port** (0 — успадковується порт inbound) служить портом за замовчуванням для записів без вбудованого порту; **Tags** (враховуються лише в RAW-підписці).
- Вкладка **Security** — `same / tls / none / reality` з SN1, відбитком (fingerprint), ALPN, прив'язкою сертифіката (pinned-cert), `allowInsecure` і ECH.
- Вкладка **Advanced** — Host header, Path, маршрут VLESS, Mux, Sockopt, Final Mask і виключення хоста з окремих форматів підписки (raw / json / clash). З 3.5.0 **Final Mask хоста потрапляє і в raw-посилання** (`fm=`; раніше — лише в JSON/Clash): TCP/UDP-маски хоста додаються до масок inbound, QUIC-параметри хоста беруться, лише якщо їх немає в inbound. Перемикач **Allow insecure** тепер діє і для **Hysteria/Hysteria2** (`insecure=1` у посиланні, `skip-cert-verify: true` у Clash).
- Вкладка **Clash (mihomo)** — версія IP, Mihomo X25519, перемішування хостів (Shuffle host).

У списку стовпець **Endpoint** показує адреси чіпами (перший видно, решта — у поповері «+N»), а стовпець **Inbounds** — чіпи inbound, розфарбовані за протоколом. З 3.5.0 хости впорядковуються **глобально** (за порядком сортування, потім за приміткою), а не в межах inbound; масові вмикання, вимикання та видалення зберігаються. Керовані хости замінюють попередній масив External Proxy.

Маршрут VLESS (VLESS route). З версії 3.4.2 це одне число `0-65535` (а не список портів; підказка — «одне значення VLESS route (0-65535), що зашивається в UUID, наприклад 443; порожньо — без нього», плейсхолдер `443`). Задане значення реально «зашивається» в UUID кожної згенерованої підписки (raw / JSON / Clash): Xray читає байти 6-7 UUID і маскує їх перед автентифікацією, тому клієнт як і раніше збігається. Порожнє або некоректне значення UUID не змінює.

Звичайні посилання (SUB) — Base64 / plain text

Базовий формат, ендпоінт `subPath` (за замовчуванням `/sub/`). Увімкнено завжди (при увімкненій підписці загалом). Повертає список посилань Xray (`vless://`, `vmess://`, `trojan://`, `ss://` тощо) — по одному на рядок. При увімкненій опції «Кодувати» (`subEncrypt`) весь список кодується в Base64; при вимкненій — видається відкритим текстом. Цей формат розуміють практично всі клієнти (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Hapn та ін.).

Приклад: тіло відповіді з вимкненим «Кодувати». При `subEncrypt = false` ендпоінт `/sub/` видає відкритий текст — по одному посиланню на рядок:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```


При `subEncrypt = true` (за замовчуванням) той самий список цілком кодується в Base64 і видається одним рядком — саме такий вигляд очікує більшість клієнтів.

JSON-підписка (sing-box і сумісні)

Ендпоінт `subJsonPath` (за замовчуванням `/json/`), вмикається окремим прапорцем.

Поле (UI)	Ключ	За замовчуванням	Опис
JSON-підписка	<code>subJsonEnable</code>	<code>false</code>	«Увімкнути/вимкнути JSON-ендпоінт підписки незалежно.».

Повертає повну JSON-конфігурацію (формат, зрозумілий sing-box і похідним клієнтам — Podkor, OpenWRT sing-box, Karing, NekoBox). З 3.5.0 у JSON-підписку потрапляють і клієнти **нативних WireGuard-inbound** (`secretKey`, `address`, `peers[]` з `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs`, `mtu`) — раніше вони мовчки пропускалися. Для цього формату доступні додаткові параметри (вкладка `subFormats`):

- **Mux** (`subJsonMux`, за замовчуванням порожньо) — JSON-налаштування мультиплексування (Mux), які впроваджуються в outbound кожного потоку JSON-підписки. «Передача кількох незалежних потоків даних в одному з'єднанні.».
- **Final Mask** (`subJsonFinalMask`, за замовчуванням порожньо) — «Маски finalmask xray (TCP/UDP) і налаштування QUIC, що додаються до кожного потоку JSON-підписки. Потрібна свіжа версія xray на клієнті.». Налаштовується через підполя: «Пакети» (`packets`), «Довжина» (`length`), «Інтервал» (`interval`), «Макс. розбиття» (`maxSplit`), «Шуми» (`noises`: «Тип»/ `type`, «Пакет»/ `packet`, «Затримка (мс)»/ `delayMs`, «Застосувати до»/ `applyTo`, кнопка «+ Шум»), а також «Паралелізм» (`concurrency`), «Паралелізм худр» (`hudrConcurrency`) і «худр UDP 443» (`hudrUdp443`).
- **Правила маршрутизації** (`subJsonRules`, за замовчуванням порожньо) — глобальні правила, що додаються в JSON-конфігурацію.

Clash / Mihomo-підписка (YAML)

Ендпоінт `subClashPath` (за замовчуванням `/clash/`), вмикається окремим прапорцем.

Поле (UI)	Ключ	За замовчуванням	Опис
Підписка Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Вмикає генерацію YAML-конфігурації для клієнтів Clash і Mihomo.
Увімкнути маршрутизацію	<code>subClashEnableRouting</code>	<code>false</code>	«Додавати глобальні правила маршрутизації Clash/Mihomo до згенерованих YAML-підписок.».
Глобальні правила маршрутизації	<code>subClashRules</code>	порожньо	«Правила Clash/Mihomo, що додаються на початок кожної YAML-підписки перед MATCH,PROXY.».

Відповідь видається з типом `application/yaml; charset=utf-8`. Якщо задано «Заголовок підписки» (`subTitle`), він передається також у заголовку `Content-Disposition (attachment; filename*=UTF-8'<title>)`, щоб Clash-клієнт назвав імпортований профіль цим іменем.

Формат генерованих посилань і YAML підтримується в актуальному стані для сучасних клієнтів: Shadowsocks-2022 (SS2022) більше не кодує userinfo в Base64; посилання Shadowsocks з http-обфускацією видаються у форматі SIP002 з плагіном `obfs-local`; для підписок Clash/Mihomo реалізовано повний набір полів XHTTP. З 3.5.0 у Clash/Mihomo-підписку потрапляють і клієнти **нативних WireGuard-inbound** (поля `private-key`, `public-key`, `pre-shared-key`, `persistent-keepalive`, `ip/ipv6`, `mtu`, `dns`) – раніше вони мовчки пропускалися. Окремих налаштувань це не потребує – посилання просто коректніше розпізнаються клієнтами.

Примітка: у цій збірці підтримуються саме три формати – звичайні посилання (Base64/текст), JSON (sing-box-сумісний) і Clash/Mihomo (YAML). Окремого формату Outline в сервері підписок немає.

10.4. Сторінка інформації про підписку та QR-коди

Якщо відкрити **будь-яке з трьох посилань підписки** – raw, JSON або Clash – у браузері (або явно додати до URL параметр `?html=1` або `?view=html`, або надіслати заголовок `Accept: text/html`), сервер замість «сирої» відповіді видає візуальну **сторінку інформації про підписку** («Інформація про підписку»). До 3.5.0 так працювало лише основне посилання `/sub/`, а `/json/` і `/clash/` віддавали в браузер сирий JSON/YAML. VPN-клієнти як і раніше отримують машинну відповідь, оскільки вони не запитують HTML.

Сторінка (односторінковий застосунок, зібраний Vite) показує:

- **Інформацію про підписку** (блок Descriptions):
 - «ID підписки» – значення `subId`;
 - «Статус» – «Активна», «Неактивна» або «Необмежено». Статус «неактивна» виставляється, якщо клієнт вимкнено, вичерпано ліміт трафіку або закінчився термін дії;
 - «Завантажено» і «Відправлено» – обсяги трафіку;
 - «Загальний ліміт» – ліміт трафіку або ∞ , якщо не обмежено;
 - «Термін дії» – дата закінчення або «Безстроково»;
 - залишок трафіку і час останнього онлайну.
 - Дати відображаються за григоріанським або джалалі-календарем залежно від налаштування «Calendar Type» панелі (`datepicker`, за замовчуванням `gregorian`).
- **Посилання на підписку**: для кожного увімкненого формату – окремий рядок з кольоровим тегом (зелений **SUB**, фіолетовий **JSON**, золотий **CLASH**), кнопкою копіювання і кнопкою **QR-коду** (спливаюче вікно, розмір 240 px). Рядок з JSON і CLASH з'являється лише якщо відповідний формат увімкнено в налаштуваннях.
- **Індивідуальні посилання** («Скопіювати посилання»): повний список окремих конфігурацій, що входять до підписки, кожна зі своїм тегом протоколу, кнопкою копіювання і QR-кодом (для post-quantum-посилань QR не будується).
- **Кнопка «Копіювати всі конфігурації»** (над списком індивідуальних посилань): одним натисканням копіює в буфер обміну одразу всі посилання конфігурацій (кожне з нового рядка), не потребуючи копіювати їх по одному; після завершення показується повідомлення «Усі конфігурації скопійовано».

- **Кнопки швидкого імпорту в застосунки** (випадаючі меню за платформами): для Android – v2box, v2rayNG (deep-link `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`), Incy (`incy://add/...`); для iOS – Shadowrocket (через параметр `flag=shadowrocket`), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ, Incy. Ці кнопки або відкривають deep-link потрібного застосунку з уже підставленою адресою підписки, або копіюють посилання в буфер обміну.

Сторінка інформації видається із заголовками заборони кешування (`Cache-Control: no-cache`), щоб клієнт завжди бачив актуальні дані про трафік і термін дії.

10.5. Кастомні шаблони сторінки підписки

Починаючи з 3.3.0 можна замінити стандартну сторінку-лендинг підписки власним HTML-шаблоном. За замовчуванням за адресою підписки видається вбудована сторінка, але якщо вказати каталог зі своїм шаблоном, панель буде рендерити його і підставляти в нього актуальні дані клієнта (трафік, термін дії, посилання тощо).

Важливо: панель **не постачає** готових шаблонів – свою тему потрібно створити самостійно. Інструкція зі створення та список доступних змінних – у [docs/custom-subscription-templates.md](https://docs.v2ray.com/custom-subscription-templates.md).

Де вмикається

Каталог теми задається в налаштуваннях панелі:

Налаштування → Підписка → розділ «Інформація про підписку», поле **«Каталог теми підписки»** (`subThemeDir`).

Опис поля в інтерфейсі: «Абсолютний шлях до папки з користувацьким шаблоном (`index.html/sub.html`) для сторінки підписки (наприклад, `/etc/3x-ui/sub_templates/my-theme/`). Залиште порожнім, щоб використовувати сторінку за замовчуванням.»

Поруч у тому самому розділі розташовані суміжні налаштування, значення яких доступні в шаблоні:

В описі поля «Каталог теми підписки» є посилання **«Посібник із шаблонів ↗»** на документацію зі створення власних шаблонів оформлення сторінки підписки.

- **«Заголовок підписки»** (`subTitle`) – назва, видима клієнту;
- **«URL підтримки»** (`subSupportUrl`) – посилання на техпідтримку.

Параметр налаштування

Параметр	Значення за замовчуванням	Призначення
<code>subThemeDir</code>	<code>""</code> (порожньо)	Абсолютний шлях до каталогу з вашим HTML-шаблоном. Порожньо = вбудована сторінка за замовчуванням.

Як підставити свій шаблон

1. Створіть на сервері папку для теми (де завгодно), наприклад `/etc/3x-ui/sub_templates/my-theme/`.
2. Покладіть усередину HTML-файл з іменем `index.html` або `sub.html`.

Приклад: шлях до теми. Фінальне розташування на сервері і значення поля в налаштуваннях:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (або sub.html – він має пріоритет)
```

Налаштування → Підписка → «Каталог теми підписки»:
`/etc/3x-ui/sub_templates/my-theme/`

Шлях має бути **абсолютним** (починатися з `/`). Якщо в папці немає ні `index.html`, ні `sub.html`, панель видасть вбудовану сторінку. 3. У панелі відкрийте **Налаштування** → **Підписка** і впишіть **абсолютний** шлях до цієї папки в поле «Каталог теми підписки». 4. Збережіть налаштування.

Поведінка вибору файлу і рендерингу:

- Якщо в каталозі є `sub.html`, використовується саме він; інакше береться `index.html`. Тобто `sub.html` має пріоритет над `index.html`.
- Шаблон рендериться стандартним рушієм `Go html/template`.
- Розібраний шаблон **кешується** і перечитується з диска лише при зміні часу модифікації файлу. Тому правки шаблону підхоплюються без перезапуску панелі, але без накладних витрат на читання/парсинг при кожному запиті.
- Відповідь формується в буфер цілком і лише потім видається клієнту: якщо шаблон впаде під час виконання, частково згенерована (зіпсована) сторінка не дійде до користувача.

Поведінка за замовчуванням і відкат (fallback)

- Поле порожнє → видається вбудована SPA-сторінка (дані впроваджуються в `window.__SUB_PAGE_DATA__`).
- Шлях не існує або це не каталог → використовується сторінка за замовчуванням.
- У каталозі немає ні `index.html`, ні `sub.html` → у лог записується попередження «subThemeDir set but no usable template found», видається сторінка за замовчуванням.
- Файл шаблону є, але не парситься → у лог записується помилка «custom template parse failed», видається сторінка за замовчуванням.
- Помилка при виконанні шаблону → у лог записується «custom template execution failed», видається сторінка за замовчуванням.

Тобто будь-яка проблема з кастомним шаблоном не «ламає» підписку — панель завжди деградує до вбудованої сторінки. Всі сторінки підписки (і кастомна, і стандартна) видаються із заголовками заборони кешування (`Cache-Control: no-cache, no-store, must-revalidate`), щоб клієнти завжди отримували свіжі дані про трафік і термін.

Доступні змінні шаблону

До контексту шаблону передається набір даних клієнта підписки. Звернення — через `{{ .ім'я }}`:

Змінна	Тип	Опис
<code>{{ .sId }}</code>	string	ID підписки (UUID).
<code>{{ .enabled }}</code>	bool	Чи увімкнено клієнта/підписку.
<code>{{ .download }}</code>	string	Форматований обсяг завантаження (напр. «2.5 GB»).
<code>{{ .upload }}</code>	string	Форматований обсяг відправки.
<code>{{ .total }}</code>	string	Форматований загальний ліміт трафіку.
<code>{{ .used }}</code>	string	Форматований використаний трафік (download + upload).
<code>{{ .remained }}</code>	string	Форматований залишок трафіку.
<code>{{ .expire }}</code>	int64	Термін дії — Unix-час у секундах (<code>0</code> = безстроково). Для JS <code>Date</code> помножте на 1000.
<code>{{ .lastOnline }}</code>	int64	Час останнього онлайну — Unix-час у мілісекундах (<code>0</code> = жодного разу).
<code>{{ .downloadByte }}</code>	int64	Завантаження в точних байтах.
<code>{{ .uploadByte }}</code>	int64	Відправка в точних байтах.
<code>{{ .totalByte }}</code>	int64	Загальний ліміт у точних байтах.
<code>{{ .subUrl }}</code>	string	URL сторінки підписки.
<code>{{ .subJsonUrl }}</code>	string	URL JSON-конфігурації підписки.
<code>{{ .subClashUrl }}</code>	string	URL конфігурації Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Заголовок підписки з налаштувань (може бути порожнім).
<code>{{ .subSupportUrl }}</code>	string	URL підтримки з налаштувань (може бути порожнім).
<code>{{ .links }}</code>	[]string	Список рядків-конфігів (VMess, VLESS тощо). Перебір: <code>{{ range .links }}</code> ... <code>{{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Список email, що належать до підписки.
<code>{{ .datepicker }}</code>	string	Поточний формат календаря панелі: <code>gregorian</code> або <code>jalali</code> (береться з налаштування «Тип календаря»; якщо порожньо — <code>gregorian</code>).

Мінімальний приклад тіла шаблону, що використовує частину змінних:

```
<h1>{{ .subTitle }}</h1>
<p>Використано: {{ .used }} із {{ .total }} (залишилося {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Приклад: дата закінчення з `expire`. Поле `{{ .expire }}` — це Unix-час у **секундах**, тому для JavaScript його множать на 1000; значення `0` означає «без терміну»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Без терміну'
    : 'До ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Зверніть увагу: `{{ .lastOnline }}` задається вже в **мілісекундах** — його множити на 1000 не потрібно.

11. Xray: маршрутизація, outbounds, DNS та розширення

Розділ «**Налаштування Xray**» — це редактор шаблону конфігурації Xray-core, на основі якого панель генерує підсумковий `config.json` для запуску ядра. Підказка до розділу шаблону: «*На основі шаблону створюється конфігураційний файл Xray.*» На відміну від inbounds (які зберігаються окремо в БД і підставляються до шаблону при збиранні конфігурації), все інше — логи, маршрутизація, outbounds, DNS, політика, статистика — задається саме тут.

Важливо: значення шаблону зберігається в БД під ключем `xrayTemplateConfig`. При збереженні панель пропускає його через ряд автоматичних перетворень (див. 11.11). Будь-який синтаксично некоректний JSON буде відхилений з помилкою «*xray template config invalid*».

Розташування в меню: «Вихідні» та «Маршрутизація»

«Вихідні» (Outbounds) та «Маршрутизація»

(Routing) — це окремі пункти бічного меню (відразу під «Хости», над «Налаштування панелі»), у кожного своя адреса: `/outbound` та `/routing`. Прямі посилання на ці сторінки та перезавантаження сторінки працюють як очікується. У підменю «**Конфігурації Xray**» при цьому залишаються лише: Основне, Балансувальник, DNS та Розширений шаблон. У описі нижче розділи 11.3 та 11.4 відповідають сторінкам «Маршрутизація» та «Вихідні».

11.1. Структура редактора: вкладки/режими

Редактор пропонує кілька режимів відображення шаблону (фільтри по розділах JSON):

Режим	Що показує
Основне	Базові секції (Лог, базова маршрутизація, основні налаштування)
Розширений шаблон	Повний JSON-шаблон Xray
Всі	Всі секції одночасно

Логічні групи налаштувань усередині редактора:

- **Основні налаштування** (підказка: «*Ці параметри описують загальні налаштування*»).
- **Лог** (див. 11.10).
- **Базові з'єднання**: блокування та прямі маршрути.
- **Вхідні** (підказка: «*Зміна шаблону конфігурації для підключення певних клієнтів*»).
- **Вихідні** (див. 11.4).
- **Балансувальник** (див. 11.5).
- **Маршрутизація** (підказка: «*Важливий пріоритет кожного правила!*», див. 11.3).
- **DNS / Fake DNS** (див. 11.6).

11.2. Основні налаштування (General)

Freedom Protocol Strategy

Поле	Підпис	Опис	За замовчуванням
FreedomStrategy	Налаштування стратегії протоколу Freedom	Стратегія виведення мережі для прямого (freedom) outbound. Підказка: «Встановлення стратегії виведення мережі в протоколі Freedom». Керує полем domainStrategy всередині settings outbound-а з протоколом freedom.	В еталонному шаблоні domainStrategy для freedom-outbound direct дорівнює AsIs (адреса не резолвиться, передається у вихідному вигляді).

domainStrategy для freedom (значення Xray-core): AsIs (не резолвити домен на стороні сервера), а також сімейство UseIP / UseIPv4 / UseIPv6 та їх «примусові» варіанти ForceIP*, які змушують вихідний сервер резолвити домен і підключатися за отриманим IP. Змінійте на UseIPv4, якщо на вихідному сервері немає IPv6 або потрібно примусово ходити лише по IPv4.

Freedom Happy Eyeballs (IPv4/IPv6)

Поле	Підпис	Опис
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Підказка: «Двостековий набір для прямого (freedom) вихідного – корисно на вихідних серверах з IPv4 та IPv6.» Вмикає алгоритм Happy Eyeballs (одночасна спроба по обох сімействах адрес) для freedom-outbound.

Окремий Happy Eyeballs є і в налаштуваннях dial (sockopt) конкретного outbound: з 3.5.0 він справді вмикається (раніше конфіг серіалізувався як вимкнений і перемикач «злітав»). «Try delay (ms)» за замовчуванням тепер 250; явно заданий 0 (= вимкнено) зберігається; також доступні Prioritize IPv6, interleave (1) і maxConcurrentTry (4). | try delay | (підказка) | «Мілісекунди перед спробою іншого сімейства адрес. 150–250 мс – гарна відправна точка.» Затримка перед переключенням на альтернативне сімейство адрес. Рекомендований діапазон – 150–250 мс. |

Overall Routing Strategy

Поле	Підпис	Опис	За замовчуванням
RoutingStrategy	Налаштування маршрутизації доменів	Загальна стратегія вирішення DNS для маршрутизації. Підказка: «Встановлення загальної стратегії маршрутизації вирішення DNS». Керує полем routing.domainStrategy.	В еталонному шаблоні routing.domainStrategy = AsIs.

routing.domainStrategy визначає, як зіставляються IP-правила маршрутизації з доменними запитамі: AsIs (лише доменні правила, без резолву), IPIfNonMatch (якщо домен не збігся з правилами – резолвити та перевірити IP-правила), IPOnDemand (резолвити відразу при зустрічі IP-правила). Для роботи IP-правил (наприклад, geoip:*) при доменному запиті зазвичай потрібен IPIfNonMatch.

Outbound Test URL

Поле	Підпис	Опис	За замовчуванням
outboundTestUrl	URL для тесту вихідного	URL для перевірки зв'язності при тестуванні outbound. Підказка: «URL для перевірки підключення вихідного». Зберігається окремо від шаблону, під ключем xrayOutboundTestUrl.	https:// www.google.com/ generate_204

Значення проходить санітизацію. При самому тесті outbound воно додатково перевіряється як публічний URL — це захист від SSRF: користувач не може підсунути довільний (зокрема внутрішній) URL через клієнта, тестовий URL завжди береться з серверного налаштування. Пусте значення при збереженні/тесті замінюється на типовий generate_204.

Default Outbound (з 3.5.0)

Перший пункт вкладки «Базова маршрутизація» — селектор **Default Outbound**: «Трафік, що не збігся з жодним правилом маршрутизації, використовує цей outbound (Xray бере перший outbound у списку)». У списку — direct, blocked і всі наявні теги; за замовчуванням direct. Вибір **переміщує вказаний outbound на першу позицію** шаблону; відсутні direct / blocked створюються на льоту, а при переключенні не видаляються.

Block BitTorrent

Поле	Підпис	Опис
Torrent	Заблокувати BitTorrent	Додає до routing.rules правило, що відправляє трафік з protocol: ["bittorrent"] на outbound blocked. В еталонному шаблоні це правило присутнє за замовчуванням.

Обмеження з'єднання (Connection Limits)

Підказка: «Політики рівня з'єднання для користувачів рівня 0. Залиште поле порожнім, щоб використовувати значення Xray за замовчуванням.» Ці параметри записуються в policy.levels.0.

Поле	Підпис	Опис	За замовчуванням
connIdle	Тайм-аут простою (секунд)	«Закриває з'єднання після простою протягом зазначеної кількості секунд. Зменшення значення швидше звільняє пам'ять і файлові дескриптори на навантажених серверах (за замовчуванням у Xray: 300).»	порожньо → типове Xray 300
bufferSize	Розмір буфера (КБ)	«Розмір внутрішнього буфера на з'єднання в КБ. Встановіть 0, щоб мінімізувати використання пам'яті на серверах з малим обсягом ОЗП (значення Xray за замовчуванням залежить від платформи).» Плейсхолдер: «авто».	порожньо → залежить від платформи; 0 — мінімізувати

Приклад (policy.levels.0). Поля з цієї групи записуються в політику рівня 0. На навантаженому сервері з малим ОЗП можна прискорити звільнення ресурсів так:

```

"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}

```

Тут з'єднання закривається вже після 120 с простою (замість типових 300), а `bufferSize: 0` мінімізує витрату пам'яті на буфери. Залишене порожнім у формі поле просто не потрапить до JSON — і Xray застосує своє значення за замовчуванням.

11.3. Правила маршрутизації (routing)

Список правил `routing.rules`. **Порядок критичний** («Важливий пріоритет кожного правила!»): правила оцінюються зверху вниз, спрацьовує перше збіглося. Підказка: «Перетягніть для зміни порядку». Кнопки керування порядком: **Перший**, **Останній**, **Підняти вгору**, **Опустити вниз**.

Кожне правило має `type: "field"`. Кнопки: **Створити правило**, **Редагувати правило**. Підказка до полів-списків: «Елементи, розділені комами».

На сторінці «Маршрутизація» кнопки **Імпорт правил** та **Експорт правил** зібрані у випадаючому меню **«ще»** (more) — як і на сторінці «Вихідні». Кнопка **Експорт правил** не завантажує файл одразу, а відкриває модальне вікно з попереднім переглядом JSON і кнопками **Копіювати** та **Завантажити**: вміст можна переглянути перед збереженням. Аналогічно працює експорт вихідних на сторінці «Вихідні».

Route Tester (тестувальник маршруту)

На вкладці Routing є підкладка **Route Tester** — вона запитує у працюючого Xray, який outbound обробив би конкретне з'єднання, не надсилаючи реального трафіку. Вкажіть домен або IP, порт, мережу (TCP/UDP) та, за потреби, inbound і перехоплений протокол (http / tls / quic / bittorrent), потім натисніть **Test Route**. Рішення береться безпосередньо з живого рушія маршрутизації.

У відповіді показується підібраний outbound, а при використанні балансувальника — ще й тег балансувальника. Якщо жодне правило не збіглося, тестувальник повідомляє, що трафік іде в outbound за замовчуванням (перший у списку `outbounds`). Це зручно для перевірки порядку правил перед тим, як покладатися на них.

Увімкнення та вимкнення окремого правила

Окреме правило маршрутизації можна тимчасово **вимкнути** перемикачем, не видаляючи його. У таблиці правил є стовпець **«Увімкнути»** з перемикачем (Switch), а у формі правила поле **«Увімкнути»** — також перемикач. Вимкнене правило не потрапляє до підсумкової конфігурації Xray, але зберігається в шаблоні і його можна знову увімкнути у будь-який момент.

Службове правило статистики (`inboundTag: ["api"] → outboundTag: "api"`) вимкнути не можна — його перемикач заблоковано, щоб не зламати облік трафіку панелі (див. 11.11).

Поля форми правила

Поле форми	Підпис	JSON-поле	Опис
Джерело	Джерело	<code>source</code>	IP-адреси/підмережі джерела. Список через кому.
Порт джерела	Порт джерела	<code>sourcePort</code>	Порт(и) джерела.
Пункт призначення	Пункт призначення	<code>domain + ip + port</code>	Цільові домени, IP та порти. Домени підтримують префікси <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , а також <code>geosite:*</code> ; IP – <code>geoip:*</code> та CIDR.
Мережа	—	<code>network</code>	<code>tcp</code> , <code>udp</code> або <code>tcp,udp</code> . 3.3.4.2 в конфіг записується малими літерами (<code>tcp</code> / <code>udp</code>), а в таблиці маршрутів відображається великими (<code>TCP</code> / <code>UDP</code>); порожньо — будь-яка мережа.
Протокол	—	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (визначається по sniffing).
Користувач	Користувач	<code>user</code>	Фільтр за e-mail/ідентифікатором користувача.
Атрибути / Значення	Атрибути / Значення	<code>attrs</code>	Атрибути HTTP-заголовків для зіставлення.
VLESS route	VLESS route	—	Маршрутизація за полем route для VLESS.
Теги вхідних	Теги вхідних	<code>inboundTag</code>	Один або кілька тегів inbound, до яких застосовується правило (включно з вбудованими <code>api</code> та DNS-тегом з налаштувань DNS). У списках inbound відображається як «tag (remark)», якщо у inbound задано окреме примітку, інакше — просто тег; у збережених правилах як і раніше зберігаються лише теги.
Тег вихідного	Тег вихідного / Вихідне підключення	<code>outboundTag</code>	Куди направити трафік, що збігся.
Тег балансувальника	Тег балансувальника / Балансувальник	<code>balancerTag</code>	Підказка: «Направляє трафік через один з налаштованих балансувальників навантаження».

Взаємовиключення `outboundTag` та `balancerTag`: «Неможливо одночасно використовувати `balancerTag` та `outboundTag`. При одночасному використанні буде працювати тільки `outboundTag`.» В одному правилі задавайте або тег вихідного, або тег балансувальника.

Вбудовані правила еталонного шаблону

У стандартному `config.json` секція `routing` містить три правила (у такому порядку):

1. `inboundTag: ["api"]` → `outboundTag: "api"` — службове правило для gRPC-API статистики панелі.
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` — блокування приватних діапазонів.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` — блокування BitTorrent.

Правило `ari` → `ari` завжди автоматично піднімається на позицію 0 при збереженні (див. [11.11](#)), щоб запит статистики не «з'їло» вищестояще catch-all-правило.

Приклад правила. Відправити весь трафік до російських сайтів та приватних мереж напряму (минаючи проксі), а решту — на балансувальник. Порядок важливий: правило «направити напряму» повинно стояти вище catch-all. У `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Щоб IP-правила (`geoip:ru`) спрацьовували і для доменних запитів, на верхньому рівні маршрутизації зазвичай потрібен `routing.domainStrategy: "IPIfNonMatch"` (див. [11.2](#)).

Попередньо налаштовані групи маршрутизації (Базові з'єднання)

У режимі «Базові з'єднання» панель допомагає зібрати типові правила з готових списків:

Група	Поля	Підказка
Блокування за протоколами/сайтами	—	«Налаштуйте, щоб клієнти не мали доступу до певних протоколів»
Блокування за країнами	Заблоковані IP-адреси, Заблоковані домени	«Ці параметри блокуватимуть трафік залежно від країни призначення.»
Прямі з'єднання	Прямі IP-адреси, Прямі домени	«Пряме з'єднання означає, що певний трафік не буде перенаправлений через інший сервер.»
Правила IPv4	—	«Ці параметри дозволять клієнтам маршрутизуватися до цільових domenів лише через IPv4»
Правила WARP	—	«Ці опції направлятимуть трафік залежно від конкретного пункту призначення через WARP.»
Маршрутизація NordVPN	—	«Ці опції направлятимуть трафік залежно від конкретного пункту призначення через NordVPN.»

MTProto-inbound: маршрутизація Telegram-трафіку через Xray

У MTProto-inbound є перемикач **«Route through Xray»** (за замовчуванням вимкнено) та необов'язковий вибір **Outbound**. При вмиканні панель додає до конфігу Xray петльовий SOCKS-міст з тегом самого inbound, і mtg направляє Telegram-трафік через нього. Після цього вихідним Telegram-трафіком керує маршрутизатор: його можна зіставляти звичайними правилами на вкладці Routing за тегом inbound або примусово направити у вибраний outbound чи балансувальник через поле **Outbound**. Залиште **Outbound** порожнім, щоб рішення приймали правила маршрутизації.

11.4. Outbounds (вихідні підключення)

Список `outbounds`. Кнопки: **Створити вихідне підключення**, **Змінити вихідне підключення**. Підказка: «Зміна шаблону конфігурації, щоб визначити вихідні підключення для цього сервера».

В еталонному шаблоні два обов'язкових `outbound`-и:

- `protocol: "freedom"`, `tag: "direct"` — прямий вихід в інтернет (з `domainStrategy: "AsIs"` та `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole"`, `tag: "blocked"` — «чорна діра» для заблокованого трафіку.

Загальні поля форми `outbound`

Поле	Підпис	Опис
Тег	Тег (підказка: «Унікальний тег»)	Унікальний ідентифікатор <code>outbound</code> . Плейсхолдер: «унікальний-тег». Валідація: «Тег обов'язковий», «Тег вже використовується іншим вихідним».
Протокол	—	Тип вихідного (див. нижче).
Адреса / Порт	Адреса / Порт	Ціль підключення. Адреса та порт обов'язкові.
Надіслати через	Надіслати через	Локальна IP-адреса вихідного інтерфейсу (<code>sendThrough</code>). Плейсхолдер: «локальний IP».
Target Strategy	Target Strategy	Нове в 3.5.0. Як резолвиться домен призначення перед підключенням. 11 значень: <code>AsIs</code> (за замовчуванням, не резолвити), <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> , <code>UseIPv6v4</code> , <code>UseIPv4v6</code> (резолв із fallback), <code>ForceIP</code> , <code>ForceIPv6v4</code> , <code>ForceIPv6</code> , <code>ForceIPv4v6</code> , <code>ForceIPv4</code> (вимагають успішного резолву). Порожньо = <code>AsIs</code> (у конфіг не пишеться); для <code>freedom</code> значення читається з fallback на попередній <code>domainStrategy</code> (легасі-ключ як і раніше записується для старих ядер).
Dialer proxy (ланцюжок)	—	Підказка: «Підключайте цей вихідний через інший вихідний (за тегом), щоб побудувати ланцюжок проксі. Залиште порожнім для прямого підключення.» Плейсхолдер: «Виберіть вихідний для ланцюжка». Реалізується через <code>streamSettings.sockopt.dialerProxy</code> .

Випадаючий список **Dialer Proxy** показує не лише локальні `outbounds`, але й теги `outbounds` з підписок — так ланцюжок можна побудувати і через вихід, отриманий по підписці. Зі списку як і раніше виключені `blackhole-outbound` та сам редагований `outbound`. Залиште поле порожнім для прямого підключення.

Підтримувані протоколи `outbound`

Підтримувані формою протоколи:

- **freedom** — прямий вихід. Поля `settings.domainStrategy`, `finalRules` (див. нижче), Happy Eyeballs. Тестуванню не підлягає («*Outbound has no testable endpoint*»).
- **blackhole** — відкидає трафік. Поле **Тип відповіді**. Не тестується.
- **socks**, **http** — список `settings.servers[]` з `address / port`; поле **Пароль авторизації**. Для протоколу **http** нижче полів **Username/Password** є редактор **Headers** (Заголовки) — пари ключ/

значення для CONNECT-заголовків, що надсилаються вищестоящому HTTP-проксі. Ці заголовки зберігаються при повторному відкритті та збереженні outbound (раніше губилися). Зверніть увагу: застосовуються лише заголовки рівня налаштувань (`settings.headers`); заголовки на рівні окремого сервера xray-core ігнорує.

- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port` .
- **trojan** , **shadowsocks** — `settings.servers[]` .
- **wireguard** — `settings.peers[]` з `endpoint` , плюс ключі (див. 11.8).
- **hysteria** — `settings.address / settings.port` (UDP-транспорт).

Для outbound типу **loopback** доступний блок **Sniffing** з тими самими параметрами, що й у inbound: увімкнення, **destOverride**, **Metadata Only**, **Route Only** та список **виключених доменів**.

У масці **UDP** (FinalMask) для **Hysteria2** доступні додаткові режими. У маски **Salamander** є селектор **Mode** зі значеннями **Salamander** та **Gecko**: режим Геско додає випадкове доповнення пакетів з полями **Min/Max** розміру (`packetSize` , діапазон 1–2048, за замовчуванням 512–1200) — це захищає від фінгерпринтингу за довжиною пакетів. У маски **Realm** (UDP hole-punching) з'явився необов'язковий блок **TLS Config** з полями **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) та перемикачем **Allow Insecure**.

З 3.5.0 додано новий тип **TCP-маски — XMC (Minecraft)** (`xmc`): потік маскується під протокол Minecraft. Поля: **Hostname** (необов'язково; «адреса сервера, що імітується в рукописі»), **Usernames** (необов'язковий список; «імена гравців, що пропонуються проберам; за замовчуванням ядро використовує Dream») і обов'язковий **Password** (16 символів, автогенерація з кнопкою 🔄; «пароль обфускації»). Важливо: **поєднання TCP-Finalmask з REALITY відхиляється при збереженні** — воно призводило до падіння Xray-core при першому підключенні (помилка: «Finalmask is not supported with REALITY security...»; див. Xray-core#6453). UDP-маска з REALITY допустима; збережені раніше некоректні комбінації автоматично «лікуються» (маска відкидається) при збиранні конфігу.

Приклад: ланцюжок через вищестоящий SOCKS. Outbound `upstream` ходить на зовнішній SOCKS5-проксі, а `chained` відправляє свій трафік через нього (`dialerProxy`), утворюючи ланцюжок. У `outbounds` :

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Тепер правило маршрутизації з `outboundTag: "chained"` виводитиме трафік в інтернет через `upstream` .

Імпорт outbound з share-посилання

Outbound можна імпортувати зі share-посилання (`vless://`, `vmess://` тощо). При імпорті зберігаються і налаштування мультіплексора **xmux** (XHTTP), передані в блоці `extra=` посилання: після імпорту їх значення підставляються у підформу **XMUX** створеного outbound.

Поля Mux (мультіплексування)

Макс. паралелізм, **Макс. з'єднань**, **Макс. повторних використань**, **Макс. запитів**, **Макс. секунд повторного використання**, **Період keep alive**. Ці параметри налаштовують mux/XUDP-поведінку вихідного.

Sockopts (налаштування сокета)

Група **Sockopts**: **Інтервал keep alive**, **Mark (fwmark)**, **Інтерфейс**, **Тільки IPv6**, **Приймати проху protocol**, **Proху protocol**, **TCP user timeout (мс)**, **TCP keep-alive idle (с)**. Тут також задається dialer-проху ланцюжка.

Freedom finalRules (перевизначення блокування приватних IP)

Для freedom-outbound доступна група **Фінальні правила**:

Поле	Підпис	Опис
<code>overrideXrayPrivateIp</code>	Перевизначити типовий блок приватних IP у Xray	Знімає вбудовану в Xray заборону на вихідні до приватних IP.
<code>action</code>	Дія	<code>allow</code> (як в еталонному шаблоні: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) або інші.
<code>blockDelay</code>	Затримка блоку (мс)	Затримка перед відкиданням з'єднання.
<code>redirect / fragment</code>	Redirect / Fragment	Дії перенаправлення та фрагментації трафіку.

Маска fragment: пофрагментні Lengths та Delays

У масці **fragment** (тип `fragment` у `FinalMask`, для TCP) одиночні поля `Length` та `Delay` замінені списками **Lengths** та **Delays**: для кожного сегмента можна задати окремий діапазон довжини (наприклад `100–200`) та затримки в мілісекундах (наприклад `10–20` або `0`). Рядки списків можна додавати та видаляти; раніше збережені одиночні значення переносяться в масив з одного елемента автоматично.

Інші поля форми

- **UDP over TCP** та **Версія UoT** — для shadowsocks-подібних протоколів.
- **Без gRPC-заголовка**, **Розмір chunk Uplink** — параметри gRPC-транспорту.
- TLS/uTLS-поля: **Перевіряти ім'я peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, плейсхолдер «ім'я сервера».

Тестування вихідних

Кнопки: **Тест**, **Тестувати всі**. Стани: **Тестування з'єднання...**, **Тест успішний**, **Тест не пройдено**, **Не вдалося протестувати вихідне підключення**. Результат: **Результат тесту**, затримка в мілісекундах.

Два режими (підказка: «TCP: швидкий dial-only probe. HTTP: повний запит через храй.»):

- **TCP** (mode=tcp) — простий dial до host:port , виконується паралельно по всіх ендпоінтах, ~таймаут 5 с. Перевіряє лише TCP-досяжність, не валідує проксі-протокол. Для freedom / blackhole / tegu blocked поверне «Outbound has no testable endpoint».
- **HTTP** (mode=http або порожньо) — піднімає тимчасовий екземпляр Храй, пропускає реальний HTTP-запит (probe URL = серверний outboundTestUrl), вимірює реальну затримку. Авторитетний, але дорогий режим: серіалізується глобальним блокуванням («Another outbound test is already running, please wait»). Таймаут однієї спроби — 10 с, вікно очікування результату — 15 с (збільшені, щоб справні outbounds на повільних або тунельованих каналах не позначалися як «Failed»). При невдачі реальна причина (помилка DNS, connection refused, вичерпання дедлайну, помилка TLS тощо) пишеться в лог панелі/Храй, на який вказують загальні повідомлення про таймаут.
- **Real delay (нове в 3.5.0)** — третій режим перемикача. Використовує той самий тимчасовий екземпляр Храй, але повідомляє **повний час «холодного» запиту, включно зі встановленням тунелю** — цифру, близьку до тієї, що показують клієнтські застосунки. Підказка перемикача: «TCP: швидкий dial-пробник. HTTP: повний запит через храй. Real delay: повний час, включно зі встановленням з'єднання». Звичайний **HTTP**-режим з 3.5.0 вимірюється за «теплим» (keep-alive) з'єднанням, тому його цифри нижчі й ближчі до затримки одного запиту.

UDP-протоколи (wireguard , hysteria) та UDP-транспорти (kcp , quic , hysteria) **завжди** тестуються в HTTP-режимі, навіть якщо запитано TCP — голий UDP-dial не відрізняє «живий» ендпоінт від «мертвого». Для wireguard у тестовій конфігурації примусово ставиться noKernelTun: true .

Стовпці Egress і Country (з 3.5.0)

Після успішного **HTTP** або **Real delay** тесту панель через той самий тимчасовий маршрут запитує метадані Cloudflare trace і показує у списку outbound два стовпці: **Egress** — IP виходу (IPv4/IPv6, кожен прихований за «оком»; поки тесту не було — риска з підказкою «Виконайте HTTP-тест, щоб показати IP виходу і країну») і **Country** — прапор і назва країни виходу; якщо вихід іде через Cloudflare WARP, додається помаранчева мітка WARP. TCP-тест ці стовпці не заповнює; на мобільних дані показуються в картці outbound.

Пакетна перевірка та розбивка за етапами

Тест та **Тестувати всі** в HTTP-режимі піднімають один спільний тимчасовий екземпляр Храй на пакет outbounds, для кожного створюють петльовий SOCKS-inbound з правилом і паралельно надсилають через нього реальний HTTP-запит; **Тестувати всі** перевіряє outbounds пачками. **Тестувати всі** перевіряє і outbounds, отримані з підписок (таблиця «з підписок», тільки для читання) — їх рядки також підсвічуються результатом тесту. При цьому outbounds freedom («direct») та dns не тестуються в жодному режимі (це не проксі): кнопка тесту у них недоступна, **Тестувати всі** їх пропускає, а серверний захист забороняє їх HTTP-

тест навіть при прямому виклику API. Крім успіху/помилки попап-результат показує HTTP-статус відповіді та розбивку часу за етапами: **Proxy connect** (підключення до проксі), **TLS via outbound** (TLS через outbound) та **First byte** (час до першого байта) — це допомагає зрозуміти, на якому кроці виникла затримка або збій.

Статистика трафіку outbounds

Панель веде лічильники трафіку за тегами (`up / down / total`). Кнопка скидання скидає лічильники для конкретного тегу або для всіх (`tag = "-alltags-"`). Поля **Інформація про обліковий запис** та **Статус вихідного підключення** показують зведення.

11.5. Балансувальники (Balancers)

Список `routing.balancers`. Кнопки: **Створити балансувальник**, **Редагувати балансувальник**.

На вкладці Balancers є стовпці живого стану: **Live Target** показує поточну активну ціль балансувальника в працюючому Xray, а **Override** дозволяє вручну перевизначити вибір цілі (значення **Auto (strategy)** повертає вибір за стратегією). Стан оновлюється окремою кнопкою. Якщо балансувальник ще не активний у працюючому Xray, панель запропонує спочатку зберегти зміни або запустити Xray.

Поле	Підпис	Опис
Тег	Тег (підказка: «Унікальний тег»)	Унікальний ідентифікатор. Плейсхолдер: «унікальний тег балансувальника». Валідація: «Тег обов'язковий», «Тег вже використовується іншим балансувальником».
Селектори	Селектори	Список тегів outbound (за підрядком), серед яких балансувальник вибирає вихід. Повинен бути обраний хоча б один: «Виберіть хоча б один вихідний».
Fallback	Fallback	Резервний тег, якщо жоден селектор не підійшов. З 3.5.0 можна вибрати й інший балансувальник (у списку такі варіанти позначені синьою міткою «Balancer», підказка «Виберіть інший балансувальник як fallback»). Xray нативно цього не вміє, тому панель сама будує прихований loopback-outbound <code>_bl_<ціль></code> і правило маршрутизації (шлях: балансувальник → loopback → сервер → цільовий балансувальник → outbound; попередження про зайвий «хоп» показується у формі). Є захист від циклів (помилка «не можна вибрати — виникла б циклічна залежність», варіант у списку блокується з підказкою шляху циклу) і від видалення використовуваного балансувальника («Не можна видалити — використовується як fallback у: ...»). Префікс тегу <code>_bl_</code> зарезервований (у тегах заборонений); службові <code>_bl_</code> -об'єкти приховані зі списків Outbounds/Routing, а стовпці Fallback/Live Target/Override показують ім'я цільового балансувальника, а не loopback-тег.
Стратегія	Стратегія	Алгоритм вибору (див. нижче).

Стратегія та параметри спостереження

Стратегія (`strategy.type`) визначає, як балансувальник вибирає outbound з селекторів. Значення Xray-core: `random` (випадково), `roundRobin` (по колу), `leastPing` (мінімальна затримка за результатами observatory), `leastLoad` (мінімальне навантаження). Для `leastLoad / leastPing` використовуються параметри з `strategy.settings`:

Поле	Підпис	Опис
expected	Очікуване	Плейсхолдер: «оптимальна кількість вузлів» — цільова кількість живих вузлів.
maxRtt	Макс. RTT	Верхня межа допустимого RTT при відборі кандидатів.
tolerance	Допуск	Допуск (tolerance) при порівнянні затримок/навантаження.
baselines	Baselines	Порогові значення затримки для групування вузлів.
costs	Costs	Вагові коефіцієнти (cost) для окремих тегів.

Приклади стратегій. Блок `strategy` живе всередині балансувальника (у JSON — поряд з `tag` та `selector`):

```
"strategy": { "type": "random" } // випадковий вибір із селекторів
"strategy": { "type": "roundRobin" } // по колу, по чергово
"strategy": { "type": "leastPing" } // мінімальна затримка (потрібен спостерігач)
```

Для `leastLoad` параметри задаються в `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

Як це відпрацьовує (на прикладі). Нехай спостерігач виміряв затримки по виходах: `A = 250 мс`, `B = 280 мс`, `C = 700 мс`, `D = 1500 мс`. З налаштуваннями вище вибір іде так:

- maxRTT: "1s"** — виходи із затримкою вище 1 с відкидаються: `D` (1500 мс) вибуває. Залишаються `A`, `B`, `C`.
- baselines + expected** — виходи групуються за порогами затримки, і береться **найменший** поріг, до якого потрапляє не менше `expected` виходів. Поріг `500ms` вже містить `A` та `B` — це 2 (= `expected`), тому вибирається група { `A`, `B` }. `C` (700 мс) у вибір не потрапляє, поки швидких вистачає (він — «гарячий резерв»).
- tolerance: 0.05** — всередині вибраної групи виходи, чиї затримки відрізняються не більше ніж на 5%, вважаються рівноцінними, і навантаження ділиться між ними порівну. `A` (250) та `B` (280) відрізняються на ~12% (> 5%), тому за інших рівних умов перевага у більш швидкого `A`; якби різниця була в межах 5% — трафік ішов би і через `A`, і через `B`.
- costs** — до порівняння коригують «вартість» окремих виходів: менше `value` робить вихід привабливішим, більше — навпаки. У прикладі `proxy-premium` отримує `0.1` (стає «дешевшим» і вибирається охочіше), а всі `proxy-cheap-*` (за регулярним виразом, `regex: true`) — `5` (стають

«дорожчими» і використовуються в останню чергу). Так можна м'яко пріоритизувати виходи, не виключаючи їх жорстко.

Підсумок: трафік піде переважно через **A** (при близьких затримках — порівну з **B**), **C** залишиться резервом, **D** виключений, доки його RTT не опуститься нижче **maxRTT**.

Спостерігач: **observatory** та **burstObservatory** (заміри для **leastPing** / **leastLoad**)

Стратегії **leastPing** та **leastLoad** самі нічого не вимірюють — їм потрібні дані про затримку та доступність кожного outbound. Їх збирає **спостерігач** (observatory): він періодично «пінгує» кожний відстежуваний outbound і запам'ятовує час відгуку та доступність. Ті самі дані показуються на вкладці «Обсерваторія» (статуси **Активний** / **Недоступний**, «Остання активність», «Остання спроба»).

З версії 3.4.2 сторінку «Балансувальники» розділено на вкладки «Налаштування балансувальника» (таблиця і додавання) та «**Observatory**», а спостерігач редагується **структурною формою** (попередній сирий JSON-редактор прибрано). Панель сама створює і видаляє спостерігачів за потреби: звичайний **observatory** — для стратегії **Least Ping**, розширений **burstObservatory** — для **Least Load**, а також для **Random/Round-robin** із заданим **fallbackTag** (цього вимагає xray-core; при очищенні останнього **fallbackTag** спостерігач прибирається). Створення або видалення спостерігача вимагає повного перезапуску Xray.

Доступні два варіанти:

- **observatory** — простий: **subjectSelector** + **probeURL** + **probeInterval**.
- **burstObservatory** — розширений, з тонким налаштуванням пінгу через **pingConfig**; зручний для кількох виходів.

Приклад блоку **burstObservatory**:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Призначення полів:

Поле	Що задає
subjectSelector	Список префіксів терів outbound для спостереження. Xray бере всі outbound, чії теги починаються з вказаних рядків. У прикладі спостерігаються виходи WS-SE... , WS-FR... , WS-PL... . Ці теги повинні збігатися з тим, що вибрано в Селекторах балансувальника.

Поле	Що задає
<code>pingConfig.destination</code>	URL, що запитується через кожний outbound для заміру затримки. Беруть «легку» сторінку з відповіддю <code>204</code> без тіла — наприклад <code>https://www.google.com/generate_204</code> . Час до відповіді і є виміряна затримка.
<code>pingConfig.interval</code>	Як часто пінгувати кожний outbound. Рядок тривалості: <code>"1m"</code> — раз на хвилину, також <code>"30s"</code> , <code>"5m"</code> тощо. Частіше — свіжіші дані, але більше фонових трафіку.
<code>pingConfig.connectivity</code>	(необов'язково) URL перевірки базової зв'язності самого сервера. Якщо він недоступний — значить проблема в мережі сервера, і спостерігач не позначає outbound як недоступний (захист від хибних спрацьовувань при локальному збої). Зазвичай теж ендпоінт з відповіддю <code>204</code> .
<code>pingConfig.timeout</code>	Скільки чекати відповідь на один пінг, перш ніж вважати спробу невдалою (наприклад <code>"5s"</code>).
<code>pingConfig.sampling</code>	Скільки останніх замірів зберігати та усереднювати по кожному outbound. <code>2</code> — враховувати два останні пінги (згладжує випадкові стрибки).
<code>pingConfig.httpMethod</code>	Нове в 3.4.2. HTTP-метод проб: <code>HEAD</code> (за замовчуванням) або <code>GET</code> .

На вкладці **«Observatory»** ті самі параметри задаються формою (а не JSON). Для звичайного `observatory` це **Watched Outbounds** (`subjectSelector`, лише читання), **Probe URL** (`probeURL`, за замовчуванням `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval`, `1m`) і **Enable Concurrency** (`enableConcurrency`, за замовчуванням увімкнено). Для `burstObservatory` — перелічені вище поля плюс нове **«HTTP-метод»**. Якщо у балансувальника є обидва спостерігачі, між формами Observatory і Burst перемикає сегментний перемикач.

Як все пов'язати:

- На вкладці **«Observatory»** налаштуйте параметри спостерігача (він створюється автоматично під вибрану стратегію; `subjectSelector` слідує за селекторами балансувальника).
- Створіть балансувальник: **Стратегія** = `leastPing`, у **Селекторах** вкажіть ті самі теги outbound (`WS-SE`, `WS-FR`, `WS-PL`).
- Направте на нього трафік правилом маршрутизації (поле **Тег балансувальника**, див. 11.3).
- Перезапустіть Xray. На вкладці **«Обсерваторія»** з'являться статуси виходів, а балансувальник почне вибирати найшвидший з живих.

В одному правилі не можна одночасно задати `balancerTag` та `outboundTag` — спрацює лише `outboundTag`.

Що відбувається при видаленні outbound або балансувальника (з 3.4.2)

При видаленні outbound або балансувальника панель у тій самій дії **вичищає посилання на нього в маршрутизації** і показує **попередній перегляд наслідків** у діалозі підтвердження (заголовок «При видаленні також оновиться ваша маршрутизація», далі по кожному правилу — «видалено (не залишилося призначення)» або «залишено (тепер використовує ...)», і «Балансувальник ... видалено (не залишилося цілей)»). Так запобігається «вісяче» посилання (`balancerTag` / `outboundTag` / `dialerProxy`), через яке раніше xray-core міг не запуститися при наступному перезапуску. Видалення балансувальника: правила

зберігають свій `outboundTag`, інакше видаляються. Видалення `outbound`: його тег прибирається із селекторів і `fallbackTag` балансувальників, очищається `dialerProxy` в інших `outbound`, спустілі балансувальники видаляються разом із тепер-безпредметними правилами, спостерігачі перезбираються.

11.6. DNS

Розділ `dns`. Увімкнення: **Увімкнути DNS** (підказка: «Увімкнути вбудований DNS-сервер»).

З 3.5.0 DNS-сервери на **приватних IP** (наприклад, свій AdGuard/Pi-hole у тій самій docker-мережі) працюють «з коробки»: панель автоматично вставляє і підтримує кероване `allow`-правило (`direct`, обмежене портом DNS-сервера) **перед** блокуючим правилом `geoip:private` і синхронізує його при зміні `dns.servers`. Раніше власні DNS-запити Xray мовчки блокувалися цим правилом, додаючи ~4 с затримки на кожен новий домен; ручні правила більше не потрібні (створені вручну не зачіпаються).

Загальні параметри DNS

Поле	Підпис	JSON	Опис / підказка
<code>tag</code>	Назва тегу DNS	<code>dns.tag</code>	«Цей тег буде доступний як вхідний тег у правилах маршрутизації» Дозволяє маршрутизувати самі DNS-запити через <code>inboundTag</code> .
<code>clientIp</code>	IP клієнта	<code>dns.clientIp</code>	«Використовується для повідомлення сервера про вказане місцеположення IP під час DNS-запитів» (EDNS Client Subnet).
<code>strategy</code>	Стратегія запиту	<code>dns.queryStrategy</code>	«Загальна стратегія вирішення доменних імен». Значення: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Вимкнути кеш	<code>dns.disableCache</code>	«Вимикає кешування DNS».
<code>disableFallback</code>	Вимкнути резервний DNS	<code>dns.disableFallback</code>	«Вимикає резервні DNS-запити».
<code>disableFallbackIfMatch</code>	Вимкнути резервний DNS при збігу	<code>dns.disableFallbackIfMatch</code>	«Вимикає резервні DNS-запити при збігу списку доменів DNS-сервера».
<code>enableParallelQuery</code>	Увімкнути паралельні запити	—	«Увімкнути паралельні DNS-запити до кількох серверів для більш швидкого вирішення».

Поле	Підпис	JSON	Опис / підказка
<code>useSystemHosts</code>	Використовувати системні Hosts	<code>dns.useSystemHosts</code>	«Використовувати файл <code>hosts</code> з встановленої системи».

Приклад блоку `dns`. Запити до доменів Google резолвуються через DoH-сервер Cloudflare, все інше — через `1.1.1.1`; на запити Google очікуються лише не-приватні IP. На верхньому рівні конфігу:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Рядок-сервер (`"1.1.1.1"`) без полів — це сервер за замовчуванням для всіх інших доменів. Тег `dns-inbound` потім можна використовувати як `inboundTag` у правилах маршрутизації, щоб направляти самі DNS-запити через потрібний outbound.

Кеш застарілих записів

Поле	Підпис	Опис
<code>serveStale</code>	Використовувати застарілі	«Повертати застарілі результати з кешу під час оновлення у фоні».
<code>serveExpiredTTL</code>	TTL застарілих	«Термін дії (секунди) застарілих записів кешу; 0 = безстроково».

DNS-сервери (список `dns.servers`)

Кнопки: **Створити DNS**, **Редагувати DNS**, **Видалити всі** (підтвердження: «Всі DNS-сервери будуть видалені зі списку. Цю дію не можна скасувати.»). Шаблони: **Використати шаблон**, вікно **Шаблони DNS**, включно з пресетом **Сімейний**.

При натисканні **Редагувати DNS** на записі DNS-сервера (як і на записі Fake DNS) вікно редагування підставляє збережені значення сервера, а не значення за замовчуванням.

Поля DNS-сервера:

Поле	Підпис	Опис
<code>address</code>	—	Адреса DNS (IP, DoH-URL, <code>localhost</code> , <code>fakedns</code> тощо).
<code>domains</code>	Домени	Список доменів, для яких використовується цей сервер.

Поле	Підпис	Опис
<code>expectIPs</code>	Очікувані IP	Приймати відповідь лише якщо IP потрапляє до списку.
<code>unexpectIPs</code>	Неочікувані IP	Відкидати відповіді з вказаними IP.
<code>skipFallback</code>	Пропустити Fallback	Не використовувати цей сервер як fallback.
<code>finalQuery</code>	Фінальний запит	Позначає сервер як фінальний у ланцюжку.
<code>timeoutMs</code>	Тайм-аут (мс)	Таймаут запиту до сервера.

Hosts (статичні записи)

Група **Hosts** (`dns.hosts`). Кнопка **Додати Host**; порожній стан **Host не визначені**. Поля: домен (плейсхолдер: «Домен (напр. `domain:example.com`)») та значення (плейсхолдер: «IP або домен – введіть та натисніть Enter»).

Логи DNS

Див. 11.10: прапор **Логи DNS** (`dnsLog`) у секції журналювання.

11.7. Fake DNS

Розділ `fakedns`. Кнопки: **Створити Fake DNS**, **Редагувати Fake DNS**.

Поле	Підпис	Опис
<code>ipPool</code>	Підмережа пулу IP	CIDR-діапазон, з якого видаються фіктивні IP (наприклад <code>198.18.0.0/15</code>).
<code>poolSize</code>	Розмір пулу	Скільки адрес тримати в кільцевому пулі.

Fake DNS використовується у зв'язці зі sniffing на inbound: ядро видає клієнту фіктивний IP, запам'ятовує відповідність домен↔IP та відновлює домен при маршрутизації. Щоб Fake DNS працював, DNS-сервер з адресою `fakedns` повинен бути доданий до списку DNS-серверів.

Приклад: зв'язка Fake DNS + DNS-сервер. Спочатку задаємо пул фіктивних адрес, потім додаємо DNS-сервер `fakedns`, щоб доменні запити отримували IP з цього пулу:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Додатково на inbound потрібно увімкнути sniffing з `destOverride: ["fakedns"]`, інакше ядру нізвідки взяти реальний домен для відновлення.

11.8. WireGuard / WARP / NordVPN

Поля WireGuard (wireguard)

Поле	Підпис	Опис
secretKey	Секретний ключ	Приватний ключ локального інтерфейсу.
publicKey	Публічний ключ	Публічний ключ peer.
psk	Спільний ключ	PreShared Key (опціонально).
allowedIPs	Дозволені IP-адреси	Діапазони, що маршрутизуються в тунель.
endpoint	Кінцева точка	host:port peer.
domainStrategy	Стратегія домену	Стратегія резолву для WireGuard-outbound.

Cloudflare WARP (warp)

Інтеграція використовує API <https://api.cloudflareclient.com/v0a4005> (client-version a-6.30-3596).
Дії контролера (/xray/warp/:action): config , reg , license , data , del .

Покроково:

1. **Створити акаунт WARP** → reg : панель генерує/приймає приватний (privateKey) та публічний (publicKey) ключі, реєструє пристрій у Cloudflare та зберігає access_token , device_id , license_key , private_key (а також client_id) у налаштуванні warp .
2. **Ліцензійний ключ WARP / WARP+** → license : встановлення 26-символьного ключа WARP+ (плейсхолдер: «26-символьний ключ WARP+»). При помилці: «Не вдалося встановити ліцензію WARP.»
Якщо конфіг ще не отримано: «Спочатку отримайте WARP-конфіг.»
3. **Інформація про акаунт: Ім'я пристрою, Модель пристрою, Пристрій увімкнено, Тип акаунту, Роль, WARP+ data, Квота, Використання.**
4. **Додати вихідний** — створює WireGuard-outbound з отриманими ключами та ендпоінтом Cloudflare.
5. **Видалити акаунт** → del : очищає збережені дані WARP.

NordVPN (nord / nordvpn)

Інтеграція використовує NordLynx (= WireGuard). Дії контролера (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Покроково:

1. **Токен доступу** → reg : панель запитує у api.nordvpn.com облікові дані NordLynx та витягує nordlynx_private_key . Зберігає private_key та token у налаштуванні nord . Альтернатива — setKey : ввести **Приватний ключ** напямучу (не може бути порожнім).
2. **Країна** → countries завантажує список країн; **Місто** (або **Усі міста**).
3. **Сервер** → servers завантажує сервери вибраної країни (countryId валідується як число — захист від ін'єкцій). Фільтр: показуються лише сервери з **Навантаженням** > 7%. Якщо серверів немає: «Серверів

для вибраної країни не знайдено». Якщо у сервера немає публічного ключа NordLynx: «Вибраний сервер не повідомляє публічний ключ NordLynx.»

4. Створення/оновлення вихідного: тости «Вихідний NordVPN додано» / «Вихідний NordVPN оновлено».

Пріоритет IPv4 та userspace TUN

Генеровані майстрами WARP та NordVPN WireGuard-outbounds використовують `domainStrategy: "ForceIPv4v6"` (пріоритет IPv4 з відкатом на IPv6 на v6-only хостах) замість `ForceIP` — це усуває «зависання» рукостискання на хостах з наполовину налаштованим IPv6, коли вибирається AAAA-запис ендпоінта Cloudflare. Крім того, для них увімкнено userspace TUN (`noKernelTun: true`) замість kernel TUN: останній потребує прав і fwmark-маршрутизації та тихо падає на багатьох VPS, тоді як вбудована перевірка з'єднання панелі завжди тестує через userspace TUN — тепер реальний трафік і перевірка йдуть одним шляхом. Зміна діє лише для новододаваних або скинутих outbounds; вже збережені шаблони зберігають свої налаштування.

11.9. Reverse-проксі та TUN

Reverse (реверс-проксі)

Розділ `reverse` конфігурації Xray. У формі outbound є перемикач на тип **Реверс-проксі**. Кнопки: **Створити реверс-проксі**, **Редагувати реверс-проксі**.

Поле	Підпис	Опис
Тип	Тип	Bridge або Portal — дві ролі реверс-проксі Xray.
Домен	Домен	Службовий домен-мітка для пари bridge [?] portal.
Тег / З'єднання	Тег / З'єднання	Теги для зв'язки bridge та portal.
Reverse Tag	Тег реверс-проксі	Підказка: «Тег вихідного підключення для простого реверс-проксі VLESS. Залиште порожнім для вимкнення.» Плейсхолдер: «тег вихідного (порожньо = вимкнено)». Реалізує спрощений VLESS reverse.

У формі outbound присутні також поля зворотного потоку: **Зворотний sniffing**, **Воркери**, **Зарезервовано**, **Мін. інтервал завантаження (мс)**, **Макс. розмір завантаження (байт)**.

TUN (tun)

Поле	Підпис	Опис	За замовчуванням
name	—	«Ім'я інтерфейсу TUN.»	xray0
mtu	—	«Максимальна одиниця передачі. Максимальний розмір пакетів даних.»	1500
<code>userLevel</code>	Рівень користувача	«Всі з'єднання, встановлені через цей вхідний потік, використовуватимуть цей рівень користувача.»	0

11.10. Логи та статистика (Stats, metrics)

Лог (log)

Підказка: «Логи можуть сповільнювати роботу сервера. Вмикайте лише потрібні вам види логів при необхідності!» Секція `log` еталонного шаблону: `access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: ""`.

Поле	Підпис	JSON	Опис	За замовчуванням
<code>logLevel</code>	Рівень логів	<code>loglevel</code>	«Рівень журналу для журналів помилок...» Значення: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	warning
<code>accessLog</code>	Логи доступу	<code>access</code>	«Шлях до файлу журналу доступу. Спеціальне значення «none» вмикає логи доступу.»	none
<code>errorLog</code>	Логи помилок	<code>error</code>	«Шлях до файлу логів помилок. Спеціальне значення «none» вмикає логи помилок.»	"" (за замовчуванням)
<code>dnsLog</code>	Логи DNS	<code>dnsLog</code>	«Увімкнути логи запитів DNS»	false
<code>maskAddress</code>	Маскування адреси	<code>maskAddress</code>	«При активації реальна IP-адреса замінюється на маскувальну в логах.»	"" (вимк.)

Статистика (stats / policy)

Група **Статистика**. Вмикає лічильники в `policy.system` та `policy.levels`. В еталонному шаблоні: `statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false`; для рівня `0` — `statsUserUplink: true, statsUserDownlink: true`.

Поле	Підпис	Опис	За замовчуванням
<code>statsInboundUplink</code>	Статистика вхідного аплінку	«Вмикає збір статистики для вихідного трафіку всіх вхідних проксі.»	true
<code>statsInboundDownlink</code>	Статистика вхідного даунлінку	«Вмикає збір статистики для вхідного трафіку всіх вхідних проксі.»	true
<code>statsOutboundUplink</code>	Статистика вихідного аплінку	«Вмикає збір статистики для вихідного трафіку всіх вихідних проксі.»	false
<code>statsOutboundDownlink</code>	Статистика вихідного даунлінку	«Вмикає збір статистики для вхідного трафіку всіх вихідних проксі.»	false

Статистика за клієнтами та inbounds (uplink/downlink) — основа відображення трафіку в дашборді та у клієнтів; вимикати її не рекомендується. Outbound-статистика за замовчуванням вимкнена і потрібна лише якщо ви відстежуєте трафік за тегами вихідних.

Metrics

В еталонному шаблоні присутня секція `metrics` (`listen: "127.0.0.1:11111"`, `tag: "metrics_out"`) та відповідний API `metrics_out`. Панель використовує цей listener для збору метрик та знімків `observatory`: вона парсить `metrics.listen` з шаблону, опитує `/debug/vars` та агрегує історію затримок за тегами. Якщо ви змінюєте адресу/порт `metrics.listen`, панель звертатиметься до нової адреси; видалення секції `metrics` вимкне збір графіків `observatory`.

Тестування `outbound` в HTTP-режимі піднімає **окремий тимчасовий** екземпляр Xray зі своїм `metrics-listener` на випадковому порту — це не той самий listener, що в основному конфізі.

11.11. Збереження, перезапуск та автоматичні перетворення

Кнопки

Кнопка	Дія
Зберегти	POST <code>/xray/update</code> : валідує та зберігає шаблон + <code>outboundTestUrl</code> .
Перезапуск Xray	Перезавантажує сервіс зі збереженою конфігурацією. Підтвердження: «Перезапустити xray?» / «Перезавантажує сервіс xray зі збереженою конфігурацією.»

Тости: успіх — «Xray успішно перезапущено», «Xray успішно зупинено»; помилки — «Виникла помилка при перезапуску Xray.», «Виникла помилка при зупинці Xray.» Вікно **Вивід перезапуску Xray** показує діагностичний вивід ядра.

Гаряче застосування змін (без повного перезапуску)

Зміни `inbounds`, `outbounds` та правил маршрутизації застосовуються «в живому режимі»: при натисканні **Зберегти** панель обчислює різницю між старим та новим конфігом і застосовує лише змінені частини через gRPC-API Xray (`HandlerService/RoutingService`), не перезапускаючи процес. Повний перезапуск виконується автоматично лише тоді, коли змінюються секції без API гарячого перезавантаження (`log`, `dns`, `policy`, `observatory` тощо). Тому на сторінці Xray окрема кнопка «Перезапустити» не потрібна — **Зберегти** само застосовує зміни. Перезапуск ядра при необхідності як і раніше виконується автоматично (див. також авто-перезавантаження при оновленні підписок та ротації WARP).

Відновлення шаблону за замовчуванням

Ендпоінт `GET /xray/getDefaultJsonConfig` повертає еталонний шаблон (`config.json`, вбудований у бінарник). Ним можна скинути конфігурацію до заводської.

Автоматичні перетворення при збереженні

При збереженні налаштувань Xray панель виконує (у такому порядку):

1. **Зняття обгортки** — знімає обгортки вигляду `{ "xraySetting": <конфіг>, "inboundTags": ..., "outboundTestUrl": ... }`, якщо вони випадково потрапили до значення (інакше шари накопичувалися б при кожному збереженні). Знімається до 8 шарів.
2. **Перевірка конфігурації** — JSON розбирається в структуру конфігурації Xray; при помилці — відмова з «*xray template config invalid*».
3. **Гарантія правила статистики** — правило `inboundTag: ["api"] → outboundTag: "api"` примусово піднімається на позицію 0 у `routing.rules` (або додається, якщо відсутнє). Це гарантує, що запит gRPC-статистики панелі не буде перехоплений вищестоящим catch-all-правилом (інакше клієнти можуть відображатися офлайн з нульовим трафіком при працюючому проксі).

Через п. 3 не намагайтеся прибрати або пересунути правило `api → api` — панель все одно поверне його на місце при наступному збереженні. Це службова інфраструктура статистики, а не користувацький маршрут.

11.12. Outbound з підписки (з авто-оновленням)

Починаючи з версії 3.3.0 панель вміє імпортувати `outbound` 'и напряду з URL підписки — того самого формату, що видають провайдери VPN для клієнтських додатків. Підписки регулярно перераховуються у фоні, тому набір `outbound` 'ів на сервері підтримується в актуальному стані без ручного редагування шаблону конфігурації. З 3.5.0 у `ss://` -посилань (SIP002) із query-параметрами (`?plugin=`, `?type=`) або кінцевим `/` порт розбирається коректно (раніше він мовчки ставав `0`); посилання з дійсно некоректним портом пропускається, а не імпортується зламаним.

В інтерфейсі розділ називається «**Підписки вихідних**», опис: «Імпорт вихідних з віддалених URL підписок (`vmess/vless/trojan/ss/...`). Теги залишаються незмінними для використання в балансувальниках та правилах маршрутизації. Оновлення виконується автоматично.» Розділ розташований на сторінці Xray, над панеллю налаштування `outbound` 'ів.

Як це влаштовано

Підписки зберігаються окремо від шаблону конфігурації Xray. Шаблон **ніколи не перезаписується**: отримані з підписок `outbound` 'и додаються до підсумкової конфігурації на льоту при кожній генерації конфігу Xray.

Додавання підписки

У формі «Додати підписку» доступні такі поля:

Поле	Ключ	За замовчуванням	Призначення
URL підписки	<code>url</code>	– (обов'язково)	Адреса підписки. Плейсхолдер: «https://... (список посилань у base64)». Приймається лише HTTP(S); адреса перевіряється на безпеку.
Примітка	<code>remark</code>	порожньо	Довільна мітка (плейсхолдер «напр. вузли НК»).
Префікс тегу	<code>tagPrefix</code>	<code>subN–</code>	Префікс, з якого починаються теги імпортованих <code>outbound</code> 'ів. Якщо залишити порожнім, панель сама підбере найменший вільний номер вигляду <code>sub1–</code> , <code>sub2–</code> тощо.
Інтервал оновлення	<code>updateInterval</code>	600 секунд (10 хвилин)	Як часто підписка перерахується. У UI задається в годинах/хвилинах.
Увімкнено	<code>enabled</code>	так (<code>true</code>)	Лише увімкнені підписки потрапляють до конфігу та оновлюються автоматично.
Дозволити приватні адреси	<code>allowPrivate</code>	ні (<code>false</code>)	Дозволяє URL на localhost, LAN та приватні IP. За замовчуванням вимкнено для захисту від SSRF — вмикайте лише для довіреного локального джерела.
Перед ручними вихідними	<code>prepend</code>	ні (<code>false</code>)	Якщо увімкнено, <code>outbound</code> 'и цієї підписки ставляться перед ручними <code>outbound</code> 'ами шаблону, і один з них може стати <code>outbound</code> 'ом за замовчуванням. Інакше вони додаються після .

Кнопка «**Попередній перегляд**» (`POST /outbound-subs/parse`) дозволяє до збереження завантажити та розпарсити URL і побачити, які `outbound` 'и та теги вийдуть; нічого до бази при цьому не записується. Якщо за URL нічого не розпізнано, виводиться «За цим URL вихідних не знайдено.»

Порядок кількох підписок у загальному списку `outbound` 'ів задається пріоритетом (`priority`) та змінюється стрілками вгору/вниз (`POST /outbound-subs/:id/move`).

Які формати підписки приймаються

Тіло відповіді за URL обробляється так:

- Вміст спочатку пробується як **base64** (стандартний та URL-safe варіанти, з автодоповненням паддингу та видаленням пробілів/переводів рядків). Якщо це base64 — воно декодується; інакше береться як є.
- Потім тіло розбивається на рядки. Кожен непустий рядок, що не починається з `#`, розбирається як посилання. Нерозпізані рядки (коментарі, непідтримувані протоколи) мовчки пропускаються.
- Підтримувані схеми посилань: `vmess://`, `vless://`, `trojan://`, `ss://` (Shadowsocks), `hysteria2://` / `hy2://`, `wireguard://` / `wg://`.

Тобто підходить звичайна підписка вигляду «base64-кодований список посилань», як у більшості провайдерів.

Стабільні теги

Кожному посиланню обчислюється стійка «ідентичність» (ядро URI без фрагмента-примітки; для vmess — внутрішній JSON без поля `ps`). Відповідність «ідентичність → тег» зберігається, і при наступному оновленні той самий сервер отримує той самий тег, навіть якщо змінилися примітка або другорядні параметри. Це спеціально зроблено, щоб балансувальники та правила маршрутизації продовжували працювати після оновлень:

- Точний тег у балансувальнику/правилі продовжить вказувати на той самий сервер.
- Префіксний/wildcard-селектор (наприклад, `hk-*`) автоматично підхопить нові сервери, які підписка поверне пізніше — це рекомендований спосіб «підписатися на пул».
- Якщо сервер зникає з підписки, його тег просто пропадає з підсумкового масиву `outbound 'iv`; при наявності `fallbackTag` у балансувальника Xray використовує його.
- Якщо провайдер змінив UUID/хост/облікові дані сервера, ідентичність змінюється — це вважається новим `outbound 'om` з новим тегом.

Всередині однієї вивантаження теги дедуплікуються суфіксом `-N`. Теги з підписок зберігають не-ASCII символи (наприклад, кирилицю) та залишаються читаємими: букви та цифри Unicode зберігаються в slug, а пунктуація замінюється на дефіс — теги з кирилических імен більше не зводяться до самих цифр.

Як працює авто-оновлення

- Фонове завдання оновлення підписок запускається за розкладом **кожні 5 хвилин**.
- На кожному запуску воно перебирає всі увімкнені підписки та оновлює лише ті, у яких вичерпано власний інтервал: підписка оновлюється, якщо ще жодного разу не оновлювалась або якщо з останнього оновлення пройшло не менше її `updateInterval`. Таким чином завдання перевіряє підписки часто, але кожна конкретна підписка перерахується не частіше свого `updateInterval` (за замовчуванням 10 хвилин). У UI це відображено відповідною підказкою.
- Оновлення: URL заново перевіряється на безпеку як публічний (приватні адреси блокуються, якщо у підписки не виставлено `allowPrivate`), запит іде через проксі-клієнт панелі з заголовком `User-Agent: 3x-ui-outbound-sub/1.0`. Ланцюжок редиректів обмежений 10 переходами, і кожний хоп теж перевіряється на приватність (захист від SSRF). Очікується HTTP 200; інакше фіксується помилка.
- Після успішного парсингу результат зберігається, проставляється час останнього оновлення, очищається помилка. При помилці її текст видно в UI як «Остання помилка», а раніше отримані `outbound 'i` залишаються в силі.
- Якщо хоча б одна підписка реально оновила, завдання позначає Xray на перезапуск та надсилає UI-інвалідацію, щоб інтерфейс підтягнув нові `outbound 'i`. Фактичне перезавантаження Xray відбувається на найближчому 30-секундному циклі менеджера.

Ручне оновлення однієї підписки — кнопка **«Оновити зараз»** (`POST /outbound-sub/:id/refresh`); вона теж позначає Xray на перезапуск. Додавання, зміна, видалення підписки також встановлюють прапор перезапуску Xray (при видаленні її `outbound 'i` випадають з конфігу на наступному перезавантаженні). UI підказує: «Після додавання або оновлення перезапустіть Xray (або зачекайте наступного авто-перезавантаження), щоб вихідні стали активними.»

Як це потрапляє до конфігу Xray

При кожній генерації конфігурації Xray активні підписочні `outbound` 'и діляться на дві групи — `prepend` (прапор «Перед ручними вихідними») та решта — і зшиваються з шаблоном: `[prepend підписок] + [outbound'и шаблону] + [решта підписок]`. Всередині кожної групи підписки йдуть за пріоритетом. Ручні `outbound` 'и з шаблону при цьому не зачіпаються; якщо масив `outbound` 'ів шаблону чомусь не парситься, підписочні `outbound` 'и до нього не підмішуються (щоб не втратити ручні).

Імпортовані `outbound` 'и додатково відображаються в самій панелі `outbound` 'ів окремим блоком **«3 підписок вихідних (тільки для читання)»** — редагувати їх там не можна, керування лише через розділ «Підписки вихідних».

11.13. Ротація IP у WARP

У 3X-UI можна підняти WARP-outbound — вихідне WireGuard-підключення до Cloudflare WARP (тег `warp` у конфізі Xray). Панель сама реєструє на серверах Cloudflare пристрій-акаунт, отримує WireGuard-ключі та адреси і підставляє їх в `outbound` з тегом `warp`. Через такий `outbound` трафік виходить в інтернет під IP-адресою Cloudflare WARP. Новинка версії 3.3.0 — можливість змінювати цей вихідний IP вручну або за розкладом, не пересоздаючи акаунт WARP вручну.

Керування знаходиться в розділі **Xray** у картці WARP (після натискання «Створити акаунт WARP» та отримання конфігу; до цього дії недоступні — панель підкаже «Спочатку отримайте WARP-конфіг»).

Що відбувається при зміні IP

Кнопка **«Змінити IP»** запускає зміну IP. Логіка:

1. Генерується нова пара ключів WireGuard.
2. З новим ключем заново реєструється пристрій WARP на серверах Cloudflare (новий `device_id`, `access_token`, адреси та peer-дані).
3. Нові дані записуються до WARP-outbound конфігу Xray: оновлюються `secretKey`, `address` (v4 /32 та v6 /128), `reserved` (з `client_id`), а також `publicKey` та `endpoint` у peer.
4. Якщо раніше був заданий ліцензійний ключ WARP+ (довжиною не менше 26 символів), він автоматично переустановлюється на новий акаунт. При невдачі це лише попередження в логах — зміна IP не скасовується.
5. Після успішної зміни Xray позначається як такий, що потребує перезапуску, щоб новий `outbound` набрав чинності.

При успіху інтерфейс показує «IP-адреса WARP успішно змінена!».

Автоматична ротація за розкладом

У картці WARP є перемикач **«Автоматичне оновлення IP-адреси»** та поле **«Інтервал (дні)»**. Підказка: «0 — вимкнути. Автоматично змінює IP-адресу.»

Параметр	Значення
Налаштування в БД	<code>warpUpdateInterval</code> (ціле, ≥ 0)

Параметр	Значення
Значення за замовчуванням	0 (авто-ротація вимкнена)
Одиниця вимірювання	дні
0	вимикає автоматичну зміну
> 0	змінювати IP кожні N днів

Запис інтервалу зберігає `warpUpdateInterval`, і при значенні більше 0 скидає «час останнього оновлення» на поточний момент — інакше планувальник змінив би IP вже на найближчому тіку.

Розклад виконує фонове завдання, що запускається раз на годину — тобто панель раз на годину перевіряє, чи не час ротувати. Алгоритм перевірки:

- якщо інтервал ≤ 0 — нічого не робить;
- якщо «час останнього оновлення» дорівнює 0 (наприклад, інтервал виставили прямим редагуванням БД) — це перший запуск: завдання лише фіксує базову відмітку часу та НЕ змінює IP одразу;
- якщо з моменту останнього оновлення пройшло не менше `інтервал × 24 × 3600` секунд — виконується та сама зміна IP, оновлюється відмітка часу та планується перезапуск Xray.

Важлива деталь: ручна зміна по кнопці «Змінити IP» теж скидає відмітку часу останнього оновлення. Тому після ручної ротації відлік автоматичного інтервалу починається заново і планова зміна не спрацює одразу після неї.

«Через проксі панелі»

Змінено у 3.3.1. Окреме налаштування «Мережевий проксі панелі» (`panelProxy`) прибрано. Вихідний трафік самої панелі (у тому числі запити до WARP API) тепер направляється через обраний **outbound для трафіку панелі** — Xray-outbound або балансувальник (див. розділ 13). Опис нижче стосується версій до 3.3.1.

Всі запити до API Cloudflare WARP (реєстрація, отримання конфігу, встановлення ліцензії, зміна IP) йдуть не напряму, а через HTTP-клієнт панелі з таймаутом 15 секунд. Цей клієнт вважає налаштування **«Мережевий проксі панелі»** (`panelProxy`) з налаштувань панелі.

З опису налаштування: проксі маршрутизує власні вихідні запити панелі (оновлення гео-баз, перевірки версій Xray/панелі, Telegram, а тепер і звернення до WARP) — щоб обходити серверну фільтрацію. Приймаються адреси вигляду `socks5://` або `http(s)://`, наприклад локальний SOCKS-вхідний самого Xray. Якщо поле порожнє або проксі заданий невірно — використовується пряме підключення (поведінка не ламається).

Користь для WARP: якщо сервер не може напряму достукатися до `api.cloudflareclient.com`, реєстрація та ротація раніше падали. Тепер, вказавши в `panelProxy` робочий проксі (зокрема власний inbound Xray), можна гарантувати доступність WARP API та працездатність як ручної кнопки, так і планової ротації.

Коли це корисно

- Регулярна зміна вихідного IP для outbound, що ходить через WARP, — знижує ризик блокувань та трекінгу за однією адресою.
 - «Освіжити» IP вручну, якщо поточна адреса Cloudflare потрапила до чорних списків або працює повільно.
 - Сервери, у яких немає прямого доступу до Cloudflare WARP API: маршрутизація запитів через `panelProxy` робить реєстрацію та ротацію працездатними.
-

12. Вузли (мультипанель, master/slave)

Розділ **Вузли** перетворює звичайну установку 3X-UI на **центральну (мастер-) панель**, яка дистанційно спостерігає за іншими (дочірніми) панелями 3X-UI та керує ними. Кожен вузол — це окрема установка 3X-UI на своєму сервері; мастер звертається до неї через її власний HTTP API, опитує її стан і синхронізує на неї inboundс і клієнтів, які їм призначені. Це і є можливість **мультипанелі**: замість того щоб заходити в кожну панель окремо, ви бачите всі сервери в одному списку і керуєте ними централізовано.

Важливий принцип: **вузол — це не агент, а повноцінна панель 3X-UI**. Мастер нічого на ній не «встановлює» — він лише підключається до її API за токеном. Видалення вузла зі списку зупиняє лише моніторинг; сама віддалена панель при цьому не зачіпається (підказка: «Це зупинить моніторинг вузла. Сама віддалена панель не буде зачеплена»).

12.1. Зведення вгорі списку

Над таблицею вузлів виводяться агреговані лічильники:

Поле	Опис
Всього вузлів	Загальна кількість вузлів у списку.
В мережі	Скільки вузлів мають статус <code>online</code> .
Не в мережі	Скільки вузлів мають статус <code>offline</code> .
Середня затримка	Усереднена затримка (ping) до вузлів, у мілісекундах.

12.2. Додавання та редагування вузла

Кнопки **Додати вузол** і **Змінити вузол** відкривають форму з полями вузла.

Обов'язкові (підказка: «Ім'я, адреса, порт і токен API обов'язкові») поля **Ім'я**, **Адреса**, **Порт** і **API Токен**.

При натисканні «Зберегти» (як при додаванні, так і при зміні) панель **спочатку перевіряє досяжність вузла** з таймаутом 6 секунд. Якщо вузол не відповідає, запис не зберігається і показується помилка. Тобто не можна додати завідомо недоступний вузол.

Поля форми

Поле	За замовчуванням	Допустимі значення	Опис
Ім'я	— (обов'язково)	непорожній рядок, унікальний	Внутрішнє ім'я вузла. На колонку імені накладена унікальність — два вузли з однаковим іменем створити не можна. Підказка-плейсхолдер: <code>napr. de-frankfurt-1</code> . При збереженні пробіли по краях обрізаються.
Примітка	порожньо	будь-який рядок	Необов'язкова нотатка/опис вузла. На роботу не впливає.

Поле	За замовчуванням	Допустимі значення	Опис
Схема	<code>https</code>	<code>http / https</code>	Протокол підключення до віддаленої панелі. Якщо залишити порожнім або вказати неприпустиме значення, нормалізація виставить <code>https</code> . Якщо вузол відповідає по звичайному HTTP, а схема стоїть <code>https</code> , панель поверне зрозумілу підказку: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Адреса	– (обов'язково)	хост або IP	Адреса віддаленої панелі. Плейсхолдер: <code>panel.example.com</code> або <code>1.2.3.4</code> . Адреса нормалізується; за замовчуванням приватні/локальні адреси заборонені заради захисту від SSRF – див. «Дозволити приватну адресу».
Порт	– (обов'язково)	ціле 1–65535	Порт веб-панелі віддаленого вузла. Значення поза діапазоном відхиляються («node port must be 1-65535»).
Базовий шлях	<code>/</code>	рядок шляху	Базовий шлях (web base path) віддаленої панелі, якщо він заданий. Нормалізується: гарантовано починається і закінчується на <code>/</code> (порожнє значення → <code>/</code>). До нього панель дописує <code>panel/api/server/status</code> при опитуванні.
API Токен	– (обов'язково)	токен віддаленої панелі	Bearer-токен для доступу до API вузла. Передається в заголовок <code>Authorization: Bearer <token></code> . Плейсхолдер: «Токен зі сторінки Налаштувань віддаленої панелі». Підказка: «Віддалена панель показує свій токен API в розділі Налаштування → Токен API». Тобто токен треба створити на самому вузлі (Налаштування → Токен API), а сюди вставити.
Увімкнений	<code>true</code>	так/ні	Вмикає моніторинг і синхронізацію вузла. Вимкнені вузли не опитуються фоновими задачами (heartbeat і traffic-sync їх пропускають) і не беруть участі в масовому оновленні панелі.
Дозволити приватну адресу	<code>false</code>	так/ні	Знімає SSRF-захист і дозволяє підключатися до вузла за приватною/локальною адресою. Підказка: «Вмикати лише для вузлів у приватній мережі або VPN». Вмикайте лише коли вузол дійсно знаходиться в приватній мережі або доступний через VPN.

Отримання та регенерація токена на стороні вузла

Токен береться на віддаленій панелі в розділі **Налаштування** → **Токен API**. Там само його можна перевипустити: кнопка **Згенерувати токен заново** з попередженням: «Повторна генерація анулює поточний токен. Будь-яка центральна панель, що використовує його, втратить доступ до оновлення. Продовжити?». Після регенерації старий токен у мастер-панелі перестане працювати – його потрібно оновити у формі вузла.

Вихідне підключення (Connection outbound)

Поле **Connection outbound** (Вихідне підключення, `outboundTag`) задає, як трафік звернень мастера до API цього вузла залишає сервер. Якщо вибрати в ньому `Http-outbound`, звернення панелі до вузла підуть не напряму, а через вказаний `outbound`; панель сама додасть до робочої конфігурації міст-`inbound` на `loopback` і застосує зміну на льоту, без перезапуску. Підказка: «Route this node's panel API traffic through the selected

Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Селектор влаштований як панельний вибір outbound: теги згруповані на **Outbounds** (звичайні вихідні) і **Balancers** (балансувальники), blackhole-вихідні зі списку приховані. Порожнє значення (плейсхолдер «Direct connection») = пряме підключення до вузла.

Імпорт inbound (вибір синхронізованих inbound-ів)

У формі вузла є налаштування **Імпорт inbound** (`inboundSyncMode`) з двома режимами: **Усі inbound** (`all` , за замовчуванням) і **Вибрані** (`selected`). За замовчуванням мастер синхронізує на вузол усі inbounds, у яких вибраний цей вузол; існуючі вузли продовжують працювати в режимі «Усі inbound».

В режимі **Вибрані** під полем з'являється мультिवибір тегів inbound. Натисніть **Завантажити inbound** — мастер за введеними (ще не збереженими) параметрами підключення запросить у вузла список його inbound-ів (ендпоінт `POST /panel/api/nodes/inbounds`) і покаже їхні теги; відмітьте потрібні. Панель синхронізуватиме і розгортатиме на вузол лише відмічені теги, а решта inbounds, що існують безпосередньо на вузлі, залишаться незачепленими — мастер їх не видаляє і ними не керує.

Приклад: запросити список inbound-ів вузла для вибіркового імпорту. В тілі передаються ще не збережені параметри підключення; у відповіді — теги доступних на вузлі inbound-ів:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Перевірка TLS (для https-вузлів)

Група полів задає, як мастер перевіряє HTTPS-сертифікат вузла. Ці налаштування **актуальні лише для схеми https** ; для `http` -вузлів вони ігноруються.

Перевірка TLS — випадний список, підказка: «Як панель перевіряє HTTPS-сертифікат вузла. Закріплення або Пропуск — для самопідписаних сертифікатів (лише https-вузли)».

Режим	Значення	За замовчуванням	Опис
Перевіряти (стандартний CA)	<code>verify</code>	так (default)	Звичайна перевірка ланцюжка сертифіката довіремим CA. Підходить для вузлів з публічним/Let's Encrypt сертифікатом. Використовується також для всіх <code>http</code> -вузлів.
Закріпити сертифікат (SHA-256)	<code>pin</code>	—	Ланцюжок CA не перевіряється, але SHA-256 листового сертифіката вузла звіряється зі збереженим відбитком (constant-time порівняння). Зберігає захист від MITM для самопідписаних сертифікатів. Вимагає заповнити поле відбитка.
Пропустити перевірку	<code>skip</code>	—	Перевірка сертифіката повністю вимикається. Попередження: «Пропуск перевірки прибирає захист від атак «людина посередині» — токен API може бути перехоплений. Краще закріпити сертифікат».

До трьох режимів вище в 3.4.0 додано четвертий — **Mutual TLS (client certificate)** (`mtls`), доступний, як і решта, лише для схеми `https`.

Режим	Значення	За замовчуванням	Опис
Mutual TLS (клієнтський сертифікат)	<code>mtls</code>	—	Окрім перевірки сертифіката вузла, мастер додатково підтверджує себе вузлу клієнтським сертифікатом , виданим його власним CA. Для вузла в цьому режимі API-токен стає необов'язковим — вузол впізнає мастер за сертифікатом. При виборі режиму показується підказка: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Щоб увімкнути взаємний TLS для вузла: на стороні вузла задайте режим **Mutual TLS**, скопіюйте CA керуючої панелі з розділу **Node mTLS** (див. нижче), пропишіть його на вузлі як **довірений батьківський CA** і перезапустіть вузол.

Якщо вибрати будь-яке значення, крім `skip`, `pin` або `mtls`, нормалізація примусово виставить `verify`.

Закріплення сертифіката

При виборі **Закріпити сертифікат** з'являються:

- **SHA-256 закріпленого сертифіката** — поле введення. Приймається відбиток у **base64** (формат `pinnedPeerCertSha256` з Xray) або в **hex** з двокрапками або без (стиль `openssl -fingerprint`). Підказка: «SHA-256 сертифіката вузла в base64 або hex. Натисніть «Отримати», щоб зчитати його з вузла зараз». Плейсхолдер: «SHA-256 в base64 або hex». При виборі `pin` порожній або некоректний відбиток викликає помилку валідації при збереженні.

Приклад: один і той самий відбиток у двох форматах. Поле приймає будь-який з варіантів — обидва означають один сертифікат:

```
# base64 (формат pinnedPeerCertSha256 з Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex з двокрапками (стиль openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Якщо відбиток ще невідомий, натисніть **Отримати** — мастер сам зчитає його з вузла по HTTPS і підставить у поле.

- Кнопка **Отримати** — підключається до вузла по HTTPS без перевірки сертифіката і зчитує SHA-256 поточного листового сертифіката (ендпоінт `POST /certFingerprint`), підставляючи його в поле. Після успіху — «Поточний сертифікат вузла отримано»; при невдачі — «Не вдалося отримати сертифікат». Доступно лише для https-вузлів.

Node mTLS (взаємна TLS-автентифікація між панелями)

На сторінці **Вузли** є окремий розділ **Node mTLS** – налаштування взаємної TLS-автентифікації, яка додає до API-токена другий фактор (клієнтський сертифікат) для викликів «панель → вузол». Взаємний TLS вмикається за бажанням; якщо поля розділу порожні, вузли працюють за попередньою схемою – **лише з API-токеном** (підказка: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). У розділі дві операції:

- **Скопіювати СА цієї панелі** (`POST /panel/api/nodes/mtls/ca`) – копіює кореневий сертифікат (CA) цієї панелі в буфер обміну. Цей СА потрібно передати керованим вузлам, щоб вони довіряли клієнтському сертифікату панелі; на самих вузлах після цього виставляється режим перевірки TLS **Mutual TLS** (підказка: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Після копіювання – «CA certificate copied to clipboard».
- **Довірений батьківський СА** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – поле, яке використовується, коли сама ця панель виступає вузлом для вищестоящої (керуючої) панелі. Вставте сюди СА керуючої панелі, щоб вимагати від неї клієнтський сертифікат, і натисніть **Save trust CA**. Зміна вимагає **перезапуску панелі** (підказка: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Що показується по кожному вузлу

Колонки таблиці та поля картки вузла (спостережуваний стан, заповнюється при кожному heartbeat-опитуванні):

Поле	Опис
Статус	<code>online</code> / <code>offline</code> / <code>unknown</code> – див. нижче.
CPU	Завантаження процесора віддаленого сервера у відсотках.
Пам'ять	Використання ОЗУ у відсотках (обчислюється як <code>current/total*100</code>).
Аптайм	Час безперервної роботи сервера (у секундах).
Затримка	Час відповіді вузла на останнє опитування (мс).
Останній пінг	Час останнього успішного heartbeat (unix-секунди; <code>0</code> = «ніколи»; недавнє значення показується як «щойно»).
Версія Xray	Версія Xray-core, запущеного на вузлі.
Версія панелі	Версія 3X-UI на вузлі – порівнюється з актуальною для індикатора оновлення.
(inbounds)	Скільки inbounds фізично розміщено на цьому вузлі.
(клієнти)	Число клієнтів на inbounds вузла.
(онлайн)	Скільки клієнтів вузла зараз у мережі.
(вичерпано)	Скільки клієнтів вузла вичерпали або перевищили ліміт трафіку . Відключені вручну клієнти в цей лічильник не входять.
(швидкість)	Поточна (жива) швидкість передачі на inbound-ах, розміщених на вузлі.

Лічильники inbounds/клієнтів/онлайн прив'язуються до вузла за його стабільним GUID (`panelGuid`), а не за локальним id, — щоб клієнт на підвузлі враховувався саме під підвузлом, а не під проміжним вузлом, через який він синхронізується.

Для inbound-ів, розміщених на вузлі, сторінка показує онлайн-клієнтів, лічильники та **поточну швидкість передачі**. Прив'язка за стабільним GUID коректно розводить і «клонівані» вузли з однаковим `panelGuid`.

Статуси вузла

Статус	Назва	Коли встановлюється
online	В мережі	Вузол відповів <code>success=true</code> на опитування <code>panel/api/server/status</code> .
offline	Не в мережі	Вузол не відповів, повернув HTTP-помилку, <code>success=false</code> або нерозпізнану відповідь.
unknown	Невідомо	Початкове значення, поки вузол ще жодного разу не опитаний.

При невдалому опитуванні текст помилки зберігається і показується зрозумілим формулюванням, що допомагає діагностувати причину «offline».

12.5. Дії над вузлом

- **Перевірити з'єднання** (`POST /test`) — у формі вузла перевіряє зв'язок за введеними (ще не збереженими) параметрами з таймаутом 6 с. Результат: «З'єднання в порядку ({ms} мс)» або «Не вдалося підключитися». Зручно для налагодження адреси/порту/токена/TLS до збереження.
- **Перевірити зараз** (кнопка «Перевірити зараз», `POST /probe/:id`) — позаплановий запит вже збереженого вузла; негайно оновлює статус і метрики (CPU/пам'ять/аптайм/затримку/версії) і записує heartbeat. При невдачі — «Перевірка не вдалася».

Приклад: перевірити та опитати вузол через API мастера. «Перевірити з'єднання» випробовує ще не збережені параметри з форми:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Позаплановий запит вже збереженого вузла з id 7:

```
POST /panel/api/nodes/probe/7
```

- **Оновити панель** (`POST /updatePanel` з тілом `{ids:[...]}`) — запускає на вузлі його штатний самооновлювач: вузол завантажує останній реліз 3X-UI і перезапускається на ньому. Кнопка **Оновити вибрані ({count})** виконує це для кількох відмічених вузлів одразу. Поруч з вузлом показується індикатор: **Доступне оновлення** або **Актуально**, виходячи з порівняння версії панелі вузла з останньою.

Приклад: оновити кілька вузлів одним запитом. В тілі передаються id відмічених вузлів; оновляться лише увімкнені та `online`, решта повернуться як пропущені.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Відповідь виду «Оновлення запущено на 2 вузлах, 1 не вдалося»: вузол 12, наприклад, міг бути offline і тому пропущений.

- Підтвердження: «Оновити {count} вузлів до останньої версії? Кожен вибраний вузол завантажить останній реліз і перезапуститься. Оновлюються лише увімкнені вузли в мережі».
- **Оновлюються лише увімкнені вузли в статусі online**. Вимкнений вузол у результатах позначається «node is disabled», offline — «node is offline». Підсумок: «Оновлення запущено на {ok} вузлах, {failed} не вдалося». Якщо не вибрано жодного підходящого вузла — «Виберіть хоча б один увімкнений вузол у мережі».

У діалозі підтвердження оновлення (як для одиночного вузла, так і для масового) є прапорець **Оновити до каналу розробки (останній коміт)**. Якщо його відмітити, вибрані вузли встановлять rolling-збірку dev-latest (останній коміт гілки main) замість стабільного релізу; при знятому прапорці вузол оновлюється за своїм звичайним каналом. При увімкненому прапорці під ним виводиться попередження: «Збірки розробки відстежують кожен коміт у main і не є стабільними релізами — автоматичного відкату немає». Прапор dev передається через POST /panel/api/nodes/updatePanel на вузол, і той запускає оновлення саме по dev-каналю.

- **Set Cert from Panel** (допоміжна, GET /webCert/:id) — при створенні inbound на вузлі дозволяє підставити шляхи до **власного** web-TLS-сертифіката вузла (а не центральної панелі), щоб файли існували саме на вузлі. Вимагає, щоб вузол був увімкнений і доступний.
- **Видалити вузол** (POST /del/:id) — підтвердження: «Видалити вузол "{name}"? Це зупинить моніторинг вузла. Сама віддалена панель не буде зачеплена». Видаляє запис вузла і його накопичену статистику трафіку; віддалена панель продовжує працювати як зазвичай. **Видалити вузол можна лише після того, як з нього зняті всі inbounds**. Якщо до вузла все ще прив'язаний хоча б один inbound (через node_id), панель відхилить видалення з помилкою виду «cannot delete node: N inbound(s) still attached to it; detach or delete them first» — спочатку відкріпіть або видаліть ці inbounds, потім видаляйте вузол. Це виключає «осиротілі» inbounds із висячим посиланням на видалений вузол.

12.6. Історія метрик

Кнопка/графік історії звертається до GET /history/:id/:metric/:bucket. Доступні метрики: **cpu** і **mem** — вони накопичуються при кожному успішному heartbeat. Розмір інтервалу агрегації (bucket, у секундах) обмежений білим списком:

Приклад: запит історії. Графік завантаження CPU вузла 7 з агрегацією по 60-секундних інтервалах (повертається до 60 точок):

```
GET /panel/api/nodes/history/7/cpu/60
```

Для пам'яті та режиму «реального часу» (2 с) — відповідно .../7/mem/60 і .../7/cpu/2. Значення поза білим списком відхиляються («invalid metric» / «invalid bucket»).

Bucket (с)	Призначення
2	Режим реального часу
30	Інтервали по 30 с
60	Інтервали по 1 хв
120	Інтервали по 2 хв
180	Інтервали по 3 хв
300	Інтервали по 5 хв

Повертається до 60 точок. Неприпустима метрика або bucket відхиляються («invalid metric» / «invalid bucket»).

12.7. Як синхронізуються inbounds і клієнти

Inbound «належить» вузлу через поле `node_id` (в редакторі inbound вибирається вузол):

Приклад: токен у формі вузла. Токен береться на дочірній панелі (Налаштування → API Токен) і вставляється в поле **API Токен** мастера. При кожному опитуванні мастер надсилає його в заголовок:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Якщо на дочірній панелі заданий **базовий шлях** (web base path), наприклад `/secret/`, мастер сам підставить його перед `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

- 1. Розгортання конфігурації (reconcile).** При будь-якій зміні inbound/клієнта, прив'язаного до вузла, вузол позначається «брудним». Фонова задача для кожного увімкненого вузла в **статусі online** за наявності змін розгортає на вузол його inbounds (по `node_id`) і потім скидає прапор «брудного» стану. Вузол, який вимкнений, offline або «брудний», вважається «очікуючим» — деплой на нього відкладається до відновлення зв'язку.
- 2. Збір трафіку.** Та сама задача запитує у вузла знімок трафіку і зливає його в локальну статистику. На основі злитого трафіку виконується перевірка вичерпання лімітів/термінів і при необхідності відключення клієнтів; лічильник «вичерпано» по вузлу відображає саме це. Якщо вузол недоступний, його онлайн-клієнти очищаються.

Для клієнта, прив'язаного одразу до кількох панелей, master у тій самій задачі додатково розсилає вузлам **сумарний по всіх панелях** розхід трафіку цього клієнта (окремою таблицею на вузлі, ключ — GUID мастера; перезаписується при кожному надсиланні, тому скидання на стороні мастера теж поширюється). На вузлі в трафіку клієнта відображається більше з двох значень — локальне або надіслане, — а при перевищенні загальної квоти клієнт відключається **локально на самому вузлі** (через той самий механізм перезапуску Xray при авто-відключенні, що розриває вже встановлені з'єднання). Це усуває ситуацію, коли вузол бачив лише свою частку трафіку, недораховував розхід і продовжував обслуговувати клієнта, що вже вичерпав загальний ліміт. При скиданні трафіку, авто-продовженні або видаленні клієнта надіслані лічильники очищаються.

При **першій** синхронізації inbound, розміщеного на вузлі (додавання нового вузла або повторний імпорт inbound), мастер ініціалізує лічильники трафіку клієнтів реальними значеннями з вузла. Раніше в цій ситуації загальний лічильник inbound переносився коректно, а індивідуальні лічильники клієнтів обнулялися, і мастер занижував розхід клієнтів на всю історію, накопичену до підключення вузла. Тепер, якщо inbound створюється в тій самій синхронізації, новий рядок `client_traffics` успадковує значення лічильника з вузла (baseline виставляється рівним йому, тому наступна дельта дорівнює нулю і трафік не враховується двічі). Засів лічильника застосовується лише для inbound, створеного в цьому ж проході: клієнт, що знову з'явився під вже існуючим inbound, як і раніше стартує з нуля (захист від «фантомного» трафіку), а щойно видалений клієнт не «воскресає» при пересозданні його inbound.

3. Heartbeat. Окрема фонові задача періодично опитує всі **увімкнені** вузли (з обмеженням паралелізму) через `panel/api/server/status`, оновлює статус/метрики/версії і, за наявності веб-клієнтів, розсилає оновлене дерево вузлів по WebSocket.

12.8. Ланцюжки вузлів (підвузли / транзитивні вузли)

Топологія може бути не плоскою: вузол сам може бути мастером для своїх вузлів. Такі нижчестоящі панелі показуються у вас як **Підвузли** — це **проекції «лише для читання»**, отримані від прямого вузла.

- Підказка: «Лише для читання: підпорядкований вузол, доступний через {parent}. Керуйте ним з власної панелі {parent}». Тобто підвузол не можна тут редагувати, видаляти або оновлювати — всі операції з ним виконуються з панелі його прямого батька.
- Ідентичність підвузла визначається його GUID; завдяки цьому онлайн-клієнти та inbounds враховуються саме під тим фізичним вузлом, який їх хостить, навіть у ланцюжку `Node1 → Node2 → Node3` (мастер «проходить» на один рівень вглиб через кожен прямий вузол).
- Якщо прямий вузол стає недоступним, його кеш підвузлів очищається, і підвузли зникають з дерева до відновлення зв'язку.

12.9. Виправлення 3.5.0 (стабільність вузлів)

Пакет виправлень у 3.5.0, помітних оператору вузлів:

- **Збереження клієнта без змін більше не руйнує трафік вузла.** Раніше будь-яке збереження клієнта (навіть «відкрив — зберіг») надсилало на вузол повне оновлення inbound з повторним створенням обробника Xray — вузол значився онлайн, але не пропускав трафік до ручного перезапуску. Тепер по-ор збереження нічого не роблять, а реальні правки йдуть одним легким оновленням без повторного створення.
- **Host-перевизначення вузла приймаються в майстер:** при першому прийнятті inbound вузла його рядки Hosts (SNI, відбиток, ALPN тощо) імпортуються, і підписки несуть правильні TLS-параметри.
- **Авто-продовження на вузлі відкриває свіже вікно квоти** (пакет виду «100 ГБ кожні 30 днів» знову видає повний обсяг).
- **Видалення клієнта на майстері повністю видаляє його на вузлах** (запис + трафік), а не лише відв'язує; залишки попередніх масових видалень можна прибрати дією «Видалити неприв'язаних клієнтів» на вузлі.
- **Inbound'и вузла не вимітаються до першого прийняття:** збереження щойно доданого вузла більше не може видалити його живі inbound'и.
- **Один некоректний inbound не зупиняє синхронізацію трафіку вузла** (заразом легасі-inbound'и `socks` на вузлах перейменовуються в `mixed`).

- **Перевірка конфлікту портів обмежена своїм вузлом** — правка адреси прослуховування вузлового inbound більше не відхиляється через такий самий порт на іншому вузлі («port ... already used by inbound ...»).

12.10. Вузли: нове в 3.3.0

У версії 3.3.0 розділ **Вузли** отримав три помітних покращення: коректну атрибуцію трафіку та онлайн-клієнтів у багаторівневих (multi-hop) топологіях, синхронізацію client-IP між вузлами та окремий індикатор статусу для випадку, коли панель вузла жива, а ядро Xray на ній впало. Слова inbound/outbound далі не перекладаються.

1. Multi-hop: правильна атрибуція трафіку по ланцюжку під-вузлів

Раніше лічильники (число inbound, онлайн-клієнтів, вичерпаних) рахувалися на рівні «прямого» вузла. Якщо у вас ланцюжок виду **Мастер → Вузол1 → Вузол2 → Вузол3**, все, що фізично живе на **Вузол2 / Вузол3**, помилково приписувалося **Вузол1**, через який воно дійшло до мастера. У 3.3.0 атрибуція йде за реальним джерелом.

Як це влаштовано:

- **Під-вузли стають видимі як окремі рядки.** Кожна панель публікує список своїх прямих вузлів; включаються лише вузли з відомим **Guid** — стабільна ідентичність потрібна, щоб приписати вузол на один «стрибок» вгору. Мастер періодично (з heartbeat-завдання) підтягує ці списки і кешує їх, а потім додає до прямих вузлів «транзитивні» під-вузли.
- **Транзитивні вузли — лише для читання.** В UI вони позначаються як **«Підвузол»** з підказкою: *«Лише для читання: підпорядкований вузол, доступний через {батько}. Керуйте ним з власної панелі {батько}.»* Кнопок керування у такого рядка немає — вузлом керують з панелі його безпосереднього батька.
- **Ієрархія через GUID.** У прямого вузла **ParentGuid** — це GUID самого мастера; у транзитивного — GUID його батьківського вузла. Так будується дерево.
- **Джерело правди для лічильників — origin_node_guid на inbound.** Це **panelGuid** вузла, який фізично тримає цей inbound. Воно пропоставляється при синхронізації inbound з вузла і **зберігається як є при подальших стрибках**, тому глибоко вкладений inbound приписується реальному вузлу, а не проміжному. За цим GUID перераховуються лічильники числа inbounds, онлайн-клієнтів і вичерпаних клієнтів. Логіка вибору ключа:

Стан inbound	Чому приписується
origin_node_guid заданий	цьому GUID (реальний вузол-джерело)
порожньо, але заданий node_id	синтетичний GUID вузла (старий білд, ще не повідомив свій panelGuid)
порожньо і node_id порожній	власному GUID мастера (inbound на локальному Xray)

Онлайн-клієнти так само групуються за GUID, тому кожен рядок вузла показує лише тих, хто реально підключений саме до нього.

Що бачить користувач: у плоскій топології (вузли напряму під мастером) нічого не змінюється — лічильники за GUID і за `id` збігаються. Але як тільки з'являється ланцюжок вузлів, у списку виникають рядки-«Підвузли», і числа inbound/онлайн/вичерпаних у кожного вузла тепер відображають саме його власне навантаження, а не суму всього, що пройшло через нього транзитом.

2. Синхронізація client-IP з access.log між вузлами

Ліміт по IP (`limitIp` у клієнта) спирається на адреси, які Xray пише у свій `access.log`. Раніше кожен вузол бачив лише підключення до себе, тому обмеження «не більше N IP на клієнта» не працювало в кластері: клієнт міг підключитися до різних вузлів і обійти ліміт. У 3.3.0 спостережені IP синхронізуються по всьому кластеру.

Як це працює:

- На кожному вузлі фонове завдання парсить `access.log`, витягуючи по кожному рядку IP, email клієнта і мітку часу, і складає їх у локальну таблицю (по одному запису на email, IP зберігаються як JSON-масив `{ip, timestamp}`). Локальні адреси `127.0.0.1` і `::1` відкидаються.
- Синхронізація **раз на 10 секунд** виконує двосторонній обмін по кожному увімкненому вузлу в мережі: тягне IP з вузла і зливає їх у локальну таблицю, а потім віддає вузлу зведену картину мастера.
- Злиття об'єднує старі та вхідні спостереження **без подвійного обліку** одного IP, побаченого на кількох вузлах, і **без воскресіння застарілих** записів: застосовується той самий поріг давності, що і в локальному завданні — **30 хвилин**. По кожному IP зберігається найсвіжіша мітка часу. Записи з чужих вузлів отримують новий локальний `id` (`id`-простори вузлів незалежні); конкурентна вставка одного і того ж email захищена від дублів.
- При підрахунку ліміту «живим» вважається IP, який або помічений у поточному скануванні локально, або має дуже свіжу мітку із синхронізованої бази (**в межах 2 хвилин**). Саме це і змушує ліміт працювати в масштабі всього кластера, навіть якщо адреса була помічена на іншому вузлі. При перевищенні ліміту найстаріші «живі» IP надсилаються до журналу `fail2ban`, а з'єднання примусово розриваються (`remove/re-add` клієнта через Xray API).

Що бачить користувач: обмеження по числу IP тепер діє на весь кластер, а не на кожен вузол окремо; в панелі по клієнту видно IP, помічені на будь-якому вузлі (в межах вікна в 30 хвилин). Окремої кнопки/налаштування для цього немає — синхронізація йде автоматично у фоні, за умови що у вузла увімкнений і доступний `access.log` (для самого ліміту також потрібен Fail2Ban на вузлі).

3. Окремий індикатор статусу: панель вузла онлайн, але Xray впав

Раніше статус вузла був, по суті, «в мережі / не в мережі». Якщо панель вузла відповідала, вузол вважався онлайн — навіть коли ядро Xray на ньому не працювало, і клієнти фактично не могли підключитися. У 3.3.0 здоров'я панелі та здоров'я ядра Xray розділені.

Як це влаштовано:

- При опитуванні вузла мастер бере з відповіді віддаленого `/panel/api/server/status` поля `xray.state` і `xray.errorMessage` і зберігає їх у вузлі. Ці поля заповнюються навіть при успішному пінгу панелі, коли ядро нездорове, — саме щоб відрізнити доступність панелі від стану Xray.
- Значення `xray.state`: `"running"` (працює), `"stop"` (зупинений), `"error"` (помилка).

- Ці значення транслюються в статуси вузла. До звичних додалися нові:

Ключ статусу	Підпис	Коли показується
online	«В мережі»	панель відповідає, Xray працює (running)
offline	«Не в мережі»	панель недоступна / пінг не пройшов
unknown	«Невідомо»	стан ще не визначено
xrayError	«Помилка Xray»	панель онлайн, але ядро Xray в стані error (є errorMsg)
xrayStopped	«Зупинений»	панель онлайн, але Xray зупинений (stop)

- Для подібного стану в UI використовується **окремий фіолетовий індикатор** (колір, відмінний від зеленого «в мережі» і червоного «не в мережі»). Фіолетовий прямо сигналізує: до вузла додзвонитися можна, проблема – в самому ядрі Xray, а не в мережі або в самій панелі.

Що бачить користувач: замість оманливого «зеленого» при впалому ядрі вузол підсвічується **фіолетовим** зі статусом **«Помилка Xray»** або **«Зупинений»**. Це одразу показує, що лагодити треба Xray на вузлі (перезапустити ядро, подивитися errorMsg), а не розбиратися з доступністю самого вузла. Той самий xrayState / xrayError передається і в транзитивні під-вузли (див. пункт 1), тому некоректний стан ядра видно по всьому ланцюжку.

13. Налаштування панелі

Розділ «Налаштування» (заголовок сторінки — **Налаштування**, англ. *Panel Settings*) керує поведінкою самої вебпанелі 3X-UI: на якій адресі та порту вона слухає, як захищена, як взаємодіє з Telegram-ботом і зовнішніми сервісами, у якому часовому поясі виконує заплановані завдання. Кожен параметр зберігається в таблиці `settings` бази даних у вигляді пари «ключ — значення»; якщо значення в БД відсутнє, застосовується значення за замовчуванням.

Важливо — застосування змін. Будь-яку зміну на цій сторінці потрібно зберегти кнопкою **Зберегти** (*Save*), а потім перезапустити панель, щоб зміни набули чинності. Дослівна підказка: «Збережіть зміни та перезапустіть панель для їх застосування.» Під час збереження показується сповіщення «Налаштування змінено».

13.1. Збереження та перезапуск панелі

Елемент	Призначення
Зберегти (<i>Save</i>)	Записує всі поля форми в БД (<code>POST /panel/setting/update</code>). Перед записом значення проходять валідацію — некоректні адреси, порти чи шляхи будуть відхилені, і панель поверне помилку.
Перезапустити панель (<i>Restart Panel</i>)	Перезапускає вебсервер панелі (<code>POST /panel/setting/restartPanel</code>). Перезапуск відбувається із затримкою 3 секунди. Підказка: «Ви впевнені, що хочете перезапустити панель? Підтвердьте, і перезапуск відбудеться через 3 секунди. Якщо панель буде недоступна, перевірте лог сервера». При успіху — «Панель успішно перезапущено».
Відновити налаштування за замовчуванням (<i>Reset to Default</i>)	Видаляє всі збережені в БД налаштування, після чого панель використовує значення за замовчуванням. Облікові дані адміністратора цією операцією не скидаються.

Перезапуск виконується надсиланням процесу панелі сигналу `SIGHUP` (або через зареєстрований хук перезапуску). На Windows автоматичний перезапуск через сигнал не підтримується. **Зміни параметрів прослуховування (IP, порт, шлях, домен, сертифікати, часовий пояс) застосовуються лише після перезапуску панелі.**

13.2. Загальні налаштування (вкладка «Панель» / *General*)

Мова інтерфейсу (*Language*)

Мова вебінтерфейсу панелі. Доступні мови: `en-US` (англійська), `ru-RU` (російська), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Це налаштування відображення і не впливає на роботу Xray.

Тип календаря (*Calendar Type*)

- **Ключ:** `datepicker`
- **Значення за замовчуванням:** `gregorian` (григоріанський).

- **Призначення:** тип календаря, що використовується у виборі дат (наприклад, при заданні терміну дії клієнтів). Підказка: «Заплановані завдання виконуватимуться відповідно до цього календаря.» Альтернативне значення — перський (джалалі) календар, що затребуваний в іранської аудиторії панелі.

Розмір нумерації сторінок (*Pagination Size*)

- **Ключ:** `pageSize`
- **Значення за замовчуванням:** `25`
- **Допустимі значення:** ціле від `0` до `1000`.
- **Призначення:** кількість рядків на сторінці в таблицях (списах підключень/inbound). Підказка: «Визначити розмір сторінки для таблиці підключень. Встановіть 0, щоб вимкнути» — при `0` посторінковий вивід вимикається, і всі записи показуються єдиним списком.
- **Перезапуск панелі не потрібен** (налаштування відображення).

Перезапустити Xray після авто-відключення (*Restart Xray After Auto Disable*)

- **Ключ:** `restartXrayOnClientDisable`
- **Значення за замовчуванням:** `true`
- **Призначення:** при автоматичному відключенні клієнта (через закінчення терміну дії або досягнення ліміту трафіку) Xray перезапускається, щоб розірвати вже встановлені з'єднання такого клієнта. Підказка: «Коли клієнт автоматично відключається через закінчення терміну дії або ліміту трафіку, перезапустити Xray.» Сама функція не змінилася — перемикач просто живе на вкладці «Панель» (*General*) поруч з іншими загальними налаштуваннями.

Модель примітки та символ розділення (*Remark Model & Separation Character*)

- **Ключ:** `remarkModel`
- **Значення за замовчуванням:** `-ieo`
- **Призначення:** задає, як формується ім'я (remark) конфігурації в підписці. Рядок складається з **першого символу** — роздільника, і далі **послідовності літер-порядку**:
 - `i` — примітка inbound (*inbound remark*);
 - `e` — email клієнта;
 - `o` — додаткова мітка (*extra*).

При значенні за замовчуванням `-ieo` роздільником служить `-`, а порядок частин: inbound → email → extra (наприклад, `MyInbound-user@mail-extra`). Порожні частини пропускаються. Поле «Приклад примітки» (*Sample Remark*) в інтерфейсі показує попередній перегляд формованого імені. Включення email в ім'я додатково залежить від параметра «Включати Email у назву» в налаштуваннях підписки (розділ про підписку).

Приклад: як значення `remarkModel` впливає на ім'я конфігурації. Припустімо, inbound називається `VLESS-Reality`, email клієнта — `alex@vpn`, а додаткова мітка — `RU`. Тоді:

Значення поля	Підсумкове ім'я (remark)
-ieo (за замовчуванням)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (пробіл-роздільник, лише мітка)	RU

Перший символ рядка — завжди роздільник; решта літер задають, які частини і в якому порядку увійдуть в ім'я.

13.3. Доступ до панелі: IP, порт, шлях, домен, сертифікат

Ця група визначає мережеву точку входу панелі. **Усі зміни тут застосовуються лише після перезапуску панелі.**

Поле	Ключ	Значення за замовчуванням	Опис
IP-адреса для керування панеллю (<i>Listen IP</i>)	webListen	"" (порожньо)	IP, на якому слухає вебпанель. Порожньо = слухати на всіх IP. Підказка: «Залиште порожнім для підключення з будь-якого IP». Якщо задано, має бути коректною IP-адресою (інакше збереження відхиляється).
Домен панелі (<i>Listen Domain</i>)	webDomain	"" (порожньо)	Доменне ім'я панелі для перевірки запиту за доменом. Порожньо = приймати підключення з будь-яких доменів та IP. Підказка: «Залиште порожнім для підключення з будь-яких доменів та IP.»
Порт панелі (<i>Listen Port</i>)	webPort	2053	Порт, на якому працює панель. Підказка: «Порт, на якому працює панель». Допустимо 1–65535. Порт має бути вільним; панель і служба підписки не можуть одночасно використовувати однакову пару IP: порт.
URI-шлях (<i>URI Path</i>)	webBasePath	/	Базовий шлях URL панелі (basePath). Підказка: «Має починатися з '/' і закінчуватися '/'». Під час збереження панель автоматично додає провідний і завершальний / , якщо вони відсутні. Заборонені символи в шляху відхиляються.

Сертифікат панелі (TLS / HTTPS)

Поле	Ключ	Значення за замовчуванням	Опис
Шлях до файлу публічного ключа сертифіката панелі (<i>Public Key Path</i>)	webCertFile	""	Повний шлях до файлу сертифіката (ланцюжка). Підказка: «Введіть повний шлях, що починається з '/'».
Шлях до файлу приватного ключа сертифіката панелі (<i>Private Key Path</i>)	webKeyFile	""	Повний шлях до файлу приватного ключа. Підказка: «Введіть повний шлях, що починається з '/'».

Якщо задано **хоча б один** зі шляхів сертифіката/ключа, панель під час збереження намагається завантажити пару «сертифікат + ключ»; при помилці (неіснуючий файл, невідповідність ключа та сертифіката) збереження відхиляється. Коли задано обидва коректні шляхи, панель переходить на HTTPS. Обидва поля порожні = панель працює за звичайним HTTP.

Попередження безпеки (*Security warnings*). Панель показує блок «Вашу панель може бути відкрито:» з попередженнями, якщо виявляє небезпечну конфігурацію:

- робота за звичайним HTTP — «налаштуйте TLS для продакшну»;
- стандартний порт 2053 — «змінить його на випадковий»;
- базовий шлях за замовчуванням / — «змінить його на випадковий»;
- стандартний шлях підписки /sub/ та JSON-підписки /json/ — «змінить його». Це рекомендації, а не блокування.

13.4. Сесія, проксі панелі та довірені проксі (вкладка «Проксі та сервер» / *Proxy and Server*)

Тривалість сесії (*Session Duration*)

- **Ключ:** `sessionMaxAge`
- **Значення за замовчуванням:** 360 (хвилин, тобто 6 годин).
- **Допустимі значення:** від 1 до 525600 хвилин (1 рік).
- **Призначення:** як довго адміністратор залишається авторизованим без повторного входу. Одиниця — **хвилина**. Підказка: «Тривалість сесії в системі (значення: хвилина)».

Outbound для трафіку панелі (*Panel Traffic Outbound*)

- **Ключ:** `panelOutbound`
- **Значення за замовчуванням:** `""` (порожньо = пряме підключення).
- **Призначення:** задає Xray-outbound, через який панель надсилає **власні запити** — перевірки версій і завантаження панелі/Xray, звернення до Telegram, звичайне оновлення гео-файлів — щоб обійти серверну фільтрацію GitHub/Telegram. Поле являє собою **випадаючий список**: у ньому перелічені outbound'и з шаблону конфігурації Xray, outbound'и з підписок на outbound, а також маршрутні **балансувальники** (окремою групою). Outbound'и типу `blackhole` зі списку виключені — маршрутизувати завантаження в «чорну діру» немає сенсу. Дослівна підказка: «Маршрутизує власні запити панелі — перевірки версій та завантаження панелі/Xray, Telegram і звичайне оновлення гео-файлів — через цей вихідний Xray для обходу серверної фільтрації GitHub/Telegram. Локальний міст-вихідний додається до робочої конфігурації автоматично і застосовується на льоту. Вбудоване в Xray автооновлення Geodata не зачіпається; у нього свій вихідний для завантаження. Залиште порожнім для прямого підключення.»

Як це працює. При виборі outbound'а панель сама додає в робочий конфіг службовий loopback-inbound (SOCKS-міст з тегом `panel-egress`) і правило маршрутизації, яке завертає власний HTTP-трафік панелі на вибраний outbound. Якщо вибрано балансувальник, у правило підставляється

balancerTag , і трафік панелі розподіляється між його учасниками. Міст і правило застосовуються **на льоту**, без повного перезапуску панелі. Залиште поле порожнім для прямого з'єднання. Вбудоване в Xray автооновлення гео-даних цим налаштуванням **не зачіпається** — у нього власний outbound усередині Xray-роутингу.

- **Формат:** socks5:// (або socks5h://) чи http(s)://, за потреби з авторизацією виду socks5://user:pass@host:port . Підтримувані схеми строго: socks5, socks5h, http, https — інші схеми вважаються неприпустимими, і панель тоді відкочується на пряме підключення. Типовий приклад — локальний SOCKS-вхідний самого Xray.
- Дослівна підказка: «Маршрутизує вихідні запити самої панелі (оновлення гео, перевірки версій Xray/панелі, Telegram) через цей проксі для обходу серверної фільтрації GitHub/Telegram. Приймає socks5:// або http(s)://, напр. локальний SOCKS-вхідний Xray. Залиште порожнім для прямого підключення.»
- Невалідний проксі не призводить до помилки збереження — панель просто використовує пряме з'єднання і пише попередження в лог.

Приклад значень поля. Якщо на сервері вже піднято локальний SOCKS-вхідний Xray на порту 10808, спрямуйте через нього власні запити панелі:

```
socks5://127.0.0.1:10808
```

Для зовнішнього HTTP-проксі з авторизацією:

```
http://user:pass@proxy.example.com:8080
```

Після збереження і перезапуску панель тягтиме оновлення гео-баз, перевірятиме версії та звертатиметься до Telegram уже через вказаний проксі.

Довірені CIDR проксі (*Trusted proxy CIDRs*)

- **Ключ:** trustedProxyCIDRs
- **Значення за замовчуванням:** 127.0.0.1/32, ::1/128 (лише локальний хост).
- **Формат:** список IP-адрес або CIDR-підмереж через кому (наприклад 10.0.0.0/8, 192.168.1.5). Кожен елемент перевіряється як IP або CIDR — некоректне значення відхиляється при збереженні.
- **Призначення:** перелічує джерела, яким дозволено встановлювати заголовки X-Forwarded-Host, X-Forwarded-Proto та заголовок реального IP клієнта. Дослівна підказка: «IP/CIDR через кому, яким дозволено встановлювати заголовки forwarded host, proto та client IP.» Потрібно налаштовувати, якщо панель працює за зворотним проксі (nginx, Caddy тощо), щоб коректно визначати клієнтські IP і схему.

Приклад: панель за зворотним проксі. Якщо nginx стоїть на тому самому хості і проксує запити на панель, залиште довіру лише локальному хосту (значення за замовчуванням):

```
127.0.0.1/32, ::1/128
```

Якщо проксі знаходиться на окремому сервері у внутрішній мережі 10.0.0.0/8, додайте його підмережу, інакше панель проігнорує передані ним заголовки і бачитиме IP проксі замість реального клієнта:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Приклад відповідного блоку nginx, який передає реальний IP і схему:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)

Увімкнути Telegram бота (*Enable Telegram Bot*)

- **Ключ:** `tgBotEnable`
- **Тип/за замовчуванням:** логічний, `false`.
- **Призначення:** вмикає роботу Telegram-бота. Підказка: «Доступ до функцій панелі через Telegram-бота».

Telegram-токен (*Telegram Token*)

- **Ключ:** `tgBotToken`
- **За замовчуванням:** `""`.
- **Призначення:** токен бота. Підказка: «Необхідно отримати токен у менеджера ботів Telegram @botfather».
- **Особливість безпеки:** токен належить до секретних значень. У відповіді панелі на читання налаштувань він не повертається (поле очищується, віддається лише прапорець «налаштований/не налаштований»). Якщо під час збереження поле залишити порожнім, попередній збережений токен **зберігається** (не затирається).

Мова Telegram-бота (*Telegram Bot Language*)

- **Ключ:** `tgLang`
- **За замовчуванням:** `en-US`.
- **Призначення:** мова повідомлень бота (незалежно від мови вебінтерфейсу). Список доступних мов збігається з мовами панелі.

User ID адміністратора бота (*Admin Chat ID*)

- **Ключ:** `tgBotChatId`
- **За замовчуванням:** `""`.
- **Формат:** один або декілька числових Telegram User ID **через кому**.
- **Призначення:** одержувачі сповіщень та адміністратори, яким дозволено керувати панеллю через бота. Підказка: «Один або декілька User ID адміністратора(-ів) Telegram-бота. Для отримання User ID використовуйте @userinfobot або команду '/id' у боті.»

Частота сповіщень (*Notification Time*)

- **Ключ:** `tgRunTime`
- **За замовчуванням:** `@daily` (раз на добу).
- **Формат:** рядок у форматі **Crontab** (підтримуються як стандартні cron-вирази, так і скорочення виду `@daily`, `@hourly`, `@every 1h`). Підказка: «Вкажіть інтервал сповіщень у форматі Crontab». Керує періодичними звітами бота.

Приклади значень поля.

Значення	Коли бот шле звіт
<code>@daily</code>	раз на добу опівночі (за замовчуванням)
<code>@hourly</code>	щогодини
<code>@every 6h</code>	кожні 6 годин
<code>0 9 * * *</code>	щодня о 09:00
<code>30 8 * * 1</code>	щопонеділка о 08:30

Час рахується в часовому поясі з налаштування «Часовий пояс» (п. 13.6).

SOCKS-проксі (*SOCKS Proxy*)

- **Ключ:** `tgBotProxy`
- **За замовчуванням:** `""`.
- **Призначення:** SOCKS5-проксі окремо для підключення бота до Telegram. Підказка: «Якщо для підключення до Telegram вам потрібен проксі Socks5, налаштуйте його параметри згідно з посібником.» Застосовується саме до трафіку бота (відрізняється від загального «Мережевого проксі панелі» з п. 13.4).

Telegram API Server (*Telegram API Server*)

- **Ключ:** `tgBotAPIServer`
- **За замовчуванням:** `""` (використовувати стандартний сервер `api.telegram.org`).
- **Формат:** URL `http(s)://...`; під час збереження проходить перевірку коректності URL — невалідна адреса відхиляється. Підказка: «Використовуваний API-сервер Telegram. Залиште порожнім, щоб використовувати сервер за замовчуванням.» Потрібен для самостійно розгорнутого Telegram Bot API server.

Сповіщення бота (група «Сповіщення» / *Notifications*)

Поле	Ключ	За замовчуванням	Опис
Резервне копіювання бази даних (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Надсилати в Telegram файл резервної копії БД разом зі звітом. Підказка: «Надсилати сповіщення з файлом резервної копії бази даних».

Поле	Ключ	За замовчуванням	Опис
Сповіщення про вхід (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Сповіщати при спробі входу в панель. Підказка: «Відображає ім'я користувача, IP-адресу та час, коли хтось намагається увійти у вашу панель.»
Затримка сповіщення про закінчення сесії (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	За скільки днів до закінчення терміну дії клієнта слати сповіщення. <code>0</code> — вимкнено. Допустимо <code>>= 0</code> . Підказка: «Отримання сповіщення про закінчення терміну дії сесії до досягнення порогового значення (значення: день)».
Поріг трафіку для сповіщення (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Поріг залишку трафіку для сповіщення. Підказка: «Отримання сповіщення про вичерпання трафіку до досягнення порога (значення: ГБ)». Допустимо <code>0–100</code> .
Поріг навантаження на ЦП (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Сповіщати адміністраторів, якщо завантаження CPU перевищує поріг (у %). Допустимо <code>0–100</code> . Підказка: «Сповіщення адміністраторів у Telegram, якщо навантаження на ЦП перевищує цей поріг (значення: %)».

13.6. Дата і час (вкладка «Дата і час» / *Date and Time*)

Часовий пояс (*Time Zone*)

- **Ключ:** `timeLocation`
- **Значення за замовчуванням:** `Local` (системний часовий пояс сервера).
- **Формат:** ім'я зони з бази IANA tz (наприклад, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Призначення:** часовий пояс, у якому панель виконує заплановані завдання (звіти бота, скидання/перевірки трафіку, закінчення термінів). Підказка: «Заплановані завдання виконуються відповідно до часу в цьому часовому поясі».
- **Валідація:** під час збереження зона перевіряється — неіснуюча зона відхиляється. Якщо пізніше в БД опиниться некоректне значення, панель у рантаймі відкотиться на `Local`, а при недоступності і його — на `UTC`.

13.7. Зовнішній трафік і поведінка Xray (вкладка «Зовнішній трафік» / *External Traffic*)

Поле	Ключ	За замовчуванням	Опис
Інформація про зовнішній трафік (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Сповіщати зовнішній API при кожному оновленні трафіку. Підказка: «Сповіщати зовнішній API при кожному оновленні трафіку.»

Поле	Ключ	За замовчуванням	Опис
URI інформації про зовнішній трафік (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL, на який панель надсилає оновлення трафіку. Проходить перевірку коректності URL при збереженні. Підказка: «Оновлення трафіку надсилаються на цей URI».
Перезапускати Xray після автовідключення (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Перезапускати Xray, коли клієнт автоматично відключається через закінчення терміну або перевищення ліміту трафіку. Підказка: «Коли клієнт автоматично відключається через закінчення терміну дії або ліміту трафіку, перезапускати Xray.» Перемикач знаходиться на вкладці «Панель» (General) — див. п. 13.2; тут наведений для повноти.

13.8. Інше: шаблон конфігурації Xray та перевірочний URL

Шаблон конфігурації Xray (*xrayTemplateConfig*)

- **Ключ:** `xrayTemplateConfig`
- **За замовчуванням:** вбудований (embedded) JSON-шаблон, що постачається зі збіркою.
- **Призначення:** базовий JSON-шаблон конфігурації Xray-core, поверх якого панель добудовує inbound/outbound. Це значення **не віддається** у звичайній видачі всіх налаштувань і редагується на окремій сторінці конфігурації Xray, а не в загальному списку полів налаштувань панелі. Стандартний шаблон за замовчуванням доступний через `GET /panel/setting/getDefaultJsonConfig`.

URL перевірки вихідних (*xrayOutboundTestUrl*)

- **Ключ:** `xrayOutboundTestUrl`
- **За замовчуванням:** `https://www.google.com/generate_204`
- **Призначення:** URL, що використовується при перевірці працездатності вихідних (outbound) підключень. При встановленні проходить санітизацію як HTTP(S)-URL.

13.9. Обліковий запис адміністратора та API-токени

Ці параметри знаходяться на суміжній вкладці («Обліковий запис» / *Authentication*) і докладно розглянуті в розділі про безпеку; тут — коротка зведення ключів.

- **Зміна облікових даних** (поля «Поточний логін», «Поточний пароль», «Новий логін», «Новий пароль») зберігається окремим запитом `POST /panel/setting/updateUser`. Потрібні правильний поточний логін і пароль; нові логін і пароль не повинні бути порожніми. Повідомлення: «Ви успішно змінили облікові дані адміністратора.» / «Невірне ім'я користувача або пароль».
- **Двофакторна автентифікація (2FA)** — ключі `twoFactorEnable` (за замовчуванням `false`) і секретний `twoFactorToken`. Токен — секрет: при увімкненому 2FA порожнє поле під час збереження не затирає

наявний токен. При **першому** увімкненні 2FA панель інвалідує поточні сесії (підвищується «епоха входу»).

- **API-токени** керуються окремими ендпоінтами (`/panel/setting/apiTokens...`): список, створення (`apiTokens/create`), видалення, увімкнення/вимкнення. Сам токен показується **лише один раз під час створення** і не зберігається в читабельному вигляді: «Скопіюйте цей токен зараз. З міркувань безпеки він не зберігається в читабельному вигляді і більше не буде показаний.»

Подробиці щодо 2FA, паролів, LDAP-синхронізації та форматів підписки (JSON/Clash, fragmentation, noises, mux) винесені у відповідні окремі розділи посібника.

13.10. Зміни API в 3.3.0 (важливо для інтеграцій)

У версії 3.3.0 змінилася структура шляхів серверного API. Якщо у вас є зовнішні інтеграції (скрипти, боти, центральні панелі, CI-завдання), які звертаються до панелі по HTTP, їх **потрібно поправити**, інакше вони перестануть працювати.

⚠ BREAKING CHANGE: ендпоінти `/panel/setting/*` і `/panel/xray/*` **переїхали під** `/panel/api`

Раніше керування налаштуваннями панелі та конфігурацією Xray жило окремо, під шляхами `/panel/setting/*` і `/panel/xray/*`. Тепер обидва набори зареєстровані всередині спільної API-групи `/panel/api`. Старі шляхи **видалені повністю** — запит за ними поверне 404.

Навіщо це зроблено: уся група `/panel/api` проходить через єдину перевірку доступу, тобто ці ендпоінти тепер приймають той самий заголовок `Authorization: Bearer <token>`, що й решта API. API-токен — це повноцінний адміністраторський доступ, і таким чином уся поверхня API стала однаковою.

Що НЕ змінилося: сторінки вебінтерфейсу (SPA-маршрути) `/panel/settings` і `/panel/xray` залишилися на місці — йдеться лише про серверні API-ендпоінти.

Таблиця відповідності шляхів (старий → новий)

Префікс до всіх шляхів нижче — просто додалося `api/` після `/panel/`.

Було (≤ 3.2.x)	Стало (3.3.0)	Метод
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST

Було (≤ 3.2.x)	Стало (3.3.0)	Метод
/panel/setting/apiTokens/delete/:id	/panel/api/setting/apiTokens/delete/:id	POST
/panel/setting/apiTokens/setEnabled/:id	/panel/api/setting/apiTokens/setEnabled/:id	POST
/panel/xray/	/panel/api/xray/	POST
/panel/xray/update	/panel/api/xray/update	POST
/panel/xray/getDefaultJsonConfig	/panel/api/xray/getDefaultJsonConfig	GET
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (i / outbound-subs/*)	/panel/api/xray/outbound-subs (i / outbound-subs/*)	GET/POST/DELETE

Самі імена під-шляхів, тіла запитів і формати відповідей не змінювалися — змінився **лише префікс**.

Як полагодити наявні інтеграції

1. Знайдіть у своїх скриптах/конфігах усі входження `/panel/setting/` і `/panel/xray/`.
2. Замініть префікс: додайте `api/` одразу після `/panel/` (наприклад, `/panel/setting/all` → `/panel/api/setting/all`).
3. Тіла запитів, параметри і формат відповідей правити не потрібно — змінюється лише URL.
4. Оскільки налаштування та Xray-конфігурація тепер під `/panel/api`, до них можна (і потрібно) звертатися за тим самим API-токеном `Authorization: Bearer <token>`, що й до `/panel/api/inbounds/*` та інших ендпоінтів. Не забувайте про CSRF-middleware, яке увімкнене на всю групу `/panel/api`.

Приклад: читання всіх налаштувань через API. Раніше (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Тепер (3.3.0) — додали `api/` після `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Аналогічно перезапуск панелі: `POST /panel/api/setting/restartPanel`. Старий шлях `/panel/setting/restartPanel` тепер поверне 404.

Типізований API: схеми та документація (Swagger / OpenAPI)

У 3.3.0 OpenAPI-специфікація стала повноцінно типізованою. Раніше типізовані відповіді описувалися порожнім об'єктом `{}`; тепер компоненти і схеми (`components.schemas`) генеруються прямо з моделей даних. Завдяки цьому:

- Swagger UI показує реальні моделі даних, а не безликі заглушки.
- Зовнішні генератори (`openapi-generator` тощо) можуть побудувати за специфікацією готові клієнти потрібною мовою.
- До кожної типізованої відповіді прикріплений `$ref` на конкретну модель і додані приклади відповідей.

Де дивитися документацію API:

- **Вбудована сторінка Swagger.** У меню панелі — пункт «Документація API» (SPA-маршрут `/panel/api-docs`). Тут інтерактивно перелічені всі ендпоінти з описами, тілами запитів і прикладами відповідей.
 - **Сира OpenAPI 3.0-специфікація** віддається за адресою `/panel/api/openapi.json` . Цей URL можна згодувати напряму в Postman, Insomnia або `openapi-generator` . Специфікація вбудована в бінарник на етапі збірки; при роботі панелі під нестандартним `webBasePath` поле `servers` у специфікації автоматично переписується під поточний базовий шлях, щоб кнопка «Try it out» і зовнішні генератори били по правильному префіксу.
-

14. Telegram-бот

Панель 3X-UI містить вбудованого Telegram-бота, через якого можна отримувати сповіщення про стан сервера та клієнтів, а також керувати окремими клієнтами прямо з месенджера. Бот працює за технологією long polling (постійне опитування Telegram), тому йому не потрібен зовнішній домен або відкритий порт – достатньо вихідного доступу до серверів Telegram.

Бот розрізняє два типи співрозмовників:

- **Адміністратор** – користувач, чий Telegram User ID вказано в налаштуваннях бота (поле «User ID адміністратора бота»). Має доступ до всіх функцій: статистики сервера, резервного копіювання, керування клієнтами, перезапуску Xray.
- **Клієнт** – будь-який інший користувач, чий Telegram User ID прив'язано до конкретного клієнта вхідного підключення (поле `tgId` клієнта). Бачить лише інформацію про власні підписки.

Приклад: прив'язка клієнта до Telegram. Щоб користувач отримував статистику за своєю підпискою, його числовий Telegram User ID записується в поле `tgId` клієнта. У JSON-налаштуваннях клієнта це виглядає так:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Після цього користувач із User ID `123456789` зможе запросити у бота `/usage ivan` і побачити свою статистику. Той самий ID адміністратор може встановити через кнопку  «Встановити користувача Telegram» у картці клієнта – вручну в JSON лізти не обов'язково.

14.1. Увімкнення та налаштування бота

Усі параметри бота задаються в панелі у розділі **Налаштування** → **Telegram-бот**. Після зміни налаштувань достатньо їх зберегти – панель застосовує їх одразу, перезапуск панелі не потрібен. Якщо змінено прапор увімкнення (`tgBotEnable`), токен, User ID адміністраторів або адресу API-сервера, панель автоматично зупиняє та заново запускає бота з новими параметрами. Попереднє правило про необхідність перезапуску панелі після зміни токена більше не діє.

Поле (UI)	Ключ налаштування	Значення за замовчуванням	Опис
Увімкнути Telegram бота	<code>tgBotEnable</code>	<code>false</code>	Головний перемикач. Підказка: «Доступ до функцій панелі через Telegram-бота». Поки вимкнено, бот не запускається і завдання сповіщень не плануються.

Поле (UI)	Ключ налаштування	Значення за замовчуванням	Опис
Telegram-токен	tgBotToken	(порожньо)	Токен бота. Підказка: «Необхідно отримати токен у менеджера ботів Telegram @botfather». Без непустиго токена бот не стартує.
SOCKS-проксі	tgBotProxy	(порожньо)	Проксі для підключення до Telegram. Підказка: «Якщо для підключення до Telegram вам потрібен проксі Socks5, налаштуйте його параметри відповідно до інструкції».
Telegram API Server	tgBotAPIServer	(порожньо)	Альтернативний API-сервер Telegram. Підказка: «API-сервер Telegram, що використовується. Залиште порожнім, щоб використовувати сервер за замовчуванням».
User ID адміністратора бота	tgBotChatId	(порожньо)	Один або кілька Telegram User ID адміністраторів через кому. Підказка: «Для отримання User ID використовуйте @userinfobot або команду /id в боті».
Частота сповіщень для адміністраторів від бота	tgRunTime	@daily	Розклад періодичного звіту у форматі crontab. Підказка: «Вкажіть інтервал сповіщень у форматі Crontab».
Резервне копіювання бази даних	tgBotBackup	false	Підказка: «Надсилати сповіщення з файлом резервної копії бази даних». Додає резервну копію до періодичного звіту.
Сповіщення про вхід	tgBotLoginNotify	true	Підказка: «Відображає ім'я користувача, IP-адресу та час, коли хтось намагається увійти у вашу панель».
Поріг навантаження на ЦП для сповіщення	tgCpu	80	Поріг завантаження CPU у відсотках (валідація 0–100). Підказка: «Сповіщення адміністраторів у Telegram, якщо навантаження на ЦП перевищує цей поріг (значення: %). При значенні 0 перевірку CPU вимкнено».
Мова Telegram-бота	—	—	Мова, якою бот формує всі повідомлення.

Отримання токена через @BotFather

1. Відкрийте в Telegram діалог з @BotFather.
2. Надішліть команду /newbot та дотримуйтесь інструкцій (ім'я бота та унікальний username , що закінчується на bot).
3. BotFather видасть токен вигляду 123456789:AA... . Скопіюйте його в поле **Telegram-токен**.

Отримання User ID адміністратора

User ID — це числовий ідентифікатор акаунта (не username). Дізнатись його можна двома способами:

- Написати боту @userinfobot.

- Запустити вже налаштованого бота і надіслати йому команду `/id` — він поверне ваш ID.

Отримане число впишіть у поле **User ID адміністратора бота**. Щоб призначити кількох адміністраторів, перерахуйте їхні ID через кому (наприклад, `11111111,22222222`). Кожен ID перевіряється як ціле число; некоректне значення призведе до помилки запуску бота.

Приклад: значення поля «User ID адміністратора бота». Один адміністратор — просто число:

123456789

Два адміністратори через кому (пробіли можна не ставити):

123456789,987654321

Кожне значення повинно бути цілим числом. Запис вигляду `@username` або `123 456` (з пробілом усередині числа) не підійде — бот не запуститься.

Проксі

Підтримуються схеми `socks5://`, `http://` та `https://`. Якщо поле проксі залишено порожнім, бот намагається використати загальний проксі панелі (якщо він заданий і його схема підтримується). URL із непідтримуваною схемою або некоректним синтаксисом ігнорується — бот підключається напряму. Проксі корисний, коли прямий доступ до API Telegram із сервера заблоковано.

Сповіщення електронною поштою (SMTP)

Крім Telegram, ті самі події можна отримувати листами. Канал налаштовується у розділі **Налаштування** → **Email** на вкладці **SMTP Settings**:

Поле (UI)	Ключ налаштування	Значення за замовчуванням	Опис
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Головний перемикач email-сповіщень через SMTP.
SMTP Host	<code>smtpHost</code>	(порожньо)	Хост SMTP-сервера (наприклад, <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Порт SMTP-сервера.
SMTP Username	<code>smtpUsername</code>	(порожньо)	Ім'я користувача для автентифікації SMTP. Використовується також як адреса відправника (From).
SMTP Password	<code>smtpPassword</code>	(порожньо)	Пароль для автентифікації SMTP. Зберігається приховано; якщо пароль уже задано, поле показує ознаку «налаштовано», і його можна залишити порожнім, щоб зберегти поточний.
Recipients	<code>smtpTo</code>	(порожньо)	Список отримувачів через кому (наприклад, <code>admin@example.com, ops@example.com</code>).

Поле (UI)	Ключ налаштування	Значення за замовчуванням	Опис
Encryption	smtpEncryptionType	starttls	Тип шифрування з'єднання: <code>none</code> (без шифрування), <code>starttls</code> (STARTTLS) або <code>tls</code> (невийнятний TLS).

Кнопка **Send Test Email** надсилає пробний лист і показує результат за етапами: **Connection** (підключення), **Authentication** (автентифікація) та **Send** (відправка). Якщо щось не так, діагностика вказує, на якому етапі виникла помилка (наприклад, «Authentication failed – check username and password» або «Server requires STARTTLS – change encryption type»), що спрощує підбір параметрів.

На другій вкладці (**Notifications**) вибираються події, про які надходять листи, – тими самими картками-групами, що й для Telegram (див. «Шина подій і вибір сповіщень» у розділі 14.5).

API-сервер Telegram

За замовчуванням бот звертається до офіційного API Telegram. У полі **Telegram API Server** можна вказати адресу власного Bot API сервера (`telegram-bot-api`). URL перевіряється на безпечність; заблокована або некоректна адреса відкидається, і використовується сервер за замовчуванням.

14.2. Головне меню та кнопки

Меню викликається командою `/start`. Кнопки – це inline-клавіатура, прикріплена до повідомлення; набір кнопок залежить від того, є ви адміністратором чи клієнтом.

Меню адміністратора

Кнопка	Дія
 Відсортований звіт про використання трафіку	Перераховує всіх клієнтів, відсортованих за трафіком, із витратою кожного; «зайві» email без даних позначаються «  Немає результатів».
 Стан сервера	Зведення по серверу (див. розділ 14.5). Кнопка «  Оновити» перемальовує дані.
Скинути весь трафік	Скидає лічильники трафіку всіх клієнтів. Запитує підтвердження («Ви впевнені?  »), потім для кожного клієнта виводить «  Успішно» або «  Невдача», наприкінці – «Скидання трафіку завершено для всіх клієнтів».
 Резервна копія БД	Надсилає файл бази даних і <code>config.json</code> (див. розділ 14.6).
 Лог банів	Надсилає файли логів забаних за IP-лімітом адрес.
 Вхідні підключення	Зведення по всіх inbound: Remark, порт, трафік, кількість клієнтів, дата закінчення.
 Незабаром кінець	Список inbound і клієнтів, у яких незабаром вичерпається трафік або термін (див. розділ 14.5).
 Команди	Виводить довідку з команд адміністратора.
 Онлайн	Кількість і список клієнтів, що знаходяться онлайн; натискання на email відкриває картку клієнта. Кнопка «  Оновити».
 Всі клієнти	Відкриває вибір inbound, потім список його клієнтів – для перегляду/керування.

Кнопка	Дія
+ Новий клієнт	Запускає майстер додавання клієнта (вибір inbound → чернетка → підтвердження).
Налаштування підписки / індивідуальні посилання / QR-код	Вибір inbound і клієнта для отримання посилання на підписку, індивідуальних посилань або QR-кодів.

Меню клієнта

Клієнту доступний обмежений набір кнопок:

Кнопка	Дія
Статистика клієнта	Показує дані по всіх підписках, прив'язаних до Telegram User ID клієнта.
Команди	Виводить довідку з команд клієнта.
Налаштування підписки	Вибір свого клієнта → посилання на підписку.
Індивідуальні посилання	Вибір свого клієнта → індивідуальні посилання.
QR-код	Вибір свого клієнта → QR-коди.

Якщо у користувача немає жодного клієнта з його Telegram User ID, бот відповідає: « Вашу конфігурацію не знайдено! Будь ласка, попросіть адміністратора використати ваш Telegram User ID у конфігурації. Ваш User ID: ...». Цей ID потрібно передати адміністратору, щоб той вписав його в поле клієнта.

14.3. Команди бота

З версії 3.5.0 у меню «/» Telegram зареєстровано **всім** команд:

Команда	Опис (з меню)	Доступ	Що робить
/start	Показати головне меню	всі	Привітання; адміністратору додатково показує «  Ласкаво просимо до бота управління!» та головне меню.
/help	Довідка по боту	всі	Виводить загальне привітання та пропозицію вибрати пункт меню.
/status	Перевірити статус бота	всі	Відповідає «  Бот функціонує нормально».
/id	Показати ваш Telegram ID	всі	Повертає «  Ваш User ID: ...». Зручно для отримання власного User ID.
/usage	Показати витрати клієнта: /usage email	всі (див. нижче)	Додана в меню в 3.5.0.
/inbound	Пошук inbound: /inbound remark (admin)	адмін	Додана в меню в 3.5.0.
/restart	Перезапустити ядро Xray (admin)	адмін	Додана в меню в 3.5.0.

Команда	Опис (з меню)	Доступ	Що робить
<code>/clearall</code>	Скинути трафік усіх клієнтів (admin)	адмін	Нова в 3.5.0: запитує підтвердження (кнопки «Скасувати» / «Підтвердити скидання трафіку») і обнуляє трафік усіх клієнтів.

Деталі команд-аргументів:

- `/usage [Email]` — пошук клієнта за email.
 - Для **адміністратора** показує повну картку клієнта (з кнопками керування).
 - Для **клієнта** показує лише його власну підписку з вказаним email (за прив'язкою Telegram User ID).
Без аргументу бот просить вказати email: « Будь ласка, вкажіть email для пошуку».
- `/inbound [ім'я підключення]` — тільки для адміністратора. Шукає inbound за Remark і виводить його параметри зі статистикою всіх клієнтів. Без аргументу (або для клієнта) — « Невідома команда».
- `/restart` — тільки для адміністратора. Перезапускає Xray Core. Можливі відповіді: « Ядро Xray успішно перезапущено», « Xray Core не запущено» (якщо ядро не працює), « Помилка при перезапуску Xray-core. <Помилка>». Будь-які аргументи після `/restart` призводять до повідомлення про невідому команду з підказкою `/restart`.

У групових чатах команда вигляду `/команда@botusername` обробляється лише якщо username збігається з іменем поточного бота.

Ще три зміни 3.5.0: у списку **онлайн-клієнтів** кнопки підписані email – примітка inbound (однакові імена в різних inbound більше не плутаються); повідомлення **бекапу** і **бан-лога** починаються з рядка `Hostname==...` — зручно, коли один бот обслуговує кілька панелей; пошук картки клієнта за **Telegram ID** працює незалежно від форматування JSON налаштувань (раніше «компактні» записи, що прийшли з вузлів або з імпорту, не знаходилися).

Довідка адміністратора (кнопка «Команди»):

 Для перезапуску Xray Core: `/restart`
 Для пошуку клієнта за email: `/usage [Email]`
 Для пошуку вхідних підключень (зі статистикою клієнтів): `/inbound [ім'я підключення]`
 Ваш Telegram User ID: `/id`

Довідка клієнта:

\$
 Для перегляду інформації про вашу підписку: `/usage [Email]`
 Ваш Telegram User ID: `/id`

14.4. Керування клієнтом (тільки адміністратор)

Відкривши картку клієнта (через «Всі клієнти», «Онлайн», «Незабаром кінець» або `/usage`), адміністратор бачить відомості про клієнта (email, прив'язані inbound, статус «Активний», статус з'єднання, дата закінчення, витрата трафіку) та inline-кнопки керування:

Кнопка	Призначення
 Оновити	Перечитати картку клієнта.

Кнопка	Призначення
 Скинути трафік	Обнулити лічильник трафіку клієнта. Потребує підтвердження «  Підтвердити скидання трафіку? ».
 Ліміт трафіку	Задати ліміт трафіку. Готові значення:  Безліміт (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB або «  Своє » — введення числа на вбудованій цифровій клавіатурі (кнопки 0–9, «  » — скидання до 0, «  » — стерти останню цифру, «  Підтвердити: N »). Значення задається в гігабайтах.
 Змінити дату закінчення	Готові варіанти:  Безліміт, «  Своє », додати 7/10/14/20 днів, 1/3/6/12 місяців. Позитивне число продовжує термін (додає дні до поточної дати закінчення або до «зараз», якщо термін уже минув); 0 знімає обмеження за терміном.
 Лог IP	Показує зафіксовані IP-адреси клієнта (з відмітками часу, якщо є). З лога доступні «  Оновити » та «  Очистити IP » (з підтвердженням «  Підтвердити очищення IP? »).
 Ліміт IP	Обмеження одночасних IP. Варіанти:  Безліміт (0), 1–10 або «  Своє » (цифрова клавіатура).
 Встановити користувача Telegram	Показує поточний прив'язаний Telegram User ID клієнта; дозволяє очистити прив'язку («  Видалити користувача Telegram » з підтвердженням). Прив'язка нового користувача виконується через системний вибір контакту Telegram.
 Вкл./Викл.	Вмикає або вимикає клієнта. Потребує підтвердження «  Підтвердити вкл/викл користувача? ».

Усі операції, що змінюють конфігурацію (ліміт трафіку/IP, дата закінчення, прив'язка/відв'язка Telegram-користувача, вкл/викл), за потреби позначають Xray на перезапуск, щоб зміни набули чинності. Після успішної операції бот виводить підтвердження вигляду «  : ... » і знову показує картку.

Будь-яке введення цифр у майстрах обмежено значенням < 9999999.

14.5. Сповіщення та звіти

Сповіщення надсилаються всім адміністраторам (усім User ID з `tgBotChatId`).

Шина подій і вибір сповіщень

Сповіщення побудовані на єдиній шині подій, а каналів доставки два — **Telegram** та **електронна пошта (SMTP)**. Для кожного каналу окремо вибирається, про які події сповіщати. У **Налаштування** → **Telegram** це робиться на вкладці **Notifications**, у **Налаштування** → **Email** — на однойменній вкладці.

Події згруповані картками; у кожній групі є головний перемикач із лічильником увімкнених подій (п/всього) та проміжним станом, коли вибрано лише частину. Доступні групи:

- **Outbound** — «Down» (`outbound.down`) та «Up» (`outbound.up`): падіння та відновлення outbound.
- **Xray Core** — «Crash» (`xray.crash`): аварійне завершення ядра Xray.
- **Nodes** — «Down» (`node.down`) та «Up» (`node.up`): вузол став недоступним або відновився.
- **System** — «CPU high (%)» (`cpu.high`) та «Memory high (%)» (`memory.high`): висока завантаженість процесора та оперативної пам'яті. У обох подій поруч знаходиться inline-поле порогу у відсотках.
- **Security** — «Login attempt» (`login.attempt`): спроба входу в панель.

Набір увімкнених подій зберігається окремо: для Telegram — у `tgEnabledEvents` , для Email — у `smtpEnabledEvents` . За замовчуванням в обох каналах увімкнено «Login attempt» та «CPU high» (значення `login.attempt,cpu.high`).

Сповіщення про вхід у панель

Керується прапором **Сповіщення про вхід** (`tgBotLoginNotify` , за замовчуванням увімкнено). При кожній спробі входу у веб-панель адміністраторам надходить повідомлення:

- При успіху: «  Успішний вхід у панель.» + хост, ім'я користувача, IP, час.
- При невдачі: «  Помилка входу в панель.» + хост, **причина** (наприклад, «Помилка 2FA» при невірному другому факторі), ім'я користувача, IP, час.

Перевищення навантаження на CPU та пам'ять

Раз на хвилину панель перевіряє завантаженість процесора та оперативної пам'яті. Якщо поріг `tgCpu` > 0 і середнє навантаження CPU за хвилину його перевищує, адміністраторам надходить: «  Завантаженість процесора становить N%, що перевищує порогове значення M%». Аналогічно перевіряється завантаженість RAM проти порогу `tgMemory` (за замовчуванням 80%) — подія «Memory high (%)».

Обидва пороги задаються inline-полями поруч з подіями «CPU high (%)» та «Memory high (%)» у групі **System** на вкладці Notifications (див. «Шина подій і вибір сповіщень» нижче). Для каналу Email діють окремі ключі `smtpCpu` та `smtpMemory` . При значенні порогу 0 відповідну перевірку вимкнено.

Періодичний звіт (за розкладом)

Планується за cron-виразом із поля **Частота сповіщень** (`tgRunTime` , за замовчуванням `@daily`). Якщо значення порожнє або некоректне, використовується `@daily` . Звіт включає:

Конструктор розкладу

Поле **Частота сповіщень для адміністраторів від бота** задається не введенням рядка вручну, а через конструктор розкладу. Спочатку у випадяючому списку вибирається режим:

- `@every` — повторювати з інтервалом — з'являються поле числа та вибір одиниці (**Секунди / Хвилини / Години**); підсумок збирається у вираз вигляду `@every 6h` .
- `@hourly` — щогодини, `@daily` — щодня о 00:00, `@weekly` — щотижня, `@monthly` — щомісяця — готові пресети, що зберігаються як відповідний макрос (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Довільний (crontab)** — поле для власного crontab-виразу. Планувальник панелі працює з увімкненими секундами, тому довільний вираз складається з **6 полів**: секунда, хвилина, година, день місяця, місяць, день тижня (наприклад, `0 30 8 * * *` — щодня о 08:30:00). При переключенні на цей режим поле заповнюється crontab-еквівалентом поточного вибору, щоб було від чого відштовхуватися.

Приклад: значення поля «Частота сповіщень» (`tgRunTime`). Підтримуються як готові скорочення, так і повний crontab-формат:

Значення	Коли спрацьовує
<code>@daily</code>	раз на добу опівночі (значення за замовчуванням)
<code>@hourly</code>	щогодини
<code>@every 6h</code>	кожні 6 годин

Значення	Коли спрацьовує
0 9 * * *	щодня о 09:00
0 9 * * 1	щопонеділка о 09:00
0 */12 * * *	кожні 12 годин (о 00:00 та 12:00)

Порядок полів у crontab: хвилина, година, день місяця, місяць, день тижня.

1. Рядок « Заплановані звіти: <розклад>» та поточні дата/час.
2. **Стан сервера** (див. нижче).
3. Блок «Незабаром кінець» по inbound та клієнтах.
4. Персональні сповіщення клієнтам із прив'язаним Telegram User ID — кожному клієнту-неадміну надходить список його підписок, у яких незабаром вичерпається трафік/термін (з урахуванням вимкнених).
5. Якщо увімкнено **Резервне копіювання бази даних** (`tgBotBackup`) — резервна копія БД адміністраторам.

Стан сервера містить: хост, версію 3X-UI та Xray, IPv4/IPv6, час роботи (у днях), середнє навантаження (Load1/2/3), RAM (поточна/всього), кількість онлайн-клієнтів, лічильники TCP/UDP-з'єднань, сумарний мережевий трафік (↑/↓) та статус Xray.

«Незабаром кінець» показує:

- по inbound: кількість вимкнених і кількість «незабаром вичерпних», потім перерахування таких inbound (Remark, порт, трафік, дата закінчення);
- по клієнтах: те саме, плюс картки клієнтів і кнопки з їхніми email (натискання відкриває картку клієнта).

Пороги «незабаром вичерпання» беруться із загальних налаштувань панелі: запас за трафіком (у ГБ) та запас за терміном (у днях). «Вичерпним» вважається inbound/клієнт, у якого до ліміту трафіку залишилося менше порогу АБО до дати закінчення залишилося менше порогу.

14.6. Резервне копіювання та логи

- **Резервна копія БД** (кнопка « Резервна копія БД» або прапор у періодичному звіті): бот надсилає час резервного копіювання, файл бази даних (`x-ui.db` , або `x-ui.dump` для PostgreSQL) та файл конфігурації Xray `config.json` .

Ім'я файлу резервної копії, який надсилає бот, формується за адресою сервера: використовується значення **webDomain**, а якщо воно не задано — публічний IP сервера. Це допомагає зрозуміти, з якого сервера надійшов файл, коли резервні копії збираються з кількох панелей. Якщо адресу визначити не вдалося, застосовується узагальнена назва.

- **Лог банів** (кнопка « Лог банів»): надсилає поточний та попередній файли логів IP-адрес, забаних через перевищення IP-ліміту. Порожні файли не надсилаються.

14.7. Особливості роботи

- **Довгі повідомлення** розбиваються на частини (поріг ~2000 символів), inline-клавіатура прикріплюється до останньої частини.
 - **Паралельність**: команди та натискання кнопок обробляються конкурентно (пул до 10 одночасних обробників).
 - **Надійність відправки**: при помилках з'єднання повідомлення надсилаються повторно з експоненціальною затримкою (1с/2с/4с, до 3 спроб).
 - **Кешування**: дані «Стан сервера» кешуються, щоб часті натискання «Оновити» не навантажували систему.
 - **Перезапуск бота**: при збереженні налаштувань, що стосуються бота (прапор увімкнення, токен, User ID адміністраторів або адреса API-сервера), панель сама зупиняє попередній цикл опитування та запускає новий з актуальними параметрами — перезавантаження панелі для цього не потрібне. Одночасно працює лише один екземпляр отримання оновлень.
-

15. Гео-бази (geoip / geosite та користувацькі)

Гео-бази — це бінарні файли `.dat`, які Xray-core використовує для маршрутизації та фільтрації трафіку за належністю до країни (IP-діапазони) або за категорією доменів. Панель уміє завантажувати й оновлювати як стандартний набір гео-файлів, так і довільні користувацькі джерела, указані за URL. Усі файли зберігаються в каталозі `bin` поруч із бінарником Xray (шлях за замовчуванням `bin`, перевизначається змінною оточення `XUI_BIN_FOLDER`).

15.1. Що таке geoip.dat і geosite.dat

- **geoip.dat** — база відповідностей «IP-адреса → код країни/регіону». Застосовується в правилах маршрутизації у вигляді `geoip:<код>`, наприклад `geoip:ru`, `geoip:cn`, а також для спеціальних міток `geoip:private` (приватні/локальні мережі). За змістом це відповідь на запитання «у якій країні розташований цей IP».
- **geosite.dat** — база відповідностей «домен → категорія/список». Застосовується у вигляді `geosite:<категорія>`, наприклад `geosite:category-ads-all` (рекламні домени), `geosite:google`, `geosite:ru`. За змістом це згруповані списки доменів.

Ці файли потрібні, щоб будувати правила на кшталт «весь трафік на російські IP/домени — напряму, решта — через outbound» та подібні. Самі правила задаються в розділі маршрутизації Xray; гео-бази лише постачають для них дані. Без актуальних гео-файлів правила, що посилаються на `geoip:` / `geosite:`, не спрацюють або спиратимуться на застарілі списки.

Приклад: правило «російські домени та IP — напряму». Таке правило в розділі маршрутизації спрямовує весь трафік до російських ресурсів в outbound з тегом `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Стандартні гео-файли та їх оновлення

Панель містить фіксований «білий список» (allowlist) із шести стандартних файлів із жорстко прописаними джерелами завантаження. Оновлення виконується через `POST /panel/api/server/updateGeofile/:fileName` (або без імені файлу — для оновлення всіх одразу).

Приклад: оновлення одного файлу й усіх одразу через API. Оновити лише `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/geoip_RU.dat' \
  -H 'Cookie: 3x-ui=<session-cookie>'
```

Оновити всі шість стандартних файлів одним запитом (ім'я файлу не вказано):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Успішна відповідь:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Ім'я файлу	Джерело (репозиторій релізів)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Особливості оновлення стандартних файлів:

- **Кнопка оновлення одного файлу.** Перед завантаженням виводиться підтвердження: «Ви дійсно хочете оновити геофайл?» з поясненням «Це оновить файл #filename#.» (англ. *Do you really want to update the geofile? This will update the #filename# file.*). При успіху спливає сповіщення «Геофайли успішно оновлено» (англ. *Geofile updated successfully*).
- **Кнопка «Обновить все»** (англ. *Update all*) завантажує всі шість файлів. Підтвердження: «Це оновить усі геофайли.» (англ. *This will update all geofiles.*).
- **Умовне завантаження.** Якщо локальний файл уже є, у запит підставляється заголовок `If-Modified-Since` з часом модифікації файлу. Відповідь сервера `304 Not Modified` означає, що файл не змінювався — повторно він не завантажується, оновлюється лише позначка часу файлу.
- **Безпека імені файлу.** Приймаються лише імена з allowlist; ім'я перевіряється на відсутність `..`, роздільників шляху `/` і `\`, абсолютних шляхів і зобов'язане відповідати шаблону `^[a-zA-Z0-9._-]+\\.dat$`. Будь-яке ім'я поза списком відхиляється з помилкою «Invalid geofile name».
- **Перезапуск Xray.** Після завантаження гео-файлів Xray-core перезапускається, щоб він перечитав оновлені бази. Якщо перезапуск не вдавсь, у повідомлення про помилку додається відповідний рядок.

Оновлення гео-баз із командного рядка (x-ui)

Гео-бази можна оновлювати й без панелі — через інтерактивне меню `x-ui` (пункт оновлення гео-файлів) або неінтерактивною командою `x-ui update-all-geofiles`. Для кожного файлу набору (geoip/geosite, включно з наборами IR і RU) виводиться окремий статус: «оновлено», «уже актуальний» або «помилка завантаження». При невдалому завантаженні хибне повідомлення про успіх не друкується. Перезапуск Xray (а отже й розрив активних з'єднань) відбувається лише якщо хоча б один файл дійсно було оновлено; якщо жоден файл не змінився (усі повернули `304 Not Modified`), панель і Xray не перезапускаються.

15.3. Авто-оновлення гео-даних засобами Xray (Geodata Auto-Update)

Додаткові `.dat` -джерела за довільним URL додаються не засобами панелі, а через нативну секцію `geodata` Xray-core. Відповідний розділ винесено в модальне вікно оновлень Xray (Дашборд → оновлення Xray, `xrayUpdates`) — це вкладка «Автообновление Geodata» (англ. *Geodata Auto-Update*). Панель тут лише редагує ключ `geodata` у шаблоні конфігурації Xray; завантаженням, перевіркою та гарячим перезавантаженням файлів займається саме ядро Xray.

У верхній частині розділу показано підказку: «Xray завантажує ці файли за розкладом і перезавантажує їх без перезапуску. URL повинні бути HTTPS. Файл має вже існувати в папці `bin`, перш ніж Xray зможе його оновлювати.» (англ. *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Поля розділу

- **Расписание (cron)** (англ. *Schedule (cron)*) — рядок `cron` із 5 полів; значення за замовчуванням `0 4 * * *` (щодня о 04:00). Під час збереження перевіряється, що рядок містить рівно 5 полів, інакше виводиться помилка «Cron должен содержать 5 полей, напр. 0 4 * * *».
- **Скачивать через outbound (необязательно)** (англ. *Download through outbound (optional)*) — розкритий список із тегами доступних `outbound` (плюс `outbound` підписок), через який Xray завантажуватиме файли; `outbound` із протоколом `blackhole` до списку не потрапляють. Поле можна залишити порожнім — тоді використовується пряме підключення. Цей вибір не залежить від `outbound` для власних запитів панелі (див. §11): в авто-оновлення `geodata` свій окремий `outbound` для завантаження.
- **Список файлов** — кожен рядок задає пару «URL + Имя файла» (англ. *File name*). URL має починатися з `https://` (інакше «Для каждого файла нужен HTTPS URL.»). Ім'я файлу вказується простим, без шляхів і роздільників — лише символи `^[A-Za-z0-9._-]+$` (інакше «Имя файла должно быть простым, например `geosite_custom.dat` (без путей).»). Під час уведення URL панель намагається підставити ім'я файлу автоматично з останнього сегмента шляху. Кнопка «Добавить файл» (англ. *Add file*) додає рядок, кнопка-кошик видаляє його.

Якщо список порожній, показується підказка: «Файли не налаштовано. У правилах маршрутизації файли вказуються як `ext:geosite_custom.dat:category`.» (англ. *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*).

Збереження

Кнопка «Сохранить и перезапустит Xray» (англ. *Save & Restart Xray*) виводить підтвердження «Зберегти налаштування `geodata`?» з поясненням «Шаблон конфігурації Xray буде оновлено, а Xray перезапущено.» (англ. *Save geodata settings? This updates the Xray config template and restarts Xray.*). Після збереження ключ `geodata` записується в шаблон конфігурації (`POST /panel/api/xray/update`) і Xray перезавантажується (`POST /panel/api/server/restartXrayService`). Якщо список файлів порожній, ключ `geodata` з шаблону видаляється.

Важливі особливості:

- **Файл має вже існувати в `bin`**. Xray оновлює лише ті `.dat` -файли, які на момент запуску вже присутні в папці `bin`. Тому новий користувацький файл спершу кладуть у `bin` вручну (або хоча б створюють

там порожню/застарілу версію під потрібним іменем), і лише потім Xray починає підтримувати його в актуальному стані за розкладом.

- **Гаряче перезавантаження.** Після планового завантаження Xray перерахує оновлені бази без повного перезапуску процесу.
- **Сумісність.** Раніше завантажені гео-файли (як стандартні, так і користувацькі) продовжують працювати в правилах маршрутизації із синтаксисом `ext:` без змін.

Якщо список порожній, відображається підказка: «Користувацьких джерел гео поки немає — натисніть «Додати», щоб створити» (англ. *No custom geo sources yet — click Add to create one*).

Колонки таблиці та поля джерела

Поле (UI)	JSON	Значення за замовчуванням	Опис
Тип (<i>Type</i>)	<code>type</code>	— (обов'язково)	Тип ресурсу: лише <code>geosite</code> або <code>geoip</code> . Визначає ім'я підсумкового файлу.
Псевдонім (<i>Alias</i>)	<code>alias</code>	— (обов'язково)	Короткий ідентифікатор джерела. З нього й типу будується ім'я файлу.
URL (<i>URL</i>)	<code>url</code>	— (обов'язково)	Пряме посилання на <code>.dat</code> -файл (http/https).
Включено (<i>Enabled</i>)	—	—	Ознака активності джерела в списку.
Оновлено (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Час останнього успішного оновлення (Unix-час; <code>0</code> — ще не оновлювався).
Маршрутизація (<code>ext:...</code>) (<i>Routing (ext:...)</i>)	—	—	Готовий рядок для правил маршрутизації: <code>ext:<файл.dat>:tag</code> .
Дії (<i>Actions</i>)	—	—	Кнопки «Изменить», «Удалить», «Обновить сейчас».

Додатково в БД зберігаються службові поля: `localPath` (фактичний шлях до файлу в каталозі `bin`), `lastModified` (значення заголовка `Last-Modified` від сервера, використовується для умовного завантаження), `createdAt` і `updatedAt`.

Іменування файлів

Ім'я підсумкового файлу формується автоматично з типу й псевдоніма:

- тип `geoip` → `geoip_<alias>.dat`;
- тип `geosite` → `geosite_<alias>.dat`.

Наприклад, джерело з типом `geosite` і псевдонімом `myads` створить файл `geosite_myads.dat`.

Приклад: додавання джерела через API. Додати свій список рекламних доменів як `geosite`-ресурс із псевдонімом `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Панель завантажить файл у каталог `bin` як `geosite_myads.dat`, збереже запис і перезапустить Xray.

Кнопки та дії

- **Добавить** (англ. *Add*) — відкриває форму «Добавить источник» (англ. *Add custom geo*). Кнопка збереження — «Сохранить» (англ. *Save*). API: `POST /add`.
- **Изменить** (англ. *Edit*) — форма «Изменить источник» (англ. *Edit custom geo*). API: `POST /update/:id`. Під час зміни типу або псевдоніма старий файл видаляється, новий завантажується заново.
- **Удалить** (англ. *Delete*) — підтвердження «Видалити це користувацьке джерело?» (англ. *Delete this custom geo source?*). Видаляє запис із БД і сам `.dat`-файл. API: `POST /delete/:id`. При успіху: «Користувацький гео-файл «<ім'я>» видалено».
- **Обновить сейчас** (англ. *Update now*) — перезавантажує конкретне джерело й оновлює позначку часу. API: `POST /download/:id`. При успіху: «Geofile «<ім'я>» оновлено».
- **Обновить все** — оновлює всі користувацькі джерела одразу. API: `POST /update-all`. При повному успіху: «Усі користувацькі джерела оновлено» (англ. *All custom geo sources updated*). Якщо хоча б одне джерело не оновилося, операція вважається частково неуспішною з повідомленням «Не вдалося оновити одне або кілька користувацьких джерел» (англ. *One or more custom geo sources failed to update*), а у відповіді перелічуються успішні й неуспішні джерела.

Після будь-якої з дій (додавання, зміна, видалення, оновлення, оновлення всіх за наявності успіхів) Xray-core перезапускається.

Покроково: додавання джерела

1. Натисніть «Добавить».
2. У полі «Тип» виберіть `geosite` або `geoip`.
3. У полі «Псевдоним» уведіть ідентифікатор (лише малі латинські літери, цифри, `-` і `_`; підказка-плейсхолдер: `a-z 0-9 _ -`).
4. У полі «URL» укажіть пряме посилання на `.dat`-файл (має починатися з `http://` або `https://`).
5. Натисніть «Сохранить». Панель одразу завантажить файл у каталог `bin`, збереже запис і перезапустить Xray.

15.4. Валідація та обмеження

Під час створення та зміни джерела виконуються суворі перевірки. Повідомлення про помилки:

Умова	Повідомлення (RU)	Повідомлення (EN)
Тип не <code>geosite</code> / <code>geoip</code>	Тип должен быть geosite или geoip	Type must be geosite or geoip

Умова	Повідомлення (RU)	Повідомлення (EN)
Порожній псевдонім	Укажите псевдоним	<i>Alias is required</i>
Неприпустимі символи в псевдонімі (не <code>^[a-z0-9_-]+\$</code>)	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
Псевдонім зарезервований	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
Порожній URL	Укажите URL	<i>URL is required</i>
URL не парситься	Некорректный URL	<i>URL is invalid</i>
Схема не http/https	URL должен использовать http или https	<i>URL must use http or https</i>
Порожній/некоректний хост, або заблокований SSRF-захистом	Некорректный хост URL	<i>URL host is invalid</i>
Дублікат «тип + псевдонім»	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
Джерело не знайдено	Источник не найден	<i>Custom geo source not found</i>
Помилка завантаження	Ошибка загрузки	<i>Download failed</i>

Підказки у формі (валідація на клієнті): «Псевдоним: только a-z, цифры, - и _» (*Alias may only contain lowercase letters, digits, - and _*) і «URL должен начинаться с http:// или https://» (*URL must start with http:// or https://*).

Додаткові технічні обмеження:

- **Зарезервовані псевдоніми.** Не можна використовувати псевдоніми, що конфліктують зі стандартними файлами. Зарезервовано (порівняння реєстронезалежне, дефіс прирівнюється до підкреслення): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Наприклад, `geosite-ru` буде відхилено як `geosite_ru`.
- **SSRF-захист.** Хост URL резолвиться в IP, і якщо він указує на приватну/внутрішню адресу, завантаження блокується (користувач бачить «Некорректный хост URL»). Це не дає використовувати панель для звернень до внутрішніх сервісів.
- **Захист від path traversal.** Кінцевий шлях файлу зобов'язаний перебувати всередині каталогу `bin` (з розв'язанням симлінків); спроба вийти за його межі відхиляється.
- **Мінімальний розмір файлу.** Завантажений файл вважається валідним лише якщо він не менший за 64 байти; занадто маленький файл відхиляється з помилкою завантаження.
- **Проксі та умовне завантаження.** Якщо в налаштуваннях панелі задано проксі, завантаження йде через нього; в інших випадках використовується пряме з'єднання з SSRF-безпечним транспортом. Як і для стандартних файлів, застосовується `If-Modified-Since / 304 Not Modified` (повторно незмінений файл не завантажується). Таймаут завантаження — 10 хвилин, проба доступності URL (HEAD, за потреби — частковий GET) — 12 секунд.

15.5. Автоперевірка під час старту панелі

Під час запуску панель проходить по всіх користувацьких джерелах і для кожного перевіряє наявність та цілісність локального файлу (файл відсутній, є каталогом або менший за 64 байти). Якщо файл відсутній або

пошкоджений, виконується проба джерела і спроба повторного завантаження. Це гарантує, що після переустановлення або втрати каталогу `bin` користувацькі гео-файли буде відновлено автоматично.

15.6. Використання гео-баз у правилах маршрутизації

У правилах маршрутизації Xray гео-бази використовуються в полях на кшталт `domain / ip` через префікси:

- **geoip:** для IP-баз — `geoip:<код>`. Приклади: `geoip:ru`, `geoip:cn`, `geoip:private`. Береться з `geoip.dat` (або `geoip_RU.dat` тощо, якщо правило вказує на конкретний файл).
- **geosite:** для доменних баз — `geosite:<категорія>`. Приклади: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Береться з `geosite.dat`.

Приклад: блокування реклами через geosite. Правило, що відправляє всі рекламні домени в «чорну діру» (передбачається `outbound` з тегом `blocked` і протоколом `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Для **користувацьких** файлів використовується синтаксис зовнішнього файлу `ext:`. Підказка в UI: «У правилах маршрутизації використовуйте значення як `ext:файл.dat:тег` (замініть тег).» (англ. *In routing rules use the value column as ext:file.dat:tag (replace tag)*). Формат:

```
ext:<имя_файла.dat>:<тег>
```

де `<имя_файла.dat>` — це `geoip_<alias>.dat` або `geosite_<alias>.dat`, а `<тег>` — конкретний список/категорія всередині файлу. Панель у колонці «Маршрутизація (ext:...)» підказує готовий шаблон вигляду `ext:geosite_myads.dat:tag` — лишається замінити `tag` на потрібний тег. Ім'я такого файлу задається в розділі «Автообновление Geodata» (див. §15.3) у полі «Имя файла» — наприклад `geosite_custom.dat`; на нього посилаються в правилах як `ext:geosite_custom.dat:category`.

Приклад: правило на основі користувацького файлу. Якщо додано джерело типу `geosite` з псевдонімом `myads`, а всередині `.dat`-файлу список позначено тегом `ads`, правило маршрутизації виглядає так:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Для IP-джерела (тип `geoip`, псевдонім `mycorp`, тег `office`) поле буде `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Експлуатація: резервні копії, логи, оновлення, CLI

Цей розділ охоплює щоденне обслуговування панелі: створення та відновлення резервних копій бази даних, перегляд журналів (логів) панелі та Xray, перезапуск і зупинку сервісів, оновлення Xray і самої панелі, periodical tasks (cron) та видалення панелі. Частина операцій виконується з веб-інтерфейсу (вкладки на сторінці «Дашборд» і «Налаштування панелі»), частина — з консольного меню `x-ui` на сервері.

16.1. Резервне копіювання та відновлення бази даних

Усі дані панелі (inbound, клієнти, групи, вузли, налаштування) зберігаються в одній базі. Управління резервними копіями доступне на сторінці «Дашборд» у вкладці «Резервна копія», заголовок блоку — «Бекап і відновлення».

Панель підтримує два рушії БД, і поведінка резервного копіювання залежить від цього:

- **SQLite** (варіант за замовчуванням) — дані знаходяться у файлі `x-ui.db`.
- **PostgreSQL** — якщо панель налаштована на PostgreSQL, у блоці відображається підказка:

«Ця панель працює на PostgreSQL. „Резервна копія“ завантажує архів `pg_dump (.dump)`, а „Відновлення“ завантажує його назад через `pg_restore`. Відновлення також приймає базу SQLite (`.db`) або міграційний дамп SQLite та імпортує їхні дані в PostgreSQL. На сервері мають бути встановлені клієнтські інструменти PostgreSQL (`pg_dump` і `pg_restore`).»

Експорт (створення копії)

Кнопка «Експорт бази даних» (англ. Back Up) завантажує файл резервної копії на ваш пристрій.

Рушій БД	Ім'я файлу	Що відбувається на сервері
SQLite	<code>{host}_PPPP-MM-DD_ГГXBCC.db</code>	Спочатку виконується checkpoint WAL, щоб файл містив останні записи, потім файл цілком читається і передається для завантаження
PostgreSQL	<code>{host}_PPPP-MM-DD_ГГXBCC.dump</code>	Запускається <code>pg_dump</code> , архів передається для завантаження

З версії 3.4.2 ім'я завантажуваного файлу резервної копії містить адресу, за якою ви відкрили панель, і **дату-час**: `{host}_PPPP-MM-DD_ГГXBCC` з розширенням `.db` (SQLite) або `.dump` (PostgreSQL) — наприклад `panel.example.com_2026-06-27_000000.db`. Так файли групуються за сервером і сортуються за часом, а кілька копій за один день не перезаписують одна одну. Те саме ім'я використовується в резервних копіях, що надсилає бот Telegram (`host` береться з домену/IP панелі, запасний варіант — `x-ui`).

Підказки в інтерфейсі:

- SQLite: «Натисніть, щоб завантажити файл `.db`, що містить резервну копію вашої поточної бази даних, на ваш пристрій.»
- PostgreSQL: «Натисніть, щоб завантажити дамп PostgreSQL (`.dump`) поточної бази даних на ваш пристрій.»

Технічно експорт — це запит `GET /panel/api/server/getDb`. Ім'я вкладення формується сервером (Content-Disposition) залежно від рушія.

Ім'я файлу резервної копії формується за адресою сервера, а не фіксованим `x-ui.db` / `x-ui.dump`. При завантаженні через браузер воно береться з адреси панелі в адресному рядку (хост запити), інакше — з налаштованого веб-домену, а при його відсутності — з публічного IP сервера (спочатку IPv4, потім IPv6), з відкатом на `x-ui`. Так резервні копії з різних серверів легко розрізняти. Розширення залишається `.db` для SQLite і `.dump` для PostgreSQL; резервні копії через Telegram іменуються за тим самим доменом/IP.

Приклад: завантаження резервної копії через API. Той самий експорт можна отримати запитом з консолі — наприклад, для скрипту автоматичного резервного копіювання. Потрібна авторизована сесія (cookie входу):

```
# 1) Логінімося і зберігаємо cookie сесії
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Завантажуємо файл бази (ім'я задає сервер: x-ui.db або x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Якщо панель відкрита за базовим шляхом (Web Base Path), його потрібно додати в URL: `...:2053/<base_path>/panel/api/server/getDb`.

Імпорт (відновлення)

Кнопка «Імпорт бази даних» (англ. Restore) відкриває вибір файлу та завантажує його на сервер для відновлення (POST `/panel/api/server/importDB`, поле форми `db`).

Підказки в інтерфейсі:

- SQLite: «Натисніть, щоб вибрати і завантажити файл `.db` або міграційний дамп (`.dump`) з вашого пристрою для відновлення бази даних.»
- PostgreSQL: «Натисніть, щоб вибрати і завантажити бекап PostgreSQL (`.dump`), базу SQLite (`.db`) або міграційний дамп SQLite для відновлення бази даних. Це замінить усі поточні дані.»

З 3.5.0 тип файлу визначається за вмістом (PGDMP — архів `pg_dump`; заголовок «SQLite format 3» — база `.db`; початкові `PRAGMA / BEGIN TRANSACTION` — текстовий SQL-дамп), а не за розширенням; діалог вибору приймає `.dump`, `.db` на обох СУБД. SQLite-файли, завантажені на PostgreSQL-панель, імпортуються в PostgreSQL однією транзакцією (див. 3.14); перед імпортом файл перевіряється як справжня панельна БД, старі схеми автоматично домігруються. Перед відновленням `pg_dump`-архіву панель заздалегідь (до зупинки Xray) перевіряє його читабельність: якщо дамп створено новішим PostgreSQL, виводиться помилка з точною командою — наприклад, «...run 'x-ui pgclient 17'...».

Процес імпорту для SQLite (важливо розуміти, що він атомарний і з відкатом):

1. Завантажений файл перевіряється на формат — це має бути валідна база SQLite; інакше повертається помилка «Invalid db file format».
2. Файл зберігається у тимчасовий `x-ui.db.temp` і проходить перевірку цілісності.
3. Xray зупиняється перед заміною БД.

4. Поточна база перейменовується в резервну `x-ui.db.backup` (fallback).
5. Тимчасовий файл переміщується на місце робочої БД, виконується ініціалізація та міграції схеми, потім міграція inbound.
6. **Якщо будь-який крок завершився помилкою** — виконується відкат: відновлюється попередня база з `x-ui.db.backup`, і Xray перезапускається на старих даних.
7. При успіху fallback-файл видаляється, і **Xray автоматично перезапускається** вже на відновлених даних.

Повідомлення інтерфейсу за підсумком:

Результат	Текст
Успіх	«База даних успішно імпортована»
Помилка імпорту	«Під час імпорту бази даних виникла помилка»
Помилка читання файлу	«Під час читання бази даних виникла помилка»

Відновлення повністю замінює поточні дані. Оскільки Xray у процесі короткочасно зупиняється, наявні підключення клієнтів на час імпорту перериваються.

Файл міграції між рушіями (SQLite ⇌ PostgreSQL)

Окремо від звичайного резервного копіювання є функція «**Завантажити файл міграції**» (Download Migration, запит `GET /panel/api/server/getMigration`). Вона формує переносимий файл для переходу на інший рушій БД:

З 3.5.0 кнопка показується **лише на PostgreSQL-панелях** (на SQLite звичайний бекап `.db` тепер відновлюється прямо в PostgreSQL — окремий експорт не потрібен):

Поточний рушій	Що завантажується	Ім'я файлу	Призначення
PostgreSQL	База SQLite, зібрана з даних PostgreSQL	<code>x-ui.db</code>	Перевести панель назад на SQLite

Підказка: «Натисніть, щоб завантажити базу даних SQLite (.db), зібрану з ваших даних PostgreSQL і готову для запуску панелі на SQLite.»

Перетворення `.db` ⇌ `.dump` для SQLite можна виконати і з CLI командою `x-ui migrateDB [file]` (див. розділ 16.7). Також з 3.5.0 міграція між рушіями стала **транзакційною і без втрат** (переносяться зокрема групи клієнтів і глобальні лічильники трафіку; невдалий імпорт не змінює цільову БД), а для оновлення клієнтських інструментів PostgreSQL з'явилася команда `x-ui pgclient [версія]` і пункт **10. Install/Upgrade client tools (pg_dump/pg_restore)** у меню PostgreSQL (за потреби підключається офіційний репозиторій PostgreSQL).

Резервна копія через Telegram-бот

Якщо налаштований Telegram-бот (див. розділ про сповіщення), він вміє надсилати резервну копію прямо в чат адміністратора. Резервна копія через Telegram включає **два файли**: саму базу даних (`x-ui.db`, або `x-`

ui.dump на PostgreSQL) і конфігурацію Xray config.json. Повідомленню передуює рядок « Час резервного копіювання: ...».

Є два способи отримати резервну копію в Telegram:

1. **За запитом.** Кнопка « **Бекап БД**» у меню бота — бот негайно надсилає файли в поточний чат.
2. **Автоматично зі звітом.** У налаштуваннях бота є перемикач «**Резервне копіювання бази даних**» (Database Backup) з описом «Надсилати сповіщення з файлом резервної копії бази даних». Коли він увімкнений, при кожному periodical розсиланні звіту бот після звіту надсилає резервну копію всім адміністраторам. Period надсилання звіту задається розкладом cron бота (див. розділ 16.6). Між файлами та між адміністраторами бот робить паузи, щоб не перевищувати ліміти Telegram.

Резервна копія через бота надсилається лише якщо бот запущений; на PostgreSQL він також вимагає наявності pg_dump на сервері.

16.2. Перегляд логів

У панелі два незалежних засоби перегляду логів, обидва відкриваються з вкладки «**Логи**» на «Дашборді». Кожне вікно вміє оновлюватися (іконка «оновити» в заголовку) і завантажувати показане у файл x-ui.log (кнопка з іконкою завантаження).

Логи панелі (застосунку / syslog)

Вікно логів панелі (POST /panel/api/server/logs/{count}). Елементи управління:

Елемент	Значення за замовчуванням	Опис
Кількість рядків	20	Випадаючий список: 20 / 50 / 100 / 500 / 1000
Рівень	Info	Мінімальний рівень: Debug / Info / Notice / Warning / Error
SysLog (галочка)	вимкнено	Звідки брати логи: з буфера застосунку або з системного журналу
Автооновлення (галочка)	вимкнено	Перечитувати лог кожні 5 секунд (див. нижче)

Поведінка залежить від галочки **SysLog**:

- **Вимкнено (за замовчуванням):** логи беруться з внутрішнього кільцевого буфера панелі, відфільтровані за вибраним рівнем. Записи відображаються з рівнем (DEBUG / INFO / NOTICE / WARNING / ERROR) і джерелом: X-UI: — повідомлення самої панелі, XRAY: — переправлені повідомлення Xray.

Прості сповіщення без мітки часу та рівня (наприклад, системне повідомлення «Syslog is not supported» на Windows) тепер показуються повністю, як є. Розпізнається строго формат YYYY/MM/DD

LEVEL – тіло ; все інше виводиться без розбору, тому такі рядки більше не обрізаються (раніше перші три слова помилково трактувалися як дата/час/рівень).

- **Увімкнено:** панель виконує на сервері `journalctl -u x-ui --no-pager -n <count> -p <level>`, тобто показує системний журнал служби `x-ui`. Допустима кількість рядків – від 1 до 10000; рівень приймає значення syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). На Windows режим SysLog не підтримується – буде показано попередження, що потрібно зняти галочку і використовувати логи застосунку. Якщо `systemd` /служба недоступні, з'явиться повідомлення про помилку запуску `journalctl`.

Приклад: той самий журнал з консолі сервера. Коли панель недоступна (наприклад, не стартує), системний журнал можна прочитати напряму – це рівно та команда, яку панель виконує в режимі SysLog:

```
# останні 100 рядків рівня warning і вище
journalctl -u x-ui --no-pager -n 100 -p warning

# стежити за журналом у реальному часі
journalctl -u x-ui -f
```

Рівень у цьому вікні фільтрує **вивід**. Який мінімальний рівень взагалі пишеться в консоль/syslog, визначається рівнем логування панелі (змінна оточення, за замовчуванням `Info` ; у файл панель завжди пише на рівні `DEBUG`).

Логи доступу Xray (журнал доступу)

Окреме вікно для access-лога Xray (`POST /panel/api/server/xraylogs/{count}`). Воно розбирає рядки журналу доступу Xray і показує їх таблицю: **Date, From, To, Inbound, Outbound, Email**.

Починаючи з 3.4.1 це вікно і кнопка його виклику на картці статусу Xray підписані **«Логи доступу»** (`Access Logs`) – раніше вони називалися просто «Логи». Перейменування зроблено, щоб не плутати переглядач access-лога Xray з переглядачем логів самої панелі, у якого раніше була така сама назва.

Елемент	Значення за замовчуванням	Опис
Кількість рядків	20	20 / 50 / 100 / 500 / 1000
Фільтр	порожньо	Текстовий пошук за підрядком (застосовується після натискання Enter)
Автооновлення (галочка)	вимкнено	Перечитувати лог кожні 5 секунд (див. нижче)
Direct (галочка)	увімкнено	Показувати прямі з'єднання (трафік через <code>freedom-outbound</code>)
Blocked (галочка)	увімкнено	Показувати заблоковані з'єднання (трафік у <code>blackhole-outbound</code>)
Proxy (галочка)	увімкнено	Показувати проксований трафік

Тип події визначається автоматично за тегом вихідного з'єднання у рядку лога: відповідність тегам `freedom` → «DIRECT» (зелений), `blackhole` → «BLOCKED» (червоний), все інше → «PROXY» (синій). Рядки `api` → `api` і порожні рядки пропускаються.

Автооновлення. В обох вікнах логів (і «Логи», і «Логи доступу») є прапорець **«Автооновлення»** (`Auto Update`). Якщо його увімкнути, вміст лога автоматично перерахується кожні 5 секунд із збереженням усіх поточних налаштувань вікна — вибраної кількості рядків, рівня/фільтра та галочок `Direct` / `Blocked` / `Proxy`. Опитування припиняється, щойно вікно закрито або прапорець знятий.

Щоб це вікно показувало записи, у Xray має бути увімкнений **журнал доступу** зі шляхом до файлу (не `none`) — див. нижче. Якщо `access-лог` вимкнено або файл недоступний, вікно буде порожнім («No Record...»).

16.3. Рівень і налаштування логування Xray

Параметри журналювання самого Xray задаються на сторінці **«Конфігурації Xray»** у блоці **«Лог»** (`Log`) з попередженням:

«Логи можуть сповільнювати роботу сервера. Вмикайте лише потрібні вам види логів при необхідності!»

Поле	Переклад	Значення за замовчуванням	Опис
Рівень логів (<code>logLevel</code>)	Log Level	<code>warning</code>	Рівень деталізації лога помилок Xray. Допустимі значення: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Підказка: «Рівень журналу для журналів помилок, що вказує інформацію, яку необхідно записувати.»
Логи доступу (<code>accessLog</code>)	Access Log	<code>none</code>	Шлях до файлу журналу доступу. Спецзначення <code>none</code> вимикає логи доступу. Підказка: «Шлях до файлу журналу доступу. Спеціальне значення „none" вимикає логи доступу.»
Логи помилок (<code>errorLog</code>)	Error Log	порожньо (шлях за замовчуванням)	Шлях до файлу логів помилок; <code>none</code> вимикає їх. Підказка: «Шлях до файлу логів помилок. Спеціальне значення „none" вимикає логи помилок.»
Логи DNS (<code>dnsLog</code>)	DNS Log	<code>false</code> (вимк.)	Увімкнути логування DNS-запитів. Підказка: «Увімкнути логи запитів DNS».
Маскування адреси (<code>maskAddress</code>)	Mask Address	порожньо (вимк.)	При активації реальна IP-адреса автоматично замінюється на маскувальну в логах. Підказка: «При активації реальна IP-адреса замінюється на маскувальну в логах.»

Оскільки за замовчуванням **«Логи доступу»** = `none` , вікно «Логи Xray» (розділ 16.2) спочатку порожнє. Щоб воно запрацювало, задайте тут шлях до `access-лога` і перезапустіть Xray.

Зверніть увагу: порожній access-лог впливає лише на це вікно. Список онлайн-клієнтів на «Дашборді» та ліміт кількості IP у формі клієнта **не залежать** від access-лога — панель визначає онлайн-клієнтів і рахує їх IP-адреси через online-stats API ядра Xray (статистику з'єднань). На старих версіях ядра, де цього API немає, панель автоматично повертається до попереднього способу (читання access-лога), і тоді для ліміту IP шлях до access-лога тут як і раніше потрібен.

Ліміт за кількістю IP і fail2ban. Саме обмеження за кількістю IP у клієнта (поле «IP Limit» у формі клієнта і при масовому додаванні) застосовується на сервері лише якщо встановлений **fail2ban** — саме він банить адреси, що перевищують ліміт. Панель перевіряє наявність fail2ban (GET /panel/api/server/fail2banStatus); якщо його немає, поле «IP Limit» стає недоступним із пояснювальною підказкою (на Windows — окремим повідомленням), а раніше задані ліміти на таких серверах автоматично обнуляються, оскільки все одно не діяли. Блокування fail2ban поширюється і на TCP, і на UDP. З 3.5.0 «мертве» з'єднання (клієнт зник без коректного закриття TCP) фіксується **один раз**, а не при кожному 10-секундному скануванні: рядок [LIMIT_IP] і цикл відключення повторюються лише при реальному перепідключенні, тому лічильник fail2ban не накручується і завищувати maxretry більше не потрібно (формат логів і failregex не змінювалися). На звичайних серверах fail2ban тепер встановлюється автоматично при встановленні та оновленні панелі (див. розділ 16.5).

Приклад: блок Log , при якому вікно «Логи Xray» почне показувати записи. У JSON-конфігурації Xray це виглядає так:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Головне — замінити "access": "none" на шлях до файлу (наприклад, "./access.log"). Після збереження перезапустіть Xray, і таблиця у вікні «Логи Xray» наповниться рядками.

16.4. Управління Xray: зупинка і перезапуск

Станом Xray управляють з картки Xray на «Дашборді». Поточний стан відображається одним із значень: **Запущений** (Running), **Зупинений** (Stopped), **Невідомо** (Unknown), **Помилка** (Error). При помилці доступна спливаюча підказка «Помилка при запуску Xray».

Кнопка	Переклад	Ендпоінт	Дія
Стоп	Stop	POST /panel/api/server/stopXrayService	Зупиняє процес Xray. При успіху — сповіщення-попередження «Xray service has been stopped».

Кнопка	Переклад	Ендпоінт	Дія
Перезапуск	Restart	POST /panel/api/server/restartXrayService	Перезапускає (або запускає) Xray із застосуванням поточної конфігурації. При успіху – сповіщення «Xray service has been restarted successfully».

Після будь-якої з операцій панель розсилає новий стан по WebSocket, тому статус на «Дашборді» оновлюється без перезавантаження сторінки. Якщо операція завершилася помилкою, стан Xray стає «Помилка», і текст помилки потрапляє до сповіщення.

Крім ручного перезапуску, панель сама перевіряє, чи не потрібно перезапустити Xray (фонове завдання кожні 30 с) і чи не впав процес (перевірка кожну секунду) – див. розділ 16.6.

Монітор здоров'я тунелю (автоперезапуск Xray)

У 3.4.1 з'явився необов'язковий **монітор здоров'я тунелю**. Якщо він увімкнений, панель periodically перевіряє доступність заданого URL і після кількох невдалих перевірок поспіль автоматично перезапускає ядро Xray – це допомагає відновити тунель, що перестав пропускати трафік. За замовчуванням монітор **вимкнений** і налаштовується **лише змінними оточення служби** (у веб-інтерфейсі його налаштувань немає – так задумано авторами).

Вмикає монітор змінна `XUI_TUNNEL_HEALTH_MONITOR=true`. Змінну `XUI_TUNNEL_HEALTH_PROXY` слід спрямувати на локальний xray-inbound (наприклад `socks5://127.0.0.1:1080`) – тоді проба йде через сам Xray і перевіряє саме тунель; без неї перевіряється лише зв'язність хоста, і перезапуск не виправить проблему мережевого підключення сервера. Інші змінні задають параметри перевірки:

Змінна	Призначення	За замовчуванням
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Увімкнути монітор (вкл/вимк)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Проксі, через який йде проба (вказіть локальний xray-inbound)	порожньо
<code>XUI_TUNNEL_HEALTH_URL</code>	URL, який перевіряється	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Інтервал між перевітками	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Таймаут одної перевірки	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	Кількість невдач поспіль до перезапуску	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	Мінімальна пауза між перезапусками	<code>5m</code>

Перезапуск Xray розриває з'єднання всіх підключених клієнтів, тому має сенс залишати інтервал і поріг кількості невдач достатньо великими, щоб випадковий збій однієї проби не призводив до зайвих перезапусків.

16.5. Перезапуск і оновлення панелі

Перезапуск панелі

На сторінці «**Налаштування панелі**» є дія «**Перезапустити панель**» (Restart Panel, POST /panel/api/setting/restartPanel). При підтвердженні панель перезапускається **через 3 секунди**.

Повідомлення:

- Підтвердження: «Ви впевнені, що хочете перезапустити панель? Підтвердіть, і перезапуск відбудеться через 3 секунди. Якщо панель буде недоступна, перевірте лог сервера.»
- Успіх: «Панель успішно перезапущена».

Технічно на Linux перезапуск виконується відправленням сигналу `SIGHUP` процесу панелі (або через зареєстрований хук). На Windows відправлення `SIGHUP` не підтримується.

Самооновлення панелі (Update Panel)

На «Дашборді» доступна функція «**Оновити панель**» (Update Panel) – оновлення 3X-UI до останнього релізу прямо з веб-інтерфейсу.

Перед оновленням панель звіряє версії (GET /panel/api/server/getPanelUpdateInfo), запитуючи останній реліз 3x-ui з GitHub:

Поле	Переклад
Поточна версія панелі	Current panel version
Остання версія панелі	Latest panel version
Панель оновлена / «Оновлено»	Panel is up to date / Up to date – показується, якщо нової версії немає

Запуск оновлення – POST /panel/api/server/updatePanel. Діалог підтвердження:

- «Ви справді хочете оновити панель?»
- «Це оновить 3X-UI до версії #version# і перезапустить сервіс панелі.»

Після запуску – спливаюче повідомлення «Оновлення панелі розпочато» (Panel update started); при невдалій перевірці версій – «Перевірка оновлення панелі не вдалася» (Panel update check failed).

Що відбувається на сервері: самооновлення підтримується **лише на Linux** (на інших ОС повернеться помилка «panel web update is supported only on Linux installations»). Панель завантажує офіційний скрипт `update.sh` з GitHub (raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh) і запускає його в окремому процесі: переважно через `systemd-run` в окремому юніті (`x-ui-web-update-<timestamp>`), а при відсутності `systemd` – як окремий відсоединений процес. Після завершення скрипт оновлює компоненти і перезапускає службу панелі. Для запуску потрібен `bash`.

Якщо під час оновлення скрипт згенерував новий випадковий базовий шлях веб-панелі (Web Base Path), служба `x-ui` перезапускається автоматично, щоб новий шлях одразу запрацював. (Без перезапуску сервер

продовжував би віддавати старий шлях, а інтерфейс показував би новий, і новий адрес був би недоступний до ручного перезапуску.)

Dev-канал (rolling), з 3.4.2. Кнопка версії панелі на «Дашборді» тепер завжди відкриває вікно оновлення (раніше на актуальній стабільній збірці вона вела на сторінку релізів GitHub), а перемикач **«Dev-канал»** (`devChannelEnable`) показується завжди — навіть на стабільній збірці. Увімкнувши його, при наступному оновленні можна перейти на «котючий» dev-канал (збірки за кожним комітом у `main`). При увімкненні виводиться попередження: «Dev-збірки відстежують кожен коміт у `main` і не є стабільними релізами — автоматичного відкату немає». Без явної дії користувача нічого не оновлюється.

Канал оновлення Dev (rolling-збірки за комітами)

Крім звичайного оновлення до стабільного релізу, є необов'язковий **«Канал розробки»** (`Dev`). Перемикач з'являється у вікні оновлення панелі **лише на dev-збірках** (CI-збірки, зібрані за окремим комітом); на стабільних релізах його не видно. При увімкненні панель буде оновлюватися до rolling-збірки `dev-latest` , яка відстежує кожен коміт гілки `main` і не є стабільним релізом — показується попередження, що dev-збірки нестабільні і автоматичного відкату немає. У режимі dev вікно показує «Поточний коміт» / «Останній коміт» замість номерів версій. Функція доступна лише на Linux із `systemd`.

На dev-збірках панель показує свою версію як `dev+<короткий-коміт>` замість оманливого стабільного номера — у бейджі бічної панелі, на картці «Дашборду», у вікні оновлення, у звіті Telegram-бота про статус і у виводі команди `x-ui -v` . На стабільних релізах вигляд версії не змінюється.

На вузлах (нодах) панель цього ж 3x-ui оновлюється централізовано через `POST /panel/api/nodes/updatePanel` — див. розділ про вузли.

Автоматичне встановлення fail2ban

Щоб ліміт за кількістю IP у клієнтів (розділ 16.3) працював з коробки, при встановленні та оновленні панелі на звичайному сервері `fail2ban` тепер встановлюється і налаштовується автоматично (раніше це відбувалося лише в Docker-образі). Поведінкою керує змінна оточення `XUI_ENABLE_FAIL2BAN` : налаштування виконується, якщо змінна не задана або дорівнює `true` . Ручний запуск доступний командою `x-ui setup-fail2ban` . Збій налаштування `fail2ban` не перериває встановлення або оновлення панелі.

Встановлення і оновлення на IPv6-only хостах

Скрипти `install.sh` і `update.sh` тепер коректно працюють на серверах лише з IPv6: завантаження релізу, скрипту `x-ui.sh` і сервіс-файлів більше не використовує примусово IPv4 (`curl -4`), а бере доступний протокол. Тому панель можна встановити і оновити і на хості без IPv4-адреси.

Виправлення скриптів у 3.5.0

- **RHEL-сімейство** (Rocky/Alma/RHEL/Oracle): локальне встановлення PostgreSQL з меню тепер працює (автентифікація за паролем замість `ident`), а `fail2ban` для ліміту IP ставиться з **EPEL**.

- **Arch/Manjaro:** встановлення та оновлення більше не запускають повний апгрейд системи `расман -Syu` — лише оновлення бази пакетів.
- **32-бітний ARM:** завантажений бінарник Xray іменується `xray-linux-arm32` — саме так, як його запускає панель (раніше оновлення клало файл під іншим ім'ям, і панель продовжувала ганяти старе ядро).
- **IP-сертифікат:** автовизначений публічний IPv4 показується для підтвердження (Enter — прийняти) перед випуском; при відмові — перевірене ручне введення.
- **Вибір ACME-порту:** Enter (прийняти порт 80 за замовчуванням) більше не виводить хибне «Your input is invalid».

Перевизначення порту панелі змінною `XUI_PORT`

Порт прослуховування веб-панелі можна перевизначити змінною оточення `XUI_PORT` — вона діє лише на час роботи поточного процесу і **не змінює** збережене значення `webPort` у базі. Допустимі значення від 1 до 65535; порожнє, некоректне або таке, що виходить за діапазон, значення ігнорується (використовується `webPort`) з попередженням у лозі. Це зручно при розгортанні, насамперед у Docker: при використанні bridge-мережі публікований порт контейнера має збігатися з `XUI_PORT` — наприклад, `XUI_PORT=8080` і `ports: "8080:8080"`.

Оновлення і перемикання версії Xray-core

Тут же на «Дашборді» можна управляти версією Xray-core окремо від панелі.

- **Оновлення Xray (Xray Updates) / Вибір версії (Version)** — випадаючий список доступних версій. Підказки: «Виберіть потрібну версію» і попередження «Важливо: старі версії можуть не підтримувати поточні налаштування».
- Встановлення/зміна версії — `POST /panel/api/server/installXray/{version}`. Діалог: «Переключити версію Xray» / «Ви точно хочете змінити версію Xray?». При успіху — «Xray успішно оновлений».

Приклад: зміна версії Xray-core запитом до API. Версія вказується тегом релізу з XTLS/Xray-core (з префіксом `v`). Наприклад, переключитися на `v1.8.24`:

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — файл з cookie з прикладу в розділі 16.1.) Після встановлення Xray перезапуститься автоматично з вибраною версією.

На сервері при зміні версії Xray спочатку зупиняється, завантажується архів потрібної версії з GitHub (XTLS/Xray-core), бінарник розпаковується і замінюється, після чого Xray перезапускається з перевіркою контрольних розмірів архіву/бінарника.

16.6. Periodical tasks (cron)

Панель реєструє ряд фонових завдань при запуску. Їх розклади фіксовані (не налаштовуються в UI, за винятком розкладу Telegram-звіту та LDAP-синхронізації). Нижче — завдання, що стосуються експлуатації.

Завдання	Розклад	Призначення
Перевірка роботи Xray	кожну 1 с	Контроль, що процес Xray запущений
Перевірка необхідності перезапуску Xray	кожні 30 с	Перезапуск, якщо конфігурацію позначено як змінену
Збір трафіку Xray	кожні 5 с (старт через 5 с після запуску)	Облік трафіку inbound/клієнтів
Перевірка IP клієнтів	кожні 10 с	Контроль ліміту IP за логом
Heartbeat і синхронізація трафіку вузлів	кожні 5 с	Обмін з вузлами (нодами)
Очищення логів	щодня (@daily)	Очищає IP-limit логи і persistent access-log, ротуючи поточний лог у *.prev.log. З 3.5.0 щоденне очищення зачіпає і error-лог Xray (раніше — лише access)
Обмеження росту логів Xray	кожні 10 хвилин (@every 10m)	Нове в 3.5.0. Обрізає access- і error-лог Xray, коли будь-який з них перевищує 64 MiB ; вимкнені логи (none /порожньо) не чіпаються
Скидання трафіку за period	@hourly , @daily , @weekly , @monthly	Скидає лічильники трафіку тих inbound (та їх клієнтів), у яких задано відповідний period автоскидання
Telegram-звіт	задається у налаштуваннях бота (за замовчуванням @daily)	Розсилання звіту адміністраторам; при увімкненій опції — з прикладеною резервною копією БД (розділ 16.1)
Скидання hash-сховища Telegram	кожні 2 м	Лише при увімкненому боті
Контроль навантаження CPU для Telegram	кожні 10 с	Лише якщо задано поріг CPU > 0

Додатково:

- **Periodical скидання трафіку** спрацьовує лише для тих inbound, у яких вибрано відповідний режим автоскидання (щогодини/щодня/щотижня/щомісяця). Завдання скидає трафік самого inbound і всіх його клієнтів.
- **Перевірка закінчення терміну та вичерпання.** Вимкнення клієнтів після закінчення терміну та при вичерпанні ліміту трафіку виконується в рамках обліку трафіку: клієнти з простроченим expiry_time або вичерпаним обсягом позначаються і вимикаються, при необхідності розраховується наступний термін (для циклічних лімітів і режиму «відлік з першого використання»). На «Дашборді» і в списках це відображається статусами «Закінчилося»/«Вичерпано»/«Скоро закінчиться».
- **Автоматичний бекап у Telegram** — це побічний ефект завдання звіту, окремого cron-розкладу лише для бекапу немає. Тому частота автобекапу дорівнює частоті звіту бота.

16.7. Консольне меню і CLI (`x-ui`)

На сервері панель управляється командою `x-ui`. Без аргументів відкривається інтерактивне меню «3X-UI Panel Management Script»; з аргументом виконується конкретна підкоманда. Пункти меню, що стосуються експлуатації:

№ у меню	Пункт	Дія
1	Install	Встановлення панелі (завантажує і запускає <code>install.sh</code>)
2	Update	Оновлення всіх компонентів <code>x-ui</code> до останньої версії без втрати даних; після – автоматичний перезапуск
3	Update to Dev Channel (latest commit)	Оновлення до rolling-збірки <code>dev-latest</code> (останній коміт гілки <code>main</code>) з підтвердженням (див. 16.5)
4	Update Menu	Оновлення лише самого скрипту меню <code>x-ui</code>
5	Legacy Version	Встановлення вказаної (старої) версії панелі за введеним номером (наприклад, <code>2.4.0</code>)
6	Uninstall	Повне видалення панелі та Xray (див. 16.8)
7	Reset Username & Password	Скидання логіну/паролю адміністратора
8	Reset Web Base Path	Скидання базового шляху веб-панелі
9	Reset Settings	Скидання налаштувань до значень за замовчуванням
10	Change Port	Зміна порту панелі
11	View Current Settings	Перегляд поточних налаштувань
12–14	Start / Stop / Restart	Запуск, зупинка, перезапуск служби панелі
15	Restart Xray	Перезапуск лише Xray
16	Check Status	Поточний статус служби
17	Logs Management	Перегляд і очищення логів (див. нижче)
18–19	Enable / Disable Autostart	Увімкнути/вимкнути автозапуск служби при старті ОС
27	Update Geo Files	Оновлення гео-файлів (GeoIP/GeoSite)
25	PostgreSQL Management	Управління PostgreSQL

Нумерація пунктів меню змінилася в 3.4.1: через додавання пункту 3 «Update to Dev Channel» всі наступні пункти зсунулися на одиницю. Всього пунктів стало 28, вибір вводиться в діапазоні `[0–28]`.

Управління логами в CLI (пункт 16)

Підменю «Logs Management» тепер відкривається пунктом **17** (раніше — 16):

- **Debug Log** — потоковий перегляд журналу служби:
`journalctl -u x-ui -e --no-pager -f -p debug` (на Alpine — `grep` по `/var/log/messages`).
- **Clear All logs** — очищення системного журналу: `journalctl --rotate + journalctl --vacuum-time=1s`, після чого служба перезапускається. (Недоступно на Alpine.)

Прямі підкоманди `x-ui`

Усі доступні підкоманди:

Команда	Опис
<code>x-ui</code>	Відкрити адміністративне меню
<code>x-ui start</code>	Запустити панель
<code>x-ui stop</code>	Зупинити панель
<code>x-ui restart</code>	Перезапустити панель
<code>x-ui restart-xray</code>	Перезапустити Xray
<code>x-ui status</code>	Поточний статус
<code>x-ui settings</code>	Показати поточні налаштування
<code>x-ui enable</code>	Увімкнути автозапуск при старті ОС
<code>x-ui disable</code>	Вимкнути автозапуск
<code>x-ui log</code>	Перегляд логів
<code>x-ui banlog</code>	Перегляд логів банів Fail2ban
<code>x-ui setup-fail2ban</code>	Встановити і налаштувати fail2ban для ліміту IP (див. 16.5)
<code>x-ui update</code>	Оновити панель

| `x-ui update-dev` | Оновити панель до каналу розробки (rolling-збірка `dev-latest`) || `x-ui update-all-geofiles` | Оновити всі гео-файли (з наступним перезапуском) || `x-ui migrateDB [file]` | Перетворення бази `.db` ➔ `.dump` (SQLite) || `x-ui legacy` | Встановити застарілу версію || `x-ui install` | Встановити панель || `x-ui uninstall` | Видалити панель |

Команда `x-ui update` завантажує і запускає офіційний `update.sh` (той самий, що й веб-оновлення з розділу 16.5), запитуючи підтвердження: «This function will update all x-ui components to the latest version, and the data will not be lost.» По завершенні панель автоматично перезапускається.

Прапори `-webCert` / `-webCertKey` у підкоманді `setting`. Шляхи до сертифіката і приватного ключа веб-панелі можна задати прямо у підкоманді `x-ui setting -webCert <шлях> -webCertKey`

<шлях> — вказання будь-якого з цих прапорів зберігає відповідний шлях (як і окрема підкоманда `cert`), і панель одразу переходить на HTTPS.

Отримання API-токена через CLI

Команда отримання API-токена через CLI (пункт меню/команда `x-ui`) не показує раніше виданий токен. API-токени зберігаються лише у вигляді хешів, тому існуючий токен не можна отримати у відкритому вигляді. Якщо токени вже налаштовані, команда повідомляє їх кількість, рекомендує управляти токенами у панелі (**Settings** → **API Tokens**, див. розділ про API-токени) і одразу генерує **новий запасний токен** з іменем виду `cli-fallback-<timestamp>` та виводить його, щоб CLI залишався корисним без заходу в інтерфейс.

16.8. Видалення панелі

Видалення виконується з CLI — пункт меню **5 (Uninstall)** або команда `x-ui uninstall`. Перед видаленням запитується підтвердження (за замовчуванням «ні»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

При підтвердженні скрипт:

1. Зупиняє службу і вимикає її автозапуск (`systemctl stop/disable x-ui`, або на Alpine — `rc-service / rc-update`), видаляє unit-файл служби і перезавантажує конфігурацію `systemd`.
2. Видаляє каталоги даних і застосунку (`/etc/x-ui/`, каталог встановлення) і env-файл служби (`/etc/default/x-ui`, `/etc/conf.d/x-ui` або `/etc/sysconfig/x-ui` — залежно від дистрибутива).
3. Видаляє сам скрипт `x-ui` і виводить повідомлення «Uninstalled Successfully», а також команду для повторного встановлення.

Якщо панель використовувала PostgreSQL (в env-файлі `XUI_DB_TYPE=postgres`), після видалення файлів панелі скрипт додатково запитує, чи потрібно видалити і сам сервер PostgreSQL разом з усіма його базами: «Also purge PostgreSQL and delete all of its data?». Запит вимагає явного підтвердження (за замовчуванням — відмова) і супроводжується попередженням: видалення торкнеться **ВСІХ** баз PostgreSQL на машині, в тому числі тих, що належать іншим застосункам, і є необоротним. При відмові PostgreSQL і його дані залишаються недоторканими.

Видалення необоротне: разом з панеллю видаляється Xray і всі дані (включно з базою). Якщо дані можуть знадобитися, заздалегідь зробіть експорт бази (розділ 16.1).

16.9. Команда `x-ui migrateDB`

Починаючи з версії 3.3.0 керуючий скрипт `x-ui.sh` отримав підкоманду `migrateDB` — обгортку навколо вбудованого бінарника `x-ui` (`x-ui migrate-db`) для конвертації бази даних панелі SQLite між двома форматами: бінарним `.db` і переносимим текстовим дампом `.dump` (звичайний SQL-текст).

Що робить команда

Команда працює в двох напрямках, причому напрямок визначається **автоматично** за вхідним файлом:

Напрямок	Як називається	Що відбувається
<code>.db → .dump</code>	dump (вивантаження)	бінарна база SQLite вивантажується в текстовий SQL-файл
<code>.dump → .db</code>	restore (відновлення)	з текстового SQL-файлу заново збирається бінарна база SQLite

Під капотом скрипт викликає бінарник панелі:

- вивантаження: `x-ui migrate-db --src <вхід> --dump <вихід>`
- відновлення: `x-ui migrate-db --restore <вхід> --out <вихід>`

Синтаксис виклику

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** — вхідний файл (перший аргумент). Якщо його не вказати, береться встановлена база панелі за замовчуванням: `/etc/x-ui/x-ui.db`.
- **[output]** — шлях до вихідного файлу (другий аргумент). Необов'язковий: за відсутності ім'я вибирається автоматично поруч з вхідним файлом (див. нижче).

Приклади:

```
x-ui migrateDB                               # вивантажити /etc/x-ui/x-ui.db -> /etc/x-
ui/x-ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # зібрати .db з дампу
```

Як визначається напрямок

Скрипт дивиться на розширення вхідного файлу:

- `*.db`, `*.sqlite`, `*.sqlite3` → режим **dump** (вивантаження в текст);
- `*.dump`, `*.sql` → режим **restore** (збірка бази).

Якщо розширення не розпізнане, скрипт читає перші 16 байт файлу: сигнатура `SQLite format 3` означає бінарну базу (режим dump), інакше файл вважається дампом (режим restore).

Ім'я вихідного файлу, якщо другий аргумент не заданий:

- при вивантаженні — те саме ім'я, що й у входу, з розширенням `.dump`;
- при відновленні — те саме ім'я з розширенням `.db`.

Захисні перевірки і поведінка

- **Наявність бінарника.** Якщо бінарник `x-ui` не знайдено або не є виконуваним — виводиться помилка «x-ui binary not found ... Is the panel installed?».
- **Підтримка функції у збірці.** Скрипт перевіряє, що бінарник вміє `migrate-db --dump/--restore` (через `x-ui migrate-db -h`). Якщо ні — пропонується спочатку оновити панель командою `x-ui update`.

- **Існування вхідного файлу.** При відсутності вхідного файлу виводиться помилка і рядок із синтаксисом виклику.
- **Перезапис виходу.** Якщо вихідний файл вже існує, запитується підтвердження (за замовчуванням «ні»); без підтвердження операція скасовується. При відновленні старий вихідний файл попередньо видаляється.
- **Захист «живої» бази.** При відновленні в базу за замовчуванням `/etc/x-ui/x-ui.db`, коли панель запущена, операція відхиляється з вимогою спочатку зупинити панель (`x-ui stop`) або вибрати інший вихідний шлях. Це запобігає перезапису робочої бази у працюючого сервісу.
- При невдачі збірки бази недописаний вихідний файл видаляється.

Навіщо це потрібно

- **Резервна копія.** Текстовий `.dump` — зрозумілий людині, зручний для зберігання в системах контролю версій і для диференційного перегляду вмісту бази.
- **Перенесення.** Дамп переносимий між машинами і стійкий до відмінностей версій формату файлу SQLite — на новому сервері з нього збирається робоча `.db`.
- **Діагностика.** З `.dump` можна на очі подивитися структуру і дані панелі, не маючи під рукою інструментів SQLite.

Інтерактивний режим

Крім прямого виклику, конвертація доступна з інтерактивного меню. У підменю PostgreSQL (`x-ui` → розділ роботи з PostgreSQL) є пункт **9. Convert SQLite .db <=> .dump**: вона запитує шлях до вхідного файлу (за замовчуванням `/etc/x-ui/x-ui.db`) і до вихідного (можна залишити порожнім для авто-іменування), а напрямок, як і в CLI-режимі, визначається автоматично.

Документ підготовлено за вихідним кодом 3X-UI. Якщо якийсь пункт інтерфейсу у вашій версії відрізняється — пріоритет у поведінки панелі та підказок у самому UI.